ACM ICPC 铜牌稳妥覆盖 算法与模型 V2

先从题意找模型，再查代码。优先直接采用我原来验证使用过的实现，只修具体问题；基础树存储和容器用法与算法同样重要。

本册按识别信号讲推导，再给个人比赛代码和调用；全部模型、来源与测试可在仓库按同一编号检索。

原版 inline ll read()保留；按题目 n/T 读取常规整数，其他输入用 cin。关闭 iostream 同步后，不混用 cin 与 getchar。

纸质资料可携带数量、规格：待赛站确认。生成文件不表示全部允许带入比赛。

先按操作选容器

题目需要	首先考虑
每次取当前最小 / 最大	小根堆 / 大根堆
动态有序去重 / 允许重复	set / multiset
动态频率计数	map / unordered_map
查前驱、后继、第一个不小于 x	set / multiset
先进先出 / 后进先出	queue / stack
两端增删 / 固定窗口候选	deque / 单调队列
字符串前缀 / 最大异或数对	字符串 Trie / 二进制 Trie
单点加、前缀与区间和	Fenwick
静态区间 min/max/gcd	ST 表
动态区间合并信息 / 整段修改	线段树 / Lazy 线段树
连通关系 / 无权最短路	DSU / BFS
0/1 权 / 非负一般权最短路	0-1 BFS / Dijkstra

复杂度先看 n、q、值域和状态数；数量级只是筛选，不能替代证明或实测。

从题意到实现的六步

1 看约束

很小先穷举；n≈40 考虑折半；集合小考虑状压；大 n 寻找线性或 nlogn；值域小计数、大值域离散化。

2 看操作

连续段→前缀/差分/数据结构；最少步→下界构造/BFS/DP；可达→不变量+充分性；轮流行动→先核对终止与胜负规则。

3 写暴力

枚举合法对象，找重复计算；问未来究竟还需要哪些历史信息。

4 补证明

构造要合法且达到目标；贪心写交换；二分证单调；DP 证状态足够并划分互斥完整。

5 选源码

先用已有熟悉代码。写清 1/0 下标、闭/半开区间、初始化、值域、输出含义。

6 找反例

最小规模、全相等、全负、边界、断图/重边、多测；小数据暴力与优化对拍，记录最小反例。

DP 状态检查：保存什么 / 漏了什么 / 初值和不可达 / 转移是否重计 / 遍历依赖 / 答案位置 / max、min 还是 sum / 是否取模 / 能否滚动。

WA 先核对语义与边界；RE 查下标、空容器和递归深度；TLE 先重算复杂度；多测清实际使用过的状态。

每个代码模块单独使用。同名全局 a/g/tree/init 等来自不同模板，拼到同一题前只保留所需实现或按题目改名。本地 #include 在打印中已递归展开；无需照抄文件依赖。

我的代码习惯与 DP 状态识别

原代码验证过就直接采用：ll、1 下标、全局 a/g/dp/cnt、独立函数；原 Fenwick 的 struct/vector 同样保留。树用 vector<ll> g[N]存双向边；新题先 init 清上一组状态。

线段树沿用 pushup、pushdown、build、modify、query；DSU 沿用 init、find、unite、same。已验证的原实现与新增修复均有来源记录，不为统一外观改写。

识别信号	必须保存的状态 / 编号
一段只能接前一段	以 i 结尾的最优值 DPC01
只能右下走	到(i,j)的方案数/代价 DPC02
每件最多一次 / 无限 / 有限 / 分组	容量与阶段，遍历方向不同 DPC03–06
最后一次合并把区间切开	区间[l,r]与切分点 k DPC07
父子选择互斥	子树根选/不选 DPC08
依赖无环	拓扑序上的最优前驱 DPC09
小集合与当前位置	mask 和 last DPC10
区间很大但数位少	pos/tight/started 和题目状态 DPC11
保留原序 / 两序列匹配	末尾值或两个前缀长度 DPC12
历史只影响当前阶段	位置与有限状态 DPC13
问方案数 / 随机收益	互斥计数 / 概率加权与自环移项 DPC14–15

状态相同必须拥有相同的后续选择。max/min 保留最优；计数相加但防重；不可达要单独表示。详推导与完整代码见 V2 相同编号。

XOR / AND / OR 思维地图

XOR 是逐位模 2 加：同数两次抵消。连续区间先写前缀 XOR；求等于 k 的数量查 freq[prefix^k]，先查后加，起点 freq[0]=1。

数对贡献：某位 c 个 1、n-c 个 0，XOR 有 c(n-c)对，AND 有 C(c,2)对，OR 有 C(n,2)-C(n-c,2)对。计数值、移位位宽和累加溢出要分别检查。

最大数对/子数组→二进制 Trie；最大子集 XOR→线性基；value[u]^value[v]=w→图上相对值；独立无环正常规则游戏→SG 异或。

构造先列长度、值域、互异、正数等全部约束；最后一个数由目标 XOR 倒推，只保证 XOR 式，不保证其他限制。

从原题提取的思维缺口：2268B 补全可达性；2268C 目标实际是区间 XOR AND 区间最大值；2218E 先核对 sum n<=3105，原双循环可用；1457D 证明压缩到小 n；909F 分开证明两种排列；2261B 给 popcount 不变量补充分性。详细个人复盘见 XOR08、12–15 与 MY_XOR_REVIEW.md。

题型识别与代码索引

稳定编号与内容	页
DPC01 连续选择只能接前一段：线性 DP	13
DPC02 只能右下到达：网格 DP	14
DPC03 每件最多选一次：0/1 背包	15
DPC04 每种可无限取：完全背包	16
DPC05 每种有数量上限：多重背包	17
DPC06 同组互斥：分组背包	18
DPC07 最后一次合并切开区间：区间 DP	19
DPC08 父子选择互相限制：树形 DP	20
DPC09 依赖无环：DAG DP	21
DPC10 小集合选到哪里：状压 DP	22
DPC11 巨大数值区间但数位少：数位 DP	23
DPC12 保持原序的序列：LIS、LCS、编辑距离	24
DPC13 历史只剩阶段与位置：状态机 DP	25
DPC14 互斥的最后一步相加：计数 DP	26
DPC15 加权求和与自环移项：概率/期望 DP	27
MEX_GLOBAL 插删后问最小缺失非负数	28
MEX_CONSTRUCT mex 反推出必须出现与必须缺失的元素	29
GAME_PN 先枚举胜败状态，再证明规律	30
GAME_NIM 独立子游戏为何异或，Nim 怎样走到败态	31
MEX_STRUCTURE 全局 MEX 与区间 MEX，数据结构不能混用	32
GAME_SYMMETRY 奇偶或镜像策略要保证每一步都有合法应答	33
PAL_CENTER 连续回文看中心，子序列不能用中心扩展	34
PAL_CONSTRUCT 回文构造先镜像，再处理唯一奇数中心	35
TRIE_PREFIX 很多字符串反复查前缀，把相同前缀只存一次	36
GEN_NAV 读完题后，把答案变成六个明确问题	37
INT_PLUS 从 0 只做区间+1：正差分是必须新开的层	38
INT_SIGNED 区间±1：两端补 0 后的总变差除以 2	39
INT_LIMITS 只能改前缀或后缀：每个长度的系数被唯一确定	40
INT_FLIP 二进制区间翻转：数初始与目标差异的 1 段	41
INT_TRANSPORT 相邻搬运一单位：每道切口必须承担前缀差	42

题型识别与代码索引 续

稳定编号与内容	页
INT_ARBITRARY 区间加任意整数、每次费用 1：小差分数的精确分组	43
INT_INVARIANT 指定位置分别加减：先写操作向量，再找不变量	44
CONSTRUCT_FRAME 先从 n=1、2、3 拆限制，再构造并独立验合法	45
CONSTRUCT_BLOCK 先造可计数的完整块，再补独立增量	46
GREEDY_DOMINANCE 选择尽量小的可行末值：让所有后续更容易	47
GREEDY_INTERVAL 不相交区间取最多：先选最早结束	48
GREEDY_COST 固定目标后选最小代价，动态候选交给堆	49
GREEDY_PAIR 从最大元素反推搭档：首尾配对压低最坏一对	50
SEARCH_CHECK 把互相牵制的优化目标固定下来，再证明单调	51
SEARCH_REAL 实数阈值用二分，单峰函数才用三分	52
SEARCH_STATE 把完整状态当点，把一次合法操作当边	53
ARRAY_PREFIX 静态问和看前缀，离线区间加看差分	54
ARRAY_2D 矩形和与矩形加：二维容斥的四个角	55
ARRAY_WINDOW 双指针成立的关键是可排除一整类候选	56
ARRAY_COMPRESS 排序去重与离散化：保留顺序，别丢掉距离和次数	57
ARRAY_INVERSION 逆序对：把二重枚举分成左右内部与跨中点	58
ARRAY_MONOTONIC 单调结构删除被支配者，每个下标最多进出一次	59
COUNT_CONTRIBUTION 从逐个统计答案，改成每个元素贡献多少次	60
COUNT_ENDPOINT 固定一个位置，让其余选择变成前后缀统计	61
COUNT_CASES 分类讨论要互斥且穷尽，用最小反例定位缺类	62
SEARCH_MITM 折半搜索：把一个大指数拆成两张半集表	63
STRING_BRACKETS 括号：总数相等还要每个前缀都不欠账	64
STRING_PERIOD 字符串边界连到周期，模环周期由 gcd 决定	65
COUNT_COMBINATIONS 排列和组合：先决定对象是否可区分、顺序是否重要	66
GRID_GRAPH 先把题意翻译成点和合法移动，再数连通块	67
GRAPH_PATH 先决定边权，再选 BFS、0-1 BFS 或 Dijkstra	68
TREE_COUNT 先定根分解子树，路径问题用 LCA 或边贡献	69
GRAPH_CYCLE 先分清有向环、无向环、基环结构与桥	70
CG01 树的存储、多测与根化	71
CG02 个人原版并查集 fa/rk	72

题型识别与代码索引 续

稳定编号与内容	页
CG03 原队列 BFS 与最短路径恢复	73
CG04 显式栈 DFS 与连通块	74
CG05 全局数组 Dijkstra	75
CG06 个人树上倍增 LCA	76
CG07 四邻接网格 BFS	77
CG08 多源 BFS	78
CG09 0-1 BFS	79
CG10 Floyd 全源最短路	80
CG11 二分图染色与冲突检测	81
CG12 有向/无向环恢复	82
CG13 无权树直径与路径	83
CG15 DAG 路径计数与最长路	84
CG16 拓扑排序：先决条件到有序执行	85
CG17 最小生成树：全连通的最小总边权	86
CG18 强连通缩点：把互相可达的一团压成点	87
CG19 桥：删除一条边会破坏多少连通性	88
CG20 Euler 子树区间与树 DP 接口	89
CM01 扩展欧几里得与一般模逆元	90
CM02 线性筛、判素、分解与约数	91
CM03 负数向下/向上整除	92
CM04 基础求和与成对差贡献	93
CM05 奇偶、模不变量与环上可达	94
CM06 整数点积、叉积与线段相交	95
CM07 gcd/lcm 与差分同余	96
CM08 模快速幂、组合数与除法条件	97
CM09 奇偶与余数分类：必要条件之后还要构造	98
RT_IO 先判断输入格式，再选原 read 或 cin	99
RT_ANGLE 从向量求距离、夹角与方向	100
RT_SEARCH 实数答案的分界，或函数的唯一谷底	101
RT_DISPLAY 显示几位小数，与数值精度和取整分开考虑	102

题型识别与代码索引 续

稳定编号与内容	页
SEG-00 一页速查：先选容器，再选维护信息	103
SEG-01 单点改值、区间求和	104
SEG-02 单点改值、区间最小值	105
SEG-03 单点改值、区间最大值	106
SEG-04 单点改值、区间 gcd	107
SEG-05 区间加、区间和	108
SEG-06 区间加、区间最小值	109
SEG-07 区间加、区间最大值	110
SEG-08 区间赋值、区间和	111
SEG-09 区间赋值与区间加混合	112
SEG-10 区间最大值及次数	113
SEG-11 最大连续子段和	114
SEG-12 非负频率第 k 个	115
SEG-13 范围内第一个达到阈值的位置	116
SEG-14 多个可合并统计量	117
SEG-15 Fenwick 调用：前缀与区间不要混淆	118
STL_HUFFMAN 合并代价为两份之和，每次拿最小两份	119
STL_TOPK 只保留最大或最小 k 项，无须完整排序	120
STL_MEDIAN 中位数就是小半集合的最大值	121
STL_LAZY 堆不能删中间元素：先记失效，露出顶再清理	122
STL_NEIGHBOR 插删后找严格前驱、后继或最近值	123
STL_RESOURCE 给需求分配最小够用资源	124
STL_WINDOW 滑窗要删一份旧值，不能删掉全部重复值	125
STL_SELECT 先列操作，再选顺序容器、有序集合或堆	126
XOR01 先写二进制：消去、逆操作和奇偶	127
XOR02 区间 XOR 的前缀抵消	128
XOR03 统计 XOR 为 k 的子数组	129
XOR04 离线区间 XOR 更新与全局未知量	130
XOR05 贡献：XOR / OR / AND 三种表	131
XOR06 构造先列限制，再留最后自由量	132

题型识别与代码索引 续

稳定编号与内容	页
XOR07 最高不同位：比较、连续值域 OR、限制反例	133
XOR08 我的删除操作为什么只剩两个原值	134
XOR09 Trie：每一步只保留能决定最高位的候选	135
XOR10 XOR 图约束：相对值与分量自由度	136
XOR11 线性基不是 Trie：选子集还是选一个	137
XOR12 我的 popcount 不变量如何补上充分性	138
XOR13 我的 AND 排列：块内补码 / 最高位循环	139
XOR14 我的 XOR-gun：小值域把大 n 压到 60	140
XOR15 我的空白 C：前缀化后还差什么	141
XOR16 Nim / SG：先核对规则再异或	142
CO01 / IO02 主程序骨架 Main	143
CO02 前缀、差分、离散化 Basics	144
CO03 答案二分、双指针、窗口 Search	146
DP01 0/1 与完全背包 Knapsack	147
DP02 线性 DP 与最长递增子序列 LIS	148
DP03 小规模状压最短路径 Hamilton DP	149
DP04 石子合并区间 DP Interval Merge	150
DS04 单调栈与单调队列 Monotonic	151
DS05 稀疏表 Sparse Table	152
G04 拓扑排序 Topological Sort	153
G05 简单无向基环图环恢复 Unicyclic Graph	154
G07 Kruskal 最小生成树与森林 Kruskal MST	155
G08 堆优化 Prim 最小生成森林 Prim MST	156
G10 Euler 子树区间与迭代树 DP Euler Subtree	157
G11 强连通分量 Iterative Kosaraju SCC	158
G12 无向图找桥 Iterative Bridges	159
M01 整数与位运算 Integer and Bits	160
M02 安全模运算与快速幂 Modular Power	161
M03 普通模矩阵快速幂 Matrix Power	162
M04 乘积递推修正版 Product Recurrence	164

题型识别与代码索引 续

稳定编号与内容	页
M05 质数模数组合数 Prime Combinations	166
S01 KMP 与 Z 函数 Prefix Matching	167
S02 双模字符串哈希 Double Hash	168
S03 基础字符串处理 String Basics	169
BM01 区间单位操作与总变差	170
BM02 最少区间操作的压缩构造与变体	172
BM03 二维前缀和与矩形差分	174
BM04 逆序对与折半枚举	175
BM05 原解直接收录 二分选题 CF913D	176
BM06 原解直接收录 选最小可行值 CF2065C2	178
BM07 区间选择、首尾配对与括号判定	179
BM08 整数状态 BFS 与非负子集回溯剪枝	181
FP01 浮点误差、距离与反三角	183
GAME01 正常规则、Nim 胜着与有限 DAG SG	184
IO01 原版 ll/getchar 快读	185
IO03 cin 字符串转 i128 与原输出函数	186
MEX01 / MEX02 静态与前缀 MEX	188
PAL01 回文判定、中心扩展与构造	189
PDP01 线性与网格：最大子段、路径计数、最小代价	192
PDP02 多重背包与分组背包	194
PDP03 序列前缀与个人状态机、候选分层 DP	196
PDP04 个人数位 DP 与基础有限状态计数	198
PDP05 树形 DP：最大权独立集	199
PDP06 独立试验概率与阶段等待期望	200
PG01 树的存储、多测与根化	201
PG02 个人原版并查集 fa/rk	203
PG03 原队列 BFS 与最短路径恢复	204
PG04 显式栈 DFS 与连通块	205
PG05 全局数组 Dijkstra	206
PG06 个人树上倍增 LCA	208

题型识别与代码索引 续

稳定编号与内容	页
PG07 四邻接网格 BFS	210
PG08 多源 BFS	211
PG09 0-1 BFS	213
PG10 Floyd 全源最短路	214
PG11 二分图染色与冲突检测	215
PG12 有向/无向环恢复	216
PG13 无权树直径与路径	217
PG15 DAG 路径计数与最长路	219
PM01 扩展欧几里得与一般模逆元	220
PM02 线性筛、判素、分解与约数	221
PM03 负数向下/向上整除	223
PM04 基础求和与成对差贡献	224
PM05 奇偶、模不变量与环上可达	225
PM06 整数点积、叉积与线段相交	226
PSD01 Fenwick：个人 add/query/rangeSum	227
PSD02 单点树公共骨架与区间和	228
PSD03 单点树：区间最小值	230
PSD04 单点树：区间最大值与阈值搜索	232
PSD05 单点树：区间 gcd	234
PSD06 单点树：最大值及出现次数	236
PSD07 单点树：最大非空连续子段和	238
PSD08 频率线段树：查第 k 个	240
PSD09 单点树：多个可合并统计量	242
PSD10 区间加：和、最小值、最大值	244
PSD11 区间赋值与区间加：和	246
STL01 可执行 STL 容器与算法短例	248
STL02 堆与有序集合的典型题型	252
TRS01 实数二分与三分	255
TRS02 个人数组字符串 Trie：前缀频次与整词频次	256
X01 前缀 XOR、子数组计数与离线区间异或	257

题型识别与代码索引 续

稳定编号与内容	页
X02 按位贡献：数对与所有子数组	258
X03 XOR 构造、未知元素与最高不同位	260
X04 数组节点二进制 Trie：插入、删除、最大/最小 XOR	261
X05 图上的 XOR 约束	263
X06 XOR 线性基：插入与最大子集 XOR	264
X07 我的偶数 popcount 计数模型（2268B）	265
X08 我的最大数对双循环（2218E）	266
X09 我的 XOR-gun：最高位分类与前缀枚举	267
X10 我的 AND 排列：补码块与同最高位环	268
X11 我的 Energy Intervals 补全（2268C / 2269E）	269

【DPC01】连续选择只能接前一段：线性 DP

识别：选一个非空连续子段，使元素和最大；加入当前元素后，只能自己开头或接在上一段后。

先看什么：连续与子序列不同；必须非空时全负答案也为负。

从暴力出发：枚举左右端点，计算每个连续区间的和；不使用前缀和时甚至会重复求和。

重复与性质：所有以 i 结尾的候选，接上未来元素的能力相同，只保留和最大的候选。

推出模型：以 i 结尾的段长度为 1 或更长：dp[i]=max(a[i],dp[i-1]+a[i])；最终对所有末尾取 max。

适用条件：a[1..n]，n>=1，选择非空连续段。；任意子段和绝对值<2^60；不是要求选 k 段，也不是子序列问题。

一句证明：最优非空段若长度大于 1，删去末位后仍是以 i-1 结尾的段；替换为该末尾最优前缀不会变差。

状态含义：dp[i]：以第 i 个元素结尾的非空连续子段的最大和；滚动变量 dp 只保存上一末尾。

转移：dp=max(a[i],dp+a[i])；ans=max(ans,dp)。

初值：dp=ans=a[1]；从 i=2 开始。

遍历顺序：下标递增，只读取上一末尾。

复杂度：O(n)时间，O(1)额外空间。

反例与易错：[-8,-2,-5]答案-2；答案初值 0 等价于偷偷允许空段。；若每 m 个数额外扣 k，必须增加长度模 m 状态；只记最大子段和不够。

练习与来源：cf/date/6/6.9/3_leaner_dp.cpp：代码中的局部模型对每段长度每满/新开 m 个收 k，dp[j]记录长度模 m；用本卡理解为何还要记余数。cf/2/1057/D.cpp：长度 2/3 分块的线性子问题，环形枚举另行处理。

代码入口：PDP01

```
#include "competition/dp/linear_grid.hpp"
int main() {
    cout << max_subarray({0, -8, -2, -5}) << '\n'; // -2
}
```

原代码：cf/date/6/6.9/3_leaner_dp.cpp；cf/2/1057/D.cpp

【DPC02】只能右下到达：网格 DP

识别：矩形网格里从左上走到右下，只能向右或向下，求路径数或最小费用。

先看什么：能否后退决定依赖是否有环；有障碍的格子不能继承上方或左方状态。

从暴力出发：枚举所有右/下步序列，遇障碍丢弃；许多路径会到达相同格子。

重复与性质：到同一格子后未来可走区域一致，计数合并方案数，费用只保留最小代价。

推出模型：最后一步只可能来自上方或左方。计数相加，最小费用取二者较小值再加当前格费用；两者都不可达时保持 INF。

适用条件：n,m>=1；g[1..n][1..m]，'#'为障碍。；只能右/下；模数 1..INT_MAX。最小费用可有负数，任意路径和绝对值<2^60，费用含起终点。

一句证明：两种最后一步互斥且覆盖全部路径；固定格子下最优前缀可以独立替换。

状态含义：计数 dp[j]：当前已处理位置的路径数；费用 dp[j]：到当前格子的最小含点费用。更新前 dp[j]是上方，dp[j-1]已是左方。

转移：计数 dp[j]=(上+左)%mod；费用 dp[j]=min(上,左)+a[i][j]，仅从可达状态转移。

初值：计数 dp[1]=1，其余 0；费用 dp[1]=0，其余 INF，统一在第一次循环加起点费用。

遍历顺序：行 1..n、列 1..m，滚动列必须正序。

复杂度：O(nm)时间，O(m)额外空间。

反例与易错：滚动时障碍格必须清 0/INF，否则上一行的旧值会穿墙。；若允许上下左右移动，不能继续按行递推；无权最短路通常转 BFS。；起点被堵答案 0/INF；1x1 可走格有 1 条空移动路径。

练习与来源：cf/date/5/5.22/2.cpp 的 solve 只抽取单条右下路径计数；其四个路径乘积作差需要额外组合解释，不当成本卡整题题解。cf/2/1078/E.cpp 只引用 dpS/dpT 的单向最大路径局部状态。

代码入口：PDP01

```
#include "competition/dp/linear_grid.hpp"
int main() {
    vector<string> g = {"", " ..", " .."};
    vector<vector<ll>> a = {{}, {0, 1, 2}, {0, 3, 4}};
    cout << grid_paths(g) << ' ' << grid_min_path(a, g) << '\n'; // 2 7
}
```

原代码：cf/date/5/5.22/2.cpp；cf/2/1078/E.cpp

【DPC03】每件最多选一次：0/1 背包

识别：若干物品有重量和价值，总重量不能超过容量，每件最多取一次。

先看什么：先问容量是否足够小；目标是重量恰好等于容量，还是不超过容量。

从暴力出发：枚举每件取/不取，共 2^n 个子集，计算重量和价值。

重复与性质：处理前 i 件后，未来只关心已用重量；相同重量只保留最大价值。

推出模型：按最后一件取或不取划分：f[i][j]=max(f[i-1][j],f[i-1][j-w[i]]+v[i])。滚动为 dp[j]后倒序更新，读取的仍是上一层。

适用条件：w[i]>0，cap>=0；物品彼此独立。；价值与任意可达总价值绝对值<2^60；恰好型不能把不可达初始化成 0。

一句证明：每个合法子集唯一属于不取第 i 件或取第 i 件两类；保留相同重量下的最大价值不会降低任何后续选择的效果。

状态含义：dp[j]：处理完当前物品后，总重量至多 j 的最大价值；exact=true 时改为重量恰好 j。

转移：dp[j]=max(dp[j],dp[j-w[i]]+v[i])，仅从可达前驱转移。

初值：至多型全 0；恰好型 dp[0]=0，其余 KNAP_NEG。

遍历顺序：物品 1..n；容量 cap..w[i]倒序。

复杂度：O(n*cap)时间，O(cap)空间。

反例与易错：w={2},v={3},cap=4：若正序会错误得到 6，正确为 3。；恰好重 3 但只有重 2 物品时不可达；负价值可达状态不能被 0 吞掉。

练习与来源：template/01bag.cpp；把容量 4、物品(2,3)手推一次，随后练习 luogu/3.cpp 的价值维度与资金到达时刻。

代码入口：DP01

```
#include "dp/knapsack.hpp"
int main() {
    auto dp = knapsack01({0, 2, 3}, {0, 3, 5}, 5, true);
    cout << dp[5] << '\n'; // 8
}
```

原代码：template/01bag.cpp；cf/luogu/3.cpp

【DPC04】每种可无限取：完全背包

识别：有限种物品，每种可用任意次；容量小，求最大价值。

先看什么：零重量正价值会无界；先确认重量正数。

从暴力出发：枚举每种物品的数量 0..cap/w[i]，枚举树会反复遇到同一剩余容量。

重复与性质：同一物品阶段、同一容量下，只需保存最优值，不需记完整数量方案。

推出模型：取第 i 种一次后，仍可从当前阶段的 j-w[i]取该种；所以 f[i][j]=max(f[i-1][j],f[i][j-w[i]]+v[i])。

适用条件：每种数量无限；w[i]>0。；只求价值最优；若改计数必须先区分排列还是组合。；任意可达总价值绝对值<2^60。

一句证明：最优方案若含第 i 种，删去一件后仍是允许继续选择第 i 种的较小容量问题。

状态含义：dp[j]：已处理物品种类中可重复选择，总重至多 j（或恰好 j）的最大价值。

转移：dp[j]=max(dp[j],dp[j-w[i]]+v[i])。

初值：与 0/1 相同：至多全 0，恰好只有 dp[0]=0。

遍历顺序：物品 1..n；容量 w[i]..cap 正序，让当前物品可继续使用。

复杂度：O(n*cap)时间，O(cap)空间。

反例与易错：(2,3),cap=4 应得到 6；倒序只会得到 3。；把物品循环和容量循环交换，在计数题里可能把组合数改成排列数。

练习与来源：template/完全背包 dp.cpp；与 0/1 例子用相同输入比较容量方向。

代码入口：DP01

```
#include "dp/knapsack.hpp"
int main() {
    cout << knapsack_complete({0, 2}, {0, 3}, 4)[4] << '\n'; // 6
}
```

原代码：template/完全背包 dp.cpp

【DPC05】每种有数量上限：多重背包

识别：每种物品最多选 cnt[i]件，重量限制仍较小。

先看什么：cnt 大时逐件展开太慢；最多只能装 cap/w[i]件，先截断无用数量。

从暴力出发：每种枚举取 0..cnt[i]件，再递归下一种。

重复与性质：同一已处理阶段与容量下只保留最大价值；数量可以拆成少量 0/1 组合包。

推出模型：把数量拆为 1,2,4,...和最后剩余包，每包重量价值乘份数；任何 0..cnt 数量都可由这些包组成，然后直接复用 0/1 倒序转移。

适用条件：w[i]>0，cnt[i]>=0，容量非负。；各件同种物品价值相同；任意可达总价值绝对值<2^60。；拆包只用于最优值/可达性；多个包可能重复表达同一件数，不能用于按件数去重计数。

一句证明：拆包的可表示数量区间每次无缝扩张到 0..累计份数，最终恰覆盖所有允许的选取数量。

状态含义：沿用 0/1：处理完当前组合包后，重量至多/恰好 j 的最大价值。

转移：每个组合包作为一件 0/1 物品，读上一层 j-包重。

初值：至多型全 0；恰好型只有 0 可达，其余 KNAP_NEG。

遍历顺序：组合包逐个处理，容量倒序。

复杂度：O(cap*Σlog(cnt[i]+1))时间，O(cap+Σlog(cnt[i]+1))空间；cnt 先按容量截断。

反例与易错：cnt=0 不能生成一包；cnt 极大时先截断避免无意义循环与乘积。；把拆出的包当完全背包会突破数量限制。

练习与来源：从 template/01bag.cpp 出发，先手算同一物品 w=2,v=3,cnt=3,cap=7；再把 cnt 设 1 与原 DP01 在同一枚举 oracle 下比较。

代码入口：PDP02、DP01

```
#include "competition/dp/knapsack_variants.hpp"
int main() {
    auto dp = knapsack_bounded({0, 2}, {0, 3}, {0, 3}, 7);
    cout << dp[7] << '\n'; // 9
}
```

原代码：template/01bag.cpp；template/dp/knapsack.hpp

【DPC06】同组互斥：分组背包

识别：物品分组，每组最多选一个，或整组不选。

先看什么：互斥限制属于整组；转移时必须保持上一组的状态快照。

从暴力出发：每组枚举“不选”或选其中一个，所有组选择作笛卡尔积。

重复与性质：处理相同组数且容量相同的方案，未来仅需知道最大价值。

推出模型：从 old[j]继承不选；每件(w,v)只用 old[j-w]+v 更新本组 dp[j]，不从同组新状态继续转移。

适用条件：groups[1..n]，组内每个重量>0；最多一个允许不选。；若要求每组恰选一个，应把每组新层初始化为 NEG 且不继承 old；本函数不提供该语义。；任意可达总价值绝对值<2^60。

一句证明：每个合法方案在当前组只有一种选择；统一从上一组转移保证不会取同组两件。

状态含义：dp[j]：已处理所有组里，每组至多选一件、重量至多/恰好 j 的最大价值。

转移：dp=old；枚举本组每件，dp[j]=max(dp[j],old[j-w]+v)。

初值：至多型全 0；恰好型 dp[0]=0，其余 NEG。

遍历顺序：先组、再组内物品、再容量；所有候选都读取固定 old，所以容量顺序不影响语义。

复杂度：O(cap*物品总数 + cap*组数)时间，O(cap)DP 空间。

反例与易错：一组两件(1,4),(1,5)、cap=2 正确答案 5，错误原地逐件会得到 9。；空组允许直接跳过；负价值且不要求装满时不选可能更好。

练习与来源：把 template/01bag.cpp 中的每件扩成一组多个替代方案；单元素组与原 0/1 在相同 oracle 下应一致。

代码入口：PDP02、DP01

```
#include "competition/dp/knapsack_variants.hpp"
int main() {
    vector<vector<pair<int,ll>>> a = {{}, {{2,3},{3,5}}, {{2,4}}};
    cout << knapsack_grouped(a, 5)[5] << '\n'; // 9
}
```

原代码：template/01bag.cpp；template/dp/knapsack.hpp

【DPC07】最后一次合并切开区间：区间 DP

识别：每次只能合并相邻部分，最终把整个序列合成一个对象。

先看什么：固定最后一步；它把[l,r]分成哪两个已经完成的子区间。

从暴力出发：枚举所有相邻合并顺序；很多顺序反复求同一段的最优合并代价。

重复与性质：当前段内部怎样合成不影响未来，只需记住边界和最小代价。

推出模型：最后一刀在 k：dp[l][r]=min(dp[l][k]+dp[k+1][r]+sum(l,r))。

适用条件：直线非负石子堆；合并代价为两堆重量和。；n*sum(a)<2^60。环形石子合并仍用区间和作费用，但要处理循环边界并对长度 n 的起点取优；矩阵链乘法则需改费用项。

一句证明：最后一刀唯一分开左右两段；若任一侧不是最优，可替换成更优方案而不影响另一侧。

状态含义：dp[l][r]：把闭区间[l,r]内的石子全部合为一堆的最小代价。

转移：枚举 l<=k<r，左右最优代价加整段重量。

初值：dp[i][i]=0；其他待求状态置 INF；空数组或一堆答案 0。

遍历顺序：长度 2..n，左端点 1..n-len+1，分割点 l..r-1。

复杂度：O(n^3)时间，O(n^2)空间。

反例与易错：先左端点再长度可能用到未算的子区间。；[10^9,10^9]答案 2*10^9；旧 int 无穷大不足，已正式修复。

练习与来源：template/区间 dp.cpp 与 cf/luogu/4.cpp 对比：后者保留端点并增加 a[l]*a[k]*a[r]，不能照抄石子合并费用。

代码入口：DP04

```
#include "dp/interval_merge.hpp"
int main() {
    cout << interval_merge({0, 1, 2, 3}) << '\n'; // 9
}
```

原代码：template/区间 dp.cpp；cf/luogu/4.cpp

【DPC08】父子选择互相限制：树形 DP

识别：在树上选点，相邻点不能同时选，最大化点权和。

先看什么：子树之间是否真的独立；固定父亲是否选后，儿子的限制就明确。

从暴力出发：枚举所有 2^n 个点集，检查是否存在同时选中的相邻点。

重复与性质：父亲只需要知道儿子是否被选和儿子子树的最优值，不需要子树内部完整方案。

推出模型：选 u 时儿子 v 必须不选；不选 u 时每个儿子可选或不选，独立取较大值并求和。

适用条件：无向简单连通树，g/a 均 1-indexed，n>=1。；允许空集，所有点权绝对值之和<2^60；一般图或有额外跨子树约束不能照用。

一句证明：删去父子边后子树互不相连；固定 u 是否选时，每个儿子的最佳合法选择可独立合并。

状态含义：dp[u][0]：u 子树内不选 u 的最大权和；dp[u][1]：u 子树内选 u 的最大权和。

转移：dp[u][0]+=max(dp[v][0],dp[v][1])；dp[u][1]+=dp[v][0]。

初值：每个点先令 dp[u][0]=0，dp[u][1]=a[u]；叶子无需额外转移。

遍历顺序：先记录父亲和父序，再逆序处理，确保所有儿子已完成。

复杂度：O(n)时间、O(n)空间。

反例与易错：全负时允许空集所以答案 0；若必须选至少一点，需要额外处理。；20 万链递归可能爆栈；实现用父序逆序作后序。；只记 dp[u]会遗漏 u 是否选，从而无法给父亲提供正确边界条件。

练习与来源：template/tree_dp.cpp 只提供儿子先算骨架；本模块补为完整最大权独立集。cf/3/1096/H_!!dp_tree.cpp 的 ct/mx 展示子树信息如何压成小状态，未确认原题，不把叶子奇偶递推解释成独立集。

代码入口：PDP05

```
#include "competition/dp/tree_independent_set.hpp"
int main() {
    vector<vector<int>> g = {{}, {2,3}, {1}, {1}};
    cout << tree_independent_set(g, {0,5,4,3}) << '\n'; // 7
}
```

原代码：template/tree_dp.cpp；cf/3/1096/H_!!dp_tree.cpp

【DPC09】依赖无环：DAG DP

识别：对象依赖形成有向无环图，求从起点到各点的路径数或最长边数。

先看什么：先确认边方向与全图是否有环；只按编号顺序未必是合法依赖顺序。

从暴力出发：从起点枚举全部有向路径，多个路径会到达同一结点。

重复与性质：同一结点之后的可选后缀相同；路径数相加，最长前缀保留最大值。

推出模型：拓扑序中对每条 u->v，把 ways[u]加到 ways[v]，把 dist[u]+1 作为 dist[v]候选；不可达点不转移。

适用条件：PG15 只求最长边数，边权均按 1；重边视为不同边选择，计数会重复贡献。；1<=s<=n<200010，1<=mod<=LLONG_MAX；全图有环返回 false，结果作废。

一句证明：拓扑序保证 u 的全部前驱先处理；每条路径按最后一条边唯一分类，所以求和不漏不重。

状态含义：dist[v]：从 s 到 v 的最长路径边数，未到达为-1；ways[v]：从 s 到 v 的路径数模 mod。

转移：每条可达 u->v：dist[v]=max(dist[v],dist[u]+1)，ways[v]+=ways[u]取模。

初值：dist 全-1，ways 全 0；dist[s]=0，ways[s]=1%mod。

遍历顺序：Kahn 拓扑顺序；出队点数不足 n 表示有环。

复杂度：O(n+m)时间、O(n+m)空间。

反例与易错：最长边数与加权最长路不同，不可直接把权值丢掉。；空路径让 ways[s]=1；不可达 dist=-1，不能设 0。；cf/date/6/6.7/1_leaner_value_dp.cpp 的整除值域递增只是一个隐式 DAG，并不保留原序列下标顺序。

练习与来源：cf/date/6/6.7/1_leaner_value_dp.cpp：只检查局部模型“已出现数值从小倍数推大倍数”；先画 2->4->8 等边，再解释为什么升序是拓扑序。

代码入口：PG15

```
#include "competition/graph/dag_dp.hpp"
int main() {
    init(4); g[1] = {2,3}; g[2] = {4}; g[3] = {4};
    if (!dag_dp(1, 1000000007)) return 1;
    cout << dist[4] << ' ' << ways[4] << '\n'; // 2 2
}
```

原代码：cf/date/6/6.7/1_leaner_value_dp.cpp；template/competition/graph/dag_dp.hpp

【DPC10】小集合选到哪里：状压 DP

识别：n 约 18 以内，要恰好访问每个点一次，代价还依赖最后停在哪个点。

先看什么：集合丢失顺序，但未来只需要已访问集合和当前末尾；这两项都不能少。

从暴力出发：枚举中间点的所有排列，累加从 1 到 n 的路径费用。

重复与性质：访问了相同集合且终点相同的前缀，未来可选点和下一条边都相同，只留最短前缀。

推出模型：从(mask,u)扩展尚未访问 v：dp[mask|1<<(v-1)][v]=min(原值,dp[mask][u]+w[u][v])。

适用条件：有向图，n<=18，固定 1 出发 n 结束，每点恰好一次。；缺边用 HAMILTON_INF；可有负边，但任意路径总权值在(-2^60,2^60)。

一句证明：按访问点数归纳；任一合法路径删掉最后一个点恰好对应一个较小状态。

状态含义：dp[mask][u]：从 1 出发，恰访问 mask 内的点，最后停在 u 的最小总费用。

转移：枚举不在 mask 中且有边 u->v 的点，将费用推给新增 v 的状态。

初值：只设 dp[{1}][1]=0，其余 INF；n=0/1 约定 0。

遍历顺序：mask 递增（每条转移都增加一位），再枚举末尾和下一点。

复杂度：O(n^2*2^n)时间，O(n*2^n)空间；n=18 约 36MiB。

反例与易错：只记 mask 会丢失末尾，下一条边费用无法确定。；点 i 对应第 i-1 位；程序内部末尾 u 为 0-indexed。；cf/luogu/5.cpp 按相同字符去重的计数语义与 Hamilton 最短路不同，不能互换。

练习与来源：cf/luogu/5.cpp 只核对局部子集删位递推，不据此猜原题；先把 Hamilton 用 n<=8 的全排列对拍。

代码入口：DP03

```
#include "dp/hamilton_path.hpp"
int main() {
    vector<vector<ll>> w(4, vector<ll>(4, HAMILTON_INF));
    w[1][2] = 4; w[2][3] = -1; w[1][3] = 1;
    cout << hamilton_path(w) << '\n'; // 3
}
```

原代码：template/dp/hamilton_path.hpp；cf/luogu/5.cpp

【DPC11】巨大数值区间但数位少：数位 DP

识别：统计[L,R]里十进制非零数位不超过 3 的整数，R 很大但至多 19 位。

先看什么：按位从高到低构造；只需记已用非零位数和是否仍贴着上界。

从暴力出发：逐个枚举[L,R]，数每个数的非零位；区间长时不可行。

重复与性质：未贴上界的同位置、同计数拥有完全一样的后缀选择，所以可以缓存。

推出模型：枚举当前位 d，下一层非零数位数加(d!=0)；超过 3 立即返回 0；走完全部位返回 1。用 F(R)-F(L-1)求区间答案。

适用条件：0<=L<=R<=LLONG_MAX；F(x<0)=0。；0 计入，前导零不增加非零计数；该条件无需额外 started 维度。；这是固定最多 3 个非零位的模板；若限制相邻数位或数位乘积，必须重新设计状态。

一句证明：每个整数在补前导零到上界位数后有唯一数位序列；逐位分支互斥，limit 确保不超过上界。

状态含义：dfs(pos,cnt,limit)：已处理 pos 位、已用 cnt 个非零数位且是否贴上界时，合法补完的方案数。dp[pos][cnt]仅缓存未贴上界情形。

转移：枚举 0..up，递归 pos+1、cnt+(d!=0)、limit&&(d==up)。

初值：从 dfs(0,0,true)出发；cnt>3 为 0，pos 到末尾为 1，缓存初值-1。

遍历顺序：递归自顶向下记忆化；pos 严格增加，天然无环。

复杂度：最多 19*4*2 个有效状态，每状态 10 次转移，O(位数*4*10)时间与 O(位数*4)空间。

反例与易错：x 换成不同位数后必须清除 dp；只缓存!limit 状态。；上界末位取相同时 limit 才继续；前导零不是实际非零位。；如果题目只要正整数，[0,0]模板答案 1，需在题目层排除 0。

练习与来源：cf/date/8/1/2_digit_dp.cpp 原个人实现，递归 lambda 和全局 s/dp 保留，仅提取 calc/range 接口。测试 [1000,1111]=111 与 LLONG_MAX。

代码入口：PDP04

```
#include "competition/dp/digit_count.hpp"
int main() {
    cout << digit_range(1000, 1111) << '\n'; // 111
}
```

原代码：cf/date/8/1/2_digit_dp.cpp

【DPC12】保持原序的序列：LIS、LCS、编辑距离

识别：从序列中删元素但保留相对次序；或比较两个序列的前缀。

先看什么：子序列不要求连续；严格递增和非递减要分开。双序列一般保留两个前缀长度。

从暴力出发：LIS 枚举所有下标子集，判断递增；LCS 枚举两个字符串子序列找相同者；编辑距离枚举删除、插入、替换操作。

重复与性质：LIS 的未来只看末尾值与长度；LCS/编辑距离的未来只看尚未处理的两个后缀。

推出模型：LIS 以最后位置 i 分类，枚举 j<i 且 a[j]<a[i]；按长度保存最小末尾可用二分优化。LCS 相等匹配，不等删去一侧末位；编辑距离枚举最后一步。

适用条件：LIS 数组 1-indexed；只返回长度。；LCS 按字符相等，编辑距离每次插入/删除/替换费用均为 1；加权版本需另改转移。

一句证明：更小的同长度末尾可接上的未来元素集合不会更少；双序列末步分类覆盖全部合法对齐。

状态含义：LIS：dp[i]为以 a[i]结尾的最长合法子序列长度。LCS：dp[i][j]为前 i、前 j 字符的 LCS 长度。编辑距离：dp[i][j]为前 i 变成前 j 的最少操作数。

转移：LIS 取 1 和 dp[j]+1 最大值；LCS 字符同则加 1，否则取删一侧的 max；编辑距离取删除、插入、匹配/替换的 min。

初值：LIS 每个非空末尾至少 1；LCS 空前缀 0；编辑距离 dp[i][0]=i，dp[0][j]=j。

遍历顺序：LIS i 递增且 j<i；双序列 i、j 递增，滚动时保留左上角旧值。

复杂度：LIS $O(n^2)$或 O(n log n)；LCS/编辑距离 O(nm)时间、空间。JAKARTA 局部模型每个字符串长度 L<=15，枚举 O(L*2^L)，排序最坏 O(L*2^L*log(2^L))，相邻层扫描线性于候选数。

反例与易错：[2,2]严格 LIS=1，非递减=2。；tails 不是实际答案序列；连续子段不能调用 LIS。；JAKARTA 的候选是不同子序列字符串，必须排序去重；用<=会把严格字典序误写成非严格。

练习与来源：template/线性 dp.cpp；cf/ICPC/JAKARTA_2018/J.cpp 理解为何枚举当前候选后只查上一层字典序前缀最大值。

我的原题延伸：JAKARTA J 只提取可从代码确认的局部模型：每个对象选一个非空子序列，候选严格字典序递增，最大化总长度。dp[j]是上一对象恰选 pre[j]时的最优总长度；cur 递增时用双指针维护 pre<cur 的前缀最大值，避免候选集合的笛卡尔积。原题与在线状态未核验。

代码入口：DP02、PDP03

```
#include "dp/lis.hpp"
#include "competition/dp/sequence_state.hpp"
int main() {
    cout << lis_length({0, 3, 1, 2, 2}) << ' '
         << lcs_length("abc", "ac") << ' '
         << edit_distance("kitten", "sitting") << ' '
         << lex_subsequence_chain({"", "ab", "bc"}) << '\n'; // 2 2 3 4
}
```

原代码：template/线性 dp.cpp；cf/ICPC/JAKARTA_2018/J.cpp

【DPC13】历史只剩阶段与位置：状态机 DP

识别：在字符串中先向右再向左、最多转向一次，问能否读出目标串。

先看什么：同样停在位置 i，是否已经转向会改变下一步合法方向，不能只存位置。

从暴力出发：枚举起点和向右走的步数（转向点），直接模拟目标串。

重复与性质：匹配了同样长度、停在同位置、方向阶段相同的历史，未来选择完全一样。

推出模型：dp0 可向右留 dp0，或向左进 dp1；dp1 只能继续向左。走到的字符必须等于目标下一位。

适用条件：每步移动相邻一个位置；可在第一步前转向，也可始终向右；最多一次右转左。；允许重新访问旧位置；空目标约定 true，非空目标配空源串为 false。

一句证明：合法路径的每步属于三种状态转移之一；阶段只能 0->1，保证绝不会二次转向。

状态含义：dp0[i]：匹配完当前目标前缀，停在 s[i]且尚未向左；dp1[i]：匹配完同一前缀，停在 s[i]且已向左。

转移：0->0 向右，0->1 向左，1->1 向左；按下一字符过滤。

初值：所有 s[i]==t[0]的位置设 dp0[i]=true，dp1 全 false；空 t 直接 true。

遍历顺序：目标前缀长度递增；每层新建 ndp，完成后交换。

复杂度：O(|s|*|t|)时间，O(|s|)空间。

反例与易错：只记位置可能允许先左再右；复用本层会在同一目标字符上走多步。；每个目标字符都要清新层；原地修改会把还未发生的移动串联起来。

练习与来源：cf/date/7/7.9/1_interesting_dp.cpp 原个人实现，保留 dp0/dp1/ndp0/ndp1。用 s=abc,t=abcba 检验右后左，用 s=abc,t=abca 检查非法绕行。

代码入口：PDP03

```
#include "competition/dp/sequence_state.hpp"
int main() {
    cout << right_then_left_match("abc", "abcba") << '\n'; // 1
}
```

原代码：cf/date/7/7.9/1_interesting_dp.cpp

【DPC14】互斥的最后一步相加：计数 DP

识别：求长度 n 的 01 串数量，要求没有相邻两个 1；相似题常是局部禁用模式。

先看什么：给不同的构造路径是否会生成同一对象；最后一位足以判断能否追加 1。

从暴力出发：枚举全部 2^n 个串，检查相邻位置是否同时为 1。

重复与性质：长度相同且最后一位相同的前缀，合法后缀集合相同，只需累计前缀数量。

推出模型：末尾补 0 可接任意旧状态，补 1 只能接旧末位 0；两类新串按最后一位互斥。

适用条件：统计不同长度 n 的二进制字符串，n>=0；空串是一种。；模数 1..INT_MAX；不返回完整精确大整数。

一句证明：每个新串删去最后一位得到唯一前缀，因而每条合法转移贡献的对象不重复。

状态含义：f[0/1]：当前前缀最后一位为 0/1 的合法串数量；空前缀虚拟放在 0 状态。

转移：new[0]=(f[0]+f[1])%mod；new[1]=f[0]。

初值：f={1%mod,0}；n=0 答案 1%mod。

遍历顺序：长度从 1 到 n，使用独立新层；最后答案两状态求和。

复杂度：O(n)时间，O(1)额外空间。

反例与易错：dp[0]的空对象初值应为 1；设 0 会让全部答案为 0。；用更新后的末位 0 计数再算末位 1，会误把本层串作为上一层。；多重背包二进制拆分只保证可表示，不保证计数唯一，不能据此数数量方案。

练习与来源：cf/luogu/5.cpp 展示按子集删位累计的局部递推及相同字符去重判定；本卡只用明确的 01 串模型练习计数不重不漏，原状压题题意 UNKNOWN。

代码入口：PDP04

```
#include "competition/dp/digit_count.hpp"
int main() {
    cout << binary_no_adjacent(3) << '\n'; // 5
}
```

原代码：cf/luogu/5.cpp

【DPC15】加权求和与自环移项：概率/期望 DP

识别：独立试验求成功次数分布；或按阶段反复尝试，成功前进、失败留在原阶段，求平均尝试次数。

先看什么：概率是方案的权重，不能按路径条数平均；有自环的期望式需要移项，不能假装拓扑递推。

从暴力出发：概率枚举每次成功/失败的 2^n 个结果并乘概率。期望枚举所有尝试历史，失败次数没有有限上界。

重复与性质：成功次数分布只依赖已做试验数和成功数；等待模型只依赖当前阶段，过去失败次数不改变固定成功概率。

推出模型：概率 dp[i][j]=dp[i-1][j]*(1-p[i])+dp[i-1][j-1]*p[i]。期望 E[i]=1+(1-p[i])*E[i]+p[i]*E[i+1]，移项得 E[i]=E[i+1]+1/p[i]。

适用条件：分布模型要求试验相互独立，0<=p[i]<=1，均为有限数。；等待模型要求每阶段各次尝试相互独立、成功概率固定，0<p[i]<=1；sum(1/p[i])须有限且落在 long double 范围。；p=0 的等待阶段可能永远过不去，期望无限，不能调用有限期望函数；概率分布函数则允许 p=0。

一句证明：概率按最后一次成败作全概率划分；期望按第一次尝试分解，再利用失败回到同一状态的条件期望。

状态含义：概率 dp[j]：做完当前前 i 次试验，恰 j 次成功的概率。期望 dp[i]：从阶段 i 开始直到全部成功所需的期望尝试次数。

转移：概率按失败留 j、成功从 j-1 加入；期望先移项自环后加 1/p[i]。

初值：概率 dp[0]=1，其余 0；期望终止状态 dp[n+1]=0。

遍历顺序：概率试验 i 递增、成功数 j 倒序；期望阶段 i 从 n 递减到 1。

复杂度：成功次数分布 $O(n^2)$时间、O(n)空间；阶段等待 O(n)时间、O(n)空间。

反例与易错：分布更新正序会把同一次试验成功多次；应倒序或新数组。；E 取 min/max 不是随机过程的期望；除以出度也不等于使用真实转移概率。；若成功率随失败次数变化，状态必须增加次数；直接 1/p 不成立。

练习与来源：明确基础例：两枚独立公平硬币，恰一正面概率 1/2；两个阶段成功率 1/2、1，期望尝试次数 2+1=3。个人目录未发现已确认的概率期望原题，本模块标记 ROUND02 基础补缺。

代码入口：PDP06

```
#include "competition/dp/probability_expectation.hpp"
int main() {
    auto dp = coin_heads_probability({0, 0.5L, 0.5L});
    cout << fixed << setprecision(6) << dp[1] << ' '
         << expected_stage_attempts({0, 0.5L, 1.0L}) << '\n'; // 0.500000 3.000000
}
```

【MEX_GLOBAL】插删后问最小缺失非负数

识别：mex、最小没有出现的整数；单点修改后问全数组 mex。

先看什么：从 0 开始；元素总数上界 N；是否有删除或重复值。

从暴力出发：每次从 0 向上检查整个数组，$O(N^2)$量级。

重复与性质：一次修改只改变两个数的出现频次，没有必要重扫其他候选。

推出模型：静态 vis 扫描；仅插入的前缀维护只增指针；任意插删维护 cnt 与缺失 set。

适用条件：n 元素 mex<=n；动态 init(N)要求活跃总数<=N 且删除值存在。

一句证明：[0,N]有 N+1 个数，N 个元素不能全覆盖；缺失 set 与 cnt==0 互补保持。

复杂度：静态/所有前缀 O(n)；动态每次 O(log N)。

反例与易错：{0,0}删一个 0 后 mex 仍 1；负数不占 mex；所有区间共享一个全局 set 无法回答每个区间的 mex。

练习与来源：cf/3/1096/D_mex_set.cpp::cal 局部模型：中心扩展时把遇到的值从候选 set 中移除；完整题面与线上 AC 未核验。

代码入口：MEX01、MEX02

调用提醒：mex_init(n); for(int x:a) mex_add(x); mex_erase(old); mex_add(value); cout<<mex_query();

原代码：template/mex.cpp；cf/3/1096/D_mex_set.cpp

【MEX_CONSTRUCT】mex 反推出必须出现与必须缺失的元素

识别：要求某段 mex 等于 k，或构造指定 mex 的排列/数组。

先看什么：mex=k 等价于 0..k-1 全部出现且 k 不出现；段长至少 k。

从暴力出发：枚举所有数组再算 mex，状态爆炸。

重复与性质：可行性直接分解为覆盖集合与禁用元素，无须枚举排列细节。

推出模型：先放齐 0..k-1，再用允许的重复值或大于 k 的值填充；额外限制需单独验证。

适用条件：仅 mex 条件时，段长 m>=k 必要；可用整数不受限时也充分。k=0 时必须避免 0；若 m=0 只有 mex0。

一句证明：mex 定义本身给出两个方向：小于 k 全在且 k 不在，当且仅当最小缺失恰为 k。

复杂度：构造后 O(m)检验。

反例与易错：长度 2 不可能 mex3；排列要求每值唯一时不能重复填 0；全数组 mex 固定不代表每个子区间 mex 固定。

练习与来源：tests/round02/test_game_string.cpp：用 static_mex 验证小值构造；排列 0..n-1 全数组 mex=n，真子区间的 mex 需重新求。

代码入口：MEX01

原代码：template/mex.cpp

【GAME_PN】先枚举胜败状态，再证明规律

识别：两人轮流、询问先手输赢/合法胜着。

先看什么：无路可走谁输？双方可用操作是否相同？状态是否终止、是否有环？

从暴力出发：从当前状态枚举所有合法后继，递归搜索双方最优行动。

重复与性质：不同走法可到同一状态，按依赖顺序记忆/DP 复用结果。

推出模型：正常规则终态败；存在败态后继则胜，否则败。小范围表格用于猜奇偶/模数规律，再对全部状态归纳证明。

适用条件：有限无环正常规则可直接递推；有环可能出现平局，不能当成普通 SG。

一句证明：能走到败态就把无解留给对手；所有后继都赢时每一步都给对手赢态。

复杂度：O(状态数+合法转移数)。

反例与易错：取 1 或 2 时败态是 3 的倍数，奇偶猜法在 2 就错；只对前 20 项成立不是证明。

练习与来源：tests/round02/test_game_string.cpp：取{1,3,4}枚举到 50，并验证双游戏的直接 P/N 递推。

代码入口：GAME01

原代码：template/mex.cpp

【GAME_NIM】独立子游戏为何异或，Nim 怎样走到败态

识别：若干堆，每步仅改变一堆；或已证明独立的公平正常子游戏之和。

先看什么：每一步是否只能影响一个分量？是否共用资源导致联动？最后一步规则是否正常？

从暴力出发：对所有堆大小构成的笛卡尔积做胜败 DP。

重复与性质：独立分量的复杂状态可分别压成 SG 值，合并只需异或。

推出模型：Nim 堆的 SG 是堆大小；一般有限公平 DAG 取后继 SG 集合的 mex。异或非零找最高为 1 的位，把相应 Nim 堆 ai 减到 ai xor sum。

适用条件：有限、无环、公平、完全信息、无随机、正常规则、每步恰改一个独立分量。

一句证明：零异或任意合法改变一个 SG 都会非零；非零异或存在一个分量可改变为使总异或零的更小 SG 值。

复杂度：Nim O(堆数)；一般 SG O(V+E)。

反例与易错：一步可同时取两堆时不是独立和；反常规则不能直接套 SG。反常 Nim 只有全 1 时按个数奇偶，其余按 xor，不能推广到任意游戏。

练习与来源：tests/round02/test_game_string.cpp：{1,2,3}败；{1,2,4}将 4 改 3；3 堆 0..5 正常/反常穷举。

代码入口：GAME01、MEX01

原代码：template/mex.cpp

【MEX_STRUCTURE】全局 MEX 与区间 MEX，数据结构不能混用

识别：问最小缺失数，但还附带区间[l,r]或按缺失秩查询。

先看什么：被询问集合是整个数组还是每次不同子区间？只问首个缺失还是第 k 个缺失?

从暴力出发：每次把当前查询区间的值标记后从 0 扫描。

重复与性质：单次全局更新只改变一个频次；不同区间的出现状态则可能完全不同。

推出模型：静态一次用 vis；前缀只插入用只增指针；全局插删用 MEX02。缺失标记 0/1 放 Fenwick 或线段树可找第 k 个缺失。任意区间 MEX 必须另做离线/区间数据结构：例如扫右端点维护每值最后位置，以值域线段树找最小 last[x]<l。

适用条件：值域截断[0,n]仍成立；Fenwick 按秩找位置要求非负 0/1 频率。区间 MEX 的 last 模型只适用于固定数组、离线按右端点推进。

一句证明：x 出现在[l,r]当且仅当扫到 r 时 last[x]>=l，故最小 last[x]<l 的值就是 mex。

复杂度：已有全局实现每次 O(log n)；区间模型本卡仅解释选择，不冒充已有可调用区间 MEX 代码。

反例与易错：全数组{0,1}的 mex 是 2，区间只含{1}的 mex 却是 0；全局 set 直接回答会错。

练习与来源：cf/3/1096/D_mex_set.cpp 只研究中心向外的当前集合，不把局部 cal 当通用区间查询。

代码入口：MEX01、MEX02

原代码：template/mex.cpp；cf/3/1096/D_mex_set.cpp

【GAME_SYMMETRY】奇偶或镜像策略要保证每一步都有合法应答

识别：可按相同两半配对，或每步取走固定奇偶性的数量。

先看什么：镜像位置是否独立？中心是否单独存在？对手能否跨两半同时操作？终局是谁输？

从暴力出发：枚举小状态胜败表，寻找偶数/模数/对称规律。

重复与性质：若每个回合都能恢复相等两半，无须区分内部操作历史。

推出模型：正常 Nim 两个相等堆：后手在另一堆镜像对手的减少量，恢复相等。只可取 1 或 3 时每步翻转奇偶，偶数败；只可取 1 或 2 时败态是 3 的倍数，显示奇偶并不通用。

适用条件：镜像动作必须存在且不与对手刚动的部分冲突；两半操作独立；正常规则；无剩余中心。

一句证明：非空相等两堆中一堆可减多少，另一堆也可同量减少；后手每次应答后恢复相等且不先无路可走。取 1/2 则从 3 倍数只能走到非倍数，非倍数可一步走回倍数。

复杂度：有证明时 O(1)或 O(堆数)；枚举仅用于发现规律。

反例与易错：允许一步同时取两堆时镜像应答可能不存在；反常规则拿最后一个输时策略结论改变。

练习与来源：tests/round02/test_game_string.cpp 验证取{1,3}/{1,2}的 40 状态递推以及 100 组相等堆。

代码入口：GAME01

原代码：template/mex.cpp

【PAL_CENTER】连续回文看中心，子序列不能用中心扩展

识别：最长连续回文子串、回文子串个数、判断一段字符串是否回文。

先看什么：连续子串还是允许跳过的子序列？按位置计数还是不同内容去重？规模是否允许平方？

从暴力出发：枚举左右端点并反转比较，每段又花线性时间。

重复与性质：同一中心的更长回文包含更短回文，可复用向外比较过程。

推出模型：单次判定用 S03 双指针；枚举奇中心(i,i)与偶中心(i,i+1)向外扩，记录长度和每次成功。

适用条件：按字节处理；$O(n^2)$适用于相应数据量；空串最长长度 0。要求子序列时换序列/区间 DP。

一句证明：每个非空回文子串恰有唯一中心和半径，扩展不会遗漏也不重复计数。

复杂度：双指针 O(n)，中心扩展 $O(n^2)$、O(1)额外空间。

反例与易错：aaa 有 6 个位置回文子串但 3 种内容；abca 的最长回文子序列可为 aca，中心扩展不能跳过 b。大量区间查询可参考 S02 双哈希但需承认碰撞；Manacher 留 OPTIONAL。

练习与来源：tests/round02/test_game_string.cpp：4000 随机短串，对每个子串反转判定和最长长度。

代码入口：PAL01、S03

调用提醒：auto info=palindrome_centers(s); s.substr(info.l,info.r-info.l);

原代码：template/string/basic_string.hpp；cf/3/1096/D_mex_set.cpp

【PAL_CONSTRUCT】回文构造先镜像，再处理唯一奇数中心

识别：重排字符串成回文；或 0/1/?固定字符与总数同时限制。

先看什么：是否允许重排？哪些位置已固定？目标各字符数量是否相容？

从暴力出发：枚举所有排列或每个?的替换再检查。

重复与性质：镜像位置必须相等；成对位置消耗偶数份字符，仅中心能消耗一份。

推出模型：任意重排时统计频次，最多一个奇数，左右各放 freq/2。固定 01?时沿原代码传播镜像，扣已用计数，填中心奇数，再填相同字符对。

适用条件：任意重排的充要条件是奇频字符<=1；固定位置还必须无镜像冲突、余量非负且匹配未知位置。

一句证明：必要性来自每个非中心位置都与镜像配对；充分性由左右半份+可选中心的实际构造给出。

复杂度：O(n+字符集大小)。

反例与易错：aabb 可重排，固定字符串 aabb 却不是回文；仅看奇偶不会发现镜像 0/1 冲突。数值串 010 还需单独检查前导零约定。

练习与来源：cf/construction/1.cpp 原镜像传播与计数代码；n<=7 所有 01?与计数穷举对全部替换。

代码入口：PAL01

调用提醒：palindrome_permutation(s); binary_palindrome_fill(pattern,zeros,ones);

原代码：cf/construction/1.cpp

【TRIE_PREFIX】很多字符串反复查前缀，把相同前缀只存一次

识别：查询多少单词以 p 开头，或某个完整词重复了几次。

先看什么：字符集大小？总字符串长度？查询完整词还是前缀？是否多测？

从暴力出发：每个查询遍历所有单词，对前缀逐字符比较。

重复与性质：许多单词的相同前缀被反复扫描。

推出模型：直接沿个人 tr[u][c]数组建边；结点代表一个前缀，cnt 记录经过次数，word_end 记录终止次数。

适用条件：小写字母；节点上界由总字符数而非单词数决定；多测清除全部已用节点。

一句证明：每个词与每个前缀结点相交至多一次，经过次数正好等于拥有该前缀的词数。

复杂度：单次 O(串长)，空间 O(26*节点数)。

反例与易错：插 ab 和 abc 后 cnt[ab]=2 但 word_end[ab]=1；只重置 tot 不清边会在下组重用脏节点。

练习与来源：cf/ICPC/2026ICPCWeb1/L.cpp 原 tr/cnt/dep/tot 局部模型；不把未核验题目猜成通用模板题。

代码入口：TRS02

调用提醒：trie_init(); trie_insert("ab"); trie_insert("abc"); trie_prefix("a")==2;

原代码：cf/ICPC/2026ICPCWeb1/L.cpp

【GEN_NAV】读完题后，把答案变成六个明确问题

识别：不知道选什么算法，或已经想到算法却无法说明它为什么适用。

先看什么：先抄清总规模、取值/符号、操作集合、目标量与是否在线；尤其区分连续段/子序列、恰好/至多、方案数/最优值。

从暴力出发：写出最直接的枚举对象：所有区间、所有子集、所有操作序列或所有状态。

重复与性质：记录哪些枚举共享同一前缀、同一状态、同一阈值或同一贡献；这才是优化的入口。

推出模型：约束→操作→暴力中的重复或不变量→证明→可靠接口→独立验证。可行集合单调才二分；未来只受少量历史影响才压成 DP；可交换才贪心。

适用条件：复杂度估算是筛选条件，不是算法正确性的充分条件；任何剪枝或状态压缩都必须保留全部可行解或最优值。

一句证明：说明被删除的候选为什么永远不可能优于保留者，或为什么每个合法答案都被恰好覆盖。

复杂度：先算状态数×每状态转移数，再算总测试规模；O(n log n)并不自动意味着能过任意时限。

反例与易错：看到“最小”就二分、看到“选择”就贪心、看到“区间”就线段树都缺少关键前提；例如含负数的限和窗口不单调。

练习与来源：用 cf/other/Hello_2018/D.cpp 的原注释逐句回答六个问题，再对 cf/date/7/7.25/1.cpp 写出“为什么保留最小可行末值”。

代码入口：CO01、CO02、CO03

调用提醒：先查 HOW_TO_THINK.md 总索引；从模块 source 复制已验证实现，并逐项核对 contract、下标和返回失败值。

原代码：cf/other/Hello_2018/D.cpp；cf/date/7/7.25/1.cpp

【INT_PLUS】从 0 只做区间+1：正差分是必须新开的层

识别：从全零数组构造非负整数目标，每次给任意连续非空区间加 1，最少多少次；只减 1 的对称版本。

先看什么：把 a[0]=a[n+1]=0 补上，d[i]=a[i]-a[i-1]；只加 1 时目标必须全非负。

从暴力出发：把每一层的连续正高度段逐个加出来，复杂度随最大高度或答案增长。

重复与性质：区间内部差分不变，一次[l,r]+1 只在 l 新开一层、在 r+1 结束一层；下降处可以关掉已有层。

推出模型：最少次数=a[1]+Σ(i=2..n)max(0,a[i]-a[i-1])。扫描时遇正差分开相应层，负差分关闭相同数量的活跃层；可将相同区间的操作压成次数。只减 1 要求 a[i]<=0，对-a 应用同式。

适用条件：非负整数目标是只加 1 可达的必要且充分条件；只减 1 对应非正。每次费用按单位操作计，端点任意，无额外长度/重叠限制。

一句证明：位置 i 比 i-1 多出的高度只能由从 i 开始的新操作提供，正差分和为下界；逐层开闭恰好达到该下界。

复杂度：计数 O(n)、额外空间 O(1)；压缩构造 O(n)时间空间、至多 n 组，不展开可能巨大的操作次数。

反例与易错：[2,1,3]需 4 次而非 max=3；下降只关层，不另开操作。若每次可加任意整数且统一费用 1，[2,2]只需 1 次，此式失效。取 LLONG_MIN 的负数或差分先提升 i128。

练习与来源：一般练习 LeetCode 1526（官方题面已核验，并非声称用户做过）：[1,2,3,2,1]答案 3；自测[2,1,3]可用[1,3]+1、[1,1]+1、[3,3]+2 达到 4 个单位操作。

代码入口：BM01、BM02

调用提醒：unit_intervals(a,1)计数；construct_unit_intervals(a,1)返回{l,r,sign,cnt}压缩清单。只减 1 传-1。a 含 0 号占位，nullopt 表示符号不可达，合法全零则答案 0、清单空。

原代码：template/competition/models/interval_count.hpp；template/competition/models/interval_construct.hpp

【INT_SIGNED】区间±1：两端补 0 后的总变差除以 2

识别：从 0 变成可正可负的整数数组，每次任选区间加 1 或减 1，求最少单位操作数。

先看什么：补 d[1]=a[1]、d[i]=a[i]-a[i-1]、d[n+1]=-a[n]，总和恒为 0；不要漏掉右边界。

从暴力出发：对小数组把整条数组编码为状态，枚举全部区间及±1 边，用 BFS 求最短操作数。

重复与性质：每次操作只移动一单位差分质量：在两个边界一加一减；内部所有点的重复修改无需单独处理。

推出模型：答案=Σmax(0,d[i])=Σ|d[i]|/2=(|a[1]|+Σ(i=2..n)|a[i]-a[i-1]|+|a[n]|)/2。把正差分和负差分逐一配对：正点在左用+区间，在右用-区间。

适用条件：任意 ll 整数目标都可达；允许任意区间、双向单位操作，且中间数组值没有额外限制。公式和差值都用 i128。

一句证明：t 次操作形成的差分 L1 范数至多 2t，给出下界；正负边界配对每次恰消掉等量需求，达到下界。

复杂度：计数 O(n)时间/O(1)额外空间；压缩构造 O(n)时间空间。

反例与易错：[1,-1]答案 2，不是只看内部变化得到 1；任意区间改成固定长度、只能前缀、或中间值受限时须重做可达性证明。不能先在 ll 里做 a[i]-a[i-1]再转 i128。

练习与来源：tests/round02/test_models.cpp 把 n<=4、每项[-2,2]全部状态与 BFS 对照，并重放压缩构造；[-2,3]的答案为 5。

代码入口：BM01、BM02

调用提醒：unit_intervals(a,0)与 construct_unit_intervals(a,0)；输出 i128 可用 CO02 内的全局函数 to_string_i128。

原代码：template/competition/models/interval_count.hpp；template/competition/models/interval_construct.hpp；template/tests/round02/test_models.cpp

【INT_LIMITS】只能改前缀或后缀：每个长度的系数被唯一确定

识别：每次只能对[1,r]或[l,n]加 1/减 1；看似区间操作，但一端固定。

先看什么：从不受其他较短操作影响的端点倒推；先确认允许加减两种方向还是只允许一种。

从暴力出发：枚举每种长度的使用次数，或在小状态图上 BFS。

重复与性质：前缀长度 i 的净操作次数必为 a[i]-a[i+1]；后缀起点 i 的净次数必为 a[i]-a[i-1]，没有自由选择。

推出模型：前缀±1：|a[n]|+Σ(i=1..n-1)|a[i]-a[i+1]|；后缀±1：|a[1]|+Σ(i=2..n)|a[i]-a[i-1]|。只能+1 时，前缀要求 a[1]>=…>=a[n]>=0 且答案 a[1]；后缀要求 0<=a[1]<=…<=a[n]且答案 a[n]。只-1 对-a 作同一判定。

适用条件：上述单向单调条件既必要又充分；双向单位前后缀对任意整数目标可达。中间状态、长度集合若另受限制，则不沿用这些充分性结论。

一句证明：相邻位置相减会消去所有共同覆盖的操作，只留下唯一长度的净系数；最少使用次数至少为系数绝对值，按其符号操作即达到。

复杂度：O(n)时间、O(1)额外空间，返回 i128。

反例与易错：[2,1,3]任意区间+1 需 4 次，前缀±1 需 6 次，后缀±1 需 5 次；不能混用。非负目标[1,2]也不能只靠前缀+1 得到。

练习与来源：tests/round02/test_models.cpp 的 prefix/suffix 独立 BFS；手算[1,2]：前缀±1 为 3，后缀±1 为 2。

代码入口：BM01

调用提醒：prefix_units(a)、suffix_units(a)实现双向计数；仅+1/-1 的额外单调判定属于本卡推导，BM01 没有单向前后缀接口。

原代码：template/competition/models/interval_count.hpp；template/tests/round02/test_models.cpp

【INT_FLIP】二进制区间翻转：数初始与目标差异的 1 段

识别：一次把某个区间的 0/1 全部翻转，要把 s 变成 t，最少几次。

先看什么：先令 b[i]=s[i] xor t[i]；问题只依赖差异数组，而不是目标 t 自身的 1 段。

从暴力出发：n 较小时把二进制串当 mask，枚举所有区间异或 mask，用 BFS 求最少翻转次数。

重复与性质：在 GF(2)差分中，一次翻转只改变 l 和 r+1 两个边界；b 中每个 1 段恰有两个边界。

推出模型：答案等于 b 中的连续 1 段数，即 b[i]=1 且 b[i-1]=0 的次数，b[0]=0；逐个翻转这些 1 段就是构造。

适用条件：s 与 t 等长且均为 0/1；可任意选非空连续区间，费用统一 1。GF(2)翻转与整数加减不是同一操作。

一句证明：2k 个目标变化边界至少需要 k 次各影响两边界的操作；直接翻转 k 个 1 段达到下界。

复杂度：O(n)时间、O(1)额外空间。

反例与易错：s=111、t=101 时差异 010，答案 1，而 t 有 2 个 1 段。只允许翻前缀需另一种系数；把一段元素次序 reverse 也不是 0/1 按位翻转，不能套用本公式。

练习与来源：tests/round02/test_models.cpp 对 n<=7 全部 mask 用 BFS 对照；自测 101 从全 0 出发需 2 次。

代码入口：BM01

调用提醒：binary_interval_flips(b)接受 1 下标差异数组；只有初始全 0 时才可直接传目标。

原代码：template/competition/models/interval_count.hpp；template/tests/round02/test_models.cpp

【INT_TRANSPORT】相邻搬运一单位：每道切口必须承担前缀差

识别：一次从某位置移 1 到相邻位置，可左右移动；将数组 a 变成 b，最小搬运步数。

先看什么：总量守恒先检查 Σa=Σb；把第 i 和 i+1 之间当作一道切口。

从暴力出发：枚举每次搬运的状态，用 BFS；或反复挑一个多余位置和一个缺少位置搬运。

重复与性质：前 i 个位置的净多余量 P[i]=Σ(j<=i)(a[j]-b[j])必须经过第 i 道切口，搬运顺序不改变净流量。

推出模型：总量相同才可达；答案=Σ(i=1..n-1)|P[i]|。按切口需求安排对应方向的流，或顺序消掉当前位置与目标的差。

适用条件：BM01 模型允许任意整数状态、两向相邻单位转移，无额外容量/禁边；在该模型下总和相同必要且充分。若题面限制库存非负、边方向或每点容量，另核可执行顺序与可达性。

一句证明：每道切口至少走|P[i]|次，且每一步只跨一道切口；按所需净流向实现这些流量即可达到总下界。

复杂度：O(n)时间、O(1)额外空间。

反例与易错：[3,0,0]到[0,0,3]要 6 步，不是 Σ|a-b|/2=3；后者忽略距离。若允许任意两点直接传送，模型已改变。

练习与来源：tests/round02/test_models.cpp 验证 3 单位跨两条边的 6 步和总量不等的失败；自行用 n=3 小状态 BFS 检查流量解释。

代码入口：BM01

调用提醒：adjacent_moves(a,b)返回 optional<i128>；长度相同、1 下标；nullopt 是总量不一致。

原代码：template/competition/models/interval_count.hpp

【INT_ARBITRARY】区间加任意整数、每次费用 1：小差分数的精确分组

识别：一次区间加 x，x 可为任意整数，费用与|x|无关；目标最少操作次数。

先看什么：补两端 0 后的非零差分只有 m 个；数值大并不等于操作多，应先看 m 是否<=20。

从暴力出发：枚举差分边界之间所有转移与权值会爆炸；较小 m 时可枚举零和子集分组。

重复与性质：一次区间加相当于差分边界图上一条带权边；每个连通分量的需求和必须为 0。

推出模型：把 m 个非零差分点划成尽量多的零和组，组数最大为 g，答案=m-g。设 sum[mask]为子集和，dp[mask]=max_i dp[mask 去掉 i]+[sum[mask]==0]；它等价于选一种排列并统计零和前缀，每个相邻零和前缀间构成一组。

适用条件：任意整数区间增量、固定单次费用 1、中间状态无限制；BM02 精确实现仅支持 m<=20。全零目标 m=0 答案 0。没有宣称一般规模存在多项式算法，也不在此断言复杂性分类。

一句证明：每个含 k 个非零需求点的连通分量至少 k-1 条边，故至少 m-g 条；每个零和组以一顶点为中心，用 k-1 条带权边满足其余点，再由和为 0 满足中心，达到下界。

复杂度：O(m·2^m)时间、O(2^m)空间；m 为含 n+1 右边界的非零差分数。

反例与易错：[2,2]仅需 1 次而单位公式为 2；[1,3,2]有差分 1,2,-1,-2，可分两零和组，答案 2。把正负点随意贪心配对可能错过更多零和组。

练习与来源：tests/round02/test_models.cpp 用独立子集分割递归对照 200 组随机目标；解释[1,3,2]=[1,2]+1 与[2,3]+2。

代码入口：BM02

调用提醒：arbitrary_intervals_small(a)只返回最少次数；星形构造是存在性证明，本函数不输出任意整数操作清单。差分、子集和用 i128。

原代码：template/competition/models/interval_construct.hpp；template/tests/round02/test_models.cpp

【INT_INVARIANT】指定位置分别加减：先写操作向量，再找不变量

识别：一次对若干指定位置加减不同数，问可达、最少操作或“不可能”的原因。

先看什么：把一次操作写成向量 v；寻找权重 w 使 w·v=0，并检查模数、奇偶、方向及中间合法性。

从暴力出发：对小范围状态枚举全部合法操作，用 BFS 画出可达集合，观察分层与剩余类。

重复与性质：许多状态共享同一线性量或同余类；不同类之间根本无边，可以先排除。

推出模型：例：只允许(x,y)→(x+2,y-1)，x+2y 不变；若允许±(2,-2)，总和不变且每坐标奇偶不变。把必要条件列全后，再尝试解操作次数或构造序列。模 m 环上反复+step 另有 gcd(step,m)整除差值的完整判据。

适用条件：一般线性不变量只给必要条件，不能自动当成充分。PM05 的 gcd 判据仅对模环可绕回模型必要且充分，不代表有限区间或任意指定位置操作。

一句证明：每步都使 w·x 增加 w·v=0，归纳得到整条操作序列不改变该量；充分性必须另给达到目标的序列或代数整数解。

复杂度：不变量计算通常 O(维度×操作种类)；模环函数 O(log m)；一般状态图规模由合法状态数决定。

反例与易错：±(2,-2)下(1,-1)总和对却不可达；只允许+(2,-1)时(-2,1)满足线性等式却需反向次数，因此不可达。

练习与来源：自测：从(0,0)分别枚举两种操作的有限状态，写出遗漏的奇偶/方向条件；PM05 测试核验模环可达与独立 BFS。

代码入口：PM05

调用提醒：reachable_mod 只实现“模 m 环上加固定 step”的可达性；任意指定位置变动在本卡是思维框架，未提供万能判定器。

原代码：template/competition/math/invariants.hpp

【CONSTRUCT_FRAME】先从 n=1、2、3 拆限制，再构造并独立验合法

识别：输出任意合法排列、字符串、图或操作序列；可能要求不存在时输出-1。

先看什么：先手算 n=1、2、3；把限制拆成数量、局部相邻、全局和/模数、唯一性、范围，确认奇偶或中心位置。

从暴力出发：小 n 枚举全部排列/二进制填法/简单图，找可行实例与最小不可能例。

重复与性质：对称位置常绑定，配对后只决定半边；排序可固定相对次序；剩余一个中心或余数组通常只需单独处理。

推出模型：优先尝试奇偶分类→镜像/两端向中间→分组配对→按值排序→模类分组→循环移位。每一步写保持的不变量；如要求最少操作，先给下界，再检查构造次数是否等于下界。

适用条件：不存在判定要覆盖所有候选而非“本算法没找到”；存在性要包含每个分支和所有边界。循环移位只有在移动非 0 模 n 且限制确实由移位满足时才可用。

一句证明：构造后的每条限制逐条验；对称保证局部相等不等于数量正确。最优构造另需“下界+达到”，不能从一个合法输出推断最优。

复杂度：套路没有统一复杂度；正式构造必须给出输出规模界。重放 BM02 压缩清单应累计差分，不能按巨大 cnt 逐次展开。

反例与易错：n=1 没有“两端配对”；奇数长度中心只能用一次；移位 k=n 等于没动；排序输出后可能丢失原下标。只验样例或只看形状不能证明合法。

练习与来源：cf/construction/1.cpp 先传播镜像字符再扣计数、处理中心和成对空位；cf/date/5/5.31/3.cpp 原注释明确“先完全再试着补”，用来练分块贡献。原来源线上 AC 均 UNKNOWN。

代码入口：BM02、BM07、STL01、PAL01

调用提醒：BM02 提供可重放的最优单位操作清单；PAL01 保留原 01?镜像填法；STL01 有排列枚举/rotate 短例。独立 checker 检查长度、范围、唯一性、原固定位、目标统计和操作成本。

原代码：cf/construction/1.cpp；cf/date/5/5.31/3.cpp；template/competition/models/interval_construct.hpp

【CONSTRUCT_BLOCK】先造可计数的完整块，再补独立增量

识别：要构造恰好 K 个局部结构，例如三角形、配对或指定贡献。

先看什么：先找贡献可精确计算的大块；补块之间是否会产生额外结构；顶点/长度等预算是多少。

从暴力出发：枚举图的全部边集或排列所有位置，再计算目标结构数量。

重复与性质：结构对称时贡献可以按组合数计算；新对象只连接受控的旧块，可隔离增量。

推出模型：cf/date/5/5.31/3.cpp 先造 p 点完全图，贡献 C(p,3)；新点连旧团中的 s 点产生 C(s,2)个三角形。新点之间没有边，因此不会产生含两个新点的三角形。

适用条件：CF232A 的 1<=k<=100000 下，初始团 p<=85；余数<C(p,2)。一次贪心取最大 C(s,2)后新余数<s，最多 6 个补点即可清零，所以总点数<=91<100。本卡完整证明模型，但旧程序本轮没有单列运行测试，仍标 MODEL_ONLY。

一句证明：新增三角形恰为一个新点和两个旧团点，共 C(s,2)个；无新新边排除交叉贡献。余数依次受界：首补后<=83，再<=12、<=4、<=2、<=1、0，因此最多 6 个补点。

复杂度：原程序输出 O(V²)，在官方范围由证明得 V<=91；贡献分解只增加至多 6 个点。

反例与易错：把新点互连会额外产生三角形；若 k 超官方上界，点数预算需重证。一般“减最大块”不自动成立，本题依赖余数<C(s+1,2)-C(s,2)=s。

练习与来源：CF232A Cycles，官方题面要求恰好 k 个三角形且最多 100 点；用独立三重枚举 checker 核对原程序输出，而不是复用构造公式作为 oracle。

代码入口：M05

调用提醒：组合计数可查 M05；原构造实码在 cf/date/5/5.31/3.cpp，保留原样。M05 是质数模组合数，不是本图构造的直接求解程序。

原代码：cf/date/5/5.31/3.cpp

【GREEDY_DOMINANCE】选择尽量小的可行末值：让所有后续更容易

识别：每个位置有若干可选值，要使最终序列非降；本位置的选择只通过末值约束后续。

先看什么：两个合法前缀若最后值分别 x<=y，是否 x 允许 y 的全部后续；可用资源是否会被消耗。

从暴力出发：对每个位置枚举不改或所有替换来源，遍历所有选择组合。

重复与性质：同样处理到 i 时，较大的末值没有额外好处；所有可行前缀可由最小末值代表。

推出模型：CF2065C2 的候选为 a[i]或 b[j]-a[i]。已选前值 pre，保留 a[i]>=pre 的原值，并在排好序的 b 中找第一个 b[j]>=pre+a[i]；取两种合法候选的较小者。

适用条件：每个位置最多改一次；同一个 b[j]可被不同位置反复使用，不消耗资源；只要求最终非降。BM06 保留原题 a,b<=1e9 的数值范围，不能当成任意 ll 安全接口。

一句证明：若某解当前选 y 而算法可选 x<=y，把 y 换成 x 仍接得上过去且不收紧任何未来下界；逐位替换得到贪心解。

复杂度：排序 b 为 O(m log m)，扫描 a 为 O(n log m)，空间 O(n+m)。

反例与易错：若 b 每个数只能用一次，当前最小可能消耗后面唯一资源，原证明失效。盲目把 a 本身排序会改变允许操作的位置关系。

练习与来源：用户原练习 cf/date/7/7.25/1.cpp；官方 CF2065C2 已核验。复述原注释“先分析贪心，再二分优化”，并检查 n=1、负替换结果和无候选分支。

代码入口：BM06、STL01

调用提醒：BM06 为用户 cf/date/7/7.25/1.cpp 完整原程序，只复制并统一 LF；独立编译运行。lower_bound 只是把可行候选搜索加速，贪心正确性来自支配关系。

原代码：cf/date/7/7.25/1.cpp

【GREEDY_INTERVAL】不相交区间取最多：先选最早结束

识别：从若干区间中选数量最多的一组，彼此不能重叠，每段价值相同。

先看什么：明确端点相接是否冲突；数量目标还是总权目标；一条时间线还是多台机器。

从暴力出发：枚举全部子集，逐对检查相交，找最大集合。

重复与性质：已选前缀只通过最后结束时间影响后续；更早结束不会减少以后可选区间。

推出模型：按右端点升序，依次选起点不早于上次终点的区间。BM07 采用非空半开[l,r)，因此 l>=last 可选。

适用条件：单资源、无权、非空半开区间，允许接触端点；这些前提下最早结束贪心充分。带权版本、机器数>1 或必须恰选某类需另建模型。

一句证明：把任意最优解的第一段替成结束最早的一段，后续仍全部可选且数量不变；对子问题重复交换。

复杂度：O(n log n)时间、O(n)空间。

反例与易错：选最早开始会被[1,10)挡住[2,3)、[3,4)、[4,5)；选最短也可能同时挡住左右两段。闭区间相接算冲突时不能照搬>=。

练习与来源：tests/round02/test_greedy.cpp 对 700 组 n<=10 区间用所有子集对照；手算[0,3)、[2,4)、[3,6)，最短的中段只能选 1 个而两边可选 2 个。

代码入口：BM07

调用提醒：interval_schedule(seg)接受 seg[1..n]，返回选择顺序下的原下标。源码比较器用结束时间、开始时间、下标稳定破同分。

原代码：template/competition/models/greedy_construct.hpp；template/tests/round02/test_greedy.cpp

【GREEDY_COST】固定目标后选最小代价，动态候选交给堆

识别：固定选 k 件、预算有限，或每轮从可用候选里取代价最小/价值最大者。

先看什么：候选资格会不会随 k 或时间变化；已选对象是否有冲突；局部代价是静态还是每次合并后改变。

从暴力出发：枚举 k 子集或所有执行顺序，计算总费用。

重复与性质：固定 k 且费用相加、资格独立时，未选的更便宜对象可以替换较贵已选对象；动态极值只需保留候选集合。

推出模型：BM05 先固定 k，再从 a[i]>=k 的题中选耗时最小的 k 题；按时间全局排序后扫描即可。候选不断插入、撤销或取 top-k 时转 STL02；Huffman 需单独的两最小合并证明。

适用条件：最小代价优先只对可交换且没有额外相互约束的选择成立。堆提供极值操作，不会自动证明贪心；Huffman 要求合并代价等于权值和且权非负。

一句证明：若选择中含昂贵对象而未选便宜的合格对象，交换后资格与数量不变、总耗时不增；重复即可得到 k 个最小。

复杂度：静态排序 O(n log n)，一次固定 k 检查 O(n)；堆每次插入/弹出 O(log n)。

反例与易错：最小费用优先只保证固定数量下的最小费用，不是一般背包最大价值：容量 3，(重量,价值)=(1,1),(3,5)，先选最轻只能得 1，而最优选后者得 5；堆顶规则不能替代证明。

练习与来源：cf/other/Hello_2018/D.cpp 固定 k 的子问题；STL02 huffman_cost 用全合并顺序 oracle 核验，具体证明与反例见 STL_TRICKS.md。

代码入口：BM05、STL02

调用提醒：BM05 的 check 只接受原 main 会调用的 1..n。堆专题直接查 STL_HUFFMAN、STL_TOPK、STL_LAZY、STL_RESOURCE 卡，不重复维护另一份实现。

原代码：cf/other/Hello_2018/D.cpp；template/competition/stl/heaps_sets.hpp

【GREEDY_PAIR】从最大元素反推搭档：首尾配对压低最坏一对

识别：偶数个数全部两两配对，目标是让最大的配对和尽量小；可任意重排。

先看什么：最大数必须与某个数配对；它与最小数配对是否能通过交换而不变差。

从暴力出发：固定一个未配元素，尝试所有搭档，递归枚举全部完美配对。

重复与性质：与最小、最大元素有关的两对可以局部交换；其余对完全不受影响。

推出模型：排序后 a[1]配 a[n]、a[2]配 a[n-1]，向中间推进。这是从最大项的不可回避约束倒推的反向贪心，也是一种两端构造。

适用条件：元素个数为偶数，每个元素恰用一次，目标恰是最大配对和；元素可正可负，和先转 i128。若固定左右阵营或禁止搭档，需重证。

一句证明：最小 m 配 x、最大 M 配 y 时，改成(m,M)、(x,y)：m+M<=y+M，且 x+y<=M+y，因此新两对最大和不超过原两对最大和；归纳剥去首尾。

复杂度：O(n log n)时间、O(n)空间（复制和输出）。

反例与易错：[1,2,100,101]相邻配最大 201，首尾配最大 102。若目标改成配对差绝对值总和，首尾策略会变坏，不能复用结论。

练习与来源：tests/round02/test_greedy.cpp 对 n<=8 三值数组枚举全部配对；[-5,-2,3,8]返回最大和 3，全负例用于发现答案误从 0 取 max。

代码入口：BM07

调用提醒：minmax_pair_sum(a,pairs)清空并写入数值对，返回 i128；a[0]占位，空输入{0}约定值 0。

原代码：template/competition/models/greedy_construct.hpp；template/tests/round02/test_greedy.cpp

【SEARCH_CHECK】把互相牵制的优化目标固定下来，再证明单调

识别：最小可行值、最大可行值、最小化最大值、最大化最小值；直接优化时变量互相制约。

先看什么：写出 check(x)的完整中文含义、可行见证和取值范围；先证明真值只能改变一次，再写二分。

从暴力出发：枚举全部候选答案，对每个候选枚举所有选择方案。

重复与性质：固定答案后条件可能解耦；如果可行方案能通过放松阈值或删对象得到另一方案，可行集合就单调。

推出模型：CF913D 删除不计分题不减分，故可设选 k 题且都计分。check(k)等价于合格题 a[i]>=k 中最小 k 个耗时和<=T；k 可行时删题得到任意更小 k 可行。一般“最大值不超 X”随 X 增大放松，“最小间距至少 X”随 X 增大收紧。

适用条件：CO03 first_true 要求假→真，last_true 要求真→假；无解返回 nullopt。check 内部算术、排序与贪心必须分别正确，不能用一个不可靠的 check 证明二分。

一句证明：从任一可行方案显式构造阈值放松后的方案，证明可行集合是前缀或后缀；二分只丢弃由该单调性排除的一半。

复杂度：O(log U)次 check；BM05 排序 O(n log n)、每次检查 O(n)，总 O(n log n)。

反例与易错：“恰好等于 x 能做到”不一定单调，例如可达和{0,2,4}；此时不能直接二分。只证明数学可行性单调，尚未证明写出的贪心 check 等价。

练习与来源：用户 cf/other/Hello_2018/D.cpp 及官方 CF913D；重新核对官方第二样例没有题可做时原程序输出 0，且不会误调用 check(0)。

代码入口：BM05、CO03

调用提醒：原 BM05 从 l=1,r=n、ans=0 搜索，check(0)从不调用，原逻辑正确且无需重写。若复用 CO03 泛型接口，必须让传入谓词支持接口可能查询的所有端点。

原代码：cf/other/Hello_2018/D.cpp；template/core/search_window.hpp

【SEARCH_REAL】实数阈值用二分，单峰函数才用三分

识别：答案是长度、时间、速度等实数，允许绝对/相对误差；或连续函数求最小值。

先看什么：能否给单调的真假判定，还是只有函数值且已知单峰；初始区间是否包住答案。

从暴力出发：等间距枚举实数网格，精度与运行时间相互牵制。

重复与性质：单调性允许排除半区；单峰性允许用两采样点排除不含最优值的一侧。两种性质不同。

推出模型：实数二分固定迭代次数维护转折点所在区间；三分仅用于先降后升的单峰函数。TRS01 提供 200 次迭代的实现，FP01 处理显示精度与三角函数边界。

适用条件：real_first_true 要求 check 假→真且右端可行，边界/宽度/check 计算有限。ternary_min 允许连续最优平底，但不允许非最优平台；多峰或数值噪声无保证。

一句证明：每次保留的一段仍包含转折点或最优点，区间收缩给误差界；迭代次数不能修复错误的单调/单峰假设。

复杂度：固定 200 次 check 或约 400 次函数评估；额外空间 O(1)。

反例与易错：任意函数并不单峰；$f(x)=(x^2-1)^2$有两个谷，不能直接三分。输出小数位数足够不等于内部计算准确；整数能准确判定的阈值优先整数。

练习与来源：tests/round02/test_runtime.cpp 的平方根阈值、二次函数与几何距离；手写一个多峰反例说明缺少哪条前提。

代码入口：TRS01、FP01

调用提醒：real_first_true(l,r,check)返回近似最小可行值；ternary_min(l,r,f)返回近似极小点。输入输出与 fixed/setprecision 说明查运行精度专题。

原代码：template/Ternary_Search.cpp；template/competition/runtime/real_search.hpp

【SEARCH_STATE】把完整状态当点，把一次合法操作当边

识别：n 很小、位数少、所有局面数量可控；求最少操作、是否可达或输出一条操作序列。

先看什么：状态必须保存影响未来合法操作的全部信息；先数状态与出边数。BM08 把整数局面 x 直接编码为下标 x，边界本身是题面约束。

从暴力出发：递归枚举所有操作序列，相同状态可能被不同序列反复到达。

重复与性质：同一完整状态的未来选择相同；无须记住到达它的全部历史，只保留最短距离或记忆化结果。

推出模型：BM08 示例：在[0,limit]内每步+1/-1/乘 2，用 BFS 和 fa/from 恢复最短操作。另一例对非负数组选/不选，sum>target 或 sum+剩余总和<target 时剪枝；保留选中的原下标。一般边权 0/1 转 PG09，非负一般权转 PG05。

适用条件：BM08 整数模型 0<=start,target<=limit<=1e6；所有合法位置因±1 可达。子集 a[1..n]非负 ll、n<=40、target>=0，空集允许，重复值按位置区分。一般压缩状态需未来完全等价，有环不能直接当 DAG 记忆化。

一句证明：BFS 按步数分层首次到达即最短，前驱逐条恢复合法操作；非负性保证当前和不会下降，剩余全取是最大可加量，两条剪枝均不删除可行解。

复杂度：BM08 状态 BFS O(limit)时间空间；子集回溯最坏 O(2^n)、额外 O(n)，剪枝不保证快速。n 约 40 普通子集和优先 BM04 折半。

反例与易错：随意截断无界状态空间可能删除真正最短路；用 vis[位置]会漏钥匙/能量状态；[5,-4]目标 1 中先超目标仍可成功，负数不能用 BM08 子集剪枝。

练习与来源：tests/round02/test_search.cpp：limit<=30 全部起终点对 Floyd 并重放路径；3→6→5→10 为 3 步最短。非负数组子集与全 mask 枚举对照，{3,3}目标 6 必须用两个位置，目标 0 可返回空集。

代码入口：BM08、PG03、PG09、PG05、M01、STL01

调用提醒：integer_state_bfs(start,target,limit)返回 states 与 operations（+/-/*）；subset_target_nonnegative(a,target)返回一组原下标或 nullopt。源码直接给出编码、选/不选递归、回退和剪枝，不只是伪流程。

原代码：template/competition/models/state_search.hpp；template/tests/round02/test_search.cpp；template/competition/stl/recipes.hpp

【ARRAY_PREFIX】静态问和看前缀，离线区间加看差分

识别：多次求区间和；或全部区间增量先给出，最后只要最终数组。

先看什么：操作顺序中有没有“修改后立即查询”；聚合能否由两份前缀作逆运算。

从暴力出发：每次查询或修改逐项扫一段，总复杂度 O(nq)。

重复与性质：区间和的两前缀共享前半段；区间加的内部变化相同，只有起止边界不同。

推出模型：前缀 s[r]-s[l-1]得到区间和；差分 d[l]+=x、d[r+1]-=x，全部处理后前缀还原。在线单点增量/区间和转 Fenwick；复杂在线区间变换查线段树菜谱。

适用条件：CO02 用 1 下标闭区间，ll 全部中间量须不溢出；Difference 初始全 0。差分只能廉价积累修改，不能 O(1)回答任意时刻区间和。

一句证明：前缀相减逐项抵消共同部分；差分还原时 l 起多出的 x 恰在 r+1 被抵消。

复杂度：前缀建 O(n)、问 O(1)；差分每改 O(1)、最终还原 O(n)；Fenwick 每操作 O(log n)。

反例与易错：min 不能由两个前缀 min 相减；对旧前缀表修改数组后不重建会读到旧快照。l>r 的空区间契约与非空下标越界要分开。

练习与来源：原 CO02 测试随机区间对拍；用[2,-1,4]核对[2,3]和 3，再在[1,2]加 3 得到[5,2,4]。

代码入口：CO02、PSD01

调用提醒：PrefixSum(a).sum(l,r)；Difference(n).add(l,r,x)，结束后 values()；若有原数组，最终增量加到原值，不能误当 Difference 自动带初值。

原代码：template/core/basics.hpp

【ARRAY_2D】矩形和与矩形加：二维容斥的四个角

识别：二维矩阵反复求子矩形总和；或大量矩形统一加后一次恢复。

先看什么：先约定坐标是否 1 下标、闭区间；查询快照与离线修改能否分批。

从暴力出发：每个矩形遍历全部格子，最坏 O(nmq)。

重复与性质：二维前缀重叠部分被减了两次，需要补回一次；矩形差分只需标记四个角。

推出模型：s[x][y]=a+s[x-1][y]+s[x][y-1]-s[x-1][y-1]；查询同样四项容斥。矩形加在 (x1,y1),(x2+1,y1),(x1,y2+1),(x2+1,y2+1)依次+,-,-,+。

适用条件：BM03 使用 1..n、1..m 且每维<=500，ll 前缀/差分中间值不溢出。矩形更新离线累积；在线二维数据结构不由这个模块覆盖。

一句证明：每个格子的系数等于它是否同时落在两个坐标范围内；四项容斥将矩形外系数抵消为 0，内部保留 1。

复杂度：预处理/恢复 O(nm)，单次查询/矩形加 O(1)，空间 O(nm)。

反例与易错：右/下边界必须有 x2+1、y2+1 哨兵；恢复差分不会自动更新 s。将闭区间右端忘加 1 会少一行/列。

练习与来源：tests/round02/test_model_tools.cpp 把二维结果与逐格矩形更新/求和比较；覆盖 1x1、负增量、边界矩形与分批恢复。

代码入口：BM03

调用提醒：init(n,m)→填 a→build_prefix→query；修改批次 add_rect→restore_difference。若随后查询新数组，必须再 build_prefix；restore_difference 后差分已清零可开下一批。

原代码：template/competition/models/prefix_grid.hpp

【ARRAY_WINDOW】双指针成立的关键是可排除一整类候选

识别：排序数组找两数和；非负数组找和不超过 K 的最长连续段；固定窗口维护极值。

先看什么：指针移动后，丢掉的端点为何永远不会再有用；窗口扩张/收缩是否保持单调。

从暴力出发：枚举所有左右端点或所有二元组，复杂度 $O(n^2)$。

重复与性质：有序两数和太小时，固定左端与任何更小右端仍太小；非负窗口和随右端扩张不减。

推出模型：两数和小于目标时增左端，大于目标时减右端。非负限和窗口每次扩右，和超 K 就缩左，维护最大长度；固定窗口最小值另查 DS04。

适用条件：two_sum_sorted 要求数组非降、使用不同下标；longest_sum_at_most 要求全非负且 1 下标。一般含负数的和约束不满足此窗口证明。

一句证明：每次移走的端点，与尚未扫描的另一端所有组合都不可能改进答案，因此丢弃不会漏解。

复杂度：已排序后双指针 O(n)，窗口 O(n)，额外空间 O(1)；需先排序时再加 O(n log n)。

反例与易错：[5,-4]、K=1 的合法最长段是整个数组，但标准非负窗口会先丢掉 5 而漏解；排序原数组会破坏原题连续区间含义。

练习与来源：CO03 原随机对拍与负数禁用说明；分别解释两数和与窗口的指针移动理由，不把两个证明混写。

代码入口：CO03、DS04

调用提醒：two_sum_sorted(a,target)无解返回{0,0}；longest_sum_at_most(a,limit)返回长度，limit<0 返回 0。

原代码：template/core/search_window.hpp

【ARRAY_COMPRESS】排序去重与离散化：保留顺序，别丢掉距离和次数

识别：值域很大但只有 n 个值；要计频、比较排名、建 Fenwick/线段树；或只需不同元素集合。

先看什么：题目需要原值距离、出现次数还是仅相对顺序；查询值是否可能不在初始集合。

从暴力出发：按原值域开数组，或每次线性查一个值的排名。

重复与性质：相等元素共享坐标，只有排序后的不同值有不同秩；大跨度空值不需要存储。

推出模型：复制原值→排序→unique→erase，保留去重表，lower_bound 取得排名。若计数对象按位置区分，只压缩值域、不删除原数组中的重复元素。

适用条件：CO02 Compression.rank 只接受初始化中出现过的值；离散化保序，不保差值、面积、区间长度。在线未知新值需换结构或离线收集。

一句证明：严格单调的秩映射保留<、=、>比较，因此基于顺序的查询不变；对值相减的结论则没有这种保证。

复杂度：O(n log n)预处理、O(log n)求秩，空间 O(n)。

反例与易错：[1,100]压成[1,2]后距离从 99 变 1；统计子集方案时两个相同值仍是两个位置，不能 unique 掉。unique 本身也不改变 vector.size。

练习与来源：CO02 原测试含重复值/负数；拿[5,-2,5,100]写出秩[2,1,2,3]，分别回答不同值数与元素总数。

代码入口：CO02、STL01

调用提醒：Compression c(a)，c.rank(x)取 1 下标秩，c.values[rank]反查原值；STL01 演示 sort+unique 与 erase-remove 的不同用途。

原代码：template/core/basics.hpp；template/competition/stl/recipes.hpp

【ARRAY_INVERSION】逆序对：把二重枚举分成左右内部与跨中点

识别：统计 i<j 但 a[i]>a[j]的对数，或排序所需相邻交换次数。

先看什么：严格大于还是大于等于；相等元素不构成普通逆序对；答案可能达 n(n-1)/2。

从暴力出发：枚举全部 i<j 逐对比较，O(n²)。

重复与性质：左右两半排好序后，右边某值小于左边当前最小未合并值时，它小于左边全部剩余值。

推出模型：归并时若 a[i]<=a[j]先取左边；否则一次加 mid-i+1，递归累计左内、右内与跨中点三类。也可离散化+Fenwick 计已见更大元素。

适用条件：BM04 处理 1 下标数组且 n<200010，函数会把 a[l..r]排序；ll 保存逆序对数。相邻交换解释要求最终按非降排列。

一句证明：每对逆序对在唯一一次递归分割时跨左右边界，归并只计这些跨边界对，因此恰计一次。

复杂度：O(n log n)时间、O(n)辅助空间。

反例与易错：[2,2]逆序数 0；归并比较写成<会把相等元素计入。最大 n=200000 时逆序数约 2e10，int 不够。

练习与来源：tests/round02/test_model_tools.cpp 与逐对 O(n²)比较；[3,1,2]答案 2，并核对输出 a 已变为[1,2,3]。

代码入口：BM04、CO02、PSD01

调用提醒：填 BM04 全局 a[1..n]后 merge_count(1,n)；若后续还要原顺序，先自行备份。

原代码：template/competition/models/enumeration.hpp

【ARRAY_MONOTONIC】单调结构删除被支配者，每个下标最多进出一次

识别：每个位置左/右最近更大或更小；固定窗口的最小/最大值；柱形边界。

先看什么：答案需要值还是下标；严格关系如何处理相等；窗口下标何时过期。

从暴力出发：每个位置向前扫描找最近更小，或每个窗口重扫 k 个元素。

重复与性质：更靠后且更小的候选，会在未来的“最近更小/窗口最小”任务中支配更早且更大的候选。

推出模型：栈从尾弹出不可能再成为答案的元素；队列除尾部支配删除外，还从头删除过期下标。DS04 previous_less 弹出>=，window_min 相等时保留新下标。

适用条件：DS04 具体实现为左侧严格更小与固定窗口最小值；转成子数组最小值贡献时，左右一边严格一边非严格，以便重复值唯一归属。

一句证明：被弹出的旧候选更差且更早失效，以后不会优于新候选；每个下标入、出各一次，摊还线性。

复杂度：O(n)时间、O(n)空间。

反例与易错：只存值无法判断哪个重复值离开窗口；贡献计数左右都严格会对[2,2]重复归属，都非严格又会漏归属。

练习与来源：DS04 原测试；[2,2,1,3]左侧严格更小下标为[0,0,0,3]（去掉返回数组的 0 号占位）。

代码入口：DS04

调用提醒：previous_less(a)无答案返回 0；window_min(a,k)返回 ans[l]对应左端 l，非法 k 返回{0}。

原代码：template/data_structure/monotonic.hpp

【COUNT_CONTRIBUTION】从逐个统计答案，改成每个元素贡献多少次

识别：所有子数组、所有二元组的和；某元素会被很多对象重复算入。

先看什么：一个固定元素作为左端、右端、最大值或最小值时各出现几次；权重是否可以独立相加。

从暴力出发：枚举全部对象，再遍历其中每个元素，常出现 $O(n^2)$或 $O(n^3)$。

重复与性质：求和可以交换顺序，元素的值不变，变的只是它被计入的次数与符号。

推出模型：排序后的所有无序对绝对差，a[i]作较大者 i 次、较小者 n-1-i 次（0 下标），系数为 2i-n+1。所有子数组元素总和则每个 a[i]出现 i(n-i+1)次（1 下标）。

适用条件：可以交换有限求和；必须先分清有序/无序、允许相等与否。PM04 pair_abs_sum 实现的是二元绝对差和，子数组公式是推导示例。

一句证明：每个二元组唯一按大下标归入一次；固定 a[i]的正负出现次数分别数清，所有对既不漏也不重。

复杂度：绝对差和 O(n log n)时间、O(n)复制空间；已知贡献系数的求和可 O(n)。

反例与易错：未排序就套 2i-n+1 会把下标当大小；用 ll 先乘系数再转 i128 仍会溢出。子数组最小值贡献还需解决重复最小值归属，不能直接用 i(n-i+1)。

练习与来源：cf/date/7/7.28/1.cpp 原注释从路径想到排列，实际 sum2 正是排序绝对差贡献；这里只抽取该局部公式，不把后续分数解释成已核验整题。

代码入口：PM04、CO02

调用提醒：pair_abs_sum(vector<ll>)接受无占位的值列表，内部排序并返回 i128；注意它与多数 1 下标模块的接口不同。

原代码：cf/date/7/7.28/1.cpp；template/competition/math/sums.hpp

【COUNT_ENDPOINT】固定一个位置，让其余选择变成前后缀统计

识别：要选两端或三元组；某个中间位置把合法选择分成左边与右边。

先看什么：固定 r 或 j 后，剩余条件能否分成只依赖左侧与只依赖右侧；需要前缀和、频率还是最优值。

从暴力出发：两端枚举 $O(n^2)$，三元组枚举 $O(n^3)$，每次重复扫描两侧。

重复与性质：相邻端点的左侧集合只新增一个元素；右侧统计可预处理或反向扫描。

推出模型：例：递增三元组 i<j<k 且 a[i]<a[j]<a[k]，固定 j 后设 L[j]为左侧更小数目、R[j]为右侧更大数目，贡献 L[j]·R[j]。离散化+Fenwick 分别扫描两遍；删除一个位置后的总信息常由 prefix[i-1]与 suffix[i+1]合并。

适用条件：左右选择在固定中点后独立，乘法才成立；若还有 i 与 k 之间的联合约束，不能只乘两侧数量。本卡是组合框架，Fenwick 等组件有测试，三元组拼装未作为独立模块宣称验证。

一句证明：每个三元组有唯一中点 j；固定 j 时左、右各选一个合法位置一一对应一组合法三元组。

复杂度：三元组框架 O(n log n)时间、O(n)空间；可直接线性合并的前后缀为 O(n)。

反例与易错：先把当前位置插入频率再查询可能把自己算入；重复值要求严格比较时不可用<=。L·R 和总答案都可能超 int。

练习与来源：自拟[1,2,3,4]共 4 个严格递增三元组、[1,1,2]为 0；写三重枚举 oracle 验证组合后的程序。

代码入口：PSD01、CO02、STL01

调用提醒：计数查询可用 PSD01 add/query，严格小于查 rank-1，严格大于用总数减前缀 rank；前后缀构造短例见 STL01。

原代码：template/competition/data_structure/fenwick.hpp；template/competition/stl/recipes.hpp

【COUNT_CASES】分类讨论要互斥且穷尽，用最小反例定位缺类

识别：正负号、奇偶、余数、相等、是否为零决定不同结果；一条贪心或公式只能覆盖部分输入。

先看什么：列出所有会改变不等号方向、除法合法性、配对余数的条件；优先检查 0、1、2、3。

从暴力出发：枚举小范围输入，按失败原因分组，而非逐个补补丁。

重复与性质：同一类通常共享一个证明；跨类的差异只发生在明确边界。

推出模型：例如无序对乘积>0，正正与负负两类贡献 C(pos,2)+C(neg,2)，0 不贡献；若目标变成>=0，所有含 0 的对必须补入，且两零对只能计一次。

适用条件：分类必须两两不交且覆盖全体；每类内部公式仍须独立正确。n=0、空集、全相等是否允许由题面决定。

一句证明：给每个合法对象一个唯一类别，再在类别内建立计数对应；唯一归类同时证明不重不漏。

复杂度：取决于分类特征统计；正负计数 O(n)，各类组合 O(1)次查询。

反例与易错：把条件写成 x<=0 与 x>=0 会把 0 重复归类；奇数分支忘记未配对元素；用试出来的有限模式直接替代证明。

练习与来源：自拟[-2,-1,0,0,3]：乘积>0 的无序对 1 个，>=0 的 8 个；枚举十对作为独立核对。

代码入口：M05、STL01

调用提醒：M05 仅提供满足质数模与 n<mod 条件的组合数，STL01 提供计频与排序；本卡是推导框架，不把所有分类问题包装成通用代码。

原代码：template/math/combinations.hpp；cf/construction/1.cpp

【SEARCH_MITM】折半搜索：把一个大指数拆成两张半集表

识别：n 约 30~40 的选/不选问题，数值大不能按容量 DP；两半可通过一个和值或少量摘要合并。

先看什么：答案是存在性、数量还是最优；相同和值来自不同子集是否必须保留；子集和与 target-s 会不会溢出。

从暴力出发：枚举全部 2^n 个子集；每个元素只有选或不选。

重复与性质：任一全子集唯一分成左右两半；半边的内部选择只通过总和参与另一边的条件。

推出模型：枚举两半全部子集和，排序右半；对左和 x 统计右半中 target-x 的出现次数。计数用 equal_range，不能把重复和值去重。

适用条件：BM04 支持 1 下标向量、n<=40；所有子集和及 target-x 在 ll 范围。子集按位置区分，包含空集。可分解摘要是折半合并的前提。

一句证明：每个全子集对应唯一的左右子集对，和值条件等价于右和=target-左和，故二分区间次数恰是合法搭配数。

复杂度：O(n·2^(n/2))量级时间、O(2^(n/2))空间；具体实现排序一半并逐项二分。

反例与易错：[1,1]目标 1 有 2 个位置子集，不是 1；target=0 时空集必须算一次。负数存在时不能用“当前和已超目标”剪枝。

练习与来源：tests/round02/test_model_tools.cpp 与全子集枚举对照；{0,1,1}、target=1 返回 2，0 号只是占位。

代码入口：BM04、STL01

调用提醒：subset_sum_count(v,target)，v[0]占位，返回 ll；n<=40 时总方案至多 2^40，计数可装 ll。

原代码：template/competition/models/enumeration.hpp

【STRING_BRACKETS】括号：总数相等还要每个前缀都不欠账

识别：判断括号串合法；匹配位置；问某前缀能否成为一个合法括号序列的开头。

先看什么：只有一种()还是多种括号；只判断合法还是要匹配下标；空串是否允许。

从暴力出发：反复删除相邻()直到不能删，或枚举匹配关系。

重复与性质：单类括号的所有未匹配左括号彼此等价，判断时只需记录数量；恢复匹配位置才需要栈。

推出模型：把(记+1、)记-1；合法当且仅当所有前缀和>=0 且总和 0。多类括号使用栈保存类型与下标。

适用条件：BM07 只支持()，空串合法，其他字符返回 false；多类或带通配符题不属于这个函数。

一句证明：任何右括号必须匹配它前面尚未使用的左括号，故前缀不能负；始终有可用左括号且最终无剩余就能逐个配完。

复杂度：单类判断 O(n)时间/O(1)空间；下标或多类匹配 O(n)栈空间。

反例与易错：“)(”左右数量相等但前缀为负；“([)]”各类数量相同也不合法，不能给每种类型独立计数后宣布成功。

练习与来源：tests/round02/test_greedy.cpp 穷举长度<=16 的全部()串，与独立栈判定比较，另查非法字符和 20 万长串。

代码入口：BM07、STL01

调用提醒：regular_brackets(s)判断；STL01 有 stack 匹配下标短例。多组输入按题面读取，不默认空白分隔能读入空串。

原代码：template/competition/models/greedy_construct.hpp；template/competition/stl/recipes.hpp

【STRING_PERIOD】字符串边界连到周期，模环周期由 gcd 决定

识别：字符串由同一块重复组成；操作在有限模环上反复加一个步长；求多久回到起点。

先看什么：“周期”允许最后一块残缺吗；是完整字符串重复还是状态重复；n=0 需单独约定。

从暴力出发：枚举块长逐字符检查；模环逐步模拟直到首次回到起点。

重复与性质：字符串最长真前后缀说明平移后有大段重合；模环回到起点等价于 t·step 被 m 整除。

推出模型：非空串取 p=n-pi[n-1]；若 n%p==0，最短整块长度 p，否则最短整块长度 n。模 m 环加 step 的循环节 =m/gcd(step,m)，可达类差值须被同一 gcd 整除。

适用条件：S01 prefix_function 为 0 下标字符串；完整重复需要整除条件。模环要求 m>0、允许绕回，step=0 时周期 1；有限线性范围不是该模型。

一句证明：长度 n-p 的前后缀相等意味着 s[i]=s[i-p]，再用整除闭合全部重复块；最小 t 满足 m|t·step 恰为 m/gcd。

复杂度：字符串 O(n)时间空间；模环 gcd O(log m)时间/O(1)空间。

反例与易错：ababa 有位移周期 2，但不能完整拼成若干个 ab；直接输出 2 漏掉 n%p 条件。重复出现一个状态能推出后续周期，还需要转移确定且时间不改变规则。

练习与来源：S01 原测试已验证 pi 与朴素真前后缀；手算 abababab→2、ababa→5（整块）；模 10 加 4 周期 5，只到达同奇偶类。

代码入口：S01、PM05

调用提醒：S01 计算 pi；周期判断按本卡推导组合，未新增重复实现。PM05 reachable_mod 判断可达剩余类，不直接返回周期。

原代码：template/string/kmp_z.hpp；template/competition/math/invariants.hpp

【COUNT_COMBINATIONS】排列和组合：先决定对象是否可区分、顺序是否重要

识别：选 k 个位置、安排次序、分配相同元素；求方案数且通常取模。

先看什么：选择的是位置还是数值；重复值是否代表不同对象；空选择与有序/无序的区别。

从暴力出发：枚举所有 k 子集或全排列，再过滤限制。

重复与性质：每个无序 k 子集对应 k!个不同排序；相同元素互换不产生新字符串，因此需除去重复排列。

推出模型：无序选 k 个为 C(n,k)，有序选为 n!/(n-k)!；多重集合排列为 n!/Πc!。带局部禁用或状态依赖时，不再只由频数决定，转计数 DP。

适用条件：M05 阶乘逆元法要求模数已知为质数、0<=预处理上限<mod；不自动检查质数。除法在模意义下需要可逆，不能直接整数除后取模。

一句证明：建立排列与“子集+内部次序”的 k!对 1 映射，或按相同元素交换产生的等价排列数消重；每个类大小必须一致才可这样除。

复杂度：M05 预处理 O(n+log mod)、查询 O(1)、空间 O(n)；完整枚举按排列数量计。

反例与易错：aab 不同字符串排列只有 3 种，不是 6；按位置选一个 a 却有 2 种。模数合数或 n>=mod 时阶乘不可逆，不能继续用这个模板。

练习与来源：M05 原测试与 Pascal 三角对照；STL01 枚举 aab 得到 3 种，用来区分重复值与不同位置。

代码入口：M05、STL01

调用提醒：PrimeComb(max_n,prime).choose(n,k)，C(0,0)=1、非法 k 返回 0；全排列自测用 STL01 的 do...while(next_permutation)，先排序且别漏初始排列。

原代码：template/math/combinations.hpp；template/competition/stl/recipes.hpp

【GRID_GRAPH】先把题意翻译成点和合法移动，再数连通块

识别：迷宫、岛屿、相邻格子传播、把一组相互可达对象合并。

先看什么：点是格子还是状态；四邻接/八邻接；障碍；是否有方向；是否多个源同时扩散。

从暴力出发：每个起点重复搜索所有路径，或反复扫描全图传播标记。

重复与性质：无向连通块中的任何点都共享同一可达集合；BFS 首次入队即可确定单位边距离。

推出模型：单位网格走路用 PG07；一般连通块用 PG04；多个起点同时从距离 0 入队用 PG08；只做动态加边连通性用 PG02 并查集。

适用条件：PG07 是四邻接、#为障碍、1 下标矩形；PG04 无向图 component 才对应普通连通块。有向可达不是对称关系，不应直接当 DSU 集合。

一句证明：搜索恰沿合法边从已到达集合扩张，结束时既不遗漏任何可达点，也不会访问不可达点；一块只在第一个未访问点启动一次。

复杂度：网格 O(nm)，邻接表搜索 O(V+E)，并查集近似 O(α(n))摊还。

反例与易错：八邻接题套四方向会少连边；入队时不标记会重复入队；能量/钥匙影响移动时点必须加状态维度。

练习与来源：tests/round02/test_graph_personal.cpp 的网格/多源与独立 Floyd 对照；自行检查 1x1、起点障碍、不可达和多源重合。

代码入口：PG07、PG04、PG08、PG02

调用提醒：PG07 每行前补空格，bfs(sx,sy)后 dist=-1 为不可达；PG08 全部源一次入队，不能把各源结果顺序覆盖。具体接口以图论专题为准。

原代码：template/competition/graph/grid_bfs.hpp；template/competition/graph/multisource_bfs.hpp

【GRAPH_PATH】先决定边权，再选 BFS、0-1 BFS 或 Dijkstra

识别：最少步数、最小花费、少用一次特殊操作，或把某条件作为状态层。

先看什么：一次行动的真正代价；代价是否只能 0/1、能否为负；是否存在不同状态但相同地点。

从暴力出发：枚举全部路径；有环时甚至无限重复，简单路径数量也可指数级。

重复与性质：到同一完整状态时，更大代价通常被更小代价支配；通过松弛逐步维护最好距离。

推出模型：全 1 边用 PG03；0/1 边用 PG09，0 边压队首、1 边压队尾；非负权用 PG05 Dijkstra。少量全源、可含负边且无负环时查 PG10 Floyd；“免费用一次”可按是否使用分两层。

适用条件：Dijkstra 需所有边非负；Floyd 遇负环不再有通常有限最短路解释。分层状态必须保留影响未来的次数/模式；数值需满足模块 INF 边界。

一句证明：非负边保证 Dijkstra 当前最小未确定距离不会被绕路变小；0-1 BFS 维持非降距离处理顺序。状态间只有可合法执行的行动才能连边。

复杂度：BFS/0-1 BFS $O(V+E)$，堆 Dijkstra $O((V+E)\log V)$，Floyd $O(V^3)$。

反例与易错：收费 0 和 1 的边混合时普通按边数 BFS 可能错误；状态只记城市会把“免费机会已用”和“未用”混在一起。最少转弯题的状态通常还需当前方向。

练习与来源：tests/round02/test_graph_personal.cpp 随机距离对照；自己画出两条边成本 1+1 与三条边 0+0+1，说明边数最少不是费用最少。

代码入口：PG03、PG09、PG05、PG10

调用提醒：查 GRAPH_TREE_COOKBOOK.md 的建图/调用；距离数组不可达标记、父边和有向/无向加边方式须按模块契约核对。

原代码：template/competition/graph/zero_one_bfs.hpp；template/competition/graph/dijkstra.hpp

【TREE_COUNT】先定根分解子树，路径问题用 LCA 或边贡献

识别：树上求子树大小、路径长度、某条边被多少对点路径经过；每条路径唯一。

先看什么：树是否连通无环、权是否非负/无权、根会不会改变答案；统计按点还是按边分解。

从暴力出发：每个点重新 DFS 子树，或对所有点对逐条找路径。

重复与性质：同一个子树只需汇总一次；删一条树边会将点集分成两个互不相交部分。

推出模型：PG01 定根得到 fa、dep、sz 及父先子后的 order，逆序合并子树。边(u,fa[u])切成 sz[u]与 n-sz[u]两部分，经过它的无序点对有乘积那么多；路径长度可由 LCA 深度公式得到。

适用条件：输入必须是真树；PG13 两遍最远点只提供无权树直径。边贡献适用于唯一简单路径，带权时再乘边权，负权路径优化不能照搬无权直径算法。

一句证明：一对点的唯一路径经过该边，当且仅当两点分处删边后两侧；两侧独立选点给出 sz·(n-sz)。

复杂度：定根/子树 O(n)，LCA 预处理 O(n log n)、每问 O(log n)；边贡献求和 O(n)。

反例与易错：把父边当儿子会重复回走；深链递归可能爆栈；n=1 直径是 0 条边。sz·(n-sz)先用足够宽的整数再乘。

练习与来源：图论随机树测试核验子树/LCA；链 1-2-3 上两条边各被 2 个无序点对经过，总距离 4。

代码入口：PG01、PG06、PG13、G10

调用提醒：PG01 read_tree/root_tree 后读 sz/order；PG06 依赖同一树存储构建倍增。Euler 子树区间仍可直接引用已验证 G10，树形 DP 转 DP_MODEL_ATLAS.md。

原代码：template/competition/graph/tree_storage.hpp；template/graph/euler_subtree.hpp

【GRAPH_CYCLE】先分清有向环、无向环、基环结构与桥

识别：加边是否成环、能否安排依赖顺序、删边后是否断开、每块恰好有一个环。

先看什么：有向/无向、重边、自环、是否连通；边数只给约束，不能代替连通性检查。

从暴力出发：枚举路径找闭环；逐边删除再搜索判断是否断开。

重复与性质：无向加边时两端已连通便形成环；有向依赖可用入度剥离；桥只需比较子树能否回到祖先。

推出模型：PG02 维护无向连通性；PG12 恢复一般有向/无向环；G04 拓扑判断 DAG；简单连通 n 点 n 边才用 G05 基环图；强连通/桥直接引用 G11/G12 旧可靠模块。

适用条件：无向多重图必须按边号跳过父边，不能跳过所有同端点边；G05 限简单连通无向基环图。SCC 与桥属于按需识别，不借此扩展更高级结构。

一句证明：树加一条连接同块两点的边形成唯一闭路；拓扑过程若仍有未移除点，它们的有限依赖链必进入环。不同结论各自依赖图类型。

复杂度：搜索、拓扑、SCC、桥 O(V+E)；DSU 近似 O(α(n))摊还。

反例与易错：n 条边但不连通可能是一块多环加一块树，不能称基环图；无向两条平行边不是桥，若按父顶点全部跳过会误判。

练习与来源：G05、G11、G12 原测试覆盖自环/重边/不连通；PG12 用恢复出来的边号逐边检查闭环，不只相信一个 true。

代码入口：PG02、PG12、G04、G05、G11、G12

调用提醒：有向环调用 PG12 对应接口；不要把 DSU 检测到无向环当成有向环判定。SCC/桥使用原模块，不再维护重复实现。

原代码：template/competition/graph/cycle.hpp；template/graph/unicyclic_graph.hpp；template/advanced/bridges.hpp

【CG01】树的存储、多测与根化

识别：输入 n-1 条无向边；题目说子树、父亲、深度。

先看什么：树是存储+选根后的方向关系；点编号仍然 1..n。

从暴力出发：对每个点重新遍历所有后代，O(n^2)。

重复与性质：同一子树会被每个祖先重复统计。

推出模型：先定根保存 fa/dep；父先子后的 order 反过来累加 sz，边只处理常数次。

适用条件：1<=n<200010；g[1..n]是双向对称邻接表，端点合法。read_tree 读 n 及 n-1 条边并先 init。root_tree(root)非树返回 false；成功时根 fa=0、dep=0，sz 包括自己，order 父亲先于儿子。

一句证明：连通无向图有 n-1 条边当且仅当是树。入栈时标记保证每点唯一父亲；父先子后的顺序逆转后，所有孩子先于父亲完成累加。

复杂度：建图/根化 O(n)，空间 O(n)。

反例与易错：每测先清 1..n，不能只清数组不清 g。；度为 1 是无根树叶子；有根树的根不能直接套此判断。；order 是父先子后的次序，不能直接当 Euler 闭区间；子树区间用 G10。

练习与来源：cf/3/1096/H_!!dp_tree.cpp：只练习读边、clear、父亲与子树汇总，不把局部 ct/mx 当通用答案。

代码入口：PG01

调用提醒：istringstream in("4 1 2 2 3 2 4");
read_tree(in);
bool ok=root_tree(1);
// ok && sz[2]==3

原代码：template/tree_storage.cpp；cf/3/1096/H_!!dp_tree.cpp

【CG02】个人原版并查集 fa/rk

识别：操作只把两组接起来，询问是否同组。

先看什么：合并关系具有传递性且不会撤销。

从暴力出发：每次查询都沿已知边 BFS，O(n+m)。

重复与性质：相同组件被查询反复遍历。

推出模型：让整块共享代表；路径压缩缩短查询，按秩避免高树。

适用条件：0<=n<200010，调用点为 1..n；init/find/unite/same 沿用原接口。unite 是 void，rk 是秩不是集合大小。

一句证明：每次只合并两个连通块，find 路径压缩不改变根所属集合；按秩合并避免构造线性高度。

复杂度：init O(n)，合并和查询均摊 O(alpha(n))，空间 O(n)。

反例与易错：原注释“挂到秩小的那里”写反，算法本身正确。；不能按 rk 推集合大小；需要大小时另设 sz。；普通 DSU 不支持删边；不能直接套带撤销题。

练习与来源：cf/xian/B.cpp：辨认第一次出现位置与第二次位置的等价合并，再理解栈顶 same 查询。

代码入口：PG02

调用提醒：init(4);
unite(1,2);
unite(2,1);
bool ok=same(1,2);

原代码：template/DSU.cpp；cf/xian/B.cpp；cf/3/1109/3.cpp

【CG03】原队列 BFS 与最短路径恢复

识别：最少走几步，所有动作代价 1。

先看什么：距离 d 的点处理完才会处理 d+1 层。

从暴力出发：枚举所有路径，指数级且会绕环。

重复与性质：同一点更晚到达不会产生更短前缀。

推出模型：首次入队即锁定最短距离，保留 fa 恢复一条见证。

适用条件：1<=start<=n<200010；点 1..n，有向/无向均可；init 只清图，bfs 重置访问/距离/父亲。不可达 dist=-1；path 返回点序列。

一句证明：FIFO 按层扩展；首次发现 v 时所有更短层都已扫描，因此 dist[u]+1 就是最短距离。

复杂度：O(n+m)时间、O(n+m)空间。

反例与易错：出队才标记会重复入队。；带 0 权或不同边权不能继续用普通 BFS。；有向边不要自动反向 push。

练习与来源：template/bfs.cpp：给一条有向链加孤立点，检查 dist=-1 与 path 空。

代码入口：PG03

调用提醒：init(4);
g[1]={2};
g[2]={3};
bfs(1);
auto p=path(3);

原代码：template/bfs.cpp；template/graph/bfs_path.hpp

【CG04】显式栈 DFS 与连通块

识别：需要分连通块或按图结构逐点处理。

先看什么：普通图会有环，需要 visited；树父边判断只是特例。

从暴力出发：对每点单独搜索其组件。

重复与性质：同一组件被 n 次重复计算。

推出模型：一次 DFS 标整个组件；外层只对未访问点启动。

适用条件：0<=n<200010，点 1..n；dfs_forest 重置状态并遍历所有组件。comp 从 1 开始，根 fa=0。有向输入只表示遍历森林，不能当强连通分量。

一句证明：每个栈帧与递归调用一一对应；visited 保证点只首次进入一次，扫完整个组件后再选新根。

复杂度：O(n+m)时间和空间。

反例与易错：只 dfs(1)会漏断图。；仅跳过父亲只适用于树，普通图要 visited。；树深度 20 万不依赖系统递归栈。

练习与来源：template/dfs.cpp：链改为 20 万点，比较递归栈与显式栈帧。

代码入口：PG04

调用提醒：init(4);
g[1]={2};
g[2]={1};
g[3]={4};
g[4]={3};
dfs_forest();

原代码：template/dfs.cpp；template/graph/dfs_forest.hpp

【CG05】全局数组 Dijkstra

识别：费用不同但都非负，问从源点最小总费用。

先看什么：首次发现不一定最短，必须允许松弛。

从暴力出发：枚举所有路径或每轮找未确定最小点 O(n^2)。

重复与性质：每轮线性扫描最小候选开销高。

推出模型：用小根堆按候选距离出队；非负性保证有效最小候选安全确定。

适用条件：0<=n<200010，点 1..n；0<=w<INF=LLONG_MAX/4；距离>=INF 视为不可达。dijkstra 返回 false 表示非法端点/源点/边权，false 后结果作废；合法空源集合全 INF。

一句证明：最小距离候选出堆时，非负边不能通过尚未出堆点改出更短路径；过期距离快照跳过。

复杂度：堆优化 O((n+m)log(n+m))时间，O(n+m)空间。

反例与易错：负边即使不在源点可达区也拒绝。；INF 不是任意大的真实距离；总路径长度须小于 INF。；等长不改父亲，避免零环导致回溯成环。

练习与来源：用 1->2 权 3、1->3 权 1、3->2 权 1 手算一次过期堆项。

代码入口：PG05

调用提醒：init(3);
g[1]={{2,3},{3,1}};
g[3]={{2,1}};
bool ok=dijkstra(1);
auto p=path(2);

原代码：template/graph/dijkstra.hpp

【CG06】个人树上倍增 LCA

识别：同一树有大量祖先或路径距离查询。

先看什么：一次向父亲走一步最坏走完整条链。

从暴力出发：每次同步爬父亲，单次 O(n)。

重复与性质：查询中反复经过同一长祖先段。

推出模型：预存 2^k 级祖先，将爬升分解为二进制，再找 LCA 两侧最后分叉。

适用条件：依赖 PG01；1<=n<200010，树双向存储。build_lca(root)必须在建图或换根后调用；非树 false。jump 越根/非法点返回 0，distance 非法返回-1。LOG=20，扩大 N 时同步调整。

一句证明：up 转移将两段 2^(k-1)跳跃拼接；先等深后从高位同时上跳，最后两点的父亲就是第一个共同祖先。

复杂度：预处理 O(n log n)，查询 O(log n)，空间 O(n log n)。

反例与易错：根祖先为 0，不是根自身。；不能拿任意图的 DFS 树距离当原图最短路。；换根或重新 init 后必须重新 build_lca。

练习与来源：cf/3/1096/H_!!dp_tree.cpp 的树存储上练 LCA，不声称原题需要 LCA。

代码入口：PG06

调用提醒：init(4);

```
g[1]={2};
g[2]={1,3,4};
g[3]={2};
g[4]={2};
bool ok=build_lca(1);
int a=lca(3,4);
```

原代码：template/graph/lca.hpp；template/tree_storage.cpp

【CG07】四邻接网格 BFS

识别：矩形地图、障碍、最少上下左右移动。

先看什么：格子坐标是顶点，四邻关系可即时计算。

从暴力出发：枚举全部走法。

重复与性质：循环路线与重复状态造成指数重复。

推出模型：每格第一次到达时入队；先判边界、障碍，再查访问状态。

适用条件：1<=n,m<1010；g 行、列均 1-indexed，每行前补空格；#障碍，其余可走。源点出界或障碍时全-1。

一句证明：四邻扩展与单位边权图 BFS 等价；先边界检查再读取 g 避免越界。

复杂度：O(nm)时间/空间。

反例与易错：string 默认 0 下标，忘补空格会整体错位。；先判边界再访问 g[nx][ny]。；斜走、传送门、钥匙状态改变建边或状态，不能原样套。

练习与来源：2x3 含障碍网格手算 3 步路径，并补起点是障碍的反例。

代码入口：PG07

调用提醒：n=2;
m=3;
g[1]=" ...";
g[2]=" .#.";
bfs(1,1);

原代码：template/bfs.cpp

【CG08】多源 BFS

识别：每个位置距最近出口/设施/初始火源。

先看什么：目标是 min_s dist(s,v)，不是各源距离之和。

从暴力出发：每个源跑一次 BFS，O(k(n+m))。

重复与性质：不同源搜索重复扫描全图。

推出模型：全部源同时放第 0 层；谁先到某点就实现最近源的最小距离。

适用条件：依赖 PG03；sources 每个点在 1..n，可重复或为空；所有源距离 0，共用 dist/fa/path。

一句证明：第 0 层就是全部源；分层不变式给出 min_s dist(s,v)，不需要每个源单独 BFS。

复杂度：O(n+m+源数)时间，O(n+m)空间。

反例与易错：每个源依次调用单源 bfs 会覆盖之前结果。；源点时间不同不能都设 0，需要最短路或分层事件。；空源集合应全部不可达。

练习与来源：5 点链两端同时为源；再把源列表重复插入一个端点。

代码入口：PG08

调用提醒：init(5);
for(int i=1;i<5;++i){g[i].push_back(i+1);g[i+1].push_back(i);}
bfs(vector<int>{1,5,1});

原代码：template/bfs.cpp

【CG09】0-1 BFS

识别：只有免费/代价 1 两种边。

先看什么：普通 BFS 的首次发现最优性质已不成立，但距离层仍只差 0/1。

从暴力出发：用 Dijkstra 正确但有堆 log。

重复与性质：堆只维护相邻两层，排序能力过强。

推出模型：0 边新候选插队头，1 边插队尾，按改善而非 visited 锁点。

适用条件：1<=start<=n<200010；每条边 w 必须是 0 或 1，函数不校验。不可达 dist=1e9。

一句证明：deque 保留当前最小层及下一层；有效弹出按距离非降序，等价于只含 0/1 边的 Dijkstra。

复杂度：O(n+m)时间和空间。

反例与易错：1->2 权 1、1->3 权 0、3->2 权 0 时，2 初次发现不是最短。；有 2 权时不能继续保证队列序。；必须跳过过期距离快照。

练习与来源：三边反例：1->2(1),1->3(0),3->2(0)，答案 0。

代码入口：PG09

调用提醒：init(3);
g[1]={{2,1},{3,0}};
g[3]={{2,0}};
bfs01(1);

原代码：template/graph/dijkstra.hpp

【CG10】Floyd 全源最短路

识别：n 约几百、全源多询问，可能有负边。

先看什么：状态可按允许经过的中间点集合递推。

从暴力出发：每对端点枚举路径。

重复与性质：同一中间路径被端点组合反复使用。

推出模型：dist^(k)[i][j]=min(旧值,dist^(k-1)[i][k]+dist^(k-1)[k][j])。

适用条件：0<=n<510，|边权|<=1e12；先 init，再 add_edge 取重边最小值，无向边加两次。floyd 有任何负环返回 false，此时全部结果不使用；不可达 INF。

一句证明：最短路按是否经过 k 分成两类；无负环可选简单最短路。k 必须是最外层才能维护中间点集合不变式。

复杂度：O(n^3)时间、O(n^2)空间。

反例与易错：不可达 INF 不参与加法。；负环可能使递推数值迅速变负；用 i128 并截到-INF 后返回 false。；不能把同一 k 放最内层当标准 Floyd。

练习与来源：手算 1->2(4),2->3(-2),1->3(9)，再添加负环观察返回 false。

代码入口：PG10

调用提醒：init(3);
add_edge(1,2,4);
add_edge(2,3,-2);
add_edge(1,3,9);
bool ok=floyd();

原代码：template/graph/dijkstra.hpp

【CG11】二分图染色与冲突检测

识别：每对约束必须分到相反两组。

先看什么：沿边翻色；一条奇环回到起点会要求颜色相反。

从暴力出发：枚举 2^n 种分组。

重复与性质：一个组件一旦选根颜色，其余点颜色被路径奇偶约束。

推出模型：任取根色 1，BFS 传播；同色边是不可行证据。

适用条件：0<=n<200010，无向边双向添加；col=1/2。成功时所有点着色；失败时剩余 col 可能为 0。支持断图、重边；自环立即失败。

一句证明：边要求两端相异，沿路径颜色由长度奇偶唯一决定；出现同色边恰好构成奇环矛盾。

复杂度：O(n+m)时间/空间。

反例与易错：只给未着色点染色但不检查已着色边，会把三角形误判可二分。；原 6.16 题可能自带可行结构，其专用代码不能直接充当通用判定。；断图必须枚举所有起点。

练习与来源：cf/date/6/6.16/2_bi_paint_.cpp：保留题内构边，另加入三角形看通用冲突检测。

代码入口：PG11

调用提醒：init(3);
g[1]={2};
g[2]={1,3};
g[3]={2};
bool ok=bipartite();

原代码：template/graph_paint.cpp；cf/date/6/6.16/2_bi_paint_.cpp

【CG12】有向/无向环恢复

识别：要求输出环上的点或边；可能有重边。

先看什么：无向父边需要按 id 区分，有向需要区分栈内与已完成。

从暴力出发：枚举简单路径直到回到旧点。

重复与性质：搜索路径前缀高度重叠。

推出模型：DFS 只保留当前祖先链，遇回边用 fa/pe 一次回溯恢复。

适用条件：0<=n<200010，输入 m 条边的唯一 id 为 0..m-1；无向双向记录共用同一个 id。find_cycle(directed)返回是否找到；cycle 首尾相同，cycle_edges 对应每一步，边号不重复。

一句证明：回边连接当前点与栈内祖先，树父链加回边形成环；无向只跳过父边 id，第二条平行边仍能闭合长度 2 的环。

复杂度：O(n+m)时间/空间。

反例与易错：无向图跳过所有 v==fa[u]会漏两条平行边构成的环。；有向图不能把指向已结束点的边判成环。；简单基环图专用剥叶算法仍见 G05，不必强行替换。

练习与来源：无向平行边长度 2、自环长度 1；有向菱形不是环。

代码入口：PG12

调用提醒：init(2);
g[1]={{2,0},{2,1}};
g[2]={{1,0},{1,1}};
bool ok=find_cycle(false);

原代码：template/graph/dfs_forest.hpp；template/advanced/bridges.hpp

【CG13】无权树直径与路径

识别：无权树上最长简单路径。

先看什么：树无环，因此任意两点路径唯一。

从暴力出发：从每点 BFS 取最大距离，O(n^2)。

重复与性质：直径端点能由一次最远搜索锁定，没必要枚举所有起点。

推出模型：任取 s 找最远 a，再从 a 找最远 b 并回溯路径。

适用条件：依赖 PG01。只接受无权无向树；tree_diameter 非树返回-1。单点长度 0、路径[1]；成功后 diameter_a/b 和 diameter_path 有效。只读 g/n，不修改 PG01 的父亲/深度。

一句证明：以初始点 s 为根，最远 a 的深度不小于任意直径端点 x/y。取 t=LCA(x,y)：若 a 在 t 子树内，选与 a 不同分支的端点；否则 a 到端点的分叉不深于 t。所得 a 到该端点距离至少为原直径，故 a 是某条直径端点；第二遍最大距离即直径。

复杂度：O(n)时间和空间。

反例与易错：本模块是无权版，不把 BFS 步数当带权距离。；非负权树可把遍历距离改为权和后用两遍法，但需另核验溢出；负边树不能直接套。；多个最远点可能并列，返回任意一条直径，不保证字典序。

练习与来源：cf/4/1090/F_tree_constrcution.cpp 的链+星形构造是直径练习输入，不猜原题完整要求。

代码入口：PG13

调用提醒：init(4);
g[1]={2,3,4};
g[2]={1};
g[3]={1};
g[4]={1};
int len=tree_diameter();

原代码：template/tree_storage.cpp；template/bfs.cpp

【CG15】DAG 路径计数与最长路

识别：所有依赖形成 DAG，统计路径或最长转移链。

先看什么：每条非空路径有唯一最后一条边。

从暴力出发：枚举所有路径，数量可能指数级。

重复与性质：相同终点的前缀只需计数/最长长度，不需保留每条路径。

推出模型：拓扑顺序累加 ways、最大化 dist；mod=1 时用 dist 判可达。

适用条件：1<=s<=n<200010，1<=mod<=LLONG_MAX；不可达 dist=-1/ways=0，起点含空路径。重边按不同边选择计数。有环返回 false，结果作废。

一句证明：每条到 v 的非空路径有唯一最后一条边 u->v；拓扑序保证所有前驱已完成，路径计数可加且最长路可取 max。

复杂度：O(n+m)时间/空间。

反例与易错：有向有环时最长简单路不能照搬此 DP。；mod=1 时 ways[s]=0 但 s 仍可达，不能用 ways==0 判断可达。；两项模数接近 LLONG_MAX 时加法先转 i128。

练习与来源：cf/1704E.cpp 只提取 tp 与 dp 累加局部；本卡通用模型是固定源的 DAG 路径。

代码入口：PG15

调用提醒：init(4);
g[1]={2,3};
g[2]={4};
g[3]={4};
bool ok=dag_dp(1,1000000007);

原代码：cf/1704E.cpp；template/graph/topological_sort.hpp

【CG16】拓扑排序：先决条件到有序执行

识别：有向依赖，要求所有前置完成后执行；判断是否有环。

先看什么：入度等于尚未处理的前驱条数。

从暴力出发：枚举 n!个排列再验证每条边。

重复与性质：排列中大量顺序明显违反前驱约束。

推出模型：只选择入度 0 的点；删除它的出边，继续剥离，输出数<n 即有环。

适用条件：有向图，n>=1；重边每条各计入入度。不是所有图都存在拓扑序。

一句证明：DAG 至少有一个入度 0 点；无入度 0 仍有点时，反向追溯前驱必进入环。

复杂度：O(n+m)。

反例与易错：入度在过程中会被修改，多测重新计算。；多个合法拓扑序无需固定字典序；需要时用最小堆。

练习与来源：cf/1704E.cpp：识别 in/e/tp 的 Kahn 段；其后逆序 DP 对应从汇点回推。

代码入口：G04、PG15

原代码：cf/1704E.cpp；template/graph/topological_sort.hpp

【CG17】最小生成树：全连通的最小总边权

识别：把所有点连接起来，成本为选边权之和。

先看什么：连通无环需要 n-1 条边；最短路树通常不最小化总边权。

从暴力出发：枚举所有边子集判连通，2^m。

重复与性质：每次切分两组，只需最便宜跨割边作为安全候选。

推出模型：Kruskal 按边排序并查集合并；Prim 从当前树取最便宜跨界边。可靠 G07/G08 直接引用。

适用条件：无向图；允许负边、重边、自环。断图返回最小生成森林，必须检查 connected/components。

一句证明：割性质：最便宜跨割边可以替换某最优树中的另一条跨割边而不增加权值。

复杂度：Kruskal O(m log m)，堆 Prim O((n+m)log n)。

反例与易错：最小生成树不是每对点最短路。；自环不会改善连通性；并查集拒绝块内边。；总权值范围由题目保证，负权不能用“初始 0 最小”误处理。

练习与来源：三点边(1,2,2),(2,3,2),(1,3,3)：MST 总和 4；根 1 最短路树边权和 5。

代码入口：G07、G08、PG02

原代码：template/kruskal MST.cpp；template/prim MST.cpp；template/graph/kruskal.hpp；template/graph/prim.hpp

【CG18】强连通缩点：把互相可达的一团压成点

识别：有向图存在环，还要做依赖关系或 DAG 分析。

先看什么：单向可达不是同一 SCC；必须双向可达。

从暴力出发：对每对点各跑可达性，O(n(n+m))。

重复与性质：许多点的双向可达关系可一次分类。

推出模型：直接用已测 G11 迭代 Kosaraju；按 comp 把跨组件边连成缩点 DAG，再按目标做 DP。

适用条件：有向图，端点合法；自环与重边可存在。缩点边需要计数时按题意去重或保留。

一句证明：若不同 SCC 组成有向环，则这些组件之间互相可达，应合并，矛盾；因此缩点无环。

复杂度：O(n+m)。

反例与易错：不能用无向 DSU 替代 SCC。；缩点后点权如何聚合必须从原目标推导。

练习与来源：1->2->1、2->3：{1,2}同 SCC，3 另一个；缩点只有一条方向边。

代码入口：G11、G04、PG15

原代码：template/advanced/scc.hpp

【CG19】桥：删除一条边会破坏多少连通性

识别：要求关键道路、删边后连通变化。

先看什么：无向重边的两个端点间仍有替代边，所以单条不是桥。

从暴力出发：逐条删边重跑连通性，O(m(n+m))。

重复与性质：每条树边下方能否绕回祖先，可用一个 low 概括。

推出模型：直接引用已测 G12；DFS 树边(u,v)满足 low[v]>tin[u]才是桥。

适用条件：无向图，输入每条边有唯一 id；自环不是桥，平行边不得按端点一起跳过。

一句证明：low[v]记录子树通过至多一条回边能触及的最早祖先；无法回到 u 或更高祖先时，唯一跨割边就是 u-v。

复杂度：O(n+m)。

反例与易错：桥判断是严格>，不是>=。；父边按 id 排除，不能排除所有父节点邻边。

练习与来源：两条平行 1-2 边都不是桥；再接 2-3，只有 2-3 是桥。

代码入口：G12、PG12

原代码：template/advanced/bridges.hpp

【CG20】Euler 子树区间与树 DP 接口

识别：子树查询/修改，或者孩子完成后汇总父亲。

先看什么：DFS 进入与退出形成连续子树区间；只有父先子的普通 order 不自动拥有这个性质。

从暴力出发：每次子树查询都遍历后代，O(nq)。

重复与性质：同一子树多次查询重复扫描。

推出模型：区间问题直接用 G10 的 tin/tout，再接树状数组/线段树；树选点等状态问题用 PDP05 的 dp[u][0/1]。

适用条件：无向树，固定根。G10 是闭区间[tin[u],tout[u]]，点权/边权映射先分清。PDP05 允许空集。

一句证明：DFS 在退出 u 前不会离开 u 子树，因此访问时间连续；树 DP 在每个子树只通过父子边与外界交互。

复杂度：根化 O(n)；区间结构依所接数据结构；独立集树 DP O(n)。

反例与易错：PG01 的 order 不作为 Euler 闭区间使用。；不能在孩子 dp 未完成时累计父亲。；全负点权允许空集时答案 0。

练习与来源：cf/3/1096/H_!!dp_tree.cpp：只读“孩子完成再累加”和父边视角；独立集是另一个明确局部模型。

代码入口：PG01、G10、PDP05

原代码：cf/3/1096/H_!!dp_tree.cpp；template/graph/euler_subtree.hpp；template/tree_dp.cpp

【CM01】扩展欧几里得与一般模逆元

识别：ax≡1(mod m)，m 不保证是质数，或需解线性丢番图方程。

先看什么：ax+my=1 把逆元改写成整数等式。

从暴力出发：枚举 x=0..m-1。

重复与性质：欧几里得余数结构可一次缩小数值规模。

推出模型：欧几里得求 gcd 时回代系数；gcd=1 才归一化 x 为逆元。

适用条件：|a|,|b|<=1e18；exgcd 返回非负 gcd 和 ll 系数，(0,0)取 x=1,y=0。inverse_mod 要求 2<=mod<=1e18，逆元归一到[0,mod)，不存在返回-1。

一句证明：把 a%b=a-(a/b)b 代回即可得到 Bezout 等式；ax≡1(mod m)有解当且仅当 gcd(a,m)=1。

复杂度：O(log max(|a|,|b|))时间、递归栈同阶。

反例与易错：费马 a^(p-2)必须模数为质数且 a 非零；复合模数用本函数。；不支持 LLONG_MIN 输入，契约上限 1e18。；整数解存在不代表存在满足非负/区间约束的解，还需解参数范围。

练习与来源：对模 8 枚举 3 和 2 的逆元，观察互质条件；再检查 30x+18y=6。

代码入口：PM01

调用提醒：ll x,y;
ll d=exgcd(30,18,x,y);
ll inv=inverse_mod(3,8);

原代码：template/gcd.cpp；template/ksm.cpp

【CM02】线性筛、判素、分解与约数

识别：大量小数判素，或少量整数要分解/列约数。

先看什么：每个合数有唯一最小质因数；约数由质因数指数独立选择。

从暴力出发：每个 n 枚举 2..n-1，列约数扫 1..n。

重复与性质：对每数重复试除，未共享最小因数信息。

推出模型：批量用线性筛；单个大数只试到 sqrt，除尽同因数后继续；约数按指数乘积扩展。

适用条件：sieve:1<=n<1000010，primes[1..cnt]和 lp[1..n]；lp[1]=0。is_prime 负数/0/1 为 false。factorize/divisors 要求 1<=n<=1e12；筛与试除函数互不依赖。

一句证明：筛循环遇整除就 break，避免重复最小质因数分解；试除结束时若剩数合成则必有不超过其平方根的因子，与循环停止矛盾。

复杂度：筛 O(n)时空；试除 O(sqrt n)时间；约数生成 O(tau(n)log tau(n))额外排序。

反例与易错：1 不是质数；factorize(1)为空，divisors(1)=[1]。；试除单个 1e12 约 1e6 步，大量查询应改筛预处理或更强算法。；多测 sieve 必须重置 cnt 与当前 1..n 的 lp。

练习与来源：60 分解为 2^2*3*5 并枚举 12 个约数；1 单独验证。

代码入口：PM02

调用提醒：sieve(1000);
bool p=is_prime(97);
auto f=factorize(60);
auto d=divisors(60);

原代码：template/Mobius_inversion.cpp

【CM03】负数向下/向上整除

识别：公式需要数学上取整/下取整，a 可能为负。

先看什么：C++的-7/3=-2 与 floor=-3 不同。

从暴力出发：用浮点再 ceil/floor。

重复与性质：浮点会丢失完整 ll 的单位精度，且不是正确整数推导。

推出模型：写 a=bq+r，看 r/b 正负后给 q 加或减 1。

适用条件：a,b 为 ll，b!=0；排除 a=LLONG_MIN,b=-1（数学商 2^63 超出 ll）。floor_div/ceil_div 返回数学下取整/上取整。

一句证明：a=bq+r，|r|<|b|；r/b 的符号确定真实商在 q 哪一侧，最多需要改 1。

复杂度：O(1)时间/空间。

反例与易错：(a+b-1)/b 只在特定非负范围成立且 a+b-1 可能溢出。；负数 a%b 可能为负。；不能用浮点 ceil 处理完整 ll 精确边界。

练习与来源：枚举(-7,3),(7,-3),(-7,-3)与恰整除(-6,3)。

代码入口：PM03

调用提醒：ll l=floor_div(-7,3);
ll r=ceil_div(-7,3);

原代码：cf/ICPC/JAKARTA_2018/H.cpp

【CM04】基础求和与成对差贡献

识别：求 1..n/平方/等差和，或所有对差值之和。

先看什么：按元素贡献重排求和可以消掉一层循环。

从暴力出发：无序对双循环 O(n^2)。

重复与性质：每个 a[i]在很多 pair 里重复参与，其正负次数可直接数。

推出模型：排序后贡献系数 i-(n-1-i)=2i-n+1；中间乘法先转 i128。

适用条件：全部返回__int128。sum1:0<=n<=1e18；sum2:0<=n<=1e12；arithmetic_sum 的 first/step 为 ll、0<=count<=1e9；pair_abs_sum 长度<=2e5、|值|<=1e18。

一句证明：每对 i<j 贡献 a[j]-a[i]，按元素收集可得线性系数；平方和可用(n+1)^3-n^3 差分求和验证。

复杂度：前三项 O(1)；成对差排序 O(n log n)、空间 O(n)拷贝。

反例与易错：必须先转 i128 再乘；最后才转换不能修复已溢出中间值。；pair_abs_sum 排序了拷贝；公式使用 0 下标 i。；不同函数的 n 上界不同，不把 sum1 的上界套给 sum2。

练习与来源：cf/date/7/7.28/1.cpp：单独提取 sum2 并用{1,3,6}核对 10，不猜整个分数目标。

代码入口：PM04

调用提醒：auto s=sum1(100);
auto q=sum2(100);
auto a=arithmetic_sum(3,2,5);
auto d=pair_abs_sum({1,3,6});

原代码：cf/date/7/7.28/1.cpp

【CM05】奇偶、模不变量与环上可达

识别：操作总改变某个量固定步长或保持奇偶。

先看什么：不变量必须来自每一步变化，而非只看几个样例。

从暴力出发：模拟所有操作序列直到目标，可能无限。

重复与性质：同余类把无限整数状态压到有限环。

推出模型：证明差能否被 gcd 整除；若模型真的是模环，Bezout 还给充分性；有边界则只保留必要性。

适用条件：norm_mod/congruent 要求 1<=mod<=LLONG_MAX；a/b 可完整 ll。reachable_mod 要求|step|<=1e18、1<=mod<=1e18，start/target 可完整 ll；允许模环反复绕回，次数为非负整数。

一句证明：若 t 可达则差为 k*step+q*mod，必被 gcd 整除；反过来 Bezout 给出整数 k，把 k 加上模环周期可变成非负次数，因此充分。

复杂度：归一化 O(1)，环可达 O(log max(|step|,mod))。

反例与易错：不变量一般只给必要条件，不能自动证明可达。；有限区间内只许+x/+y 时 gcd 整除未必充分；例如步长 4/6 从 0 不能到 2。；模环中 step=0 只可达原剩余类。

练习与来源：cf/2/873/B.cpp 提取位移 gcd；用步长 4/6 且禁止负走的到 2 反例区分有限范围模型。

代码入口：PM05

调用提醒：ll r=norm_mod(-3,5);
bool ok=reachable_mod(1,7,4,10);
bool same=congruent(-3,2,5);

原代码：cf/2/873/B.cpp；cf/2/750/A.cpp；cf/3/1109/3.cpp

【CM06】整数点积、叉积与线段相交

识别：整数坐标要精确判左右、共线、相交。

先看什么：向量叉积符号无需角度/斜率；垂直线也自然处理。

从暴力出发：求两条直线浮点交点后判断范围。

重复与性质：浮点除法与竖直线分支增加误差和特例。

推出模型：用双方叉积符号判断跨立；零叉积时加包围盒检查，涵盖端点与点段。

适用条件：|每个坐标|<=1e18；Point 存 ll，先提升到__int128 再减。dot/cross 以 a 为共同起点；orientation 返回-1/0/1。segments_intersect 把端点接触、共线重叠与退化点段都算相交。

一句证明：两段各自跨过另一条所在直线时交点落在双方区间；共线时退化为一维区间重叠。差分<=2e18，两个乘积绝对值和<=8e36，小于有符号 128 位上界。

复杂度：全部 O(1)时间/空间。

反例与易错：任意完整 ll 坐标不在契约内；其差的叉积可能超过 128 位。；只比较 orientation 乘积<=0 会把共线但分离的段误判相交。；整型坐标用精确符号；浮点三角与 epsilon 规则另见对应浮点模块。

练习与来源：穷举[-1,1]^2 的 9 个点形成的 9^4 线段组合，并检查共线分离。

代码入口：PM06

调用提醒：Point a{0,0},b{2,2},c{0,2},d{2,0};
bool ok=segments_intersect(a,b,c,d);
int turn=orientation(a,b,c);

原代码：template/math/integer_bits.hpp

【CM07】gcd/lcm 与差分同余

识别：求共同步长、整除结构或最小共同周期。

先看什么：若所有数模 k 同余，则所有差被 k 整除；反过来相邻差被 k 整除也能望远镜推出两两同余。

从暴力出发：枚举 k 再逐项检查。

重复与性质：所有 k 的可行性由一个差的 gcd 概括。

推出模型：先将约束转成差，再累积 gcd；范围 gcd 可接 ST 表。完整 ll 符号边界直接引用 M01。

适用条件：必要/充分性来自“差能被 k 整除”等价于同余。lcm 先除 gcd 再乘且仍需检查溢出。

一句证明：k 整除全部差当且仅当 k 整除它们的最大公约数。

复杂度：gcd O(log V)，n 项 O(n log V)。

反例与易错：gcd(0,0)=0 的语义要回到题意解释。；不要对 LLONG_MIN 直接 abs。

练习与来源：cf/date/6/6.1/3.cpp：询问[l,r]对应相邻差[l+1,r]；l=r 时没有差。

代码入口：M01

原代码：cf/2/873/B.cpp；cf/date/6/6.1/3.cpp；template/gcd.cpp

【CM08】模快速幂、组合数与除法条件

识别：指数很大、答案模 p，或选 k 个位置。

先看什么：乘法可快速幂；模除法实质是乘逆元。

从暴力出发：连乘 O(exp)，组合枚举 2^n。

重复与性质：指数二进制和阶乘前缀共享重复乘法。

推出模型：直接引用已测 M02/M05；复合模逆元接 PM01。先核验素数模与阶乘范围。

适用条件：M05 要求质数模且预处理最大 n<mod；M02 的负底数归一化与乘法 i128 保护已实现。

一句证明：二进制展开把 a^e 拆为 a^(2^i)乘积；C(n,k)=n!/(k!(n-k)!)只有分母可逆时能模除。

复杂度：快速幂 O(log exp)；组合预处理 O(n)，单次 O(1)。

反例与易错：mod=1 答案全 0。；模数不为质数不能盲用 p-2。；n>=p 时阶乘出现 0，普通阶乘逆元不适用。

练习与来源：对模 8 求 3 逆元可行、2 逆元不存在；对 n=p 比较普通阶乘公式失效。

代码入口：M02、M05、PM01

原代码：template/ksm.cpp；template/math/modular.hpp；template/math/combinations.hpp

【CM09】奇偶与余数分类：必要条件之后还要构造

识别：源码只统计奇偶或按 2/3/6 分组就给答案。

先看什么：奇偶压缩只保留某个特征，未必保留所有可达性信息。

从暴力出发：枚举所有原状态与操作。

重复与性质：如果能证明等价状态的后继结果相同，才可以压缩。

推出模型：先写每一步对模数的变化，再证明剩余类条件必要；若要判 YES，再给实际构造或等价性证明。

适用条件：只有“不变量”通常不足以推出可达；完整原题约束未核验时只把本地源码作为模型练习。

一句证明：不变量通过对操作次数归纳保持；充分性须另证，不能由归纳保持自动获得。

复杂度：依模型；状态特征压缩通常 O(n)。

反例与易错：从有限样例的奇偶规律不能推出普遍结论。；源代码的题目保证可能强于通用模型。

练习与来源：cf/3/1096/C.cpp 练 2/3/6 互斥分类；cf/2/1129/C.cpp 和 3/1103/4_game_theory.cpp 只读计数奇偶局部，不复述未核验题解。

代码入口：PM05、PG02

原代码：cf/3/1096/C.cpp；cf/2/1129/C.cpp；cf/3/1103/4_game_theory.cpp；cf/date/6/6.8/1.cpp

【RT_IO】先判断输入格式，再选原 read 或 cin

识别：大量已知数量的普通整数；或题面含单词、字符、整行、EOF。

先看什么：整数按题目 n/T 次数读取；文字输入先区分单词、字符和整行。

从暴力出发：常规整数直接用熟悉的原 read 逐项读入，其他格式使用 cin。

重复与性质：算法若 $O(n^2)$，换快读不能消除主要计算；输入输出混用还会引入缓冲顺序问题。

推出模型：原 read 用于数量和整数范围已知的合法输入；混合/EOF/特殊边界使用 cin。关闭同步写在 main 开头，整行前明确处理残留换行。

适用条件：普通整数按 n/T 调用原 read；其他格式 cin；关闭同步后同一输入/输出流不能混用。

一句证明：逐位累积保持已读前缀数值；整行读取以换行为边界，与空白分词语义不同。

复杂度：按输入字符数线性。

反例与易错：读取次数与题面一致；cin>>ws 会丢失有意义的前导空格。

练习与来源：template/tests/round02/test_runtime.cpp::test_cin_recipe/test_original_read_valid_integers：7、-9000000000、hello、Z 与保留前导空格。

代码入口：IO01、IO02、CO02

原代码：template/read.cpp；template/core/main.cpp；template/core/basics.hpp

【RT_ANGLE】从向量求距离、夹角与方向

识别：两点距离、两个非零向量夹角、点绕原点的方位。

先看什么：要的是夹角[0,π]还是带象限方向[0,2π)？零向量是否可能？输入单位是弧度还是角度？

从暴力出发：手写 sqrt(dx*dx+dy*dy)，再用 atan(y/x)猜象限。

重复与性质：平方和可能更易溢出；atan(y/x)丢掉象限并在 x=0 时无定义。

推出模型：距离用 hypot；方向用 atan2(y,x)后归一化；夹角用单位向量点积再 acos。

适用条件：向量非零且有限；acos 数学参数应位于[-1,1]，夹紧只修微小舍入误差。

一句证明：点积=|a||b|cosθ；atan2 结合 x/y 符号保留象限。

复杂度：O(1)。

反例与易错：把 atan2 参数反过来；对零向量强行定义角；把 EPS 加入排序比较器破坏严格弱序。

练习与来源：tests/round02/test_runtime.cpp：三四五距离、四象限、微小越界、2000 随机单位向量。

代码入口：FP01

调用提醒：distance_2d(0,0,3,4)==5；degrees(angle_between(1,0,0,1))≈90；direction_angle(0,-1)≈3π/2。

原代码：template/Ternary_Search.cpp

【RT_SEARCH】实数答案的分界，或函数的唯一谷底

识别：求最小可行实数；或一个连续函数先下降后上升求最小值。

先看什么：先证明 check 单调或 f 单谷；边界是否覆盖答案？函数求值是否稳定？

从暴力出发：均匀枚举很多小数点，精度越高枚举越多且可能漏答案。

重复与性质：单调性或单峰性允许一批区间整体排除。

推出模型：二分比较可行性留下转折点；三分比较 m1/m2 函数值留下最小区间。

适用条件：二分 false->true 且右端可行；三分连续、最小区间外严格单调、无非最优平台；不能由画几个点证明全区间性质。

一句证明：m1<m2 且 f(m1)<=f(m2)时，单谷性质说明 m2 右侧没有更优最小点。

复杂度：固定 200 次迭代乘单次 check/f 复杂度。

反例与易错：f 有两个谷时会删掉全局最优；整数答案不应用浮点二分再直接强转；无限精度迭代并不存在。

练习与来源：template/Ternary_Search.cpp 只借用 200 轮循环，题目几何性质未在本卡确认；测试对 $x^2>=v$ 与已知顶点二次函数。

代码入口：TRS01

原代码：template/Ternary_Search.cpp

【RT_DISPLAY】显示几位小数，与数值精度和取整分开考虑

识别：要求保留 k 位小数，或需要对负数取整；题目涉及距离、比例或角度。

先看什么：输出格式是小数位还是有效数字？计算需要多少精度？数学上向下/向上/就近取整分别是什么？

从暴力出发：依赖默认输出格式，或把 double 直接强转 int 当作所有取整。

重复与性质：显示格式只决定打印文本，不能修复前面计算丢失的精度；四种取整方向不同。

推出模型：fixed+setprecision(k)固定小数位；单独 setprecision(k)默认有效数字。double 通常约 15 位有效数字，long double 精度由实现决定。floor 向负无穷取整，ceil 向正无穷，trunc 向 0，round 的半数远离 0。

适用条件：printf 的%f 接 double，%Lf 接 long double；需要精确整数计数/取模时保留整数。本例保留默认同步用于展示 cout/printf；关闭同步后不混同一输出流。

一句证明：格式化在已有二进制浮点值上产生十进制文本，不改变该值；例如-1.5 在四种取整下分别得到-2/-1/-2/-1。

复杂度：每个数值操作 O(1)，打印复杂度随输出字符数。

反例与易错：显示 k 位不是十进制精确舍入的通用解法；float 精度较少；超出整数范围时强转不安全；负数不能把/或强转误当 floor。

练习与来源：tests/round02/test_runtime.cpp 实际执行格式、snprintf 与四种取整断言；本卡 main 另独立编译并核对完整输出。

代码入口：IO02、FP01

```
#include <bits/stdc++.h>
using namespace std;
int main() {
    // 本例保留默认同步；因此 cout/printf 输出顺序可一起演示。
    double x = 12.3456;
    cout << setprecision(3) << x << '\n';
    cout << fixed << setprecision(3) << x << '\n';
    printf("%.10f\n", 0.125);
    printf("%.3Lf\n", 1.0L / 8);
    long double y = -1.5L;
    cout << setprecision(0) << floor(y) << ' ' << ceil(y) << ' '
         << round(y) << ' ' << trunc(y) << '\n';
}
```

原代码：template/core/main.cpp；template/Ternary_Search.cpp

【SEG-00】一页速查：先选容器，再选维护信息

识别：看到区间修改或区间询问时，先判断是否在线、操作能否合并。

先看什么：静态还是动态？只问前缀、固定窗口、还是任意[L,R]？更新是单点还是整段？

从暴力出发：逐元素改或逐元素查；先算 n*q 是否能接受。

重复与性质：很多查询重复用到相同区间；静态、前缀可减、固定窗口各有更简单的专用结构。

推出模型：静态区间和→前缀和；只在最后要结果的区间加→差分。；单点加 + 前缀/区间和→Fenwick：前缀可相减；sum 以外的操作先检查逆运算。；静态 min/max/gcd→ST 表：可合并且重复覆盖不改变结果；静态 sum 不能用重叠两块相加。；固定长度窗口 min/max→单调队列：每个下标至多进出一次；不是任意区间结构。；动态任意区间、复杂 Node→线段树；整段更新还需能 O(1)作用于 Node 的 lazy。；点改 sum/min/max/gcd→一个字段；max+次数→mx,cnt；最大子段→sum,pref,suff,best。；区间加→sum 加 len*x、min/max 各加 x；区间赋值→has_set，不能用 0 代表无标记。；set 后 add 和 add 后 set 不同：下传先 set 再 add；kth 必须非负频率；阈值搜索明确[L,R]。

适用条件：Fenwick 差两个前缀需可消去；ST 重叠查询需幂等；lazy 需对整段摘要直接计算；每一条都是要检查的条件。

一句证明：先利用操作的额外限制减少维护量；没有这些限制时用可合并区间摘要，且以结合律保证划分无关。

复杂度：Fenwick/线段树单次 O(log n)；ST 预处理 O(n log n)、min/max 查询 O(1)；单调队列全程 O(n)。

反例与易错：前缀 min 相减无意义：前缀 min 相同也不能推出中间最小值。；ST 两块若求和会重复计数重叠部分。；普通 gcd/最大子段 Node 不能直接接上区间加 lazy。

练习与来源：对同一数组分别加入：无修改、点加、区间加、固定窗口四种约束，先写出能淘汰哪些结构。

代码入口：PSD01、PSD02、PSD10、PSD11、DS04、DS05

原代码：template/Fenwick_tree.cpp；template/segmen tree1.cpp；template/Segment tree2.cpp；cf/date/6/6.9/1_special_segment_tree.cpp

【SEG-01】单点改值、区间求和

识别：每次只改一个位置，却反复问一段总量。

先看什么：一次改值影响哪些区间；输入下标是闭区间还是[l,r)。

从暴力出发：每次查询逐个相加，O(n)。

重复与性质：同一段会被反复求和，且 sum(左+右)=sum(左)+sum(右)。

推出模型：把数组二分成固定区间，修改只重算包含该点的 O(log n)层；若只需加法可先选 Fenwick。；Node：sum：区间元素和。；pushup：sum=x.sum+y.sum。；lazy：否：只有单点赋值，不需要 pushdown。；apply：apply(node,val)把该叶改为 make_node(val)。；pushdown：无；modify 沿一条根到叶路径递归。；查询：合法非空闭区间；只进相交儿子，跨 mid 时按左、右顺序 merge。

适用条件：必要：只作单点赋值，所问答案确为元素和。充分：父段 sum 可由左右 sum 相加得到。 边界契约：1<=n<N；1 下标闭区间；modify 是赋值；a 仅是建树输入；所有中间和须在 ll 内。

一句证明：每次修改重算该叶到根路径；查询按从左到右的互不相交段合并。

复杂度：建树 O(n)，修改/查询 O(log n)，空间 O(n)。

反例与易错：modify 为赋值；加 delta 需用当前值+delta。；n=0 不能 build；查询不接受 L>R。；a 仅供建树，修改后请读 tree 或另行同步 a。

练习与来源：本地练习：cf/segment_tree_cf_edu/1.cpp 的递归骨架变体。本轮可运行模型：competition/data_structure/examples/point_sum.cpp；仅承诺此处模型，不猜测缺失的原题题面。

代码入口：PSD02

调用提醒：build(1,1,n); modify(1,1,n,pos,val); cout<<query(1,1,n,L,R).sum;

原代码：template/segmen tree1.cpp；cf/segment_tree_cf_edu/1.cpp

【SEG-02】单点改值、区间最小值

识别：在线改变点值，并问任意区间的最低值。

先看什么：有修改就不能直接沿用静态 ST 表；min 没有前缀相减法。

从暴力出发：遍历[L,R]取 min。

重复与性质：左右最小值足够确定父区间最小值，原始元素不必反复扫描。

推出模型：把 Node 从 sum 换成 mn，叶子就是当前点值，其余单点骨架完全复用。；Node：mn：区间最小值。；pushup：mn=min(x.mn,y.mn)。；lazy：否：只有单点赋值，不需要 pushdown。；apply：apply(node,val)把该叶改为 make_node(val)。；pushdown：无；modify 沿一条根到叶路径递归。；查询：合法非空闭区间；只进相交儿子，跨 mid 时按左、右顺序 merge。

适用条件：必要：只有单点改值；静态时可用 ST 替代。充分：左右 mn 的 min 决定父段 mn。 边界契约：沿用 PSD02 边界；允许负数及 LLONG_MIN；无空区间查询。

一句证明：每个元素恰属一个查询子段，min 满足结合律。

复杂度：建树 O(n)，修改/查询 O(log n)，空间 O(n)。

反例与易错：最小值不能把不相交区间返回 0。；旧 4e18 有限哨兵不足以覆盖所有 ll；本版用分支查询避免哨兵。

练习与来源：本地练习：cf/segment_tree_cf_edu/2.cpp 的点改/最小值模型。本轮可运行模型：competition/data_structure/examples/point_min.cpp；仅承诺此处模型，不猜测缺失的原题题面。

代码入口：PSD03

调用提醒：build(1,1,n); modify(1,1,n,pos,val); cout<<query(1,1,n,L,R).mn;

原代码：template/segmen tree1.cpp；cf/segment_tree_cf_edu/1.cpp；cf/segment_tree_cf_edu/2.cpp

【SEG-03】单点改值、区间最大值

识别：每次改一个点，并问一段当前的最大高度/价值。

先看什么：最大值可能全为负数；不能用 0 冒充空段。

从暴力出发：逐个枚举区间元素取 max。

重复与性质：父区间最大值只依赖两个儿子的最大值。

推出模型：Node 换成 mx；查询骨架只进入相交儿子，因此无需人为负无穷哨兵。；Node：mx：区间最大值。；pushup：mx=max(x.mx,y.mx)。；lazy：否：只有单点赋值，不需要 pushdown。；apply：apply(node,val)把该叶改为 make_node(val)。；pushdown：无；modify 沿一条根到叶路径递归。；查询：合法非空闭区间；只进相交儿子，跨 mid 时按左、右顺序 merge。

适用条件：必要：只有单点改值；负数不能当无贡献。充分：左右 mx 的 max 决定父段 mx。 边界契约：沿用 PSD02 边界；first_ge 允许 L>R 并返回-1；值可为任意 ll。

一句证明：每个区间元素位于且只位于一个儿子；两侧最大值的较大者就是整个区间最大值。

复杂度：建树 O(n)，修改/区间查询/first_ge 均 O(log n)，空间 O(n)。

反例与易错：max 不能用 0 代表空段，全负区间会错误。；first_ge 只查给定[L,R]；必须先左后右才是第一个。

练习与来源：本地练习：cf/segment_tree_cf_edu/1.cpp 的递归骨架变体。本轮可运行模型：competition/data_structure/examples/point_max.cpp；仅承诺此处模型，不猜测缺失的原题题面。

代码入口：PSD04

调用提醒：build(1,1,n); modify(1,1,n,pos,val); cout<<query(1,1,n,L,R).mx;

原代码：template/segmen tree1.cpp；cf/segment_tree_cf_edu/1.cpp

【SEG-04】单点改值、区间 gcd

识别：一段数反复求最大公约数，中间会改一个数。

先看什么：先看是否允许负数；gcd 结果统一非负，0 与其他数的 gcd 等于后者绝对值。

从暴力出发：逐个用 Euclid 累计 gcd。

重复与性质：gcd 有结合律，左右各自 gcd 足够；但区间加不能只由旧 gcd 算出新 gcd。

推出模型：叶子先取绝对值；merge 取 gcd，继续复用单点骨架。；Node：g：区间所有元素绝对值的 gcd。；pushup：g=gcd(x.g,y.g)。；lazy：否：只有单点赋值，不需要 pushdown。；apply：apply(node,val)把该叶改为 make_node(val)。；pushdown：无；modify 沿一条根到叶路径递归。；查询：合法非空闭区间；只进相交儿子，跨 mid 时按左、右顺序 merge。

适用条件：必要：点改；所有绝对值能用 ll 表示。充分：gcd 满足结合律，叶子统一非负。 边界契约：沿用 PSD02 边界；负数先取绝对值，gcd(0,0)=0；LLONG_MIN 不在契约内。

一句证明：gcd 结合律使分段方式无关；取绝对值不改变共同正因数。

复杂度：建树 O(n log V)，修改/查询 O(log n log V)，空间 O(n)。

反例与易错：std::gcd 返回非负；LLONG_MIN 的绝对值不能用 ll 表示，不支持。；区间加无法只用旧 gcd 推出新 gcd；[2,4]与[2,6]原 gcd 同为 2，加 2 后分别为 2 和 4。

练习与来源：本地练习：cf/segment_tree_cf_edu/1.cpp 的递归骨架变体。本轮可运行模型：competition/data_structure/examples/point_gcd.cpp；仅承诺此处模型，不猜测缺失的原题题面。

代码入口：PSD05

调用提醒：build(1,1,n); modify(1,1,n,pos,val); cout<<query(1,1,n,L,R).g;

原代码：template/segmen tree1.cpp；cf/segment_tree_cf_edu/1.cpp

【SEG-05】区间加、区间和

识别：每次对整段加同一个数，而且修改间还要在线查询任意子段和。

先看什么：若只在最后看数组用差分；若更新后立刻查询，需可延后下传的信息。

从暴力出发：每次区间修改逐元素加，每次查询逐元素加总。

重复与性质：整段长度已知，整体加 x 使和恰好增加 len*x，无须访问每个叶子。

推出模型：完全覆盖时改当前 sum 并记 lazy；部分覆盖前 pushdown，返回后 pushup。；Node：sum,mn,mx；本题读取 sum。；pushup：sum=x.sum+y.sum。；lazy：是；lazy[node]为本节点已处理、尚未传子的累计增量。；apply：sum+=(r-l+1)*val；mn+=val；mx+=val；lazy+=val。；pushdown：先 apply 左右儿子，再 lazy[node]=0；部分修改和部分查询都先下传。；查询：完整覆盖返回 Node；部分覆盖先 pushdown，再按左右顺序 merge。

适用条件：必要：每次覆盖段内所有元素增加同一 val。充分：len 已知，sum 可直接增加 len*val，标记可相加。 边界契约：1<=n<N；修改和查询是合法非空闭区间；a 为初始输入；和、标记、长度*x 及中间量在 ll 内。

一句证明：整段平移保持大小顺序，标记相加表示时间上的连续平移；下传后子树与父树状态一致。

复杂度：建树 O(n)，区间修改/查询 O(log n)，空间 O(n)。

反例与易错：查询部分覆盖也必须 pushdown。；sum 加 len*x，mn/mx 只加 x。；build 清 lazy 后才可多测复用；不要直接调用旧 update 跳过标记。

练习与来源：本地练习：cf/date/6/6.9/1_special_segment_tree.cpp 的区间加/最小值局部模型。本轮可运行模型：competition/data_structure/examples/range_add.cpp；仅承诺此处模型，不猜测缺失的原题题面。

代码入口：PSD10

调用提醒：build(1,1,n); modify(1,1,n,L,R,val); cout<<query(1,1,n,L,R).sum;

原代码：template/Segment tree2.cpp；cf/date/6/6.9/1_special_segment_tree.cpp

【SEG-06】区间加、区间最小值

识别：给一段整体平移，同时问最低值或某个前缀约束是否被破坏。

先看什么：整段统一加法是否保持元素相对顺序；原 6.9 练习维护 d[x]=x-count(<=x)。

从暴力出发：逐个加、逐个扫描最小值。

重复与性质：共同加 x 不改变谁最小，min 只增加 x。

推出模型：复用区间加骨架中的 mn 字段；需要最小位置时可在 Node 额外记录并处理相等取左。；Node：mn（本实现同时保留 sum,mx）。；pushup：mn=min(x.mn,y.mn)。；lazy：是；lazy[node]为本节点已处理、尚未传子的累计增量。；apply：sum+=(r-l+1)*val；mn+=val；mx+=val；lazy+=val。；pushdown：先 apply 左右儿子，再 lazy[node]=0；部分修改和部分查询都先下传。；查询：完整覆盖返回 Node；部分覆盖先 pushdown，再按左右顺序 merge。

适用条件：必要：每次整段加同一 val 而非按位置加不同量。充分：共同平移保持相对大小，mn 直接增加 val。 边界契约：1<=n<N；修改和查询是合法非空闭区间；a 为初始输入；和、标记、长度*x 及中间量在 ll 内。

一句证明：整段平移保持大小顺序，标记相加表示时间上的连续平移；下传后子树与父树状态一致。

复杂度：建树 O(n)，区间修改/查询 O(log n)，空间 O(n)。

反例与易错：查询部分覆盖也必须 pushdown。；sum 加 len*x，mn/mx 只加 x。；build 清 lazy 后才可多测复用；不要直接调用旧 update 跳过标记。

练习与来源：本地练习：cf/date/6/6.9/1_special_segment_tree.cpp 的区间加/最小值局部模型。本轮可运行模型：competition/data_structure/examples/range_add.cpp；仅承诺此处模型，不猜测缺失的原题题面。

代码入口：PSD10

调用提醒：build(1,1,n); modify(1,1,n,L,R,val); cout<<query(1,1,n,L,R).mn;

原代码：template/Segment tree2.cpp；cf/date/6/6.9/1_special_segment_tree.cpp

【SEG-07】区间加、区间最大值

识别：整段增减后反复询问最大负载/最大高度。

先看什么：整体平移不改变大小关系；单独的最大值足以快速 apply。

从暴力出发：逐个改、再扫描 max。

重复与性质：max(a_i+x)=max(a_i)+x。

推出模型：复用区间加的 mx；与 sum 不同，mx 只加 x，不能乘区间长度。；Node：mx（本实现同时保留 sum,mn）。；pushup：mx=max(x.mx,y.mx)。；lazy：是；lazy[node]为本节点已处理、尚未传子的累计增量。；apply：sum+=(r-l+1)*val；mn+=val；mx+=val；lazy+=val。；pushdown：先 apply 左右儿子，再 lazy[node]=0；部分修改和部分查询都先下传。；查询：完整覆盖返回 Node；部分覆盖先 pushdown，再按左右顺序 merge。

适用条件：必要：每次整段加同一 val。充分：max(a_i+val)=max(a_i)+val。 边界契约：1<=n<N；修改和查询是合法非空闭区间；a 为初始输入；和、标记、长度*x 及中间量在 ll 内。

一句证明：整段平移保持大小顺序，标记相加表示时间上的连续平移；下传后子树与父树状态一致。

复杂度：建树 O(n)，区间修改/查询 O(log n)，空间 O(n)。

反例与易错：查询部分覆盖也必须 pushdown。；sum 加 len*x，mn/mx 只加 x。；build 清 lazy 后才可多测复用；不要直接调用旧 update 跳过标记。

练习与来源：本地练习：cf/date/6/6.9/1_special_segment_tree.cpp 的区间加/最小值局部模型。本轮可运行模型：competition/data_structure/examples/range_add.cpp；仅承诺此处模型，不猜测缺失的原题题面。

代码入口：PSD10

调用提醒：build(1,1,n); modify(1,1,n,L,R,val); cout<<query(1,1,n,L,R).mx;

原代码：template/Segment tree2.cpp；cf/date/6/6.9/1_special_segment_tree.cpp

【SEG-08】区间赋值、区间和

识别：把一整段全部覆盖成同一个值，再查询和。

先看什么：覆盖会抹去之前所有修改；0 也是一个有效赋值。

从暴力出发：遍历区间每个元素赋新值。

重复与性质：覆盖后的和只取决于长度和值，为 len*x。

推出模型：用 has_set 表示有没有赋值，不能拿 set==0 当无标记；复用混合骨架时始终传 is_set=true。；Node：tree 为 ll 区间和；lazy 含 has_set/set/add，本变体 add 保持 0。；pushup：tree[node]=tree[left]+tree[right]。；lazy：是；has_set 独立于数值 0，set/add 分别记录赋值和后续增量。；apply：is_set=true：tree=len*val，lazy={true,val,0}；否则 tree+=len*val，lazy.add+=val。；pushdown：若 has_set 先给左右孩子赋 set，再给左右孩子加 add，最后 lazy={}。；查询：完整覆盖返回 tree[node]；部分覆盖先 pushdown 再累加相交儿子的和。

适用条件：必要：一次操作将段内全部元素覆盖成同一 val。充分：len*val 确定新和，has_set 记录赋值 0。 边界契约：1<=n<N；合法非空闭区间；true 表示赋值、false 表示加；所有中间量在 ll 内；允许赋值 0。

一句证明：覆盖后每个元素均为 val，所以整段和为 len*val；下传覆盖会正确抹除儿子的旧信息。

复杂度：建树 O(n)，修改/查询 O(log n)，空间 O(n)。

反例与易错：不能用 set!=0 判断有无赋值，set=0 也是有效操作。；set(v)后 add(d)得到 v+d；add(d)后 set(v)只剩 v。；下传先 set 再 add，新的 set 必须清掉旧 add。

练习与来源：本地练习：cf/segment_tree_cf_edu/1.cpp 的递归骨架变体。本轮可运行模型：competition/data_structure/examples/range_set.cpp；仅承诺此处模型，不猜测缺失的原题题面。

代码入口：PSD11

调用提醒：build(1,1,n); modify(1,1,n,L,R,0,true); cout<<query(1,1,n,L,R);

原代码：template/Segment tree2.cpp；cf/date/6/6.9/1_special_segment_tree.cpp

【SEG-09】区间赋值与区间加混合

识别：题目同时有整段覆盖、整段增加和区间求和。

先看什么：操作顺序是否可交换：set(5)再 add(2)=7，add(2)再 set(5)=5。

从暴力出发：逐元素按时间顺序执行每次操作。

重复与性质：一串操作可压成：可选最后一次赋值，再加其后的累计增量。

推出模型：标签 T=(has_set,set,add)表示先 set 后 add；compose(old,new)即先 old 后 new。新 set 直接覆盖旧 T；新 add 只累加到 old.add。；Node：tree 区间和；Tag{has_set,set,add}。；pushup：tree[node]=tree[left]+tree[right]。；lazy：是；has_set 独立于数值 0，set/add 分别记录赋值和后续增量。；apply：is_set=true：tree=len*val，lazy={true,val,0}；否则 tree+=len*val，lazy.add+=val。；pushdown：若 has_set 先给左右孩子赋 set，再给左右孩子加 add，最后 lazy={}。；查询：完整覆盖返回 tree[node]；部分覆盖先 pushdown 再累加相交儿子的和。

适用条件：必要：操作只有赋值与加法，按输入时间顺序执行。充分：任意操作序列能压成可选 set 后累计 add。 边界契约：1<=n<N；合法非空闭区间；true 表示赋值、false 表示加；所有中间量在 ll 内；允许赋值 0。

一句证明：新 set 覆盖全部旧操作，新 add 接在旧操作后；下传先 set 后 add，等价于原时间顺序。

复杂度：建树 O(n)，修改/查询 O(log n)，空间 O(n)。

反例与易错：不能用 set!=0 判断有无赋值，set=0 也是有效操作。；set(v)后 add(d)得到 v+d；add(d)后 set(v)只剩 v。；下传先 set 再 add，新的 set 必须清掉旧 add。

练习与来源：本地练习：cf/segment_tree_cf_edu/1.cpp 的递归骨架变体。本轮可运行模型：competition/data_structure/examples/set_add.cpp；仅承诺此处模型，不猜测缺失的原题题面。

代码入口：PSD11

调用提醒：build(1,1,n); modify(1,1,n,L,R,0,true); modify(1,1,n,L,R,5,false); cout<<query(1,1,n,L,R);

原代码：template/Segment tree2.cpp；cf/date/6/6.9/1_special_segment_tree.cpp

【SEG-10】区间最大值及次数

识别：问最大值是多少，以及恰好等于这个最大值的元素有多少。

先看什么：次数依赖最大值本身，不能独立相加所有儿子次数。

从暴力出发：扫描区间求 max，再统计相等个数。

重复与性质：较小最大值一侧不贡献；两侧最大值相等才合并次数。

推出模型：Node 保存 mx,cnt；三种比较分支构成准确的 merge。；Node：mx：最大值；cnt：该最大值出现次数。；pushup：x.mx>y.mx 返回 x；小于返回 y；相等返回{mx,x.cnt+y.cnt}。；lazy：否：只有单点赋值，不需要 pushdown。；apply：apply(node,val)把该叶改为 make_node(val)。；pushdown：无；modify 沿一条根到叶路径递归。；查询：合法非空闭区间；只进相交儿子，跨 mid 时按左、右顺序 merge。

适用条件：必要：cnt 只统计等于区间 mx 的元素。充分：比较两侧 mx 后仅保留达父最大值一侧的 cnt，相等才相加。 边界契约：沿用 PSD02 边界；单个叶 cnt=1；cnt 不超过 n；值可为任意 ll。

一句证明：所有最大值只可能来自最大值等于父最大值的子段。

复杂度：建树 O(n)，修改/查询 O(log n)，空间 O(n)。

反例与易错：子段最大值不等时不能把两边次数相加。；点改叶子必须把 cnt 重置为 1。

练习与来源：本地练习：cf/segment_tree_cf_edu/1.cpp 的递归骨架变体。本轮可运行模型：competition/data_structure/examples/max_count.cpp；仅承诺此处模型，不猜测缺失的原题题面。

代码入口：PSD06

调用提醒：build(1,1,n); modify(1,1,n,pos,val); auto ans=query(1,1,n,L,R); cout<<ans.mx<<' '<<ans.cnt;

原代码：template/segmen tree1.cpp；cf/segment_tree_cf_edu/1.cpp

【SEG-11】最大连续子段和

识别：点改后，问某区间内部收益最大的连续一段。

先看什么：答案可能跨左右分界；题意是否允许选空段决定全负时的输出。

从暴力出发：枚举子段左右端点并求和，$O(n^2)$到 $O(n^3)$。

重复与性质：跨中点的段必为左侧某后缀加右侧某前缀；只记 best 不够。

推出模型：保存 sum/pref/suff/best；按三类：全左、全右、跨中点合并。查询仍保持左右顺序。；Node：sum 总和；pref 最大非空前缀和；suff 最大非空后缀和；best 最大非空子段和。；pushup：sum=x.sum+y.sum；pref=max(x.pref,x.sum+y.pref)；suff=max(y.suff,y.sum+x.suff)；best=max(x.best,y.best,x.suff+y.pref)。；lazy：否：只有单点赋值，不需要 pushdown。；apply：apply(node,val)把该叶改为 make_node(val)。；pushdown：无；modify 沿一条根到叶路径递归。；查询：合法非空闭区间；只进相交儿子，跨 mid 时按左、右顺序 merge。

适用条件：必要：单点改值、连续且默认非空子段。充分：sum/pref/suff/best 覆盖全左、全右、跨界三类候选。 边界契约：非空子段；全负返回最大负数。sum/pref/suff/best 及相加中间量均在 ll 内；无任意区间加。

一句证明：任一最优子段恰好属于全在左、全在右、跨中点三类之一。

复杂度：建树 O(n)，修改/查询 O(log n)，空间 O(n)。

反例与易错：只存 sum 与 best 无法计算跨中点最优值。；全负结果是最大负数；允许空段时应在输出取 max(0LL,best)。；任意区间加不能只修改这四个字段。

练习与来源：本地练习：cf/segment_tree_cf_edu/1.cpp 的递归骨架变体。本轮可运行模型：competition/data_structure/examples/max_subarray.cpp；仅承诺此处模型，不猜测缺失的原题题面。

代码入口：PSD07

调用提醒：build(1,1,n); modify(1,1,n,pos,val); cout<<query(1,1,n,L,R).best;

原代码：template/segmen tree1.cpp；cf/segment_tree_cf_edu/1.cpp

【SEG-12】非负频率第 k 个

识别：维护每个值的出现次数，要找排序后的第 k 个元素/第 k 个可用位置。

先看什么：位置是自然顺序还是离散化值序；所有频率必须非负。

从暴力出发：从左往右累计频率，直到累计值达到 k。

重复与性质：整棵左子树的总频率告诉我们前多少个元素可以一次跳过。

推出模型：k<=left.sum 就向左，否则 k-=left.sum 向右；合法 k 需 1<=k<=总频率。；Node：sum：区间总频率。；pushup：sum=x.sum+y.sum。；lazy：否：只有单点赋值，不需要 pushdown。；apply：apply(node,val)把该叶改为 make_node(val)。；pushdown：无；modify 沿一条根到叶路径递归。；查询：query 返回总频率；kth 按左子树总量下降，非法 k 返回-1。

适用条件：必要：每个频率非负，k 以 1 为起点，合法时不超过总频率。充分：前缀频率单调且左右子树总量准确。 边界契约：所有频率非负，总频率在 ll 内；k 从 1 开始；k<=0 或 k>总量返回-1；modify 为频率赋值。

一句证明：非负性保证前缀和单调，左段包含的正是最前面的 left.sum 个元素。

复杂度：建树 O(n)，修改/query/kth 均 O(log n)，空间 O(n)。

反例与易错：负频率破坏前缀单调；如[2,-1,2]不能据左段总和可靠排位。；这是第 k 个元素所在下标；若下标来自离散化，还需反查原值。

练习与来源：本地练习：cf/segment_tree_cf_edu/1.cpp 的递归骨架变体。本轮可运行模型：competition/data_structure/examples/frequency.cpp；仅承诺此处模型，不猜测缺失的原题题面。

代码入口：PSD08

调用提醒：build(1,1,n); modify(1,1,n,pos,new_count); cout<<kth(1,1,n,k);

原代码：template/segmen tree1.cpp；cf/segment_tree_cf_edu/1.cpp

【SEG-13】范围内第一个达到阈值的位置

识别：从某下标开始，找第一个 a[i]>=x；或要求答案只落在[L,R]。

先看什么：需最左位置而非任意位置；寻找范围可能不是全数组。

从暴力出发：从 L 扫到 R，遇到第一个达标元素即返回。

重复与性质：某整段 max<x 时整段都能排除，max>=x 仅说明该段存在候选。

推出模型：先剪去不相交段与 mx<x 的段，再先左后右下探；找到就立即返回。；Node：mx：判断本段有没有候选。；pushup：mx=max(x.mx,y.mx)。；lazy：否：只有单点赋值，不需要 pushdown。；apply：apply(node,val)把该叶改为 make_node(val)。；pushdown：无；modify 沿一条根到叶路径递归。；查询：first_ge 剪去不相交或 mx<x 的段，先左后右，首次找到返回；无解或空搜索范围返回-1。

适用条件：必要：寻找条件是 a[i]>=x，限定[L,R]且要求最左。充分：mx<x 可排除整段，先左后右下降。 边界契约：沿用 PSD02 边界；first_ge 允许 L>R 并返回-1；值可为任意 ll。

一句证明：剪去不相交段与不达标段，按下标递增搜索，第一个叶就是最左合法位置。

复杂度：建树 O(n)，修改/区间查询/first_ge 均 O(log n)，空间 O(n)。

反例与易错：max 不能用 0 代表空段，全负区间会错误。；first_ge 只查给定[L,R]；必须先左后右才是第一个。

练习与来源：本地练习：cf/segment_tree_cf_edu/1.cpp 的递归骨架变体。本轮可运行模型：competition/data_structure/examples/first_ge.cpp；仅承诺此处模型，不猜测缺失的原题题面。

代码入口：PSD04

调用提醒：build(1,1,n); modify(1,1,n,pos,val); cout<<first_ge(1,1,n,L,R,x);

原代码：template/segmen tree1.cpp；cf/segment_tree_cf_edu/1.cpp

【SEG-14】多个可合并统计量

识别：同一个区间要同时问 sum/min/max/gcd，或合并需要补充信息。

先看什么：每个字段能否只由左右同类字段确定；merge 是否有顺序。

从暴力出发：每次为每项统计都扫描区间。

重复与性质：四个字段各自满足结合律，可共享同一棵区间树与查询划分。

推出模型：Node 存四项并逐项 merge；这个事实只证明点改可用，不证明所有字段都支持同一种 lazy。；Node：sum,mn,mx,g；如矩阵题还需保存 mat 和翻转后的 rev。；pushup：分别+、min、max、gcd；非交换字段必须保留左右顺序。；lazy：否：只有单点赋值，不需要 pushdown。；apply：apply(node,val)把该叶改为 make_node(val)。；pushdown：无；modify 沿一条根到叶路径递归。；查询：合法非空闭区间；只进相交儿子，跨 mid 时按左、右顺序 merge。

适用条件：必要：每个统计量都能仅用左右摘要合并。充分：各字段 merge 满足结合律，查询顺序正确。 边界契约：沿用 PSD02 边界；受 gcd 约束禁止 LLONG_MIN；所有和和中间量在 ll 内；不支持区间加。

一句证明：每个字段独立正确且查询划分相同，整组答案同时正确。

复杂度：建树 O(n log V)，修改/查询 O(log n log V)，空间 O(n)。

反例与易错：可合并不等于可做 lazy；gcd 不支持从 gcd 本身处理区间加。；矩阵等非交换 merge 必须保留左右顺序。

练习与来源：本地练习：cf/segment_tree_cf_edu/1.cpp 的递归骨架变体。本轮可运行模型：competition/data_structure/examples/point_stats.cpp；仅承诺此处模型，不猜测缺失的原题题面。

代码入口：PSD09

调用提醒：build(1,1,n); modify(1,1,n,pos,val); auto ans=query(1,1,n,L,R); cout<<ans.sum<<' '<<ans.mn<<' '<<ans.mx<<' '<<ans.g;

原代码：template/segmen tree1.cpp；cf/segment_tree_cf_edu/1.cpp

【SEG-15】Fenwick 调用：前缀与区间不要混淆

识别：单点加、动态前缀和或区间和。

先看什么：改的是增量还是新值？下标是否从 1 开始？

从暴力出发：每次查询累加区间元素。

重复与性质：前缀可拆为 O(log n)个 lowbit 块；闭区间和是两个前缀之差。

推出模型：沿用原 Fenwick 结构及 tr 数组。；add(x,delta)增加点值；query(x)查询[1,x]；rangeSum(l,r)=query(r)-query(l-1)。；若题目是赋新值，调用 add(pos,new-old)并同步自己存的 old。

适用条件：0<=n<2^30 且内存足够；1 下标，add 只能 1..n；query(0)=0；空 rangeSum 返回 0；所有中间量在 ll 内。

一句证明：query 访问的 lowbit 块恰好划分[1,x]，两个前缀相减得到闭区间和。

复杂度：更新/查询 O(log n)，逐点初始化 O(n log n)，空间 O(n)。

反例与易错：add(0,x)非法；add 是增量，赋新值需 new-old。；query(x)是前缀，不是原来的 query(l,r)；区间用 rangeSum。；不可用两个前缀 min 相减得到区间 min。

练习与来源：旧 template/Fenwick_tree.cpp 的主程序模型；本轮 examples/fenwick.cpp 已统一接口。

代码入口：PSD01

调用提醒：Fenwick fen(n); for(int i=1;i<=n;++i)fen.add(i,a[i]); fen.add(pos,delta); cout<<fen.query(r); cout<<fen.rangeSum(l,r);

原代码：template/Fenwick_tree.cpp

【STL_HUFFMAN】合并代价为两份之和，每次拿最小两份

识别：将所有对象合成一个，合并两份花费两份权重之和，问最小总代价。

先看什么：是否可任意选两份？权重非负？是否要求相邻合并？

从暴力出发：枚举每一步选哪两份，合并顺序数目爆炸。

重复与性质：总成本等于每个叶权重乘合并树深度；小权重应安排得更深。

推出模型：交换论证让两个最小权重处于最深的一对兄弟；先合并它们，缩成一个权重和，递归求剩余问题。

适用条件：任意两份可合并、代价严格为和、非负权重。相邻合并限制不满足此模型。

一句证明：最深兄弟的两个叶可与两个最小权重交换，成本不增；收缩这对叶后仍是同类子问题。

复杂度：小根堆 O(n log n)，空间 O(n)。

反例与易错：相邻合并如 1,100,1 不能先合并两端 1；那是区间 DP 模型。按最大两份先合并会让大权重反复付费。

练习与来源：tests/round02/test_stl.cpp::brute_merge 穷举所有合并选择，与 100 组小数据比较。

代码入口：STL02

调用提醒：huffman_cost({1,2,3,4})==19。

原代码：template/data_sturcture.cpp

【STL_TOPK】只保留最大或最小 k 项，无须完整排序

识别：数据流到来，始终问最大/最小的 k 个，或第 k 大/小。

先看什么：k 是否固定？是否还要删除历史项？重复值是否算不同项？

从暴力出发：每加入一个数重排全部历史数据。

重复与性质：前 k 名以外元素不可能影响当前答案，只需记住已保留候选中最小者。

推出模型：前 k 大用小根堆淘汰最小；前 k 小用大根堆淘汰最大。新元素入堆后若超 k 个弹出当前最差候选；最终按需要反转堆弹出序列。

适用条件：只插入、k>=0；重复值作为多个元素；需要第 k 大时先确认当前数目>=k。

一句证明：若候选有 k+1 项，最小项前面已有 k 个不小于它的值，删去它不会失去前 k 大。

复杂度：O(n log(k+1))，空间 O(k)。

反例与易错：保留前 k 大才用小根堆淘汰；保留前 k 小方向相反。k=0 应返回空。若只一次静态第 k 小，nth_element 更直接。

练习与来源：tests/round02/test_stl.cpp：前 k 大/小各 30000 次前缀与完整降/升序排序截断对照。

代码入口：STL02

调用提醒：top_k_largest({3,1,9,9,-2},3)返回 9,9,3；top_k_smallest 同输入返回-2,1,3。

原代码：template/data_sturcture.cpp

【STL_MEDIAN】中位数就是小半集合的最大值

识别：在线插入后问中位数，或求到某点绝对距离和最小的点。

先看什么：偶数长度要下中位数、上中位数还是平均？是否有删除？

从暴力出发：每次排序后取中间下标。

重复与性质：只有小半与大半的分界影响中位数，不需要两半内部完整排序。

推出模型：大根堆存小半、小根堆存大半，保证小半数量等于大半或多 1；堆顶为下中位数。有删除改用双 multiset。

适用条件：在线堆版本只插入，非空查询；绝对距离最小模型要求一维 L1 代价。

一句证明：分界两侧元素有序且小半有 ceil(n/2)项，所以小半最大正是第 ceil(n/2)小；移动点越过中位数后两边数量差使总代价不再下降。

复杂度：每次 O(log n)，查 O(1)；滑窗双 multiset O(n log k)。

反例与易错：偶数长度的中位数不一定唯一；平方距离最小在均值而非中位数。两个中间数相加求平均可能溢出。

练习与来源：tests/round02/test_stl.cpp：30000 次在线插入与排序对照，下中位数含负数/重复。

代码入口：STL02

调用提醒：median_init(); for(ll x:a) median_add(x); ll lower=median_lower();

原代码：template/data_sturcture.cpp

【STL_LAZY】堆不能删中间元素：先记失效，露出顶再清理

识别：任务优先级变化/取消，仍要反复取当前最大优先级。

先看什么：任务是否有稳定 id？旧记录会不会与新记录同优先级？总操作数多少？

从暴力出发：每次修改遍历堆找旧任务并重建。

重复与性质：只有堆顶参与当前决策，深处旧记录可以暂时不处理。

推出模型：每次更新增版本号并压新记录；取顶时，若已取消或版本不等于当前版本，就弹弃继续看。

适用条件：id 在 1..n；版本不溢出；空间允许按总更新次数增长。

一句证明：返回前排除全部顶部过期项，剩余堆顶是所有有效候选中最大者；每条过期记录最多弹出一次。

复杂度：Q 次操作总 O(Q log Q)，空间 O(Q)；单次清理可能很多项。

反例与易错：只按 value 判过期可能把重新出现的相同 value 误判；用 id+版本。查询不是最坏 O(log n)，只能按总操作摊还。

练习与来源：tests/round02/test_stl.cpp：20000 次更新/取消与 map 全扫描比较。

代码入口：STL02

调用提醒：lazy_init(2); lazy_set(1,7); lazy_set(1,3); lazy_top()->first==3; lazy_erase(1);

原代码：template/data_sturcture.cpp

【STL_NEIGHBOR】插删后找严格前驱、后继或最近值

识别：需要小于 x 的最大值、大于 x 的最小值，或动态集合最近值。

先看什么：严格还是允许相等？重复值是否要保留？两端是否可能不存在？

从暴力出发：每次扫描所有值取最大的小值和最小的大值。

重复与性质：有序集合把候选分隔成连续两段，答案就是分界的邻居。

推出模型：严格前驱取 lower_bound(x)前一个；严格后继取 upper_bound(x)。允许相等的前驱改 upper_bound 后退。最近值比较左右候选距离。

适用条件：前驱前检查 it!=begin；后继检查 it!=end；距离计算扩大类型避免端点溢出。

一句证明：lower_bound 前一项是所有<x 项中最大的，upper_bound 第一项是所有>x 项中最小的。

复杂度：O(log n)，有序迭代器后退 O(1)。

反例与易错：直接--begin 或解引用 end 是错误；std::lower_bound(set.begin,set.end)有线性步进，应用 set.lower_bound。

练习与来源：tests/round02/test_stl.cpp：相等命中和左右端不存在。

代码入口：STL02

调用提醒：strict_neighbors(set<ll>{1,4,7},4)返回 1 和 7。

原代码：template/data_sturcture.cpp

【STL_RESOURCE】给需求分配最小够用资源

识别：每份资源只用一次，容量至少需求，想尽量满足请求。

先看什么：是否只有下界限制？一个需求是否只取一个资源？有额外收益/区间上界吗？

从暴力出发：每个请求扫描所有剩余容量，选一个可用项。

重复与性质：超过需求的富余容量没有即时价值，小的够用资源应优先使用。

推出模型：multiset.lower_bound(need)找到最小够用容量并仅删除这一份，保留更大容量给未来。

适用条件：一对一资源、仅容量>=需求、每项使用一次；无额外收益/组合限制。

一句证明：若最优方案此处用更大资源，可与以后使用较小够用资源的请求交换；较大资源仍能满足后者，不减少满足数。

复杂度：每个请求 O(log n)。

反例与易错：erase(value)会删掉多份相同容量；若还要求容量<=上界，交换论证要重新检查。

练习与来源：tests/round02/test_stl.cpp：重复资源、恰好够用、无人可满足。

代码入口：STL02

调用提醒：slots={2,2,5}; assign_at_least(slots,2)取 2；再要 3 取 5；剩 2。

原代码：template/data_sturcture.cpp

【STL_WINDOW】滑窗要删一份旧值，不能删掉全部重复值

识别：长度 k 的每个连续窗口求极差或中位数。

先看什么：窗口固定长度；相同数字是否多次出现；要极值还是秩统计？

从暴力出发：每个窗口排序，O(n*k log k)。

重复与性质：相邻窗口仅删一个旧值、加一个新值。

推出模型：极差用 multiset 最前/最后；下中位数用大小平衡的左右 multiset；每次只删一个迭代器。

适用条件：k>=0；k=0 或 k>n 返回空；极差需 ll 可表示。

一句证明：维护集合的重数与窗口恰好一致；左右分界和数量不变量确保输出的是目标秩。

复杂度：O(n log k)，空间 O(k)；只需最值还可改为单调队列 O(n)。

反例与易错：{2,2,5}滑走一个 2 后仍应留一个 2；忽略重复会让窗口大小与中位秩失真。

练习与来源：tests/round02/test_stl.cpp：2000 随机窗口与逐窗排序比较。

代码入口：STL02

调用提醒：window_spread(a,k); window_lower_medians(a,k);

原代码：template/data_sturcture.cpp

【STL_SELECT】先列操作，再选顺序容器、有序集合或堆

识别：题目反复加入、删除、取最值、计频或查阈值。

先看什么：是否需要随机访问/两端访问/键有序/重复值/任意删除/前驱？

从暴力出发：无论需求都用 vector 扫描，或者看见动态就上线段树。

重复与性质：每种容器为一组操作保存不同信息；只保存答案需要的信息即可。

推出模型：顺序表 vector；两端 deque；最近未匹配 stack；按到达处理 queue；只取最优 priority_queue；有序阈值 set/multiset；键到值 map；仅等值查询 unordered。

适用条件：容器复杂度与数据分布匹配；哈希平均快但最坏线性；固定小值域计数数组通常更直接。

一句证明：先确认每次操作需要的信息，所选结构必须支持全部必要操作，不因某一个 O(1)接口忽略其余昂贵操作。

复杂度：由操作序列决定，不只看单次 API。

反例与易错：堆不支持有序前驱；set 不提供 O(1)第 k 项；unordered 没有排序；vector 中间 erase 搬移 O(n)。

练习与来源：template/data_sturcture.cpp 原示例扩展为 STL01 可执行场景。

代码入口：STL01、STL02

调用提醒：STL01 所有短例已实际运行；STL02 给出对应模型函数。

原代码：template/data_sturcture.cpp

【XOR01】先写二进制：消去、逆操作和奇偶

识别：题面有反复翻转、同一个掩码、出现偶数次、恢复未知数。

先看什么：5^5=0，5^0=5，3^5=6；每位是模 2 加法；同掩码再做一次就恢复。

从暴力出发：按每一次操作逐项模拟，或逐位展开列真值表。

重复与性质：同一项出现两次完全抵消，顺序不会改变最终 XOR。

推出模型：x^y=z 可得 x=z^y；3^5=6 所以 6^5=3。；popcount(x^y)的奇偶等于两者奇偶异或；整数奇偶只看最低位。；同时给两个位置^=t 保留全局 XOR；每个位置 popcount 奇偶仅在 t 为偶 popcount 时保留。

适用条件：不变量只是必要条件，必须再证所有允许目标可达。；XOR 是无进位加法，不能当普通加法、OR 或 AND。

一句证明：逐位真值表：不同为 1、相同为 0；结合律/交换律来自模 2 加法。每个偶数重数项消成 0。

复杂度：性质判断 O(位宽)，64 位视作常数。

反例与易错：3 是奇数但 popcount 为 2；不要混淆两种奇偶。；a^b<=max(a,b)错误：3^4=7。

练习与来源：cf/1/1124/B.cpp：原先已找到 popcount 不变量；补上可达性见 XOR12。

代码入口：X03、X07

原代码：cf/1/1124/B.cpp

【XOR02】区间 XOR 的前缀抵消

识别：多次查询连续区间；操作可合并成一段的 XOR。

先看什么：p[0]=0，p[i]=p[i-1]^a[i]；[3,1,2]前缀为[0,3,2,0]。

从暴力出发：每次区间从 l 循环到 r，O(nq)。

重复与性质：每次都重新异或公共前缀；两份相同前缀可以消去。

推出模型：xor(l,r)=p[r]^p[l-1]；区间为 0 等价于两前缀相同；区间为 k 等价于互补前缀。

适用条件：1-indexed、合法非空区间；静态查询。若原数组修改，旧 p 不再有效。

一句证明：1..l-1 在表达式中各出现两次，l..r 各出现一次。

复杂度：O(n)预处理，O(1)每查询，O(n)空间。

反例与易错：是 p[l-1]不是 p[l]。；cf/1/1124/C 的完整目标还包含区间 max，前缀化不等于已经解完。

练习与来源：原 C 已写出 pr 和 p(l-1)，用[3,1,2]验证 xor(2,3)=3；真实 2268C 见 XOR15。

代码入口：X01

原代码：cf/1/1124/C.cpp

【XOR03】统计 XOR 为 k 的子数组

识别：问多少个连续区间 XOR 等于 0/k，n 达到 2e5。

先看什么：固定右端 r，需要以前的 p[l-1]=p[r]^k；频次代表不同左端数量。

从暴力出发：枚举 l,r，用前缀使 O(n³)变 O(n²)，仍要枚举所有左端。

重复与性质：许多左端前缀相同，只需记录次数，不必记录位置。

推出模型：cnt[0]=1；逐个计算 p，ans+=cnt[p^k]，再 cnt[p]++。

适用条件：统计非空子数组；重复前缀必须累加。答案可达 n(n+1)/2，使用 ll。

一句证明：每个右端恰把所有满足等式的历史前缀配对一次，不遗漏也不重计。

复杂度：unordered_map 期望 O(n)、最坏 O(n²)；空间 O(n)。需要最坏保证可用 map O(n log n)。

反例与易错：先插后查会在 k=0 多算 n 个空区间。；n 个 0 且 k=0 答案 n(n+1)/2，不是 n。

练习与来源：局部练习：a=[0,0,0],k=0 得 6；a=[3,1,2],k=0 得 1，不冒充 2268C 原题。

代码入口：X01

原代码：cf/1/1124/C.cpp

【XOR04】离线区间 XOR 更新与全局未知量

识别：每次把[l,r]都^=x，所有更新后输出数组；或知道全体 XOR 且缺一项。

先看什么：标记在区间开头出现，在区间结束之后再出现一次就被取消。

从暴力出发：逐操作扫描区间，O(nq)；缺一项则尝试每个可能值。

重复与性质：更新影响只在两端发生切换；唯一未知量可直接移项。

推出模型：d[l]^=x，d[r+1]^=x；扫描 tag^=d[i]，a[i]^=tag。；unknown=target ^ 所有已知项。例总 XOR=6，已知 3、5，则未知 0。；把相同 t 异或整个长度 len 的区间，区间总 XOR 仅在 len 为奇数时改变。

适用条件：差分适用于只要最终数组；更新和询问交错时不能据此获得 O(1)在线区间查询。

一句证明：某操作的 x 在覆盖点前缀中出现一次，区间外出现 0 或 2 次；全局等式中已知项各被异或两次。

复杂度：离线 O(n+q)；补唯一未知项 O(n)。

反例与易错：r+1 必须预留。；已知总 XOR 且未知两项时只有一条约束，不能唯一恢复两者。

练习与来源：局部练习：两次给同一[l,r]^=7 应还原；全局目标不能代替每项范围与互异检查。

代码入口：X01、X03

原代码：cf/1/1124/C.cpp；cf/1/1124/B.cpp

【XOR05】贡献：XOR / OR / AND 三种表

识别：所有数对求位运算值的普通和；或所有子数组的 XOR 聚合。

先看什么：第 k 位 one 个 1、zero 个 0：XOR 只取不同，AND 只取双 1，OR 排除双 0。

从暴力出发：O(n²)枚举数对，子数组 O(n²)枚举。

重复与性质：每位贡献只依赖数量，不依赖具体数对下标。

推出模型：无序 i<j：XOR=one*zero；AND=C(one,2)；OR=one*zero+C(one,2)，再乘 2^k。；子数组 XOR 普通和：把 n+1 个前缀套同一位的不同数对计数。；所有子数组 XOR 再 XOR：a[i]出现 i(n-i+1)次，仅两因子都奇时保留。

适用条件：位运算值非负，明确是普通和还是再次 XOR；模块 uint64、n<=1e9，用 u128 承接乘积与总和。

一句证明：每个数对的每位独立，加法求和可以交换求和顺序。每对不同前缀按较早/较晚唯一对应一个非空区间。

复杂度：O(64n)时间、O(64)额外空间；再次 XOR O(n)。

反例与易错：普通加法有进位，但贡献按权重求和仍正确；最大值目标则不能逐位独立选不同数对。；{1,2,3}的数对 XOR/OR/AND 和分别 6/9/3。

练习与来源：从 cf/4/1090/E 的 n²枚举迁移；本卡是求和扩展，不将原 max 题误写成 sum。

代码入口：X02

原代码：cf/4/1090/E_xor_find_pattern.cpp

【XOR06】构造先列限制，再留最后自由量

识别：给 n 和总 XOR 目标，要求非负/正数/互异/上界/排列。

先看什么：只满足总 XOR 时，固定前 n-1 个后，最后值被唯一决定。

从暴力出发：枚举全部数组，检验 XOR 与额外限制。

重复与性质：异或等式只留一个未知数即可解；额外限制需要预留更多自由位。

推出模型：允许 0 和重复：n>=1 可取[0,...,0,target]。；互异 n=2 且 target=0 无解；target!=0 取[0,target]。；n>=3 且目标<2^60：前 n-3 取 1..n-3，末三项用第 60/61 位标签 01、10、11。；排列 1..n 总 XOR 固定，必须等于 xor_1_to_n(n)，重新排列不能改变它。

适用条件：自由高位方案只充分保证每项<2^62、互异非负，不保证正数或较小上界。

一句证明：最后三项标签互异，与前面低位数隔离；三项标签总 XOR 为 0，低位恰好补足目标。

复杂度：O(n)时间/空间；排列总 XOR O(1)。

反例与易错：两个不同数不能 XOR 成 0；一个正整数不能 XOR 为 0。；取 a1=1、target=0 得到末项 1 会撞值。；若每项<4，目标 4 不可能；上界不能在构造完成后才检查。

练习与来源：局部练习：n=3,target=0 允许[1,2,3]；排列 1..3 只能总 XOR0，不能构造总 XOR1。

代码入口：X03

原代码：cf/1/1124/B.cpp；cf/date/7/7.9/2_and.cpp

【XOR07】最高不同位：比较、连续值域 OR、限制反例

识别：需要比较两个 XOR 结果，或操作使值在[min,max]范围内变化。

先看什么：最高不同位决定非负整数大小；其权重大于所有更低位之和。

从暴力出发：枚举值域全部可能值求 OR；任意用最大元素代替最大 XOR。

重复与性质：公共高位永远相同；跨越首个不同位后，低位可形成全 1。

推出模型：令 d=mn^mx，从低位扩出与 d 同位宽的全 1mask，答案 mx|mask；2261B 还需证明操作能达到。

适用条件：区间 OR 需完整连续整数域；2261B 的相邻递增能将最小位置升到最高不同位边界前一项，保留最大值。

一句证明：所有数在[mn,mx]内给出上界；传播最大值到最小位置旁，再将最小值升到公共前缀后的 011..1，可与最大值共同覆盖上界。

复杂度：扫描极值 O(n)，掩码 O(64)。

反例与易错：{1,2,3}中 mx^mn=2 而最优数对为 3。；{4,6}静态 OR 为 6，连续区间[4,6]的 OR 为 7；必须有可达 5 的操作依据。

练习与来源：cf/2/1129/B.cpp = CF2261B，原代码正确；题目并非 1129B。

代码入口：X03

原代码：cf/2/1129/B.cpp

【XOR08】我的删除操作为什么只剩两个原值

识别：选 x，让所有当前数^=x，再删 x，最终最大剩余值。

先看什么：如果当前数是原值^t，选原值 a[i]后公共 t 会消去。

从暴力出发：枚举删除顺序，n!级别；原作者已降到所有数对。

重复与性质：所有更早选择都被下一步抵消，最后只取决于最后两个原下标。

推出模型：最终 a[i]^a[j]；每一对都能留到最后，因此双循环即完整正确算法。

适用条件：真实 CF2218E：2<=n<=3105，sum n<=3105，ai<=1e9；原 O(n²)可用。

一句证明：代入(a[j]^t)^(a[i]^t)=a[j]^a[i]；保留任意两点到最后证明可达上界。

复杂度：原双循环 O(n²)；n 扩大时 Trie O(nW)。

反例与易错：原注释“最后一次”指最后选的原元素与幸存者，不能把当前选值和原值混用。；没有在线 AC 证据，不因本地代码存在就标 AC。

练习与来源：cf/4/1090/E_xor_find_pattern.cpp；[1,2,3]可得 3，数对极值不等于数值极值。

代码入口：X08、X04

原代码：cf/4/1090/E_xor_find_pattern.cpp

【XOR09】Trie：每一步只保留能决定最高位的候选

识别：集合里找一个 y 最大/最小化 x^y；最大数对；最大子数组。

先看什么：最大化时先争取最高位为 1，再争次高位；最小化相反。

从暴力出发：每次查所有已存元素 O(n)，整体 O(n²)。

重复与性质：同一二进制前缀的候选共享高位决策，Trie 一次排除整组。

推出模型：插入沿位建数组节点；cnt 维护重复与删除。最大数对先查后插；子数组先插 0、查询每个前缀。

适用条件：无符号位宽 1..64；查询只走 cnt>0；空树返回 false；容量包含根。

一句证明：当前位更优的差为 2^k，所有低位最多 2^k-1，所以不可能被低位反超。

复杂度：单操作 O(W)，空间最多 1+W*累计新增不同路径。

反例与易错：先删除不存在元素会破坏 cnt；实现先验证完整路径再扣。；删除只降计数不释放节点，容量不是当前集合大小。；掩码 AND 改变了允许忽略的位，不能沿普通 Trie 随便选分支。

练习与来源：从 2218E 双循环延伸；{1,2,3}数对最大 3；最大子数组需查询历史前缀而非原数组。

代码入口：X04

原代码：cf/4/1090/E_xor_find_pattern.cpp；cf/1/1124/C.cpp

【XOR10】XOR 图约束：相对值与分量自由度

识别：value[u]^value[v]=w，需要赋值或检测矛盾。

先看什么：已知一端即可恢复另一端；沿路径累计 XOR。

从暴力出发：枚举所有顶点标签，B 位时 2^(Bn)种。

重复与性质：同一连通块只需选根的标签，其余相对值被边确定。

推出模型：每分量根置 0；BFS 遇未赋值点推值，遇已赋值点校验；任何矛盾立即失败。

适用条件：静态无向约束；允许重边/自环。若 B 位域完整，每分量可独立选择 B 位平移。

一句证明：路径给出相对根值；所有环 XOR 必须为 0，反过来环一致保证路径无关；整体加同一 XOR 不改变边关系。

复杂度：O(n+m)时间/空间。

反例与易错：矛盾例 1^2=1、2^3=2、1^3=0，因为 1^2!=0。；K 不是 2^B-1 时数值界会限制自由度；还有其他顶点约束时另算。

练习与来源：自建局部模型：边(1,2,5),(2,3,7)得到[0,5,2]；再加(1,3,3)矛盾。

代码入口：X05

原代码：cf/1/1124/B.cpp

【XOR11】线性基不是 Trie：选子集还是选一个

识别：任意子集 XOR 最大；向量是否由已有数 XOR 表示。

先看什么：64 位数是 GF(2)向量；两个数相同的最高位可互相消掉。

从暴力出发：枚举 2^n 个子集。

重复与性质：许多数在已有数张成空间内，只有新增主元才扩张可达集合。

推出模型：从高位到低位消元：有主元就异或，没有就存下；查询按高位尝试增大答案。

适用条件：允许空子集；64 位无符号；静态插入，不支持普通删除。

一句证明：消元保持张成空间；每个非零主元有不同最高位，保证独立。高位贪心不会被较低主元影响。

复杂度：插入/查询 O(64)，空间 O(64)。

反例与易错：{1,2,4}最大数对 6，最大子集 7；不能拿子集答案答数对。；P2 长期内容，FWT OPTIONAL；赛前先掌握前缀、贡献与构造。

练习与来源：局部练习：插入 3、5、6，第三项相关；秩 2，可达{0,3,5,6}。

代码入口：X06

原代码：cf/4/1090/E_xor_find_pattern.cpp

【XOR12】我的 popcount 不变量如何补上充分性

识别：0..15 的数组，相邻两位置同时异或 3、6、9、12、15；追求最多 3 的倍数。

先看什么：原注释正确发现每个位置 popcount 奇偶不变，路径操作可以只改变两个端点。

从暴力出发：在 16^n 状态图枚举所有合法操作。

重复与性质：允许掩码张成 8 个偶 popcount 数；大数组只剩每个位置的奇偶类别与全局 XOR。

推出模型：上界是偶 popcount 位置数。若有奇类点留作吸收；全偶类时留两点，用两个 3 倍数表示任意偶类总 XOR。

适用条件：n>=2、值<16；零是 3 的倍数；掩码集合固定，换操作集合要重证张成空间。

一句证明：6 个目标值{0,3,6,9,12,15}都为偶类；未直接包含的 5=3^6、10=3^9，故两个目标槽足够覆盖所有偶类余量。

复杂度：初始 O(n)，更新 O(1)。

反例与易错：原“前面清 0、处理最后一个”漏了总 XOR=5/10 的情况，但原计数结果没有错。；int popcount 在<16 范围正确，无需改成复杂封装。

练习与来源：cf/1/1124/B.cpp =2268B；[5,0]不能全清 0，却能变成[6,3]保住 2 个好位置。

代码入口：X07

原代码：cf/1/1124/B.cpp

【XOR13】我的 AND 排列：块内补码 / 最高位循环

识别：1..n 的排列，禁止不动点，分别要求 i&p[i]=0 或!=0。

先看什么：原代码用全 1mask 把一块映射为补码；另一构造按最高位分块循环。

从暴力出发：枚举 n!种排列，逐项检查。

重复与性质：同一个全 1 掩码的互补关系天然双射；同最高位块天然共有 1 位。

推出模型：零 AND 当且仅当 n 偶数；从末尾补码块向前做。非零 AND 当且仅当 n>=6 且非 2 的幂；保留原 6/7 基础环，其余高位块轮换。

适用条件：两个任务独立判断；要求的是下标 i 与值 p[i]，不是相邻 p[i]与 p[i+1]。

一句证明：n 奇时奇数下标多于偶数候选；n 偶每补码块留下偶数长度。非零 AND 时 2 的幂末项只有自己能共享位；3、5 候选冲突；其他块至少两项且共享最高位。

复杂度：O(n)时间/空间。

反例与易错：互补块必须在 1..n 内；此处 r 偶、mask 奇，所以 l=mask-r>=1。；最高位块长度 1 会产生禁用不动点，恰是 2 的幂 n。

练习与来源：cf/date/7/7.9/2_and.cpp =909F，原两段构造直接收录；n=6/n=8 检验分界。

代码入口：X10

原代码：cf/date/7/7.9/2_and.cpp

【XOR14】我的 XOR-gun：小值域把大 n 压到 60

识别：已排序正数，合并相邻两项为 XOR，最小操作制造逆序。

先看什么：最终一项对应一段原数组 XOR；两个相邻结果对应两段相邻原区间。

从暴力出发：枚举左右两段，并对段内直接循环求 XOR，实际 $O(n^4)$。

重复与性质：段内查询反复扫描；最高位只有 30 种，数量足够时不用枚举。

推出模型：n>60 必有连续三项同最高位，一次合并右两项即逆序；否则前缀 XOR 枚举 l,m,r，成本 r-l-1。

适用条件：原题 1<=ai<=1e9 且非降序；没有 0；原 60 封顶暴力本来就可用。

一句证明：三同最高位右两项异或后低于该位，左项仍有该位；任何首次逆序两块都可由枚举覆盖，合并共耗两段长度减 2。

复杂度：O(n)读入，n<=60 时 $O(n^3)$。

反例与易错：61 个 0 不满足正数条件且不能制造逆序。；最高位分类无需把 XOR 大小误当数值大小；这里只用已证明的该位消失。

练习与来源：cf/date/5.15/3.cpp =1457D；原 ca 换前缀为最小改动，原逻辑与 BFS 同数据核验。

代码入口：X09

原代码：cf/date/5.15/3.cpp

【XOR15】我的空白 C：前缀化后还差什么

识别：真实 2268C：最大化长度至少 2 的 (区间 XOR & 区间 max)。

先看什么：原注释已发现结合律、前缀异或和需要维护 max；这只是每个区间的 O(1)表达式。

从暴力出发：枚举所有 l<r，直接逐项 AND 区间 max 再 XOR，$O(n^3)$；前缀化可到 $O(n^2)$。

重复与性质：最大值依赖区间，无法把最优 XOR 与最大值分开各自求。

推出模型：按位试答案 mask；最大笛卡尔树分配区间负责人 v；检查 a[v]包含 mask，且两侧投影前缀 XOR=mask。保留大侧频次，遍历小侧。

适用条件：ai<2^18，n<=2e5；必须排除单元素；P2 进阶，不冒充赛前简单最大子数组 XOR。

一句证明：每个区间有唯一的笛卡尔树最高负责人，端点范围分属两侧；候选位全部出现是必要充分条件；高位优先给出数值最优。

复杂度：O(18n log n+18*2^18)时间、O(n+2^18)空间。

反例与易错：对 0 和候选{4,3}，普通 Trie 选 4；再&3 得 0，实际应选 3 得 3。；(v-1,v)是禁用单元素对，重复前缀时仅扣这一个下标对。；原 solve 为空，没有可以“保留原样直接运行”的完整解。

练习与来源：cf/1/1124/C.cpp；官方样例[1,7,3,7,2]应输出 6；原 solve 空输出的红测已记录。

代码入口：X11

原代码：cf/1/1124/C.cpp

【XOR16】Nim / SG：先核对规则再异或

识别：多个独立子游戏，一步只改变一个；无法行动者输；双方同样合法动作。

先看什么：单堆 Nim 大小 x 的后继 SG 是 0..x-1，所以 SG=x；总局面用异或。

从暴力出发：递归枚举所有合法后继：有一步去必败就是必胜，否则必败。

重复与性质：把每个有限无环子游戏压成一个 SG 值，独立组合只需合并这些值。

推出模型：SG(u)=mex{SG(v)}。Nim 总 S!=0 时取最高 1 位所在堆 x，将其降到 x^S<x；S=0 时任何一堆变化都令总 XOR 非 0。

适用条件：有限无环、公平、正常规则、子游戏独立且一步只动一个。反常 Nim、共享资源、多堆一起动、循环游戏需单独分析。

一句证明：SG 总 XOR 为 0 时，单分量 SG 必须改变所以不再为 0；非 0 时选最高位分量，把目标 SG 降到 g^S<g，mex 保证有该后继。

复杂度：Nim O(堆数)；DAG SG O(状态数+边数)，mex 实现需去重/标记。

反例与易错：[1,2,3]的 Nim-sum 为 0；[1,2,4]为 7，可把 4 降成 3。；误将只可取 1 或 2 颗的堆 SG 当成堆大小：应为 x%3。；最后一步输不等于标准 Nim；非标准规则不能无证据硬套。

练习与来源：独立小例子从 0、1、2 堆状态推 mex；具体代码由 GAME01 提供，XOR 专题不维护第二份博弈实现。

代码入口：GAME01、MEX01

原代码：template/mex.cpp

【CO01 / IO02】主程序骨架 Main

识别：每场比赛新建文件；单测、多测都从这份骨架开始。

约定：GNU++20；默认单测，只有题面给出测试组数时才取消 cin >> T 注释。

调用：把题目逻辑写进 solve()；需要多测时启用 cin >> T。

例子：在 solve 内先读 n，再声明 vector<ll> a(n+1)并读入 a[1..n]。仓库实例：cf/date/7/7.25/1.cpp，main 读取 T 并逐组调用 solve。

复杂度：骨架 O(T) 次调用；具体复杂度由 solve 决定。

易错：把单测题第一个数字误读为 T。；关闭同步后不要混用 scanf 与 cin。；多测数据、答案和容器应在 solve 内初始化。

规范源码：core/main.cpp；完整独立代码如下；测试：tests/test_core_data.cpp

```
#include <bits/stdc++.h>
using namespace std;
using ll = long long;

// GNU++20；数组常用 vector<ll> a(n + 1)，有效范围 [1,n]。
// 和、乘积可能超过 ll 时，先转 __int128 再运算。
void solve() {
    // 在这里读入并处理一组数据；每组状态尽量使用局部变量。
}

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int T = 1;
    // 多测题取消下一行注释；单测题保持 T=1。
    // cin >> T;
    while (T--) solve();
    return 0;
}
```

【CO02】前缀、差分、离散化 Basics

识别：静态区间和用前缀；离线区间加用差分；值域太大先离散化。

约定：a[1..n]，{}或{0}为空；非空查询须 1<=l<=r<=n，l>r 为 0 或不修改。Difference 初值全 0；所有 ll 中间值须不溢出。rank 只查已存在值。

调用：PrefixSum p(a); p.sum(l,r); Difference d(n); d.add(l,r,x); auto b=d.values(); Compression c(a); c.rank(a[i]); cout<<to_string_i128(x);

例子：a={0,4,-2,4}：PrefixSum(a).sum(2,3)=2；Compression(a).rank(4)=2。实题：洛谷 P8218 求区间和（静态闭区间求和）：https://www.luogu.com.cn/problem/P8218 。

复杂度：前缀建表 O(n)、查询 O(1)；差分修改 O(1)、还原 O(n)；离散化 O(n log n)、rank O(log n)；空间 O(n)。

易错：前缀差是 pre[r]-pre[l-1]。；离散化保顺序，不保原距离；values[rank]还原原值。；__int128 最小值不能直接取负；输出前必须转字符串。

规范源码：core/basics.hpp；完整独立代码如下；测试：tests/test_core_data.cpp

```
#include <bits/stdc++.h>
using namespace std;
using ll = long long;
// 数组 a[1..n]；{} 或 {0} 表示 n=0。所有 ll 中间结果须不溢出。
inline string to_string_i128(__int128 x) {
    bool neg = x < 0;
    // 无符号取反能处理 -2^127，禁止直接对最小有符号数取负。
    __uint128_t u = neg ? -(__uint128_t)x : (__uint128_t)x;
    string s;
    do { s += char('0' + u % 10); u /= 10; } while (u);
    if (neg) s += '-';
    reverse(s.begin(), s.end());
    return s;
}
struct PrefixSum {
    int n;
    vector<ll> pre;
    explicit PrefixSum(const vector<ll>& a)
        : n(max(0, int(a.size()) - 1)), pre(n + 1) {
        for (int i = 1; i <= n; ++i) pre[i] = pre[i - 1] + a[i];
    }
    ll sum(int l, int r) const {
        if (l > r) return 0; // 空区间；非空时 1<=l<=r<=n。
        assert(1 <= l && r <= n);
        return pre[r] - pre[l - 1];
    }
};
// 离线区间加；初值全 0。values() 得到 1 下标数组，可重复调用。
struct Difference {
    int n;
    vector<ll> d;
    explicit Difference(int size = 0) : n(size), d(size + 1) {
        assert(size >= 0);
    }
    void add(int l, int r, ll x) {
        if (l > r) return;
        assert(1 <= l && r <= n);
        d[l] += x;
        if (r < n) d[r + 1] -= x;
    }
    vector<ll> values() const {
        vector<ll> a(n + 1);
        for (int i = 1; i <= n; ++i) a[i] = a[i - 1] + d[i];
        return a;
    }
};
// 离散化保序不保距离。rank(x) 只接收初始化时存在的值。
struct Compression {
    vector<ll> values; // values[rank] 为原值；values[0] 不使用。
    explicit Compression(const vector<ll>& a) {
        values.assign(a.begin() + min<size_t>(1, a.size()), a.end());
```

【CO02】前缀、差分、离散化 Basics 续

```
        sort(values.begin(), values.end());
        values.erase(unique(values.begin(), values.end()), values.end());
        values.insert(values.begin(), 0);
    }
    int rank(ll x) const {
        auto it = lower_bound(values.begin() + 1, values.end(), x);
        assert(it != values.end() && *it == x);
        return int(it - values.begin());
    }
};
```

【CO03】答案二分、双指针、窗口 Search

识别：答案满足单调判断时二分；有序数组找二元和用双指针；非负数组求最长受限和区间用窗口。

约定：二分范围为整数闭区间；first_true 要求假到真，last_true 要求真到假，无解/空区间为 nullopt。数组 1 下标；two_sum 需非降序，窗口元素须非负。

调用：auto ans=first_true(lo,hi,check); if(ans) cout<<*ans; auto [i,j]=two_sum_sorted(a,target); int len=longest_sum_at_most(a,k);

例子：first_true(1,10,[](ll x){return x>=7;})得到 7；窗口{0,2,1,3}、limit=3 的最长长度为 2。仓库实例：cf/other/Hello_2018/D.cpp，按时间排序后 check(x)判断能否选 x 题，二分最大可行 x。

复杂度：二分 O(log 范围)次 check、O(1)空间；双指针和窗口 O(n)时间、O(1)额外空间。

易错：必须先证明 check 单调；check 自己的乘加仍可能溢出。；two_sum 返回不同下标 i<j；无解为{0,0}。；窗口含负数时不能套用；limit<0 返回 0。

规范源码：core/search_window.hpp；完整独立代码如下；测试：tests/test_core_data.cpp

```
#include <bits/stdc++.h>
using namespace std;
using ll = long long;
// 整数闭区间 [lo,hi]；空区间或无解返回 nullopt。
// ok 确定且单调；check 内部的乘加仍需自行防溢出。
template<class F>
optional<ll> first_true(ll lo, ll hi, F ok) {
    if (lo > hi || !ok(hi)) return nullopt; // false...false,true...true
    while (lo < hi) {
        ll mid = midpoint(lo, hi); // 向 lo 取整，全 ll 范围不溢出。
        if (ok(mid)) hi = mid;
        else lo = mid + 1;
    }
    return lo;
}
template<class F>
optional<ll> last_true(ll lo, ll hi, F ok) {
    if (lo > hi || !ok(lo)) return nullopt; // true...true,false...false
    while (lo < hi) {
        ll mid = midpoint(hi, lo); // 向 hi 取整，避免死循环。
        if (ok(mid)) lo = mid;
        else hi = mid - 1;
    }
    return lo;
}
// a[1..n] 非降序，找不同位置 i<j 且 a[i]+a[j]=target；无解 {0,0}。
inline pair<int, int> two_sum_sorted(const vector<ll>& a, ll target) {
    int l = 1, r = max(0, int(a.size()) - 1);
    while (l < r) {
        __int128 s = (__int128)a[l] + a[r];
        if (s == target) return {l, r};
        if (s < target) ++l;
        else --r;
    }
    return {0, 0};
}
// a[1..n] 必须非负；返回和<=limit 的最长连续段长度，可为 0。
// 负数会破坏窗口单调性；{}、{0} 均返回 0。和用 i128 防溢出。
inline int longest_sum_at_most(const vector<ll>& a, ll limit) {
    if (limit < 0) return 0;
    int n = max(0, int(a.size()) - 1), l = 1, best = 0;
    __int128 sum = 0;
    for (int r = 1; r <= n; ++r) {
        assert(a[r] >= 0);
        sum += a[r];
        while (sum > limit) sum -= a[l++];
        best = max(best, r - l + 1);
    }
    return best;
}
```

【DP01】0/1 与完全背包 Knapsack

识别：按物品选择、总重量受容量限制，价值最大化；能区分每件一次与无限次。

约定：w、v 从 1 开始，w[i]>0，cap>=0；价值和任意可达总价值绝对值<2^60。默认总重量不超过容量；exact=true 表示恰好重量，不可达为 KNAP_NEG。

调用：vector<int> w(n+1); vector<ll> v(n+1); 读入 1..n；auto f=knapsack01(w,v,m,true); 判断 f[m]==KNAP_NEG 后输出。无限次改调 knapsack_complete。

例子：原仓库 01bag.cpp：template/01bag.cpp 。保留原正确转移，新增 64 位与恰好容量契约；不宣称在线 AC。

复杂度：时间 O(n*cap)，空间 O(cap)。

易错：0/1 容量倒序，完全背包容量正序。；恰好装满必须只初始化 dp[0]=0，其他为负无穷；不能把负无穷直接相加。；不支持零重量物品；完全背包零重正价值会使答案无界。

规范源码：dp/knapsack.hpp；完整独立代码如下；测试：tests/test_dp_math_string.cpp

```
#include <bits/stdc++.h>
using namespace std;
using ll = long long;

// w、v 为 1-indexed；w[i]>0，cap>=0，价值及可达总和绝对值<2^60。
// exact=false：总重量<=j；exact=true：总重量恰好为 j。
// 不可达为 KNAP_NEG；不能直接对不可达状态加价值。
inline constexpr ll KNAP_NEG = -(1LL << 60);
inline vector<ll> knapsack01(const vector<int>& w,
                             const vector<ll>& v, int cap,
                             bool exact = false) {
    assert(cap >= 0 && !w.empty() && w.size() == v.size());
    vector<ll> dp(cap + 1, exact ? KNAP_NEG : 0);
    dp[0] = 0;
    for (int i = 1; i < int(w.size()); ++i) {
        assert(w[i] > 0);
        // 倒序：dp[j-w[i]] 仍属于上一轮，物品最多取一次。
        for (int j = cap; j >= w[i]; --j)
            if (dp[j - w[i]] != KNAP_NEG)
                dp[j] = max(dp[j], dp[j - w[i]] + v[i]);
    }
    return dp;
}

inline vector<ll> knapsack_complete(const vector<int>& w,
                                    const vector<ll>& v, int cap,
                                    bool exact = false) {
    assert(cap >= 0 && !w.empty() && w.size() == v.size());
    vector<ll> dp(cap + 1, exact ? KNAP_NEG : 0);
    dp[0] = 0;
    for (int i = 1; i < int(w.size()); ++i) {
        assert(w[i] > 0);
        // 正序：dp[j-w[i]] 可以已经取过本物品，允许继续取。
        for (int j = w[i]; j <= cap; ++j)
            if (dp[j - w[i]] != KNAP_NEG)
                dp[j] = max(dp[j], dp[j - w[i]] + v[i]);
    }
    return dp;
}
// auto best = knapsack01(w, v, capacity, true);
// best[capacity]==KNAP_NEG 表示不能恰好装满；否则就是最大价值。
```

【DP02】线性 DP 与最长递增子序列 LIS

识别：寻找保持原顺序的最长严格递增或非递减子序列；先用朴素状态理解，再用二分优化。

约定：a 从 1 开始，空序列传 {0}；strict=true 严格递增，false 非递减。只返回长度，不恢复实际序列。

调用：vector<ll> a(n+1); 读入 a[1..n]；int ans=lis_length(a); 初学调试可用 lis_quadratic(a)对照。

例子：原仓库线性 dp.cpp：template/%E7%BA%BF%E6%80%A7dp.cpp 。保留原 O(n^2)状态，增加二分版。

复杂度：lis_quadratic：O(n^2) 时间；lis_length：O(n log n) 时间；空间均 O(n)。

易错：严格递增对应 lower_bound；重复值可接在后面时用 upper_bound。；tails 表示各长度的最小末尾，最终 tails 通常不是原数组的一个子序列。

规范源码：dp/lis.hpp；完整独立代码如下；测试：tests/test_dp_math_string.cpp

```
#include <bits/stdc++.h>
using namespace std;
using ll = long long;

// a 为 1-indexed；仅求长度。strict=true 严格递增，否则非递减。
// 最直观状态：dp[i] 为以 a[i] 结尾的最优长度，O(n^2)。
inline int lis_quadratic(const vector<ll>& a, bool strict = true) {
    assert(!a.empty());
    int n = int(a.size()) - 1, ans = 0;
    vector<int> dp(n + 1, 1);
    for (int i = 1; i <= n; ++i) {
        for (int j = 1; j < i; ++j) {
            bool can = strict ? a[j] < a[i] : a[j] <= a[i];
            if (can) dp[i] = max(dp[i], dp[j] + 1);
        }
        ans = max(ans, dp[i]);
    }
    return ans;
}

// tails[k]：长度 k+1 的序列中，最小的末尾值；它不是答案序列。
// 严格递增用 lower_bound，非递减用 upper_bound；O(n log n)。
inline int lis_length(const vector<ll>& a, bool strict = true) {
    assert(!a.empty());
    vector<ll> tails;
    for (int i = 1; i < int(a.size()); ++i) {
        auto it = strict ? lower_bound(tails.begin(), tails.end(), a[i])
                         : upper_bound(tails.begin(), tails.end(), a[i]);
        if (it == tails.end()) tails.push_back(a[i]);
        else *it = a[i];
    }
    return int(tails.size());
}
// vector<ll> a(n+1); 读入 a[1..n]; cout << lis_length(a);
```

【DP03】小规模状压最短路径 Hamilton DP

识别：n 很小，需要每个点恰好访问一次；用 mask 记录已选集合，末尾点作为第二维。

约定：w 为 1-indexed 有向图方阵，缺边为 HAMILTON_INF；从 1 到 n，每点恰好一次。0<=n<=18，n=0/1 约定为 0；允许负边，任意路径总权值在(-2^60,2^60)。

调用：vector<vector<ll>> w(n+1,vector<ll>(n+1,HAMILTON_INF)); 填边；ll ans=hamilton_path(w); ans==HAMILTON_INF 表示无解。

例子：相关变体 CSES Hamiltonian Flights：https://cses.fi/problemset/task/1690 。原题计数且 n<=20，需把 min 转移改成求和并重新评估复杂度；当前模板是最短路，未在线提交。

复杂度：时间 O(n^2*2^n)，空间 O(n*2^n)；n=18 约 36MiB。

易错：点 i 对应第 i-1 位，状态里的 u 是 0-indexed。；最短路、方案计数、是否可行的转移不同，不能原封不动混用。；终点固定为 n；任意终点需对最终 mask 的末尾状态取最小值。

规范源码：dp/hamilton_path.hpp；完整独立代码如下；测试：tests/test_dp_math_string.cpp

```
#include <bits/stdc++.h>
using namespace std;
using ll = long long;

// w 为 1-indexed 方阵；有向图，缺边用 HAMILTON_INF。
// 从 1 出发，到 n 结束，每个点恰好一次。n<=18；n=0/1 约定答案 0。
// 可以有负边；任意路径的权值总和须在 (-2^60,2^60) 内。
inline constexpr ll HAMILTON_INF = 1LL << 60;
inline ll hamilton_path(const vector<vector<ll>>& w) {
    assert(!w.empty());
    int n = int(w.size()) - 1;
    assert(n <= 18);
    for (const auto& row : w) assert(row.size() == w.size());
    if (n <= 1) return 0;
    int states = 1 << n;
    // 点 i 对应第 i-1 位；dp[mask*n+u] 中 u 是 0-indexed。
    // 状态表示已经访问 mask、当前停在 u+1 的最短路。
    vector<ll> dp(size_t(states) * n, HAMILTON_INF);
    dp[n] = 0;  // mask=1, 停在点 1。
    for (int mask = 1; mask < states; mask += 2) {
        for (int u = 0; u < n; ++u) {
            ll cur = dp[size_t(mask) * n + u];
            if (cur == HAMILTON_INF) continue;
            for (int v = 0; v < n; ++v) {
                if ((mask >> v & 1) || w[u + 1][v + 1] == HAMILTON_INF)
                    continue;
                int next = mask | (1 << v);
                ll& dst = dp[size_t(next) * n + v];
                dst = min(dst, cur + w[u + 1][v + 1]);
            }
        }
    }
    return dp[size_t(states - 1) * n + n - 1];
}
// O(n^2*2^n) 时间、O(n*2^n) 空间；n=18 的 dp 约 36MiB。
// 返回 HAMILTON_INF 表示不存在；求任意终点则改取最后一行最小值。
```

【DP04】石子合并区间 DP Interval Merge

识别：只能合并相邻部分，最后一次操作把区间分成左右两段，总代价可以由子区间转移。

约定：a 从 1 开始且非负；直线石子合并，单次代价为两堆之和；n=0/1 返回 0。要求 n*sum(a)<2^60；本函数不处理环形合并。

调用：vector<ll> a(n+1); 读入 a[1..n]；cout<<interval_merge(a); 该函数求最小代价。

例子：原仓库区间 dp.cpp：template/%E5%8C%BA%E9%97%B4dp.cpp 。两堆各 10^9 的正确答案 2*10^9，旧版返回 1061109567；保留转移并修复整数范围。

复杂度：时间 O(n^3)，空间 O(n^2)。

易错：先枚举长度，再枚举左端点和分割点。；前缀和、状态和候选代价均须 64 位；无穷大必须大于合法答案。；环形题通常先复制数组，再取所有长度 n 区间的最优值，不能直接调用直线版。

规范源码：dp/interval_merge.hpp；完整独立代码如下；测试：tests/test_dp_math_string.cpp

```
#include <bits/stdc++.h>
using namespace std;
using ll = long long;

// a 为 1-indexed 非负石子堆。每次只能合并相邻两堆，代价为两堆之和。
// 求全部合成一堆的最小总代价；n=0/1 返回 0；不是环形合并。
// 要求 n*sum(a)<2^60，使前缀和、候选代价及无穷大不混淆。
inline ll interval_merge(const vector<ll>& a) {
    assert(!a.empty());
    int n = int(a.size()) - 1;
    if (n <= 1) return 0;
    vector<ll> sum(n + 1);
    for (int i = 1; i <= n; ++i) {
        assert(a[i] >= 0);
        sum[i] = sum[i - 1] + a[i];
    }
    vector<vector<ll>> dp(n + 1, vector<ll>(n + 1));
    // dp[l][r]：合并闭区间[l,r]；先枚举长度，保证子区间已算出。
    for (int len = 2; len <= n; ++len)
        for (int l = 1, r = len; r <= n; ++l, ++r) {
            dp[l][r] = 1LL << 60;
            for (int k = l; k < r; ++k)
                dp[l][r] = min(dp[l][r], dp[l][k] + dp[k + 1][r]
                               + sum[r] - sum[l - 1]);
        }
    return dp[1][n];
}
// 时间 O(n^3)，空间 O(n^2)；n 达到千级时应重新评估是否需要优化。
// vector<ll> a(n+1); 读 a[1..n]; cout << interval_merge(a);
```

【DS04】单调栈与单调队列 Monotonic

识别：找每个位置左侧最近更小元素；固定窗口查询最小值。

约定：输入和输出均 1 下标；{}或{0}为空。previous_less 无解下标为 0；window_min 返回 ans[l]=min(a[l..l+k-1])，k<=0 或 k>n 返回{0}。

调用：auto left=previous_less(a); auto mn=window_min(a,k); // mn[l]对应窗口左端 l

例子：a={0,2,2,1,3}的左侧严格更小下标为{0,0,0,0,3}；a={0,4,2,2,5},k=2 的最小值为{0,2,2,2}。实题：洛谷 P1886 滑动窗口，求固定窗口最小/最大值：https://www.luogu.com.cn/problem/P1886 。

复杂度：均 O(n)时间，每个下标至多进出一次；额外空间 O(n)。

易错：严格更小必须弹出>=当前值，重复值不能留下。；队列保存下标，才能删除离开窗口的元素。；窗口最小值改最大值时将队尾比较>=改为<=。

规范源码：data_structure/monotonic.hpp；完整独立代码如下；测试：tests/test_core_data.cpp

```
#include <bits/stdc++.h>
using namespace std;
using ll = long long;
// 输入 a[1..n]；{}、{0} 均为空数组；返回值也保留 0 号占位。
// 每个位置左侧最近的严格更小值的下标；不存在为 0。
inline vector<int> previous_less(const vector<ll>& a) {
    int n = max(0, int(a.size()) - 1);
    vector<int> ans(n + 1), st;
    for (int i = 1; i <= n; ++i) {
        while (!st.empty() && a[st.back()] >= a[i]) st.pop_back();
        ans[i] = st.empty() ? 0 : st.back();
        st.push_back(i);
    }
    return ans;
}
// 固定长度 k 的窗口最小值；ans[l]=min(a[l..l+k-1])。
// k<=0 或 k>n 返回 {0}。相等时保留较新的下标。
inline vector<ll> window_min(const vector<ll>& a, int k) {
    int n = max(0, int(a.size()) - 1);
    vector<ll> ans(1);
    if (k <= 0 || k > n) return ans;
    deque<int> q;
    for (int i = 1; i <= n; ++i) {
        while (!q.empty() && q.front() <= i - k) q.pop_front();
        while (!q.empty() && a[q.back()] >= a[i]) q.pop_back();
        q.push_back(i);
        if (i >= k) ans.push_back(a[q.front()]);
    }
    return ans;
}
```

【DS05】稀疏表 Sparse Table

识别：数组不修改，需要大量区间 min/max/gcd 查询。

约定：a[1..n]，{}或{0}为空；非空 query 需 1<=l<=r<=n，空区间返回传入 identity。op 必须结合且幂等；GcdOp 只接收非负 ll。

调用：SparseTable<MinOp> st(a,LLONG_MAX); st.query(l,r); SparseTable<GcdOp> sg(diff,0); sg.query(l+1,r);

例子：仓库实例 cf/template/ST 表.cpp 对应 CF2050F：https://codeforces.com/contest/2050/problem/F；相邻差上建 gcd 表，原区间[l,r]转查差分[l+1,r]，单点输出 0。

复杂度：预处理 O(n log n)时间和空间；每次查询 O(1)。

易错：查询的两块区间可能重叠，不能用于求和。；ST 表只适合静态数组；修改一个元素后不能继续查旧表。；差分 gcd 题查原数组[l,r]时要转为差分[l+1,r]，单点答案为 0。

规范源码：data_structure/sparse_table.hpp；完整独立代码如下；测试：tests/test_core_data.cpp

```
#include <bits/stdc++.h>
using namespace std;
using ll = long long;
// 静态区间查询；op 必须满足结合律、幂等性，如 min/max/gcd。
// 不支持修改，也不能用两块重叠区间求和。GcdOp 要求输入非负。
struct MinOp {
    ll operator()(ll a, ll b) const { return min(a, b); }
};
struct GcdOp {
    ll operator()(ll a, ll b) const { return gcd(a, b); }
};
template<class Op>
struct SparseTable {
    int n;
    vector<int> lg;
    vector<vector<ll>> st;
    ll identity;
    Op op;
    // a[1..n]，{}、{0} 均为空；empty_value 是空区间返回值。
    SparseTable(const vector<ll>& a, ll empty_value, Op merge = Op{})
        : n(max(0, int(a.size()) - 1)), lg(n + 1),
          identity(empty_value), op(merge) {
        for (int i = 2; i <= n; ++i) lg[i] = lg[i / 2] + 1;
        st.assign(lg[n] + 1, vector<ll>(n + 1));
        for (int i = 1; i <= n; ++i) st[0][i] = a[i];
        for (int j = 1; j <= lg[n]; ++j) {
            for (int i = 1; i <= n - (1 << j) + 1; ++i) {
                st[j][i] = op(st[j - 1][i],
                              st[j - 1][i + (1 << (j - 1))]);
            }
        }
    }
    ll query(int l, int r) const {
        if (l > r) return identity;
        assert(1 <= l && r <= n);
        int k = lg[r - l + 1];
        return op(st[k][l], st[k][r - (1 << k) + 1]);
    }
};
```

【G04】拓扑排序 Topological Sort

识别：DAG 处理顺序、依赖任务、DAG DP、有向环检测。

约定：n>=1，g 长度 n+1，点 [1,n]，有向边；返回全部 n 点的一个合法顺序，有环返回空。允许重边，自环判为环。

调用：auto order=cp::topological_sort(g); if(order.empty()) { /* 有环 */ }

例子：仓库实例：cf/1704E.cpp ，使用拓扑序逆序做 DAG DP；已读源码，AC 状态未核验。

复杂度：O(n+m) 时间，O(n) 额外空间。

易错：重边也要各自计入入度和扣减。；不要把无向边双向加入后直接求拓扑。；普通队列不保证字典序最小；需要时换小根堆。

规范源码：graph/topological_sort.hpp；完整独立代码如下；测试：tests/test_graph.cpp

```
#include <bits/stdc++.h>
using namespace std;
using ll = long long;

namespace cp {
// 有向图，点 [1,n]，n>=1；返回空表示有环；重边计入入度。
inline vector<int> topological_sort(const vector<vector<int>>& g) {
    int n = (int)g.size() - 1;
    vector<int> in(n + 1), order;
    for (int u = 1; u <= n; ++u)
        for (int v : g[u]) ++in[v];
    queue<int> q;
    for (int u = 1; u <= n; ++u) if (!in[u]) q.push(u);
    while (!q.empty()) {
        int u = q.front(); q.pop(); order.push_back(u);
        for (int v : g[u]) if (--in[v] == 0) q.push(v);
    }
    if ((int)order.size() != n) return {};
    return order;
}
} // namespace cp
```

【G05】简单无向基环图环恢复 Unicyclic Graph

识别：连通简单无向图恰有一个环，找环点、按环顺序做双指针或 DP。

约定：点 [1,n]，每条无向边输入一次；检查 n>=3、边数=n、合法端点、无自环重边与连通性。无效时 ok=false 且 cycle 为空；成功时 cycle 不重复首点，末尾与首点有边。

调用：vector<pair<int,int>> edges={{1,2},{2,3},{3,1}}; auto r=cp::unicyclic_graph(3,edges); if(r.ok) for(int u:r.cycle) cout<<u<<' ';

例子：手算例：边 (1,2),(2,3),(3,1),(3,4)，返回三角环 [1,2,3] 或反向；(1,2) 重复两次的图拒绝。

复杂度：含 set 重边验证 O(n log n) 时间，O(n) 空间；剥叶与环恢复本身 O(n)。

易错：本模板明确拒绝自环、平行边形成的 1/2 长度环。；只有 m=n 不够，还必须连通。；叶子剥除后只剩环点，但仍需恢复相邻顺序。；cycle 下标为 0..长度-1，点编号仍为 1..n。

规范源码：graph/unicyclic_graph.hpp；完整独立代码如下；测试：tests/test_graph.cpp

```
#include <bits/stdc++.h>
using namespace std;
using ll = long long;

namespace cp {
struct Unicycle {
    bool ok = false;
    vector<int> cycle; // 环顺序，不重复首点；末点与首点相邻
};
// 连通简单无向基环图，点 [1,n]；每条无向边只输入一次。
// 非 n 条边、重边、自环、越界、不连通均返回 {false,{}}。
inline Unicycle unicyclic_graph(int n, const vector<pair<int, int>>& edges) {
    if (n < 3 || (int)edges.size() != n) return {};
    vector<vector<int>> g(n + 1);
    vector<int> deg(n + 1), seen(n + 1);
    set<pair<int, int>> used;
    for (auto [u, v] : edges) {
        if (u < 1 || u > n || v < 1 || v > n || u == v) return {};
        if (u > v) swap(u, v);
        if (!used.insert({u, v}).second) return {};
        g[u].push_back(v); g[v].push_back(u);
        ++deg[u]; ++deg[v];
    }
    queue<int> q; q.push(1); seen[1] = 1;
    int count = 0;
    while (!q.empty()) {
        int u = q.front(); q.pop(); ++count;
        for (int v : g[u]) if (!seen[v]) seen[v] = 1, q.push(v);
    }
    if (count != n) return {};
    for (int u = 1; u <= n; ++u) if (deg[u] == 1) q.push(u);
    while (!q.empty()) {
        int u = q.front(); q.pop(); deg[u] = 0;
        for (int v : g[u]) if (deg[v] && --deg[v] == 1) q.push(v);
    }
    int start = 0, length = 0;
    for (int u = 1; u <= n; ++u) if (deg[u]) {
        if (deg[u] != 2) return {};
        if (!start) start = u;
        ++length;
    }
    if (length < 3) return {};
    Unicycle r; int prev = 0, u = start;
    for (int i = 0; i < length; ++i) {
        r.cycle.push_back(u);
        int next = 0;
        for (int v : g[u]) if (deg[v] && v != prev) { next = v; break; }
        if (!next) return {};
        prev = u; u = next;
    }
    if (u != start) return {};
    r.ok = true;
    return r;
}
} // namespace cp
```

【G07】Kruskal 最小生成树与森林 Kruskal MST

识别：无向带权图求最小生成树；图可能不连通时求最小生成森林。

约定：n>=0，合法点 [1,n]；每条无向边输入一次，允许负权、自环、重边。返回 weight、components、connected 和原边数组 0-based 下标 edges；不连通时 weight 是森林总权；n=0 时 connected=false。总权和中间累加必须在 ll 范围内。

调用：vector<cp::KruskalEdge> e={{1,2,3},{2,3,-1}}; auto r=cp::kruskal(3,e); if(r.connected) cout<<r.weight;

例子：|旧来源：template/kruskal%20MST.cpp ，原文件求指定 k 个连通块；新模块提供通常的 MST/森林契约，不冒充原题 AC。

复杂度：O(m log m + (n+m)alpha(n)) 时间，O(n+m) 额外空间。

易错：只比较权重，不能将权重强转 int；总权使用 ll。；自环不会被选中，平行边只按最优连接选择。；输出 MST 前先判断 connected；不连通时返回的是森林。；edges 是输入边下标，不是点编号。

规范源码：graph/kruskal.hpp；完整独立代码如下；测试：tests/test_graph.cpp

```
#include <bits/stdc++.h>
using namespace std;
using ll = long long;

namespace cp {
struct KruskalEdge { int u, v; ll w; };
struct KruskalResult {
    bool connected = false;
    ll weight = 0; // 不连通时仍返回最小生成森林的总权
    int components = 0;
    vector<int> edges; // 选中边在原输入中的 0-based 下标
};
// 点 [1,n]；无向边只输入一次，支持负权、重边、自环。
// 端点须合法；总权及累加过程须在 ll 范围内。
inline KruskalResult kruskal(int n, const vector<KruskalEdge>& e) {
    vector<int> p(n + 1), sz(n + 1, 1), id(e.size());
    iota(p.begin(), p.end(), 0); iota(id.begin(), id.end(), 0);
    auto find = [&](int x) {
        while (x != p[x]) { p[x] = p[p[x]]; x = p[x]; }
        return x;
    };
    sort(id.begin(), id.end(), [&](int a, int b) {
        return e[a].w < e[b].w;
    });
    KruskalResult r; r.components = n;
    for (int i : id) {
        int a = find(e[i].u), b = find(e[i].v);
        if (a == b) continue;
        if (sz[a] < sz[b]) swap(a, b);
        p[b] = a; sz[a] += sz[b]; --r.components;
        r.weight += e[i].w; r.edges.push_back(i);
    }
    r.connected = (n > 0 && r.components == 1);
    return r;
}
} // namespace cp
```

【G08】堆优化 Prim 最小生成森林 Prim MST

识别：已有带权无向邻接表，需要最小生成树或所有分量的最小生成森林。

约定：g 长度 n+1，合法点 [1,n]，无向边双向加入，允许负权、自环、重边。返回 weight、components、connected、parent；每个分量根 parent=0；不连通时仍返回森林权重；n=0 时 connected=false。累加需在 ll 范围内。

调用：vector<vector<pair<int,ll>>> g(n+1); g[u].push_back({v,w}); g[v].push_back({u,w}); auto r=cp::prim(g);

例子：旧来源：template/prim%20MST.cpp 。新实现清除多测全局状态依赖，并将总权改为 ll。

复杂度：O((n+m)log(n+m)) 时间，O(n+m) 额外空间；使用边堆，重复候选出堆后忽略。

易错：选点依据跨越已选/未选集合的最小边，不是到起点的累计路程。；负权可以用 Prim；这一点与 Dijkstra 不同。；无向边必须双向加入；只从 1 遍历不能统计所有分量。；parent 只记录父点，若需具体重边编号可将边 ID 加入堆。

规范源码：graph/prim.hpp；完整独立代码如下；测试：tests/test_graph.cpp

```
#include <bits/stdc++.h>
using namespace std;
using ll = long long;

namespace cp {
struct PrimResult {
    bool connected = false;
    ll weight = 0; // 不连通时为最小生成森林总权
    int components = 0;
    vector<int> parent; // 每个连通块的根为 0
};
// 点 [1,n]；g[u] 存 (v,w)，无向边双向加入；负权/重边可用。
// 端点须合法；累加过程和总权须在 ll 范围内。
inline PrimResult prim(const vector<vector<pair<int, ll>>>& g) {
    int n = (int)g.size() - 1;
    PrimResult r; r.parent.resize(n + 1);
    vector<int> vis(n + 1);
    using State = tuple<ll, int, int>; // 权、点、来自哪个父亲
    priority_queue<State, vector<State>, greater<State>> q;
    for (int s = 1; s <= n; ++s) if (!vis[s]) {
        ++r.components; q.push({0, s, 0});
        while (!q.empty()) {
            auto [w, u, p] = q.top(); q.pop();
            if (vis[u]) continue;
            vis[u] = 1; r.weight += w; r.parent[u] = p;
            for (auto [v, cost] : g[u])
                if (!vis[v]) q.push({cost, v, u});
        }
    }
    r.connected = (n > 0 && r.components == 1);
    return r;
}
} // namespace cp
```

【G10】Euler 子树区间与迭代树 DP Euler Subtree

识别：将树上子树映射为数组闭区间，配合前缀和/Fenwick/线段树；逆序处理简单树形 DP。

约定：合法点 [1,n]，无向邻接表双向对称；root 默认 1；检查边数和连通性。每点仅计入 Euler 序一次，order[1..n]，order[tin[u]]=u；子树闭区间 [tin[u],tout[u]]。subtree_sum 输入长度 n+1，返回含自身的子树和。无效树或长度不符返回空。

调用：cp::EulerTree e(g,root); if(e.ok) { auto sums=e.subtree_sum(value); int L=e.tin[u], R=e.tout[u]; int size=e.subtree_size(u); }

例子：仓库树 DP 实例：cf/3/1096/H_%21%21dp_tree.cpp ，旧代码递归合并儿子状态；已读源码，AC 未核验。新模板示范同类依赖顺序的子树求和。

复杂度：预处理、一次子树和 DP 均 O(n)，空间 O(n)，subtree_size 查询 O(1)。

易错：tout 为子树最后一点的编号，不是第二次访问的时间戳；区间包含两端。；逆序 order 才能先计算儿子，再将状态合并给父亲。；先确认 ok；求和过程须在 ll 范围内。；这不是用于 RMQ 求 LCA 的长度 2n-1 欧拉游走。

规范源码：graph/euler_subtree.hpp；完整独立代码如下；测试：tests/test_graph.cpp

```
#include <bits/stdc++.h>
using namespace std;
using ll = long long;

namespace cp {
// 无向树：点 [1,n]，双向邻接，端点合法且对称由调用方保证。
// Euler 序仅入点一次；子树对应闭区间 [tin[u],tout[u]]。
struct EulerTree {
    int n;
    bool ok = false;
    vector<int> tin, tout, parent, order; // order[1..n], order[0]=0
    explicit EulerTree(const vector<vector<int>>& g, int root = 1)
        : n((int)g.size() - 1) {
        if (n < 1 || root < 1 || root > n) return;
        ll count = 0;
        for (int u = 1; u <= n; ++u) count += (ll)g[u].size();
        if (count != 2LL * (n - 1)) return;
        tin.resize(n + 1); tout.resize(n + 1); parent.resize(n + 1);
        vector<pair<int, int>> st{{root, 0}};
        order = {0, root}; tin[root] = 1;
        int timer = 1;
        while (!st.empty()) {
            auto& [u, i] = st.back();
            if (i == (int)g[u].size()) {
                tout[u] = timer; st.pop_back(); continue;
            }
            int v = g[u][i++];
            if (tin[v]) continue;
            parent[v] = u; tin[v] = ++timer;
            order.push_back(v); st.push_back({v, 0});
        }
        ok = (timer == n);
    }
    int subtree_size(int u) const {
        return ok && u >= 1 && u <= n ? tout[u] - tin[u] + 1 : 0;
    }
    // 简单树 DP：逆序合并子状态；权值/中间和需在 ll 范围内。
    vector<ll> subtree_sum(const vector<ll>& value) const {
        if (!ok || (int)value.size() != n + 1) return {};
        vector<ll> sum = value;
        for (int i = n; i >= 1; --i) {
            int u = order[i];
            if (parent[u]) sum[parent[u]] += sum[u];
        }
        return sum;
    }
};
} // namespace cp
```

【G11】强连通分量 Iterative Kosaraju SCC

识别：有向图中两两互相可达的分量；缩点成 DAG 的第一步。

约定：g 长度 n+1，合法点 [1,n]；允许重边自环。返回 count、comp[1..n]、groups[1..count]；0 为保留位置。

调用：auto r=cp::scc(g); bool mutual=r.comp[u]==r.comp[v]; for(int u:r.groups[c]) { /* 分量内结点 */ }

例子：手算例：1->2,2->1,2->3,3->3；分量为 {1,2} 和 {3}，缩点后只有一条跨分量边。

复杂度：O(n+m) 时间和空间；两次显式栈遍历，额外存反图。

易错：第一遍要记录退出顺序，不能使用刚发现结点的顺序。；第二遍使用反图并按第一遍退出顺序的逆序搜索。；缩点时需另外遍历原边，过滤同分量边；是否去重依题意。

规范源码：advanced/scc.hpp；完整独立代码如下；测试：tests/test_graph.cpp

```
#include <bits/stdc++.h>
using namespace std;
using ll = long long;

namespace cp {
struct SCCResult {
    int count = 0;
    vector<int> comp;
    vector<vector<int>> groups{{}}; // groups[1..count]
};
// Kosaraju 两遍迭代 DFS；有向图，合法点 [1,n]，可有重边自环。
inline SCCResult scc(const vector<vector<int>>& g) {
    int n = (int)g.size() - 1;
    vector<vector<int>> rev(n + 1);
    for (int u = 1; u <= n; ++u)
        for (int v : g[u]) rev[v].push_back(u);
    vector<int> seen(n + 1), finish;
    vector<pair<int, int>> st;
    for (int s = 1; s <= n; ++s) if (!seen[s]) {
        seen[s] = 1; st.push_back({s, 0});
        while (!st.empty()) {
            auto& [u, i] = st.back();
            if (i == (int)g[u].size()) {
                finish.push_back(u); st.pop_back(); continue;
            }
            int v = g[u][i++];
            if (!seen[v]) { seen[v] = 1; st.push_back({v, 0}); }
        }
    }
    SCCResult r; r.comp.resize(n + 1);
    vector<int> stack;
    reverse(finish.begin(), finish.end());
    for (int s : finish) if (!r.comp[s]) {
        ++r.count; r.groups.push_back({});
        r.comp[s] = r.count; stack.push_back(s);
        while (!stack.empty()) {
            int u = stack.back(); stack.pop_back();
            r.groups.back().push_back(u);
            for (int v : rev[u]) if (!r.comp[v]) {
                r.comp[v] = r.count; stack.push_back(v);
            }
        }
    }
    return r;
}
} // namespace cp
```

【G12】无向图找桥 Iterative Bridges

识别：找删除后连通块数增加的边；边双连通分量与桥树的基础。

约定：n>=0，合法点 [1,n]；无向边每条输入一次，允许重边、自环和不连通图。返回桥在原 edges 中的 0-based 下标，按输入顺序排列。

调用：vector<pair<int,int>> e={{1,2},{1,2},{2,3},{3,3}}; auto ids=cp::bridges(3,e);

例子：手算例：两条平行边 (1,2)、一条 (2,3)、一条自环 (3,3)，只有原输入下标 2 的 (2,3) 是桥。

复杂度：O(n+m) 时间与空间；迭代 low-link，不占递归调用栈。

易错：必须按边 ID 跳过唯一父边；直接跳过父点会把平行边误判为桥。；判断桥使用 low[child] > tin[parent]，不是 >=。；退栈后才把儿子的 low 合并到父亲；自环永远不是桥。

规范源码：advanced/bridges.hpp；完整独立代码如下；测试：tests/test_graph.cpp

```
#include <bits/stdc++.h>
using namespace std;
using ll = long long;

namespace cp {
// 无向图，合法点 [1,n]；每条边输入一次，允许重边和自环。
// 返回桥在原 edges 中的 0-based 下标，按输入顺序排列。
inline vector<int> bridges(int n, const vector<pair<int, int>>& edges) {
    int m = (int)edges.size();
    vector<vector<pair<int, int>>> g(n + 1);
    for (int i = 0; i < m; ++i) {
        auto [u, v] = edges[i];
        g[u].push_back({v, i}); g[v].push_back({u, i});
    }
    vector<int> tin(n + 1), low(n + 1), parent(n + 1);
    vector<int> pe(n + 1, -1), next(n + 1), mark(m), st;
    int timer = 0;
    for (int s = 1; s <= n; ++s) if (!tin[s]) {
        tin[s] = low[s] = ++timer; st.push_back(s);
        while (!st.empty()) {
            int u = st.back();
            if (next[u] < (int)g[u].size()) {
                auto [v, id] = g[u][next[u]++];
                if (id == pe[u]) continue; // 只跳过同一条父边
                if (!tin[v]) {
                    parent[v] = u; pe[v] = id;
                    tin[v] = low[v] = ++timer; st.push_back(v);
                } else low[u] = min(low[u], tin[v]);
            } else {
                st.pop_back(); int p = parent[u];
                if (p) {
                    if (low[u] > tin[p]) mark[pe[u]] = 1;
                    low[p] = min(low[p], low[u]);
                }
            }
        }
    }
    vector<int> ans;
    for (int i = 0; i < m; ++i) if (mark[i]) ans.push_back(i);
    return ans;
}
} // namespace cp
```

【M01】整数与位运算 Integer and Bits

识别：最大公约数、最小公倍数、二进制计数、取最低位、枚举 mask 的所有子集。

约定：gcd_abs 接受完整 ll 范围并返回 unsigned long long；gcd(0,0)=0。lcm_checked 只接受能放进非负 ll 的结果，否则 nullopt。位操作用 ull，位号 0..63；最低位不存在返回-1。

调用：ull g=gcd_abs(a,b); auto l=lcm_checked(a,b); if(l) cout<<*l; int cnt=bit_count(mask); int k=lowest_bit_index(mask);

例子：原仓库 gcd.cpp：template/gcd.cpp 。旧递归在(-6,-4)返回-2；新版返回非负 gcd 并覆盖完整 64 位输入。

复杂度：gcd/lcm 为 O(log max(|a|,|b|))；64 位基础位操作 O(1)；子集枚举 O(2^popcount(mask))。

易错：LLONG_MIN 不能直接取负或 abs；其绝对值 2^63 不能装进 ll。；使用 1ULL<<k 且 k<64，不要用有符号 1 处理高位。；子集枚举必须在 sub==0 时结束，否则会回到 mask。

规范源码：math/integer_bits.hpp；完整独立代码如下；测试：tests/test_dp_math_string.cpp

```
#include <bits/stdc++.h>
using namespace std;
using ll = long long;
using ull = unsigned long long;

// 绝对值用无符号保存，避免 -LLONG_MIN 的有符号溢出。
inline ull unsigned_abs(ll x) {
    return x < 0 ? 0ULL - ull(x) : ull(x);
}
inline ull gcd_abs(ll a, ll b) {
    ull x = unsigned_abs(a), y = unsigned_abs(b);
    while (y) { ull r = x % y; x = y; y = r; }
    return x;  // gcd(0,0)=0；结果可能为 2^63，不能总转回 ll。
}
inline optional<ll> lcm_checked(ll a, ll b) {
    if (a == 0 || b == 0) return 0;
    ull x = unsigned_abs(a), y = unsigned_abs(b);
    __int128 z = (__int128)(x / gcd_abs(a, b)) * y;
    if (z > LLONG_MAX) return nullopt;
    return ll(z);
}

// 位编号从 0 开始；无符号运算可以安全处理第 63 位。
inline ull lowbit(ull x) { return x & (0ULL - x); }
inline int bit_count(ull x) { return popcount(x); }
inline int lowest_bit_index(ull x) {
    return x == 0 ? -1 : int(countr_zero(x));
}
// 第 k 位：1ULL<<k，要求 0<=k<64；不可写 1<<k 处理高位。
// 判断位：(mask>>k)&1ULL；设置位：mask|=(1ULL<<k)。
// 枚举全部子集(含 0)，顺序从大到小，O(2^popcount(mask))：
// for (ull sub=mask;; sub=(sub-1)&mask) {
//     ... 使用 sub ...
//     if (sub==0) break;  // 必须先终止，否则又回到 mask。
// }
```

【M02】安全模运算与快速幂 Modular Power

识别：大整数模乘、非负整数快速幂、负数归一化，以及质数条件成立时求逆元。

约定：1<=mod<=LLONG_MAX，底数可为任意 ll；结果在[0,mod)。指数为非负 unsigned long long；0^0 按 1 处理后取模，mod=1 返回 0。

调用：ll y=mod_pow(a,e,mod); ll p=mod_mul(a,b,mod); 若确认 mod 为质数且 a 不为 0 模 mod，可用 mod_pow(a,mod-2,mod)求逆元。

例子：原仓库 ksm.cpp：template/ksm.cpp 。旧(-2)^3 mod5 为-3、3^0 mod1 为 1；新版分别为 2 和 0。

复杂度：归一化/模加/模乘 O(1)，快速幂 O(log exp)。

易错：先转__int128 再乘；乘完再转类型已来不及。；费马逆元 a^(mod-2)只适用于 mod 为质数且 a%mod!=0。；不能把负指数直接转成 unsigned 传入；指数取模也需要额外数论前提。

规范源码：math/modular.hpp；完整独立代码如下；测试：tests/test_dp_math_string.cpp

```
#include <bits/stdc++.h>
using namespace std;
using ll = long long;

// 1<=mod<=LLONG_MAX；返回[0,mod)。支持负数和 LLONG_MIN。
inline ll mod_norm(ll a, ll mod) {
    assert(mod >= 1);
    a %= mod;
    if (a < 0) a += mod;
    return a;
}
inline ll mod_add(ll a, ll b, ll mod) {
    return ll(((__int128)mod_norm(a, mod) + mod_norm(b, mod)) % mod);
}
inline ll mod_mul(ll a, ll b, ll mod) {
    return ll((__int128)mod_norm(a, mod) * mod_norm(b, mod) % mod);
}
// exp 是非负整数；0^0 定义为 1 再取模；模 1 时答案恒为 0。
inline ll mod_pow(ll a, unsigned long long exp, ll mod) {
    a = mod_norm(a, mod);
    ll ans = 1 % mod;
    while (exp) {
        if (exp & 1ULL) ans = mod_mul(ans, a, mod);
        a = mod_mul(a, a, mod);
        exp >>= 1;
    }
    return ans;
}
// 费马逆元：仅当 mod 为质数且 a%mod!=0 时，inv=mod_pow(a,mod-2,mod)。
// 复合模数不能套此公式；负指数不能直接传入本函数。
```

【M03】普通模矩阵快速幂 Matrix Power

识别：线性递推阶数很小但 n 很大，能把一次转移写成固定矩阵乘状态列向量。

约定：矩阵 0-indexed，只接受同阶方阵，元素可为任意 ll，1<=mod<=LLONG_MAX；非负 unsigned 指数，0 次幂为单位矩阵。依赖 M02。

调用：Matrix t={{1,1},{1,0}}; auto r=matrix_pow(t,n,1000000007); cout<<r[0][1]; 工程中头文件自动包含 M02，纸面抄写需带上 M02。

例子：CSES Fibonacci Numbers：https://cses.fi/problemset/task/1722 。用{{1,1},{1,0}}的 n 次幂，结果[0][1]是 F[n]，包含 n=0；未在线提交。

复杂度：d 阶矩阵乘法 O(d^3)，幂 O(d^3 log exp)，空间 O(d^2)。

易错：先手算一次转移检查状态顺序，行向量和列向量不能混用。；这是普通模加模乘；最短路 min-plus 和最大值 max-plus 须修改运算及单位元，不能直接用于对应赛题。；负指数不合法；矩阵及中间乘积使用 64 位和__int128。

规范源码：math/matrix_power.hpp；完整独立代码如下；测试：tests/test_dp_math_string.cpp

```
#include <bits/stdc++.h>
#include <bits/stdc++.h>
using namespace std;
using ll = long long;

// 1<=mod<=LLONG_MAX；返回[0,mod)。支持负数和 LLONG_MIN。
inline ll mod_norm(ll a, ll mod) {
    assert(mod >= 1);
    a %= mod;
    if (a < 0) a += mod;
    return a;
}
inline ll mod_add(ll a, ll b, ll mod) {
    return ll(((__int128)mod_norm(a, mod) + mod_norm(b, mod)) % mod);
}
inline ll mod_mul(ll a, ll b, ll mod) {
    return ll((__int128)mod_norm(a, mod) * mod_norm(b, mod) % mod);
}
// exp 是非负整数；0^0 定义为 1 再取模；模 1 时答案恒为 0。
inline ll mod_pow(ll a, unsigned long long exp, ll mod) {
    a = mod_norm(a, mod);
    ll ans = 1 % mod;
    while (exp) {
        if (exp & 1ULL) ans = mod_mul(ans, a, mod);
        a = mod_mul(a, a, mod);
        exp >>= 1;
    }
    return ans;
}
// 费马逆元：仅当 mod 为质数且 a%mod!=0 时，inv=mod_pow(a,mod-2,mod)。
// 复合模数不能套此公式；负指数不能直接传入本函数。
using namespace std;
using ll = long long;
using Matrix = vector<vector<ll>>;

// 矩阵下标 0-indexed；只接受同阶方阵；1<=mod<=LLONG_MAX。
// 普通模加/模乘，不是最短路 min-plus 或最大值 max-plus 矩阵。
inline Matrix matrix_mul(const Matrix& a, const Matrix& b, ll mod) {
    int n = int(a.size());
    assert(mod >= 1 && b.size() == a.size());
    for (const auto& row : a) assert(int(row.size()) == n);
    for (const auto& row : b) assert(int(row.size()) == n);
    Matrix c(n, vector<ll>(n));
    for (int i = 0; i < n; ++i)
        for (int k = 0; k < n; ++k) {
            ll x = mod_norm(a[i][k], mod);
            if (x == 0) continue;
            for (int j = 0; j < n; ++j) {
                ll y = mod_norm(b[k][j], mod);
                c[i][j] = ll((c[i][j] + (__int128)x * y) % mod);
            }
```

【M03】普通模矩阵快速幂 Matrix Power 续

```
        }
    return c;
}
inline Matrix matrix_pow(Matrix a, unsigned long long exp, ll mod) {
    assert(mod >= 1);
    int n = int(a.size());
    for (const auto& row : a) assert(int(row.size()) == n);
    Matrix ans(n, vector<ll>(n));
    for (int i = 0; i < n; ++i) ans[i][i] = 1 % mod;
    while (exp) {
        if (exp & 1ULL) ans = matrix_mul(ans, a, mod);
        a = matrix_mul(a, a, mod);
        exp >>= 1;
    }
    return ans;
}
// 例：Matrix t={{1,1},{1,0}}; auto p=matrix_pow(t,n,mod);
// 先写清状态列向量，再验证转移一次；指数 0 返回单位矩阵。
```

【M04】乘积递推修正版 Product Recurrence

识别：原 1182E 乘积递推的指数转移示例；用矩阵求各底数指数，再做模幂。理解 M02、M03 后再读。

约定：f[n]=f[n-1]*f[n-2]*f[n-3]*c^(2n-6)，固定模 P=10^9+7；n>=1，四个底数可为任意 ll；n<=3 直接返回初值。依赖 M02、M03。

调用：ll ans=product_recurrence(n,f1,f2,f3,c); 固定题型示例，不是所有乘积递推通用公式；更换递推时重新推导两个转移矩阵。

例子：Codeforces 1182E Product Oriented Recurrence：CF 1182/E 。已核对原递推与两条官方样例；不宣称在线 AC。

复杂度：矩阵阶数最多 5，时间 O(5^3 log n)，额外空间 O(5^2)。

易错：累乘答案从 1 开始；原文件从 0 开始会使所有答案为 0。；n>=4 时四个底数的真实指数均为正，先判底数 modP 是否为 0，再把指数模 P-1；不能只凭取模后指数 0 判断 0^0。；原题 n>=4；新版的 n=1/2/3 是明确添加的边界支持，不把原题未要求的输入当作原提交失败证据。

规范源码：math/product_recurrence.hpp；完整独立代码如下；测试：tests/test_dp_math_string.cpp

```
#include <bits/stdc++.h>
#include <bits/stdc++.h>
#include <bits/stdc++.h>
using namespace std;
using ll = long long;

// 1<=mod<=LLONG_MAX；返回[0,mod)。支持负数和 LLONG_MIN。
inline ll mod_norm(ll a, ll mod) {
    assert(mod >= 1);
    a %= mod;
    if (a < 0) a += mod;
    return a;
}
inline ll mod_add(ll a, ll b, ll mod) {
    return ll(((__int128)mod_norm(a, mod) + mod_norm(b, mod)) % mod);
}
inline ll mod_mul(ll a, ll b, ll mod) {
    return ll((__int128)mod_norm(a, mod) * mod_norm(b, mod) % mod);
}
// exp 是非负整数；0^0 定义为 1 再取模；模 1 时答案恒为 0。
inline ll mod_pow(ll a, unsigned long long exp, ll mod) {
    a = mod_norm(a, mod);
    ll ans = 1 % mod;
    while (exp) {
        if (exp & 1ULL) ans = mod_mul(ans, a, mod);
        a = mod_mul(a, a, mod);
        exp >>= 1;
    }
    return ans;
}
// 费马逆元：仅当 mod 为质数且 a%mod!=0 时，inv=mod_pow(a,mod-2,mod)。
// 复合模数不能套此公式；负指数不能直接传入本函数。
using namespace std;
using ll = long long;
using Matrix = vector<vector<ll>>;

// 矩阵下标 0-indexed；只接受同阶方阵；1<=mod<=LLONG_MAX。
// 普通模加/模乘，不是最短路 min-plus 或最大值 max-plus 矩阵。
inline Matrix matrix_mul(const Matrix& a, const Matrix& b, ll mod) {
    int n = int(a.size());
    assert(mod >= 1 && b.size() == a.size());
    for (const auto& row : a) assert(int(row.size()) == n);
    for (const auto& row : b) assert(int(row.size()) == n);
    Matrix c(n, vector<ll>(n));
    for (int i = 0; i < n; ++i)
        for (int k = 0; k < n; ++k) {
            ll x = mod_norm(a[i][k], mod);
            if (x == 0) continue;
            for (int j = 0; j < n; ++j) {
```

【M04】乘积递推修正版 Product Recurrence 续

```
                ll y = mod_norm(b[k][j], mod);
                c[i][j] = ll((c[i][j] + (__int128)x * y) % mod);
            }
        }
    return c;
}
inline Matrix matrix_pow(Matrix a, unsigned long long exp, ll mod) {
    assert(mod >= 1);
    int n = int(a.size());
    for (const auto& row : a) assert(int(row.size()) == n);
    Matrix ans(n, vector<ll>(n));
    for (int i = 0; i < n; ++i) ans[i][i] = 1 % mod;
    while (exp) {
        if (exp & 1ULL) ans = matrix_mul(ans, a, mod);
        a = matrix_mul(a, a, mod);
        exp >>= 1;
    }
    return ans;
}
// 例：Matrix t={{1,1},{1,0}}; auto p=matrix_pow(t,n,mod);
// 先写清状态列向量，再验证转移一次；指数 0 返回单位矩阵。
using namespace std;
using ll = long long;

// CF1182E：f[n]=f[n-1]*f[n-2]*f[n-3]*c^(2n-6)，n>=4。
// n>=1；f1,f2,f3,c 可为任意 ll；固定模 P=1e9+7。
// 这里额外支持 n=1/2/3 和零底数，原题输入从 n=4 开始。
inline ll product_recurrence(unsigned long long n, ll f1, ll f2,
                             ll f3, ll c) {
    assert(n >= 1);
    const ll p = 1000000007, q = p - 1;
    array<ll, 4> f = {f1, f2, f3, c};
    for (ll& x : f) x = mod_norm(x, p);
    if (n <= 3) return f[n - 1];
    // n>=4 时四个底数的真实指数都为正。先判零，再取指数模 p-1。
    for (ll x : f) if (x == 0) return 0;
    Matrix t = {{1, 1, 1}, {1, 0, 0}, {0, 1, 0}};
    Matrix a = matrix_pow(t, n - 3, q);
    // c 的指数 E：E[t+1]=E[t]+E[t-1]+E[t-2]+2*t-4。
    // 状态列向量为 [E[t],E[t-1],E[t-2],t,1]，初态[0,0,0,3,1]。
    Matrix u = {
        {1, 1, 1, 2, -4}, {1, 0, 0, 0, 0},
        {0, 1, 0, 0, 0}, {0, 0, 0, 1, 1}, {0, 0, 0, 0, 1}
    };
    Matrix b = matrix_pow(u, n - 3, q);
    ll ec = mod_add(mod_mul(b[0][3], 3, q), b[0][4], q);
    array<ll, 4> e = {a[0][2], a[0][1], a[0][0], ec};
    ll ans = 1;  // 连乘单位元是 1，不能从 0 开始。
    for (int i = 0; i < 4; ++i)
        ans = mod_mul(ans, mod_pow(f[i], e[i], p), p);
    return ans;
}
```

【M05】质数模数组合数 Prime Combinations

识别：大量询问 C(n,k)，所有 n 有可承受的预处理上限，模数已知为质数且大于该上限。

约定：prime 必须为质数且 0<=max_n<prime；模板不检测质数。有效查询须 0<=n<=max_n；k<0 或 k>n 返回 0。依赖 M02。

调用：PrimeComb comb(max_n,1000000007); cout<<comb.choose(n,k); 纸面抄写同时带上 M02。

例子：Codeforces 1475E Advertising Agency：CF 1475/E 。对选取边界的相同值计算组合数；未在线提交。

复杂度：预处理 O(max_n+log prime) 时间、O(max_n) 空间；每次查询 O(1)。

易错：阶乘含模数因子时没有逆元：n>=prime 不能继续套此模板。；复合模数不能使用费马逆元；多测可按全体最大 n 预处理一次。；C(0,0)=1，查询上限不能大于构造时的 max_n。

规范源码：math/combinations.hpp；完整独立代码如下；测试：tests/test_dp_math_string.cpp

```
#include <bits/stdc++.h>
#include <bits/stdc++.h>
using namespace std;
using ll = long long;

// 1<=mod<=LLONG_MAX；返回[0,mod)。支持负数和 LLONG_MIN。
inline ll mod_norm(ll a, ll mod) {
    assert(mod >= 1);
    a %= mod;
    if (a < 0) a += mod;
    return a;
}
inline ll mod_add(ll a, ll b, ll mod) {
    return ll(((__int128)mod_norm(a, mod) + mod_norm(b, mod)) % mod);
}
inline ll mod_mul(ll a, ll b, ll mod) {
    return ll((__int128)mod_norm(a, mod) * mod_norm(b, mod) % mod);
}
// exp 是非负整数；0^0 定义为 1 再取模；模 1 时答案恒为 0。
inline ll mod_pow(ll a, unsigned long long exp, ll mod) {
    a = mod_norm(a, mod);
    ll ans = 1 % mod;
    while (exp) {
        if (exp & 1ULL) ans = mod_mul(ans, a, mod);
        a = mod_mul(a, a, mod);
        exp >>= 1;
    }
    return ans;
}
// 费马逆元：仅当 mod 为质数且 a%mod!=0 时，inv=mod_pow(a,mod-2,mod)。
// 复合模数不能套此公式；负指数不能直接传入本函数。
using namespace std;
using ll = long long;

// 仅适用于已知质数 prime，并且 0<=预处理上限 n<prime。
// 本模板不检测质数；复合模数或 n>=prime 不能用阶乘逆元法。
struct PrimeComb {
    ll mod;
    vector<ll> fac, ifac;
    PrimeComb(int n, ll prime) : mod(prime) {
        assert(n >= 0 && prime >= 2 && ll(n) < prime);
        fac.resize(n + 1); ifac.resize(n + 1);
        fac[0] = 1;
        for (int i = 1; i <= n; ++i)
            fac[i] = mod_mul(fac[i - 1], i, mod);
        ifac[n] = mod_pow(fac[n], mod - 2, mod);
        for (int i = n; i >= 1; --i)
            ifac[i - 1] = mod_mul(ifac[i], i, mod);
    }
    ll choose(int n, int k) const {
        if (k < 0 || k > n) return 0;
        assert(n >= 0 && n < int(fac.size()));
        return mod_mul(fac[n], mod_mul(ifac[k], ifac[n - k], mod), mod);
    }
};
// PrimeComb comb(max_n,1000000007); cout << comb.choose(n,k);
// C(0,0)=1；无效 k 返回 0；有效查询的 n 不能超过预处理上限。
// 预处理 O(n+log mod)时间/O(n)空间，每次查询 O(1)。
```

【S01】KMP 与 Z 函数 Prefix Matching

识别：线性时间寻找全部模式串出现位置、分析前后缀相等长度，或比较字符串每个后缀与全串前缀。

约定：字符串按字节、0-indexed，长度能放进 int。KMP 返回所有匹配起点，允许重叠；空模式匹配 0..n。pi 为最长相等真前后缀长度；Z 约定 z[0]=0。

调用：auto pos=kmp_find_all(text,pattern); auto pi=prefix_function(s); auto z=z_function(s); 字符串无需人为补第 0 个字符。

例子：CSES String Matching：https://cses.fi/problemset/task/1753 。调用 kmp_find_all(text,pattern).size()；函数来自原 KMP.cpp 的转移提炼并修复空模式边界，未在线提交。

复杂度：KMP O(|s|+|p|) 时间，O(|p|+匹配数)空间；pi/Z 为 O(n) 时间和空间。

易错：完全匹配后令 j=pi[j-1]，否则会漏重叠匹配或越界。；不同资料可能定义 z[0]=n，使用公式时先核对约定。；空模式不能直接访问 pi[-1]；新版先单独处理。

规范源码：string/kmp_z.hpp；完整独立代码如下；测试：tests/test_dp_math_string.cpp

```
#include <bits/stdc++.h>
using namespace std;
using ll = long long;

// 字符串统一 0-indexed。pi[i]是 s[0..i]最长相等真前后缀的长度。
inline vector<int> prefix_function(const string& s) {
    vector<int> pi(s.size());
    for (int i = 1; i < int(s.size()); ++i) {
        int j = pi[i - 1];
        while (j && s[i] != s[j]) j = pi[j - 1];
        if (s[i] == s[j]) ++j;
        pi[i] = j;
    }
    return pi;
}
// 返回 p 在 s 中所有起点，允许重叠；空 p 匹配 0..s.size()共 n+1 处。
inline vector<int> kmp_find_all(const string& s, const string& p) {
    vector<int> ans;
    if (p.empty()) {
        ans.resize(s.size() + 1); iota(ans.begin(), ans.end(), 0);
        return ans;
    }
    auto pi = prefix_function(p);
    for (int i = 0, j = 0; i < int(s.size()); ++i) {
        while (j && s[i] != p[j]) j = pi[j - 1];
        if (s[i] == p[j]) ++j;
        if (j == int(p.size())) {
            ans.push_back(i - j + 1);
            j = pi[j - 1];
        }
    }
    return ans;
}
// z[i]是 s 与 s[i..]的最长公共前缀长度；本实现约定 z[0]=0。
inline vector<int> z_function(const string& s) {
    int n = int(s.size());
    vector<int> z(n);
    for (int i = 1, l = 0, r = 0; i < n; ++i) {
        // [l,r) 是已知最靠右的匹配区间。
        if (i < r) z[i] = min(r - i, z[i - l]);
        while (i + z[i] < n && s[z[i]] == s[i + z[i]]) ++z[i];
        if (i + z[i] > r) l = i, r = i + z[i];
    }
    return z;
}
// kmp_find_all(text,pattern)无需选分隔符，适合直接查找。
```

【S02】双模字符串哈希 Double Hash

识别：大量查询子串指纹、子串去重或快速筛选相等候选；可以接受碰撞风险时使用。

约定：字符串 0-indexed，get(l,r)取半开区间[l,r)，0<=l<=r<=n；长度能放进 int。按 unsigned char+1 编码，固定 base 与两组模数。比较哈希时也比较长度。

调用：DoubleHash h(s); auto key=h.get(l,r); 若比较区间先确认长度相等，再比较 key；空区间的 key 为(0,0)。

例子：Codeforces 271D Good Substrings：CF 271/D 。可用坏字符前缀和筛选，再对(长度,双哈希)去重；这仍是有碰撞风险的方案，未在线提交。

复杂度：预处理 O(n) 时间、O(n) 空间；子串哈希 O(1)，确定性字符串核验另计。

易错：双模仍有碰撞，固定 base 还可能被针对构造；随机测试不能证明无碰撞。；需要确定性匹配可用 KMP；重要哈希命中要按原串核验。；闭区间[l,r]调用时右端改为 r+1；不同哈希参数的对象不能互相比。

规范源码：string/double_hash.hpp；完整独立代码如下；测试：tests/test_dp_math_string.cpp

```
#include <bits/stdc++.h>
using namespace std;
using ll = long long;

// 0-indexed 字符串；get(l,r)取半开区间[l,r)，要求 0<=l<=r<=n。
// 双模哈希仍可能碰撞，不能作为无碰撞证明；重要比较须直接核验。
// 比较前同时比较长度；不同对象使用相同 base 和模数才可比较。
struct DoubleHash {
    static constexpr ll mod1 = 1000000007, mod2 = 1000000009;
    static constexpr ll base = 911382323;
    vector<ll> h1, h2, p1, p2;
    explicit DoubleHash(const string& s) {
        int n = int(s.size());
        h1.assign(n + 1, 0); h2.assign(n + 1, 0);
        p1.assign(n + 1, 1); p2.assign(n + 1, 1);
        for (int i = 0; i < n; ++i) {
            // unsigned char 避免高位字节被解释成负数；+1 不把零字节当 0。
            ll x = (unsigned char)s[i] + 1;
            h1[i + 1] = (h1[i] * base + x) % mod1;
            h2[i + 1] = (h2[i] * base + x) % mod2;
            p1[i + 1] = p1[i] * base % mod1;
            p2[i + 1] = p2[i] * base % mod2;
        }
    }
    pair<ll, ll> get(int l, int r) const {
        assert(0 <= l && l <= r && r < int(h1.size()));
        ll a = (h1[r] - h1[l] * p1[r - l] % mod1 + mod1) % mod1;
        ll b = (h2[r] - h2[l] * p2[r - l] % mod2 + mod2) % mod2;
        return {a, b};
    }
};
// DoubleHash h(s); auto key=h.get(l,r); 空区间返回(0,0)。
// 固定 base 方便复现，但可被针对构造；需要确定性匹配优先用 KMP。
```

【S03】基础字符串处理 String Basics

识别：回文检查、单字符分割、ASCII 大小写、相邻相同字符分组，以及含空格输入。

约定：按字节处理而非 Unicode 字符，长度能放进 int；回文使用 0-indexed 半开区间[l,r)。分割保留所有空字段；大小写转换只处理 ASCII 大写字母。

调用：bool ok=is_palindrome(s,0,int(s.size())); auto fields=split_char(s,','); auto groups=run_length(s); string lower=ascii_lower(s);

例子：Codeforces 266A Stones on the Table：CF 266/A 。相邻同色分组后累加每组长度减 1；未在线提交。

复杂度：各函数线性时间；回文 O(1)额外空间，其余按返回内容 O(n)空间。

易错：cin>>后接 getline 要处理残留换行；cin>>ws 会吞掉有意义的前导空白。；split_char(空串)返回一个空字段；末尾分隔符也产生空字段。；UTF-8 中文由多个字节构成，不能把逐字节操作当成逐汉字处理。

规范源码：string/basic_string.hpp；完整独立代码如下；测试：tests/test_dp_math_string.cpp

```
#include <bits/stdc++.h>
using namespace std;
using ll = long long;

// 基础字符串均按字节处理；0-indexed，区间[l,r)，不是 Unicode 码点。
inline bool is_palindrome(const string& s, int l, int r) {
    assert(0 <= l && l <= r && r <= int(s.size()));
    for (--r; l < r; ++l, --r) if (s[l] != s[r]) return false;
    return true;
}
// 按单个字符分割，保留首尾及连续分隔符产生的空串。
inline vector<string> split_char(const string& s, char sep) {
    vector<string> parts;
    size_t start = 0;
    for (size_t i = 0; i <= s.size(); ++i)
        if (i == s.size() || s[i] == sep) {
            parts.push_back(s.substr(start, i - start));
            start = i + 1;
        }
    return parts;
}
inline string ascii_lower(string s) {
    for (char& c : s) if ('A' <= c && c <= 'Z') c = char(c - 'A' + 'a');
    return s;
}
inline vector<pair<char, int>> run_length(const string& s) {
    vector<pair<char, int>> groups;
    for (char c : s) {
        if (groups.empty() || groups.back().first != c)
            groups.push_back({c, 1});
        else ++groups.back().second;
    }
    return groups;
}
// 读含空格整行：getline(cin,s)；刚用 cin>>后先丢弃残留换行。
// cin.ignore(numeric_limits<streamsize>::max(),'\n');
// cin>>ws 会跳过全部前导空白；若空白有意义，不要使用它。
```

【BM01】区间单位操作与总变差

识别：从零变目标，问任意区间/前缀/后缀±1 或二进制翻转的最少次数；相邻转移。

思路：将区间操作映射为差分两端的一增一减，区分操作集合和单次费用。

证明：每次单位区间操作至多消去一单位正差分，故正差分和是下界；正负边界配对的区间构造达到它。前后缀系数则由相邻差唯一确定。

约定：a[0]=0、a[1..n]为 ll；任意区间 mode=1 要求非负目标，-1 要求非正，0 允许任意符号且中间状态无额外限制；计数 i128。相邻转移要求总和相同。

调用：auto ans=unit_intervals({0,2,1,3},1); // 4；prefix_units({0,2,1,3})=6；suffix_units(...)=5

例子：[2,1,3]只加 1 最少 4 次；[1,-1]允许±1 最少 2 次；从[3,0,0]搬到[0,0,3]需 6 次相邻转移。

复杂度：每个计数函数 O(n)时间、O(1)额外空间。

易错：不能把任意区间公式用于只能前缀的题。；每次可加任意整数且费用为 1 时，[2,2]只需 1 次，单位公式 2 失效。；LLONG_MIN 取绝对值前必须提升 i128；不直接 ll abs。

来源：cf/date/6/6.9/1_special_segment_tree.cpp

规范源码：competition/models/interval_count.hpp；完整独立代码如下；测试：tests/round02/test_models.cpp

```
#include <bits/stdc++.h>
using namespace std;
using ll = long long;
using i128 = __int128_t;
// a[1..n], a[0]=0。中间值和操作次数用 i128。
// mode=1 只加 1；-1 只减 1；0 两者均可。无其他中间状态限制。
optional<i128> unit_intervals(const vector<ll>& a, int mode = 0) {
    assert(!a.empty() && -1 <= mode && mode <= 1);
    i128 ans = 0, pre = 0;
    for(int i = 1; i < (int)a.size(); i++) {
        if((mode == 1 && a[i] < 0) || (mode == -1 && a[i] > 0))
            return nullopt;
        i128 d = (i128)a[i] - pre;
        if(d > 0) ans += d;
        pre = a[i];
    }
    if(pre < 0) ans -= pre; // 补上最后边界 d[n+1]=-a[n]
    return ans;
}
// 只能操作前缀/后缀，每次可加 1 或减 1。
i128 prefix_units(const vector<ll>& a) {
    i128 ans = 0, next = 0;
    for(int i = (int)a.size() - 1; i >= 1; i--) {
        i128 d = (i128)a[i] - next;
        ans += d < 0 ? -d : d;
        next = a[i];
    }
    return ans;
}
i128 suffix_units(const vector<ll>& a) {
    i128 ans = 0, pre = 0;
    for(int i = 1; i < (int)a.size(); i++) {
        i128 d = (i128)a[i] - pre;
        ans += d < 0 ? -d : d;
        pre = a[i];
    }
    return ans;
}
// 从全 0 二进制数组出发，任意区间按位翻转：答案是连续 1 段数。
int binary_interval_flips(const vector<int>& a) {
    int ans = 0, pre = 0;
    for(int i = 1; i < (int)a.size(); i++) {
        assert(a[i] == 0 || a[i] == 1);
        if(a[i] == 1 && pre == 0) ans++;
        pre = a[i];
    }
    return ans;
```

【BM01】区间单位操作与总变差 续

```
}
// 相邻位置转移 1 单位；允许两个方向。总和相同才可达。
optional<i128> adjacent_moves(const vector<ll>& a, const vector<ll>& b) {
    assert(a.size() == b.size());
    i128 pre = 0, ans = 0;
    for(int i = 1; i < (int)a.size(); i++) {
        pre += (i128)a[i] - b[i];
        if(i + 1 < (int)a.size()) ans += pre < 0 ? -pre : pre;
    }
    if(pre != 0) return nullopt;
    return ans;
}
```

【BM02】最少区间操作的压缩构造与变体

识别：不仅问最少次数，还要构造区间；或每次区间可加任意整数。

思路：正负差分边界配对；任意整数费用 1 改成最大零和分组，不能复用单位次数。

证明：单位配对每步同时消去相同质量的正负差分。任意整数操作是一条边，k 个边界连通至少 k-1 条；每个连通分量差分和 0，星形实现恰 k-1 条，所以答案 m 减最多零和组数。

约定：a[0]=0；单位版 mode=-1/0/1 同 BM01，输出(l,r,sign,cnt)压缩组；任意整数版精确求解仅限非零边界差分 m<=20。

调用：auto ops=construct_unit_intervals({0,2,1,3},1); // 各 op 重复 cnt 次；arbitrary_intervals_small({0,2,2})=1

例子：[1,3,2]可[1,2]加 1、[2,3]加 2，两次任意整数操作；若按单位费用则要 3 次。

复杂度：单位构造 O(n)时间空间、至多 n 个压缩组；小规模任意整数版 O(m*2^m)时间、O(2^m)空间。

易错：cnt 可很大，不能逐次展开输出导致超时；题面若要求逐操作输出还受输出量限制。；小规模指数 DP 不可用于大 m；m 限制是非零差分数而非数组长度。；任意整数费用与单位费用两个问题不可混用。

来源：template/competition/models/interval_count.hpp

规范源码：competition/models/interval_construct.hpp；完整独立代码如下；测试：tests/round02/test_models.cpp

```
#include <bits/stdc++.h>
using namespace std;
using ll = long long;
using i128 = __int128_t;
struct IntervalOp { int l, r, sign; i128 cnt; };
// 输出压缩操作：[l,r] 加 sign，重复 cnt 次；至多 n 组，不逐次展开。
// a[0]=0。mode=1 只加，-1 只减，0 任意；空结果也可能是合法全 0 目标。
optional<vector<IntervalOp>> construct_unit_intervals(
        const vector<ll>& a, int mode = 0) {
    assert(!a.empty() && -1 <= mode && mode <= 1);
    vector<pair<int,i128>> pos, neg;
    int n = (int)a.size() - 1;
    for(int i = 1; i <= n; i++) {
        if((mode == 1 && a[i] < 0) || (mode == -1 && a[i] > 0))
            return nullopt;
    }
    for(int i = 1; i <= n + 1; i++) {
        i128 d = (i <= n ? (i128)a[i] : 0) - a[i - 1];
        if(d > 0) pos.push_back({i,d});
        if(d < 0) neg.push_back({i,-d});
    }
    vector<IntervalOp> ans;
    int p = 0, q = 0;
    while(p < (int)pos.size()) {
        auto &x = pos[p]; auto &y = neg[q];
        i128 cnt = min(x.second, y.second);
        if(x.first < y.first) ans.push_back({x.first,y.first-1,1,cnt});
        else ans.push_back({y.first,x.first-1,-1,cnt});
        x.second -= cnt; y.second -= cnt;
        if(x.second == 0) p++;
        if(y.second == 0) q++;
    }
    return ans;
}
// 每次区间加任意整数，费用固定为 1：不能套单位操作公式。
// 仅用于非零差分个数 m<=20：精确子集 DP，O(m*2^m)，空间 O(2^m)。
int arbitrary_intervals_small(const vector<ll>& a) {
    assert(!a.empty());
    vector<i128> d;
    int n = (int)a.size() - 1;
    for(int i = 1; i <= n + 1; i++) {
        i128 x = (i <= n ? (i128)a[i] : 0) - a[i - 1];
        if(x != 0) d.push_back(x);
    }
```

【BM02】最少区间操作的压缩构造与变体 续

```
    int m = (int)d.size(); assert(m <= 20);
    int all = 1 << m;
    vector<i128> sum(all);
    vector<int> dp(all);
    for(int mask = 1; mask < all; mask++) {
        int bit = __builtin_ctz((unsigned)mask);
        sum[mask] = sum[mask ^ (1 << bit)] + d[bit];
        for(int i = 0; i < m; i++) if(mask >> i & 1)
            dp[mask] = max(dp[mask], dp[mask ^ (1 << i)]);
        if(sum[mask] == 0) dp[mask]++;
    }
    return m - dp.back();
}
```

【BM03】二维前缀和与矩形差分

识别：静态矩形求和；离线对多个矩形整体加值后恢复网格。

思路：二维容斥；矩形修改只改变四个角。

证明：前缀查询两次减去外部条带，再补回重复减的角；差分的两重前缀只在矩形内部保留 val。

约定：1<=坐标<=n,m<=500；先 init 再填 a；所有前缀和、差分和及中间表达式在 ll 内。restore 每批调用一次，重新查询需 build_prefix。

调用：init(2,2); a[1][1]=1; build_prefix(); query(1,1,2,2); add_rect(1,1,2,2,3); restore_difference();

例子：2x2 全 0 加整个矩形 3，再[2,2]加-1，恢复[[3,3],[3,2]]，总和 11。

复杂度：预处理/恢复 O(nm)，一次矩形查询/修改 O(1)，空间 O(nm)。

易错：x2+1/y2+1 哨兵需要留容量。；二维前缀快照不会随 a 自动更新。；不能把离线差分当在线任意矩形求和结构。

来源：template/core/basics.hpp

规范源码：competition/models/prefix_grid.hpp；完整独立代码如下；测试：tests/round02/test_model_tools.cpp

```
#include <bits/stdc++.h>
using namespace std;
using ll = long long;
const int N = 505;
int n, m;
ll a[N][N], s[N][N], d[N][N];
// 1..n,1..m; n,m<=500; 所有前缀和、差分累计值处于 ll 范围。
void init(int rows, int cols) {
    assert(0 <= rows && rows <= 500 && 0 <= cols && cols <= 500);
    n = rows; m = cols;
    memset(a, 0, sizeof(a));
    memset(s, 0, sizeof(s));
    memset(d, 0, sizeof(d));
}
void build_prefix() {
    for(int i = 1; i <= n; i++) for(int j = 1; j <= m; j++)
        s[i][j] = a[i][j] + s[i-1][j] + s[i][j-1] - s[i-1][j-1];
}
ll query(int x1, int y1, int x2, int y2) {
    if(x1 > x2 || y1 > y2) return 0;
    return s[x2][y2] - s[x1-1][y2] - s[x2][y1-1] + s[x1-1][y1-1];
}
// 离线矩形加；处理完全部操作后调用 restore_difference 一次。
void add_rect(int x1, int y1, int x2, int y2, ll val) {
    if(x1 > x2 || y1 > y2) return;
    d[x1][y1] += val; d[x2+1][y1] -= val;
    d[x1][y2+1] -= val; d[x2+1][y2+1] += val;
}
void restore_difference() {
    for(int i = 1; i <= n; i++) for(int j = 1; j <= m; j++) {
        d[i][j] += d[i-1][j] + d[i][j-1] - d[i-1][j-1];
        a[i][j] += d[i][j];
    }
    memset(d, 0, sizeof(d)); // 下一批操作可重新累计
    // s 仍是旧快照；若要查询新数组，重新 build_prefix。
}
```

【BM04】逆序对与折半枚举

识别：数对顺序违背大小关系；n 约 40 而直接枚举 2^n 不可行。

思路：归并时一批统计左侧剩余逆序；指数枚举拆成两个半边后做有序匹配。

证明：归并遇右元素小于左最小剩余，左剩余全与它构成逆序且只统计一次；每个完整子集唯一分解为左右两个子集。

约定：逆序对 a[1..n],n<200010 并原地排序；折半 v[1..n],n<=40，所有子集和及 target-x 在 ll 内，按位置区分子集并含空集。

调用：a[1]=3;a[2]=1;a[3]=2;merge_count(1,3); //2；subset_sum_count({0,1,1},1); //2

例子：重复值不算逆序；两个值均为 1 的不同位置构成两个和为 1 的子集。

复杂度：逆序对 O(nlogn)/O(n)；MITM O(n*2^(n/2))时间、O(2^(n/2))空间。

易错：归并必须用<=使相等值不计逆序。；MITM 求方案数不能 unique 掉重复和。；本模块逆序对会修改数组顺序，需要原序时先备份。

来源：template/core/basics.hpp

规范源码：competition/models/enumeration.hpp；完整独立代码如下；测试：tests/round02/test_model_tools.cpp

```
#include <bits/stdc++.h>
using namespace std;
using ll = long long;
const int N = 200010;
ll a[N], b[N];
// a[1..n], n<N；返回逆序对数，同时把 a 排序。
ll merge_count(int l, int r) {
    if(l >= r) return 0;
    int mid = (l + r) / 2;
    ll ans = merge_count(l, mid) + merge_count(mid + 1, r);
    int i = l, j = mid + 1, k = l;
    while(i <= mid && j <= r) {
        if(a[i] <= a[j]) b[k++] = a[i++];
        else { ans += mid - i + 1; b[k++] = a[j++]; }
    }
    while(i <= mid) b[k++] = a[i++];
    while(j <= r) b[k++] = a[j++];
    for(int p = l; p <= r; p++) a[p] = b[p];
    return ans;
}
// 折半搜索：下标 1..n，n<=40；所有子集和以及 target-s 必须在 ll 范围。
// 统计按位置区分的子集个数，包含空集，不把相同和值去重。
ll subset_sum_count(const vector<ll>& v, ll target) {
    int n = (int)v.size() - 1;
    assert(0 <= n && n <= 40);
    auto sums = [&](int l, int r) {
        vector<ll> ans{0};
        for(int i = l; i <= r; i++) {
            int old = (int)ans.size();
            for(int j = 0; j < old; j++) ans.push_back(ans[j] + v[i]);
        }
        return ans;
    };
    int mid = n / 2;
    vector<ll> left = sums(1, mid), right = sums(mid + 1, n);
    sort(right.begin(), right.end());
    ll ans = 0;
    for(ll x : left) {
        auto range = equal_range(right.begin(), right.end(), target - x);
        ans += range.second - range.first;
    }
    return ans;
}
```

【BM05】原解直接收录 二分选题 CF913D

识别：选择 k 个对象后，单个对象资格又依赖最终 k，且总时间有限。

思路：保留原注释的突破：不计分对象只会有负作用，先猜最终得分 k，再挑资格满足且时间最少的 k 个。

证明：删除不得分对象不减得分且放松限制；固定 k 时选最小耗时 k 个最优；可行 k 的任意更小值可通过删对象得到。

约定：原 CF913D 题面：n<=2e5，a_i<=n，t_i<=1e4,T<=1e9。完整独立程序，入口 main；check 只由原二分对 1..n 调用。

调用：独立编译运行；输入 5 300 与五组(a,t)，输出得分 2 及任意合法选择。不要与另一份 main 直接拼接。

例子：官方第二例 n=2,T=100，耗时 787/788，答案 0；第三例 42+58=100，两题资格 2，答案 2。

复杂度：O(nlogn)时间、O(n)空间。

易错：不能只按时间贪心而忽略资格随总数变化。；check(0)并非该原接口支持场景，原 main 从 1 开始搜索。；源码存在不等于线上 AC；当前验证为 300 组全子集差分。

来源：cf/other/Hello_2018/D.cpp

规范源码：competition/models/choose_tasks.cpp；完整独立代码如下；测试：tests/round02/test_model_tools.cpp

```
#include <bits/stdc++.h>
using namespace std;
using ll = long long;

/*
初步想法
贪心时间做题数多使得总分高
但是做题数多会导致其中有些题目不算数
做题数最终会等于分数（不算数的题只有负作用）

选出 x 题，让它们都得分，时间不超
也就是选 x 道满足 a_i >= x 的题，并且总时间 <= T
关键在于 k 是取得题数，正向不好想，会打架
所以反向验证

二分答案的思路
如果 x 可行，那么比 x 更小的值一定可行
所以可以二分最大可行的 x
*/

struct node {
    ll a;
    ll id;
    ll t;
};


ll n, T;
node p[200010];

bool check(ll x) {
    ll cnt = 0;
    ll sum = 0;
    for(ll i = 1; i <= n; i++) {
        if(p[i].a >= x) {
            cnt++;
            sum += p[i].t;
            if(cnt == x) return sum <= T;
        }
    }
    return false;
}

int main() {
    ios_base::sync_with_stdio(false);
    cin.tie(0);
```

【BM05】原解直接收录 二分选题 CF913D 续

```
    cin >> n >> T;

    for (ll i = 1; i <= n; i++) {
        p[i].id = i;
        cin >> p[i].a >> p[i].t;
    }

    sort(p + 1, p + n + 1, [&](node &x, node &y) {
        return x.t < y.t;
    });

    ll l = 1, r = n;
    ll ans = 0;

    while(l <= r) {
        ll mid = l + (r - l) / 2;
        if(check(mid)) {
            ans = mid;
            l = mid + 1;
        }
        else {
            r = mid - 1;
        }
    }

    vector<ll> w;

    for(ll i = 1; i <= n; i++) {
        if((ll)w.size() < ans && p[i].a >= ans) w.push_back(p[i].id);
    }

    cout << ans << '\n';
    cout << w.size() << '\n';

    for(ll id : w) cout << id << ' ';

    cout << '\n';

    return 0;
}
```

【BM06】原解直接收录 选最小可行值 CF2065C2

识别：每个位置可保持 a_i 或改成某个 b_j-a_i，问能否得到非降序列。

思路：原思路保留：当前选不小于 pre 的最小候选；b 排序后二分 pre+x。

证明：任意合法方案当前值不会小于贪心值，换成更小贪心值只会放松后面下界；归纳可保留所有可行后缀。

约定：原题 a_i,b_j 在 1..1e9，sum n 和 sum m 各<=2e5；原完整 solve+多测 main。

调用：独立程序输入 T，再每组 n,m,a,b；输出 YES 或 NO。原 solve 不需要额外初始化。

例子：a=[3,1],b=[4]：第一个变成 1，第二个保持 1，得到非降序列。

复杂度：O(mlogm+nlogm)时间，O(n+m)空间。

易错：b_j 可复用，不能当一次性资源删除。；必须同时比较保留 x 和转换候选。；原 4e18 哨兵和 pre+x 只在已写明输入范围下使用，不泛化成任意 ll 无溢出接口。

来源：cf/date/7/7.25/1.cpp

规范源码：competition/models/ordered_choice.cpp；完整独立代码如下；测试：tests/round02/test_model_tools.cpp

```
#include <bits/stdc++.h>
using namespace std;
using ll = long long;

/*
分析题目需要什么贪心就行
然后在这之后有个二分优化
*/

void solve() {
    ll n, m;
    cin >> n >> m;
    vector<ll> a(n), b(m);
    for(ll i = 0; i < n; i++) cin >> a[i];
    for(ll i = 0; i < m; i++) cin >> b[i];

    sort(b.begin(), b.end());

    ll pre = -4e18;

    ll f = 1;

    for(auto x : a) {
        ll k = 4e18; // k 为这个位置的目标值
        if(x >= pre) k = x;

        auto it = lower_bound(b.begin(), b.end(), pre + x);
        if(it != b.end()) {
            k = min(k, *it - x);
        }
        if(k == 4e18) {
            f = 0;
            break;
        }
        pre = k;
    }
    cout << (f ? "YES" : "NO") << '\n';
}


int main() {
    ios_base::sync_with_stdio(false);
    cin.tie(0);

    ll T = 1;
    cin >> T;
    while(T--) solve();
    return 0;
}
```

【BM07】区间选择、首尾配对与括号判定

识别：互不重叠的区间取最多；偶数元素配对使最大配对和最小；检查单类括号串合法性。

思路：最早结束给后续留下最多空间；大数配小数可用交换论证；括号用前缀余额表达可匹配性。

证明：把最优区间集第一段替成最早结束者不减后续容量；把最小与最大元素配对，两次原配对交换后最大和不增；括号每个右括号必须有尚未匹配的左括号。

约定：区间和数列均 1 下标、0 号占位。区间是非空半开[l,r)，l<r，端点相接允许同时选。配对要求偶数个 ll，所有元素恰用一次，返回 i128 并输出数值对；空数组约定 0。括号只接受()，空串合法。

调用：interval_schedule(seg)返回原下标；minmax_pair_sum(a,pairs)返回最优最大和并清空重写 pairs；regular_brackets(s)返回 bool。具体有效调用均在 tests/round02/test_greedy.cpp。

例子：区间[1,10),[2,3),[3,4),[4,5)最多选后 3 段；[-5,-2,3,8]配(-5,8)、(-2,3)，最优最大和 3；)(不合法，(()())合法。

复杂度：区间选择与配对 O(n log n)时间、O(n)空间；括号 O(n)时间、O(1)空间。

易错：闭区间端点相接仍重叠时，不能直接用本函数的>=判定。；按区间长度或起点贪心不保证数量最多；加权区间选择需另建 DP。；配对只对这个目标证明成立，不能据此解决任意成本函数；全负数组最大和不能初始化后一直取 max(0,sum)。；()总数相等不是充分条件，)(在第一步就违法；多种括号要用类型栈。

来源：template/competition/models/greedy_construct.hpp

规范源码：competition/models/greedy_construct.hpp；完整独立代码如下；测试：tests/round02/test_greedy.cpp

```
#include <bits/stdc++.h>
using namespace std;
using ll = long long;
using i128 = __int128_t;

// seg[1..n]为非空半开区间[l,r)，l<r；端点相接允许同时选择。
// 返回数量最多的不相交区间原下标，按被选择的时间顺序排列。
vector<int> interval_schedule(const vector<pair<ll,ll>>& seg) {
    assert(!seg.empty());
    int n = (int)seg.size() - 1;
    vector<int> p(n);
    iota(p.begin(), p.end(), 1);
    for(int i = 1; i <= n; i++) assert(seg[i].first < seg[i].second);
    sort(p.begin(), p.end(), [&](int x, int y) {
        return tie(seg[x].second, seg[x].first, x)
             < tie(seg[y].second, seg[y].first, y);
    });
    vector<int> ans;
    for(int id : p) {
        if(ans.empty() || seg[id].first >= seg[ans.back()].second)
            ans.push_back(id);
    }
    return ans;
}

// a[1..n]，n 为偶数，所有元素恰好配对；最小化最大配对和。
// pairs 输出数值对（重复值保留）；空数组约定答案为 0。和用 i128。
i128 minmax_pair_sum(vector<ll> a, vector<pair<ll,ll>>& pairs) {
    assert(!a.empty() && (a.size() - 1) % 2 == 0);
    sort(a.begin() + 1, a.end());
    pairs.clear();
    int n = (int)a.size() - 1;
    i128 ans = 0;
    for(int l = 1, r = n; l < r; l++, r--) {
        i128 sum = (i128)a[l] + a[r];
        if(pairs.empty() || sum > ans) ans = sum;
        pairs.push_back({a[l], a[r]});
    }
    return ans;
}

// 空串合法；只支持一类括号()，其他字符返回 false。
```

【BM07】区间选择、首尾配对与括号判定 续

```
bool regular_brackets(const string& s) {
    size_t balance = 0;
    for(char c : s) {
        if(c == '(') balance++;
        else if(c == ')') {
            if(balance == 0) return false;
            balance--;
        } else return false;
    }
    return balance == 0;
}
```

【BM08】整数状态 BFS 与非负子集回溯剪枝

识别：状态可以编码为有限整数，求最少操作并恢复路径；小 n 非负子集达到给定目标，练习选/不选回溯与安全剪枝。

思路：BFS 把重复操作序列合并到同一完整局面；回溯仍枚举选/不选，但用非负性和后缀总量排除不可能分支。

证明：BFS 按步数分层，首次访问状态给出最短距离，fa 与 from 逐边恢复合法路径。非负子集的 sum 超目标后不会下降，sum+剩余总和不足时任何选择都达不到目标，故两条剪枝不删可行解。

约定：整数状态 x 直接编码为 x，0<=start,target<=limit<=1e6；操作仅+1、-1、乘 2，结果须留在[0,limit]，每步费用 1。子集 a[1..n]、0 号占位，0<=n<=40、每项非负 ll、target>=0；按下标区分重复值，target=0 允许空集。

调用：auto p=integer_state_bfs(3,10,20)，p.states 首尾为起终点，p.operations 中的+/-/*分别表示+1/-1/乘 2；subset_target_nonnegative({0,3,3},6)返回含下标 1,2 的可行子集。

例子：3→6→5→10 为 3 步最短操作；起终点相同返回一个状态及空操作。{3,3}达到 6 必须用两个不同下标；{2,4}到 3 无解；空数组到 0 为空子集。

复杂度：整数 BFS O(limit)时间空间；子集最坏 O(2^n)时间、O(n)额外空间，剪枝无多项式保证。

易错：有限边界是题面模型的一部分，不能把无界整数问题随便截到某个 limit 后宣称全局最短。；未来还依赖钥匙/次数时，不能只编码一个整数位置；必须加状态维度。；含负数时 sum>target 后可能再下降，sum+后缀总和也不再是最大可达上界，不能用本剪枝。；n<=40 不是复杂度保证；最坏指数，普通 40 元素子集和优先 BM04 折半。

来源：template/competition/stl/recipes.hpp；template/competition/graph/bfs.hpp

规范源码：competition/models/state_search.hpp；完整独立代码如下；测试：tests/round02/test_search.cpp

```
#include <bits/stdc++.h>
using namespace std;
using ll = long long;
using i128 = __int128_t;

struct IntegerPath {
    vector<int> states;
    vector<char> operations; // '+'为+1，'-'为-1，'*'为乘 2。
};

// 整数局面 x 直接编码为数组下标 x；0<=start,target<=limit<=1e6。
// 每次+1/-1/乘 2，结果必须仍在[0,limit]；每步费用 1。
IntegerPath integer_state_bfs(int start, int target, int limit) {
    assert(0 <= limit && limit <= 1000000);
    assert(0 <= start && start <= limit && 0 <= target && target <= limit);
    vector<int> dist(limit + 1, -1), fa(limit + 1, -1);
    vector<char> from(limit + 1);
    queue<int> q;
    dist[start] = 0; q.push(start);
    while(!q.empty()) {
        int x = q.front(); q.pop();
        if(x == target) break;
        int next[3] = {x + 1, x - 1, x * 2};
        const char op[3] = {'+', '-', '*'};
        for(int k = 0; k < 3; k++) {
            int y = next[k];
            if(y < 0 || y > limit || dist[y] != -1) continue;
            dist[y] = dist[x] + 1; fa[y] = x; from[y] = op[k];
            q.push(y);
        }
    }
    IntegerPath ans;
    for(int x = target; x != start; x = fa[x]) {
        ans.states.push_back(x); ans.operations.push_back(from[x]);
    }
    ans.states.push_back(start);
    reverse(ans.states.begin(), ans.states.end());
    reverse(ans.operations.begin(), ans.operations.end());
    return ans; // +/-1 保证边界内任意两状态可达。
}

// a[1..n]非负、n<=40、target>=0；返回一个子集的原下标或 nullopt。
```

【BM08】整数状态 BFS 与非负子集回溯剪枝 续

```
// 重复值按不同位置保留；target=0 允许空集。最坏仍 O(2^n)。
optional<vector<int>> subset_target_nonnegative(const vector<ll>& a, ll target) {
    assert(!a.empty() && a.size() <= 41 && target >= 0);
    int n = (int)a.size() - 1;
    vector<i128> remain(n + 2);
    for(int i = n; i >= 1; i--) {
        assert(a[i] >= 0);
        remain[i] = remain[i + 1] + a[i];
    }
    vector<int> chosen;
    auto dfs = [&](auto&& self, int i, i128 sum) -> bool {
        if(sum > target || sum + remain[i] < target) return false;
        if(sum == target) return true;
        if(i > n) return false;
        chosen.push_back(i);
        if(self(self, i + 1, sum + a[i])) return true;
        chosen.pop_back();
        return self(self, i + 1, sum);
    };
    if(dfs(dfs, 1, 0)) return chosen;
    return nullopt;
}
```

【FP01】浮点误差、距离与反三角

识别：二维距离、向量夹角、方向角和数值比较。

思路：先写清数学定义域和量纲，再用 hypot、atan2 与夹紧消除常见数值边界错误。

证明：点积除以模长积是夹角余弦；atan2 同时利用 x/y 的符号确定象限。

约定：long double，输入与中间结果有限；几何向量非零且 hypot 结果有限；acos/asin 数学参数本来应在[-1,1]。EPS 默认 1e-12，需按误差预算调整。

调用：auto d=distance_2d(0,0,3,4); auto rad=angle_between(1,0,0,1); auto deg=degrees(rad); auto dir=direction_angle(0,-1);

例子：(0,0)到(3,4)距离 5；(1,0)与(0,1)夹角 π/2；(0,-1)方向 3π/2。floor(-1.2)=-2，ceil(-1.2)=-1，trunc(-1.8)=-1，round(-1.5)=-2。

复杂度：各函数 O(1)。

易错：atan2(y,x)不是 atan2(x,y)，零向量没有方向。；EPS 比较不满足严格弱序，不能作 sort/set 比较器。；float 常仅约 6~7 位有效数字；double 通常约 15 位；long double 精度依平台。；整数判等/奇偶/模运算应保留整数；大整数转浮点可能丢失末位。；sqrt 要求非负；仅对已证明应非负而出现微小误差的值夹 0。

来源：template/Ternary_Search.cpp

规范源码：competition/runtime/floating.hpp；完整独立代码如下；测试：tests/round02/test_runtime.cpp

```
#include <bits/stdc++.h>
using namespace std;

// EPS 随量级/误差预算选；排序比较器仍用精确 <，不要掺 EPS。
constexpr long double EPS = 1e-12L;
const long double PI = acosl(-1.0L);
inline bool fp_equal(long double a, long double b) {
    return fabsl(a - b) <= EPS * max({1.0L, fabsl(a), fabsl(b)});
}
inline long double radians(long double deg) { return deg * PI / 180; }
inline long double degrees(long double rad) { return rad * 180 / PI; }
inline long double normalize_angle(long double rad) {
    rad = fmodl(rad, 2 * PI);
    if (rad < 0) rad += 2 * PI;
    if (rad >= 2 * PI) rad = 0; // 极小负数加一周可能舍入成 2π。
    return rad;
}
// 只修正舍入造成的微小越界；原数学输入本来就必须在 [-1,1]。
inline long double safe_acos(long double x) {
    return acosl(clamp(x, -1.0L, 1.0L));
}
inline long double safe_asin(long double x) {
    return asinl(clamp(x, -1.0L, 1.0L));
}
inline long double distance_2d(long double x1, long double y1,
                               long double x2, long double y2) {
    return hypotl(x1 - x2, y1 - y2);
}
// (x,y) 非零；atan2 的参数是 (y,x)，返回方向角 [0,2π)。
inline long double direction_angle(long double x, long double y) {
    assert(x != 0 || y != 0);
    return normalize_angle(atan2l(y, x));
}
// 两个向量均非零且有限；先单位化，避免直接点积/模长乘积溢出。
inline long double angle_between(long double ax, long double ay,
                                 long double bx, long double by) {
    long double na = hypotl(ax, ay), nb = hypotl(bx, by);
    assert(na > 0 && nb > 0 && isfinite(na) && isfinite(nb));
    return safe_acos((ax / na) * (bx / nb) + (ay / na) * (by / nb));
}
```

【GAME01】正常规则、Nim 胜着与有限 DAG SG

识别：两人轮流、完全信息、可枚举合法后继；独立取石堆或可拆成独立公平子游戏。

思路：败态没有败态后继，胜态至少能走到一个败态；mex 为每个状态赋予可合并的 SG 值。

证明：Nim 异或非零时取最高为 1 的位所在堆，将 ai 改 ai xor sum 会严格变小并使总异或为 0；任何一步都会破坏原来的零异或。

约定：正常规则无棋可走者输；SG 图有限无环，状态编号保证后继 u<v。异或定理要求每步仅改变一个独立子游戏，双方可走集合相同，无随机。反常 Nim 函数仅用于取任意正石的 Nim 规则。

调用：auto [i,left]=nim_winning_move(a); if(i!=-1) a[i]=left; auto sg=sg_ordered_dag(moves); bool win=(sg[s1]^sg[s2])!=0;

例子：Nim {1,2,3}异或 0 必败；{1,2,4}可把 4 改 3。取{1,3,4}枚举 SG，用独立两堆直推 P/N 验证异或定理的使用。

复杂度：Nim O(堆数)；SG O(状态数+边数)，额外 O(状态数+最大出度)。

易错：SG=0 不是'没有可走'，而是所有可走后继都非 0。；奇偶不是通用判据：取 1 或 3 为奇偶；取 1 或 2 由模 3 决定。；有环可能平局，SG 递推不能直接调用；双方操作不同也不能套公平游戏 SG。；反常规则不能照搬正常 SG；反常 Nim 全空返回赢是明确的终局约定。

来源：template/mex.cpp

规范源码：competition/game/normal_play.hpp；完整独立代码如下；测试：tests/round02/test_game_string.cpp

```
#include <bits/stdc++.h>
using namespace std;

// 正常 Nim：每步选择一堆，取走任意正数；无棋可走者输。
inline unsigned long long nim_xor(const vector<unsigned long long>& a) {
    unsigned long long sum = 0;
    for (auto x : a) sum ^= x;
    return sum;
}
// 返回 {0-based 堆下标，操作后堆大小}；必败态返回 {-1,0}。
inline pair<int, unsigned long long>
nim_winning_move(const vector<unsigned long long>& a) {
    auto sum = nim_xor(a);
    if (!sum) return {-1, 0};
    for (int i = 0; i < int(a.size()); ++i)
        if ((a[i] ^ sum) < a[i]) return {i, a[i] ^ sum};
    return {-1, 0};
}

// 有限、无环、公平、无随机、完全信息、正常规则的有向状态图。
// 状态已按依赖编号：moves[v] 内每个 u 必须满足 0<=u<v。
// sg[v]=mex({sg[u]})；独立子游戏之和的 SG 才可异或。
inline vector<int> sg_ordered_dag(const vector<vector<int>>& moves) {
    int n = int(moves.size());
    vector<int> sg(n);
    for (int v = 0; v < n; ++v) {
        vector<bool> vis(moves[v].size() + 1);
        for (int u : moves[v]) {
            assert(0 <= u && u < v);
            if (sg[u] < int(vis.size())) vis[sg[u]] = true;
        }
        while (sg[v] < int(vis.size()) && vis[sg[v]]) ++sg[v];
    }
    return sg;
}

// 反常 Nim：拿最后一个的人输，仅限 Nim 取石规则。
// 若所有非空堆都是 1，偶数个 1 必胜；否则仍看 xor。
// 全空时返回 true：按反常约定，无棋可走者赢。
inline bool misere_nim_win(const vector<unsigned long long>& a) {
    int ones = 0;
    bool large = false;
    for (auto x : a) {
        ones += (x == 1);
        large |= (x > 1);
    }
    return large ? nim_xor(a) != 0 : ones % 2 == 0;
}
```

【IO01】原版 ll/getchar 快读

识别：读入格式已知、整数个数已知、需要快速读普通合法整数。

思路：沿用用户熟悉的 getchar 逐字符累积，遇到负号记录符号。

证明：每读入一位 d 后 x=10*x+d，循环结束时 x 等于数字绝对值；最后乘符号。

调用：ll n=read(); for(int i=1;i<=n;++i) a[i]=read();

例子：依次读取 0 -1 1 42 -20261024 1000000000000000000 -1000000000000000000，结果逐项相同。

复杂度：每个字符 O(1)，额外空间 O(1)。

来源：template/read.cpp

规范源码：read.cpp；完整独立代码如下；测试：tests/round02/test_runtime.cpp

```
#include <bits/stdc++.h>
using namespace std;
using ll = long long;


inline ll read() {
    ll x = 0, f = 1; char ch = getchar();
    while(ch < '0' || ch > '9') {if(ch == '-') f = -1; ch = getchar();}
    while(ch >= '0' && ch <= '9') {x = (x << 3) + (x << 1) + (ch ^ 48); ch = getchar();}
    return x * f;
}
```

【IO03】cin 字符串转 i128 与原输出函数

识别：输入数值超过 ll，但不超过有符号 128 位范围。

思路：用 cin 读十进制字符串，再逐位累加；输出直接复用 CO02。普通 ll 继续原 read。

证明：读完每个数字后 value 等于该十进制前缀；负数用-(value-1)-1 处理符号，不先对最小有符号值取负。

约定：s 非空，为可带一个前导+/-的合法十进制整数，至少一位数字，范围[-2^127,2^127-1]；允许前导 0。

调用：string s; cin >> s; __int128 x=parse_i128(s); cout << to_string_i128(x);

例子：输入+00017 和-42，输出 17 和-42；完整 main 和极值样例附在本模块。

复杂度：O(十进制位数)时间，解析器 O(1)额外空间；字符串占 O(位数)。

易错：cin 不能直接提取__int128，先读 string。；乘法扩大类型必须发生在运算前；不要先 ll 相乘再转 i128。；本函数只转换合法约定输入，不是通用文本校验器。

来源：template/core/basics.hpp

规范源码：competition/runtime/int128_input.hpp；完整独立代码如下；测试：tests/round02/test_int128_input.cpp

```
#include <bits/stdc++.h>
using namespace std;
using ll = long long;
// 数组 a[1..n]；{} 或 {0} 表示 n=0。所有 ll 中间结果须不溢出。
inline string to_string_i128(__int128 x) {
    bool neg = x < 0;
    // 无符号取反能处理 -2^127，禁止直接对最小有符号数取负。
    __uint128_t u = neg ? -(__uint128_t)x : (__uint128_t)x;
    string s;
    do { s += char('0' + u % 10); u /= 10; } while (u);
    if (neg) s += '-';
    reverse(s.begin(), s.end());
    return s;
}
struct PrefixSum {
    int n;
    vector<ll> pre;
    explicit PrefixSum(const vector<ll>& a)
        : n(max(0, int(a.size()) - 1)), pre(n + 1) {
        for (int i = 1; i <= n; ++i) pre[i] = pre[i - 1] + a[i];
    }
    ll sum(int l, int r) const {
        if (l > r) return 0; // 空区间；非空时 1<=l<=r<=n。
        assert(1 <= l && r <= n);
        return pre[r] - pre[l - 1];
    }
};
// 离线区间加；初值全 0。values() 得到 1 下标数组，可重复调用。
struct Difference {
    int n;
    vector<ll> d;
    explicit Difference(int size = 0) : n(size), d(size + 1) {
        assert(size >= 0);
    }
    void add(int l, int r, ll x) {
        if (l > r) return;
        assert(1 <= l && r <= n);
        d[l] += x;
        if (r < n) d[r + 1] -= x;
    }
    vector<ll> values() const {
        vector<ll> a(n + 1);
        for (int i = 1; i <= n; ++i) a[i] = a[i - 1] + d[i];
        return a;
    }
};
// 离散化保序不保距离。rank(x) 只接收初始化时存在的值。
struct Compression {
```

【IO03】cin 字符串转 i128 与原输出函数 续

```
    vector<ll> values; // values[rank] 为原值；values[0] 不使用。
    explicit Compression(const vector<ll>& a) {
        values.assign(a.begin() + min<size_t>(1, a.size()), a.end());
        sort(values.begin(), values.end());
        values.erase(unique(values.begin(), values.end()), values.end());
        values.insert(values.begin(), 0);
    }
    int rank(ll x) const {
        auto it = lower_bound(values.begin() + 1, values.end(), x);
        assert(it != values.end() && *it == x);
        return int(it - values.begin());
    }
};

// cin 先读字符串；s 为合法十进制整数，可带+/-，数值在有符号 i128 范围。
// 普通 ll 仍直接使用原 read()，这里只补 i128 的字符串转换。
inline __int128 parse_i128(const string& s) {
    int i = 0;
    bool neg = s[0] == '-';
    if (s[0] == '+' || s[0] == '-') ++i;
    __uint128_t value = 0;
    for (; i < int(s.size()); ++i) value = value * 10 + (s[i] - '0');
    if (neg && value) return -(__int128)(value - 1) - 1;
    return (__int128)value;
}
```

接在上述代码后面的完整 main（已独立运行样例）：

样例输入：+00017 / -42 / 170141183460469231731687303715884105727 / -170141183460469231731687303715884105728 /

样例输出：17 / -42 / 170141183460469231731687303715884105727 / -170141183460469231731687303715884105728 /

```
int main() {
    ios::sync_with_stdio(false);
    cin.tie(0);
    string s;
    while (cin >> s) {
        __int128 x = parse_i128(s);
        cout << to_string_i128(x) << '\n';
    }
}
```

【MEX01 / MEX02】静态与前缀 MEX

识别：问最小未出现的非负整数；只有插入的前缀序列。

思路：n 个元素不可能覆盖[0,n]全部 n+1 个候选，值域可截断；前缀 mex 只增。

证明：鸽巢原理给 mex<=n；vis 精确记录相关值，最小未标记位置即定义。

约定：vector<int>可含负数和很大值；静态 n 个数仅需看[0,n]。前缀不删除。

调用：int x=static_mex(a); auto pm=prefix_mex(a);

例子：{-1,0,0,2,INT_MAX}的 mex=1；{1,0,3,2}的前缀 mex 为 0,2,2,4。

复杂度：静态 O(n)时间空间；所有前缀合计 O(n)，因为 now 只向右移动。

易错：mex 从 0 开始；{1,2}的 mex 是 0。；重复值不增加覆盖长度；mex<=不同非负值数量<=元素数。；允许删除时 now 不能只增，需要 MEX02。

来源：template/mex.cpp；cf/3/1096/D_mex_set.cpp

规范源码：competition/game/mex.hpp；完整独立代码如下；测试：tests/round02/test_game_string.cpp

```
#include <bits/stdc++.h>
using namespace std;

// n 个元素的 mex 在 [0,n]；负数和大于 n 的值不影响答案。
inline int static_mex(const vector<int>& a) {
    int n = int(a.size());
    vector<bool> vis(n + 1);
    for (int x : a) if (0 <= x && x <= n) vis[x] = true;
    for (int x = 0; x <= n; ++x) if (!vis[x]) return x;
    return n + 1; // n 个元素不可能占满 n+1 个不同值。
}
inline vector<int> prefix_mex(const vector<int>& a) {
    int n = int(a.size()), now = 0;
    vector<bool> vis(n + 1);
    vector<int> ans;
    for (int x : a) {
        if (0 <= x && x <= n) vis[x] = true;
        while (now <= n && vis[now]) ++now;
        ans.push_back(now);
    }
    return ans;
}

// 动态多重集合：全过程元素总数不超过 mex_limit。
// init(N) 建 [0,N] 缺失值；忽略区间外值；删值必须确实存在。
inline int mex_limit;
inline vector<int> cnt;
inline set<int> missing;
inline void mex_init(int limit) {
    assert(limit >= 0);
    mex_limit = limit;
    cnt.assign(limit + 1, 0);
    missing.clear();
    for (int x = 0; x <= limit; ++x) missing.insert(x);
}
inline void mex_add(int x) {
    if (x < 0 || x > mex_limit) return;
    if (cnt[x]++ == 0) missing.erase(x);
}
inline void mex_erase(int x) {
    if (x < 0 || x > mex_limit) return;
    assert(cnt[x] > 0);
    if (--cnt[x] == 0) missing.insert(x);
}
inline int mex_query() {
    assert(!missing.empty());
    return *missing.begin();
}
```

【PAL01】回文判定、中心扩展与构造

识别：连续回文子串的数量/最长长度，字符重排成回文，或 0/1/?给定数量填回文。

思路：回文由中心或成对镜像位置决定；每对消耗偶数次，最多一个中心消耗奇数次。

证明：每个非空回文子串有唯一奇/偶中心，扩展恰计一次；频次至多一个奇数是重排可行的充要条件。01?先传播被迫镜像，再用唯一中心处理奇数，余量成对放置。

约定：字符串按字节；区间 0-based 半开；回文数按非负标准十进制表示，禁止多位前导零。中心扩展 $O(n^2)$，不当成大规模线性算法。01?填法仅接收规定字符且 a+b 等于长度。

调用：bool ok=is_palindrome(s,0,int(s.size())); auto info=palindrome_centers(s); string longest=s.substr(info.l,info.r-info.l); auto p=palindrome_permutation(s); auto b=binary_palindrome_fill(s,zeros,ones);

例子：aaa 有 6 个回文子串，最长 3；aabb 可排 abba；aabc 不行；0?1?0 按计数与镜像约束填充；任意猜出的构造仍须逐位验证。

复杂度：双指针 O(n)；中心扩展 $O(n^2)$、额外 O(1)；排列/01?构造 O(n+256)。

易错：子串连续；子序列可跳过字符。中心扩展不能求最长回文子序列。；按位置计数不等于不同回文内容数，例如 aaa 有 6 个位置子串但 3 种内容。；字符串 010 是回文串，但不是本接口接受的标准十进制回文数。；哈希相等有碰撞风险；大量区间判定可引用 S02，严格无碰撞需其他方法。

来源：template/string/basic_string.hpp；cf/construction/1.cpp；cf/3/1096/D_mex_set.cpp

规范源码：competition/string/palindrome.hpp；完整独立代码如下；测试：tests/round02/test_game_string.cpp

```
#include <bits/stdc++.h>
#include <bits/stdc++.h>
using namespace std;
using ll = long long;

// 基础字符串均按字节处理；0-indexed，区间[l,r)，不是 Unicode 码点。
inline bool is_palindrome(const string& s, int l, int r) {
    assert(0 <= l && l <= r && r <= int(s.size()));
    for (--r; l < r; ++l, --r) if (s[l] != s[r]) return false;
    return true;
}
// 按单个字符分割，保留首尾及连续分隔符产生的空串。
inline vector<string> split_char(const string& s, char sep) {
    vector<string> parts;
    size_t start = 0;
    for (size_t i = 0; i <= s.size(); ++i)
        if (i == s.size() || s[i] == sep) {
            parts.push_back(s.substr(start, i - start));
            start = i + 1;
        }
    return parts;
}
inline string ascii_lower(string s) {
    for (char& c : s) if ('A' <= c && c <= 'Z') c = char(c - 'A' + 'a');
    return s;
}
inline vector<pair<char, int>> run_length(const string& s) {
    vector<pair<char, int>> groups;
    for (char c : s) {
        if (groups.empty() || groups.back().first != c)
            groups.push_back({c, 1});
        else ++groups.back().second;
    }
    return groups;
}
// 读含空格整行：getline(cin,s)；刚用 cin>>后先丢弃残留换行。
// cin.ignore(numeric_limits<streamsize>::max(),'\n');
// cin>>ws 会跳过全部前导空白；若空白有意义，不要使用它。
using namespace std;

struct PalindromeInfo {
    long long count; // 非空回文子串个数：按位置计数，不按内容去重。
    int l, r;        // 最长回文子串的 0-based 半开区间[l,r)。
```

【PAL01】回文判定、中心扩展与构造 续

```
};
// 奇中心(i,i)与偶中心(i,i+1)分别扩展；空串返回{0,0,0}。
inline PalindromeInfo palindrome_centers(const string& s) {
    PalindromeInfo ans{0, 0, 0};
    int n = int(s.size());
    auto extend = [&](int l, int r) {
        while (l >= 0 && r < n && s[l] == s[r]) {
            ++ans.count;
            if (r - l + 1 > ans.r - ans.l) { ans.l = l; ans.r = r + 1; }
            --l; ++r;
        }
    };
    for (int i = 0; i < n; ++i) { extend(i, i); extend(i, i + 1); }
    return ans;
}
// 重排成回文：最多一种字符出现奇数次；按字节，不按 Unicode 字符。
inline optional<string> palindrome_permutation(const string& s) {
    array<int, 256> freq{};
    for (unsigned char c : s) ++freq[c];
    string left, middle;
    for (int c = 0; c < 256; ++c) {
        left.append(freq[c] / 2, char(c));
        if (freq[c] % 2) middle += char(c);
    }
    if (middle.size() > 1) return nullopt;
    string right = left; reverse(right.begin(), right.end());
    return left + middle + right;
}
// 非负整数的标准十进制表示：单独 0 允许，其他前导零不允许。
inline bool is_decimal_palindrome(const string& s) {
    if (s.empty() || (s.size() > 1 && s[0] == '0')) return false;
    for (char c : s) if (c < '0' || c > '9') return false;
    return is_palindrome(s, 0, int(s.size()));
}

// 直接保留 cf/construction/1.cpp：镜像传播、扣已用计数、填中心/配对。
// s 仅含'0'/'1'/'?'，要恰好 a 个 0、b 个 1；失败返回 nullopt。
inline optional<string> binary_palindrome_fill(string s, int a, int b) {
    if (a < 0 || b < 0 || (long long)a + b != (long long)s.size())
        return nullopt;
    for (int times = 0; times < 2; ++times) {
        for (int i = 0; i < int(s.size()); ++i) {
            int j = int(s.size()) - i - 1;
            if (s[i] != '?') {
                if (s[j] == '?') s[j] = s[i];
                else if (s[i] != s[j]) return nullopt;
            }
        }
        reverse(s.begin(), s.end());
    }
    a -= int(count(s.begin(), s.end(), '0'));
    b -= int(count(s.begin(), s.end(), '1'));
    int ques = int(count(s.begin(), s.end(), '?'));
    bool empty_mid = s.size() % 2 == 1 && s[s.size() / 2] == '?';
    if (a < 0 || b < 0 || a + b != ques ||
        (empty_mid && a % 2 == 0 && b % 2 == 0)) return nullopt;
    if (a % 2 == 1 || b % 2 == 1) {
        int i = int(s.size()) / 2;
        if (s[i] != '?') return nullopt;
        s[i] = (a % 2 == 1 ? '0' : '1');
        if (a % 2 == 1) --a;
        else --b;
    }
    if (a % 2 == 1 || b % 2 == 1) return nullopt;
    for (int i = 0; i < int(s.size()); ++i) {
        if (s[i] == '?') {
            int j = int(s.size()) - i - 1;
```

【PAL01】回文判定、中心扩展与构造 续

```
            if (a > 0) { a -= 2; s[i] = s[j] = '0'; }
            else { b -= 2; s[i] = s[j] = '1'; }
        }
    }
    return s;
}
```

【PDP01】线性与网格：最大子段、路径计数、最小代价

识别：选择连续子段，或只能右下移动的网格；未来只依赖前一段末尾或上方/左方。

思路：先固定当前末尾或当前格，合并未来相同的历史；最优值取 max/min，路径数相加。

证明：最后一步分类互斥完备，且同一末尾的最优前缀替换不影响未来。

约定：max_subarray 输入 a[1..n]且 n>=1，必须非空；grid 输入行列均 1-indexed 且 n,m>=1，'#'障碍。mod 在 [1,INT_MAX]；任意子段/路径和绝对值<2^60。GRID_INF 表示无路，格点费用含起终点。

调用：#include "competition/dp/linear_grid.hpp"
int main() {
  vector<string> g = {"", " ..", " .."};
  vector<vector<ll>> a = {{}, {0, 1, 2}, {0, 3, 4}};
  cout << grid_paths(g) << ' ' << grid_min_path(a, g) << '\n'; // 2 7
}

例子：DPC01：[-8,-2,-5]返回-2。DPC02：2x2 无障碍两条路径；点权 1,2/3,4 的最小含点费用为 7。

复杂度：最大子段 O(n)/O(1)；网格 O(nm)/O(m)。

易错：非空全负子段不能让 0 参与答案。；网格障碍必须清空滚动状态；允许回走时失去该依赖顺序。；无法到达的 INF 先判断，再加费用。

来源：cf/date/6/6.9/3_leaner_dp.cpp；cf/date/5/5.22/2.cpp

规范源码：competition/dp/linear_grid.hpp；完整独立代码如下；测试：tests/round02/test_dp.cpp

```
#include <bits/stdc++.h>
using namespace std;
using ll = long long;

inline constexpr ll GRID_INF = 1LL << 60;

// a[1..n], n>=1；选非空连续子段，任意子段和绝对值<2^60。
inline ll max_subarray(const vector<ll>& a) {
    assert(a.size() >= 2);
    ll dp = a[1], ans = a[1];
    for (int i = 2; i < int(a.size()); ++i) {
        // dp：必须以当前位置结尾；只能自己开头或接前一段。
        dp = max(a[i], dp + a[i]);
        ans = max(ans, dp);
    }
    return ans;
}

// g[1..n][1..m]，行列均从 1 开始；'#'不能走，只能向右/下。
// n,m>=1；mod>=1。dp[j]更新前是上方，dp[j-1]是左方。
inline int grid_paths(const vector<string>& g, int mod = 1000000007) {
    assert(g.size() >= 2 && g[1].size() >= 2 && mod >= 1);
    int n = int(g.size()) - 1, m = int(g[1].size()) - 1;
    vector<ll> dp(m + 1);
    dp[1] = 1 % mod;
    for (int i = 1; i <= n; ++i) {
        assert(int(g[i].size()) == m + 1);
        for (int j = 1; j <= m; ++j) {
            if (g[i][j] == '#') dp[j] = 0;
            else dp[j] = (dp[j] + dp[j - 1]) % mod;
        }
    }
    return int(dp[m]);
}

// 路径代价含起终点；a 也 1-indexed，可有负数。
// 任意路径和绝对值<2^60；无路返回 GRID_INF。
inline ll grid_min_path(const vector<vector<ll>>& a,
                        const vector<string>& g) {
    assert(g.size() >= 2 && g[1].size() >= 2 && a.size() == g.size());
    int n = int(g.size()) - 1, m = int(g[1].size()) - 1;
```

【PDP01】线性与网格：最大子段、路径计数、最小代价 续

```
    vector<ll> dp(m + 1, GRID_INF);
    dp[1] = 0;
    for (int i = 1; i <= n; ++i) {
        assert(int(g[i].size()) == m + 1 && int(a[i].size()) == m + 1);
        for (int j = 1; j <= m; ++j) {
            ll best = min(dp[j], dp[j - 1]);
            dp[j] = g[i][j] == '#' || best == GRID_INF
                  ? GRID_INF : best + a[i][j];
        }
    }
    return dp[m];
}
```

【PDP02】多重背包与分组背包

识别：有限数量重复物品，或同组互斥的容量最优化。

思路：多重二进制拆包后复用原 0/1；分组保留上一组快照，避免选中同组两个物品。

证明：拆包完整表示 0..cnt 的每个数量；按组转移只从上一组进入，严格满足互斥。

约定：数组和组从 1 开始；w>0，cnt>=0，cap>=0。exact=false 至多、true 恰好；不可达 KNAP_NEG。每组最多选一个，允许不选。任意可达总价值绝对值<2^60。

调用：#include "competition/dp/knapsack_variants.hpp"
int main() {
  auto dp = knapsack_bounded({0, 2}, {0, 3}, {0, 3}, 7);
  cout << dp[7] << '\n'; // 9
}

例子：DPC05：重 2 值 3 最多 3 件、容量 7，答案 9。DPC06：组{(2,3),(3,5)}与{(2,4)}容量 5，答案 9。

复杂度：多重 O(cap*Σlog(cnt+1))，计数先截到 cap/w；分组 O(cap*(总物品数+组数))；DP 空间 O(cap)。

易错：二进制拆分可能重复表示件数，不能照搬来计数。；组内逐件原地 0/1 更新会取多个同组物品。；每组恰选一个与最多一个不同，本函数不提供前者。

来源：template/dp/knapsack.hpp

规范源码：competition/dp/knapsack_variants.hpp；完整独立代码如下；测试：tests/round02/test_dp.cpp

```
#include <bits/stdc++.h>
using namespace std;
using ll = long long;

// w、v 为 1-indexed; w[i]>0, cap>=0, 价值及可达总和绝对值<2^60。
// exact=false: 总重量<=j; exact=true: 总重量恰好为 j。
// 不可达为 KNAP_NEG; 不能直接对不可达状态加价值。
inline constexpr ll KNAP_NEG = -(1LL << 60);
inline vector<ll> knapsack01(const vector<int>& w,
                             const vector<ll>& v, int cap,
                             bool exact = false) {
    assert(cap >= 0 && !w.empty() && w.size() == v.size());
    vector<ll> dp(cap + 1, exact ? KNAP_NEG : 0);
    dp[0] = 0;
    for (int i = 1; i < int(w.size()); ++i) {
        assert(w[i] > 0);
        // 倒序: dp[j-w[i]] 仍属于上一轮，物品最多取一次。
        for (int j = cap; j >= w[i]; --j)
            if (dp[j - w[i]] != KNAP_NEG)
                dp[j] = max(dp[j], dp[j - w[i]] + v[i]);
    }
    return dp;
}

inline vector<ll> knapsack_complete(const vector<int>& w,
                                    const vector<ll>& v, int cap,
                                    bool exact = false) {
    assert(cap >= 0 && !w.empty() && w.size() == v.size());
    vector<ll> dp(cap + 1, exact ? KNAP_NEG : 0);
    dp[0] = 0;
    for (int i = 1; i < int(w.size()); ++i) {
        assert(w[i] > 0);
        // 正序: dp[j-w[i]] 可以已经取过本物品，允许继续取。
        for (int j = w[i]; j <= cap; ++j)
            if (dp[j - w[i]] != KNAP_NEG)
                dp[j] = max(dp[j], dp[j - w[i]] + v[i]);
    }
    return dp;
}
// auto best = knapsack01(w, v, capacity, true);
// best[capacity]==KNAP_NEG 表示不能恰好装满；否则就是最大价值。
```

【PDP02】多重背包与分组背包 续

```
// w/v/cnt 从 1 开始；w[i]>0，cnt[i]>=0，cap>=0。
// 契约沿用 DP01：价值与任意可达总价值绝对值<2^60。
inline vector<ll> knapsack_bounded(const vector<int>& w,
                                   const vector<ll>& v,
                                   const vector<int>& cnt, int cap,
                                   bool exact = false) {
    assert(cap >= 0 && !w.empty());
    assert(w.size() == v.size() && w.size() == cnt.size());
    vector<int> a(1);
    vector<ll> b(1);
    for (int i = 1; i < int(w.size()); ++i) {
        assert(w[i] > 0 && cnt[i] >= 0);
        int rest = min(cnt[i], cap / w[i]);
        // 二进制拆成 0/1 包；最后剩余的一包使 0..cnt 全部可表示。
        for (ll k = 1; rest > 0; k *= 2) {
            int take = int(min<ll>(k, rest));
            a.push_back(w[i] * take);
            b.push_back(v[i] * take);
            rest -= take;
        }
    }
    return knapsack01(a, b, cap, exact); // 复用已验证 DP01
}

// groups[1..n]；每组若干{重量,价值}，最多选一个，也能整组不选。
// 组内空集合合法；必须从 old 转移，不能读本组刚更新的 dp。
inline vector<ll> knapsack_grouped(
        const vector<vector<pair<int, ll>>>& groups, int cap,
        bool exact = false) {
    assert(!groups.empty() && cap >= 0);
    vector<ll> dp(cap + 1, exact ? KNAP_NEG : 0);
    dp[0] = 0;
    for (int i = 1; i < int(groups.size()); ++i) {
        auto old = dp;
        for (auto [w, v] : groups[i]) {
            assert(w > 0);
            for (int j = w; j <= cap; ++j)
                if (old[j - w] != KNAP_NEG)
                    dp[j] = max(dp[j], old[j - w] + v);
        }
    }
    return dp;
}
```

【PDP03】序列前缀与个人状态机、候选分层 DP

识别：LCS、单位代价编辑距离、先右后左匹配、多个字符串各选严格递增子序列候选。

思路：两个前缀存双下标；方向存 dp0/dp1；候选已排序时用双指针累积上一层可用前缀最大值。

证明：状态保存了影响未来的末尾/阶段；每个目标候选可用的前驱集合随字典序递增而单调扩大。

约定：LCS/编辑距离字符串可空；编辑操作单位代价。方向匹配最多一次右转左，空目标 true。候选链 a[1..n]，每串长度 <=15，选非空子序列，字典序严格递增；空链返回 0，无法完成返回-1。

调用：#include "dp/lis.hpp"
#include "competition/dp/sequence_state.hpp"
int main() {
  cout << lis_length({0, 3, 1, 2, 2}) << ' '
    << lcs_length("abc", "ac") << ' '
    << edit_distance("kitten", "sitting") << ' '
    << lex_subsequence_chain({"", "ab", "bc"}) << '\n'; // 2 2 3 4
}

例子：DPC12 输出 LIS=2、LCS(abc,ac)=2、编辑距离 kitten/sitting=3、候选链 ab/bc=4；DPC13 abc 可右后左读 abcba。

复杂度：LCS/编辑距离 O(nm)时间空间；方向匹配 O(nm)时间 O(n)空间；候选链每层枚举 O(L2^L)、排序 O(L2^L log2^L)，再对相邻层线性扫描。

易错：LCS 是子序列，不是连续子串。；方向更新必须新层，不能二次转向。；字典序条件严格<；候选去重且不可达状态不加长度。

来源：cf/date/7/7.9/1_interesting_dp.cpp；cf/ICPC/JAKARTA_2018/J.cpp

规范源码：competition/dp/sequence_state.hpp；完整独立代码如下；测试：tests/round02/test_dp.cpp

```
#include <bits/stdc++.h>
using namespace std;
using ll = long long;

// dp[i][j]：s 的前 i 个字符、t 的前 j 个字符的最长公共子序列长度。
inline int lcs_length(const string& s, const string& t) {
    int n = int(s.size()), m = int(t.size());
    vector<vector<int>> dp(n + 1, vector<int>(m + 1));
    for (int i = 1; i <= n; ++i)
        for (int j = 1; j <= m; ++j) {
            if (s[i - 1] == t[j - 1]) dp[i][j] = dp[i - 1][j - 1] + 1;
            else dp[i][j] = max(dp[i - 1][j], dp[i][j - 1]);
        }
    return dp[n][m];
}

// 插入、删除、替换各花 1；dp[i][j]：s 前 i 个变成 t 前 j 个的最少操作。
inline int edit_distance(const string& s, const string& t) {
    int n = int(s.size()), m = int(t.size());
    vector<vector<int>> dp(n + 1, vector<int>(m + 1));
    for (int i = 0; i <= n; ++i) dp[i][0] = i;
    for (int j = 0; j <= m; ++j) dp[0][j] = j;
    for (int i = 1; i <= n; ++i)
        for (int j = 1; j <= m; ++j)
            dp[i][j] = min({dp[i - 1][j] + 1, dp[i][j - 1] + 1,
                dp[i - 1][j - 1] + (s[i - 1] != t[j - 1])});
    return dp[n][m];
}

// 从 s 任意位置开始，只能先向右、再向左，最多转向一次，匹配 t。
// 保留 cf/date/7/7.9/1_interesting_dp.cpp 的 dp0/dp1 递推。
// 空 t 约定可匹配；字符串下标遵从原稿 0-indexed。
inline bool right_then_left_match(const string& s, const string& t) {
    if (t.empty()) return true;
    int n = int(s.size()), m = int(t.size());
```

【PDP03】序列前缀与个人状态机、候选分层 DP 续

```
    vector<int> dp0(n), dp1(n); // 尚未向左 / 已经向左
    for (int i = 0; i < n; ++i) dp0[i] = s[i] == t[0];
    for (int j = 1; j < m; ++j) {
        vector<int> ndp0(n), ndp1(n);
        for (int i = 0; i < n; ++i) {
            if (dp0[i]) {
                if (i + 1 < n && s[i + 1] == t[j]) ndp0[i + 1] = 1;
                if (i > 0 && s[i - 1] == t[j]) ndp1[i - 1] = 1;
            }
            if (dp1[i] && i > 0 && s[i - 1] == t[j]) ndp1[i - 1] = 1;
        }
        dp0 = move(ndp0); dp1 = move(ndp1);
    }
    for (int i = 0; i < n; ++i) if (dp0[i] || dp1[i]) return true;
    return false;
}

// 保留 JAKARTA_2018/J.cpp：非空子序列排序去重 + 前缀最大值。
// 单个字符串长度<=15；每个候选按字典序严格递增，最大化总长度。
inline vector<string> nonempty_subsequences(const string& s) {
    int n = int(s.size());
    assert(n <= 15);
    vector<string> v;
    for (int mask = 1; mask < (1 << n); ++mask) {
        string t;
        for (int i = 0; i < n; ++i) if (mask >> i & 1) t += s[i];
        v.push_back(t);
    }
    sort(v.begin(), v.end());
    v.erase(unique(v.begin(), v.end()), v.end());
    return v;
}
inline ll lex_subsequence_chain(const vector<string>& a) {
    assert(!a.empty()); // a[1..n]；无对象传{""}。
    if (a.size() == 1) return 0;
    const ll NEG = -(1LL << 60);
    auto pre = nonempty_subsequences(a[1]);
    vector<ll> dp(pre.size());
    for (int i = 0; i < int(pre.size()); ++i) dp[i] = pre[i].size();
    for (int i = 2; i < int(a.size()); ++i) {
        auto cur = nonempty_subsequences(a[i]);
        vector<ll> ndp(cur.size(), NEG);
        int p = 0;
        ll best = NEG;
        for (int j = 0; j < int(cur.size()); ++j) {
            while (p < int(pre.size()) && pre[p] < cur[j]) {
                best = max(best, dp[p]);
                ++p;
            }
            if (best != NEG) ndp[j] = best + ll(cur[j].size());
        }
        pre = move(cur); dp = move(ndp);
    }
    ll ans = NEG;
    for (ll x : dp) ans = max(ans, x);
    return ans == NEG ? -1 : ans;
}
```

【PDP04】个人数位 DP 与基础有限状态计数

识别：区间数字统计非零位不超过 3；或统计无相邻 1 的定长二进制串。

思路：数字只有 19 位，非 limit 相同后缀可复用；计数按互斥的末位类别相加。

证明：补齐前导零后每个整数有唯一路径；不贴上界的同 pos/cnt 拥有相同可行后缀。

约定：digit_range 要求 0<=l<=r<=LLONG_MAX，0 计入。全局 s/dp 每次 calc 清空；非重入。binary_no_adjacent 要求 n>=0、mod∈[1,INT_MAX]，空串计 1。

调用：#include "competition/dp/digit_count.hpp"
int main() {
  cout << digit_range(1000, 1111) << '\n'; // 111
}

例子：DPC11：[1000,1111]有 111 个符合数；DPC14：长度 3 无相邻 1 有 5 种。

复杂度：数位 O(19*4*10)时间，O(19*4)缓存；二进制计数 O(n)时间 O(1)空间。

易错：count>3 先返回，避免访问 dp[][4]。；换上界清缓存；数 0 是否应扣掉由题意决定。；binary 计数新层不能读已更新的当前层；mod=1 答案 0。

来源：cf/date/8/1/2_digit_dp.cpp

规范源码：competition/dp/digit_count.hpp；完整独立代码如下；测试：tests/round02/test_dp.cpp

```
#include <bits/stdc++.h>
using namespace std;
using ll = long long;

// 原 date/8/1/2_digit_dp.cpp 的全局 s/dp 与递归 lambda 保持。
// 数 0 也算；最多 3 个非零数位。每次上界变化都清缓存。
inline string s;
inline ll dp[20][4];
inline ll digit_calc(ll x) {
    if (x < 0) return 0;
    s = to_string(x);
    memset(dp, -1, sizeof(dp));
    auto dfs = [&](auto&& self, int pos, int cnt, bool limit) -> ll {
        if (cnt > 3) return 0;
        if (pos == int(s.size())) return 1;
        if (!limit && dp[pos][cnt] != -1) return dp[pos][cnt];
        int up = limit ? s[pos] - '0' : 9;
        ll ans = 0;
        for (int d = 0; d <= up; ++d)
            ans += self(self, pos + 1, cnt + (d != 0),
                        limit && d == up);
        if (!limit) dp[pos][cnt] = ans;
        return ans;
    };
    return dfs(dfs, 0, 0, true);
}
inline ll digit_range(ll l, ll r) {
    assert(0 <= l && l <= r); // 0<=l<=r<=LLONG_MAX，l-1 不会溢出。
    return digit_calc(r) - digit_calc(l - 1);
}

// 长度 n 的 01 串，不允许相邻两个 1；空串计 1 种，模 mod>=1。
// dp[0/1]：当前前缀末位为 0/1 的方案数，空前缀放入状态 0。
inline int binary_no_adjacent(int n, int mod = 1000000007) {
    assert(n >= 0 && mod >= 1);
    if (mod == 1) return 0;
    array<ll, 2> f = {1 % mod, 0};
    for (int i = 0; i < n; ++i) {
        array<ll, 2> ndp = {(f[0] + f[1]) % mod, f[0]};
        f = ndp;
    }
    return int((f[0] + f[1]) % mod);
}
```

【PDP05】树形 DP：最大权独立集

识别：树上选点，父子不可同时选，求总点权最大。

思路：把子树压成根不选/根选择两个最优值，父节点只需读取这两个边界状态。

证明：固定父亲是否选后各儿子子树互不影响，可以分别选最优再求和。

约定：g 和 a 从 1 开始，g 为无向简单连通树，n>=1，root∈[1,n]；允许空集，sum(abs(a))<2^60。

调用：#include "competition/dp/tree_independent_set.hpp"
int main() {
  vector<vector<int>> g = {{}, {2,3}, {1}, {1}};
  cout << tree_independent_set(g, {0,5,4,3}) << '\n'; // 7
}

例子：根权 5、两个孩子权 4 和 3，选两个孩子得到 7；20 万节点单位权链得到 100000。

复杂度：O(n)时间、O(n)空间；迭代父序逆序不消耗递归栈。

易错：不是一般图最大独立集；有环不能使用。；全负返回 0 因为允许空集；必须非空需另加语义。；若还有跨子树限制，两个状态不再充分。

来源：template/tree_dp.cpp；cf/3/1096/H_!!dp_tree.cpp

规范源码：competition/dp/tree_independent_set.hpp；完整独立代码如下；测试：tests/round02/test_dp.cpp

```
#include <bits/stdc++.h>
using namespace std;
using ll = long long;

// g、a 从 1 开始，g 为无向简单连通树；n>=1。允许不选任何点。
// 相邻点不能都选，求最大权和；sum(abs(a[i]))<2^60。
// 按原 tree_dp.cpp“儿子先算，再合并父亲”的思路补完整状态。
inline ll tree_independent_set(const vector<vector<int>>& g,
                               const vector<ll>& a, int root = 1) {
    int n = int(g.size()) - 1;
    assert(n >= 1 && g.size() == a.size() && 1 <= root && root <= n);
    vector<int> fa(n + 1), order = {root};
    fa[root] = -1;
    for (int i = 0; i < int(order.size()); ++i) {
        int u = order[i];
        for (int v : g[u]) if (v != fa[u]) {
            assert(1 <= v && v <= n && fa[v] == 0);
            fa[v] = u;
            order.push_back(v);
        }
    }
    assert(int(order.size()) == n);
    vector<array<ll, 2>> dp(n + 1);
    for (int i = n - 1; i >= 0; --i) {
        int u = order[i];
        dp[u][1] = a[u]; // 选 u；dp[u][0]不选 u。
        for (int v : g[u]) if (fa[v] == u) {
            dp[u][0] += max(dp[v][0], dp[v][1]);
            dp[u][1] += dp[v][0];
        }
    }
    return max(dp[root][0], dp[root][1]);
}
```

【PDP06】独立试验概率与阶段等待期望

识别：独立伯努利成功次数分布；按顺序阶段重复尝试，失败留原阶段、成功前进。

思路：概率按最后一次成败加权求和；期望先列包含自环的全方程，再移项。

证明：全概率公式分类互斥完备；条件期望失败后回到相同阶段，解线性式得 E[i]=1/p[i]+E[i+1]。

约定：p[1..n]均有限。分布要求独立且 p∈[0,1]；等待要求每次独立、同阶段 p 固定且 p∈(0,1]，Σ1/p 有限且落在 long double 范围。空试验分布{1}，空阶段期望 0。

调用：#include "competition/dp/probability_expectation.hpp"
int main() {
  auto dp = coin_heads_probability({0, 0.5L, 0.5L});
  cout << fixed << setprecision(6) << dp[1] << ' '
    << expected_stage_attempts({0, 0.5L, 1.0L}) << '\n'; // 0.500000 3.000000
}

例子：两枚公平硬币恰一正面概率 0.5；阶段成功率 0.5 与 1，平均尝试次数 3。

复杂度：概率 O(n²)时间、O(n)空间；期望 O(n)时间、O(n)空间。

易错：不是所有随机过程都独立，条件概率变化时必须加状态。；p=0 等待期望无限，禁止传入有限期望函数。；概率相加需权重，不能平均后继，也不能取最小值替代期望。

规范源码：competition/dp/probability_expectation.hpp；完整独立代码如下；测试：tests/round02/test_dp.cpp

```
#include <bits/stdc++.h>
using namespace std;

// p[1..n]是相互独立的伯努利试验成功概率，0<=p[i]<=1 且有限。
// 返回 dp[k]：恰有 k 次成功的概率；空试验 dp[0]=1。
inline vector<long double> coin_heads_probability(
        const vector<long double>& p) {
    assert(!p.empty());
    int n = int(p.size()) - 1;
    vector<long double> dp(n + 1);
    dp[0] = 1;
    for (int i = 1; i <= n; ++i) {
        assert(isfinite(p[i]) && 0 <= p[i] && p[i] <= 1);
        for (int j = i; j >= 0; --j)
            dp[j] = dp[j] * (1 - p[i]) + (j ? dp[j - 1] * p[i] : 0);
    }
    return dp;
}

// 按顺序完成 n 阶段。阶段 i 每次独立尝试成功概率固定为 p[i]，
// 成功去 i+1，失败留 i，每次耗时 1。要求 0<p[i]<=1，sum(1/p)有限。
// E[i]=1+(1-p[i])*E[i]+p[i]*E[i+1]，移项得到下式。
inline long double expected_stage_attempts(const vector<long double>& p) {
    assert(!p.empty());
    int n = int(p.size()) - 1;
    vector<long double> dp(n + 2);
    for (int i = n; i >= 1; --i) {
        assert(isfinite(p[i]) && 0 < p[i] && p[i] <= 1);
        dp[i] = dp[i + 1] + 1 / p[i];
    }
    return dp[1];
}
```

【PG01】树的存储、多测与根化

识别：n 个点 n-1 条无向边；先建立树，再求父亲、深度、子树大小或树 DP。

思路：原 vector<ll> g[N]双向存边；显式栈给父亲和深度，逆序把 sz 加给父亲。

证明：连通无向图有 n-1 条边当且仅当是树。入栈时标记保证每点唯一父亲；父先子后的顺序逆转后，所有孩子先于父亲完成累加。

约定：1<=n<200010；g[1..n]是双向对称邻接表，端点合法。read_tree 读 n 及 n-1 条边并先 init。root_tree(root)非树返回 false；成功时根 fa=0、dep=0，sz 包括自己，order 父亲先于儿子。

调用：istringstream in("4 1 2 2 3 2 4");
read_tree(in);
bool ok=root_tree(1);
// ok && sz[2]==3

例子：边 1-2、2-3、2-4，根 1：fa[2]=1，dep[3]=2，sz[2]=3；换根 3 后应重做 root_tree。

复杂度：建图/根化 O(n)，空间 O(n)。

易错：每测先清 1..n，不能只清数组不清 g。；度为 1 是无根树叶子；有根树的根不能直接套此判断。；order 是父先子后的次序，不能直接当 Euler 闭区间；子树区间用 G10。

来源：template/tree_storage.cpp；cf/3/1096/H_!!dp_tree.cpp

规范源码：competition/graph/tree_storage.hpp；完整独立代码如下；测试：tests/round02/test_graph_personal.cpp

```
#include <bits/stdc++.h>
using namespace std;
using ll = long long;

// PG01. 原 tree_storage.cpp 的双向邻接表；点 1..n，1<=n<N。
const int N = 200010;
int n;
vector<ll> g[N];
ll fa[N], dep[N], sz[N];
vector<ll> order; // 父亲先于儿子；逆序可做子树 DP。

void init(int count) {
    n = count;
    for (int i = 1; i <= n; ++i) {
        g[i].clear();
        fa[i] = sz[i] = 0;
        dep[i] = -1;
    }
    order.clear();
}

void read_tree(istream& in = cin) {
    int count;
    in >> count;
    init(count);
    for (int i = 1; i <= n - 1; ++i) {
        ll u, v;
        in >> u >> v;
        g[u].push_back(v);
        g[v].push_back(u);
    }
}

// 双向对称的邻接表由调用方保证。非树返回 false。
bool root_tree(int root = 1) {
    order.clear();
    if (root < 1 || root > n) return false;
    ll edges = 0;
    for (int i = 1; i <= n; ++i) {
        fa[i] = 0; dep[i] = -1; sz[i] = 1;
        edges += (ll)g[i].size();
    }
```

【PG01】树的存储、多测与根化 续

```
    if (edges != 2LL * (n - 1)) return false;
    vector<ll> st{root};
    dep[root] = 0;
    while (!st.empty()) {
        ll u = st.back(); st.pop_back();
        order.push_back(u);
        for (ll v : g[u]) if (dep[v] == -1) {
            fa[v] = u; dep[v] = dep[u] + 1;
            st.push_back(v);
        }
    }
    if ((int)order.size() != n) return false;
    for (int i = n - 1; i > 0; --i) {
        ll u = order[i];
        sz[fa[u]] += sz[u];
    }
    return true;
}
```

【PG02】个人原版并查集 fa/rk

识别：只增加等价关系，查询两个位置是否连通；组件内置换、Kruskal 合并。

思路：先找根，把较低秩的根挂到较高秩根；同秩合并后新根秩加一。

证明：每次只合并两个连通块，find 路径压缩不改变根所属集合；按秩合并避免构造线性高度。

约定：0<=n<200010，调用点为 1..n；init/find/unite/same 沿用原接口。unite 是 void，rk 是秩不是集合大小。

调用：init(4);
unite(1,2);
unite(2,1);
bool ok=same(1,2);

例子：init(4); unite(1,2); unite(2,1)不改变集合；same(1,2)=true 而 same(1,3)=false。

复杂度：init O(n)，合并和查询均摊 O(alpha(n))，空间 O(n)。

易错：原注释“挂到秩小的那里”写反，算法本身正确。；不能按 rk 推集合大小；需要大小时另设 sz。；普通 DSU 不支持删边；不能直接套带撤销题。

来源：template/DSU.cpp；cf/xian/B.cpp；cf/3/1109/3.cpp

规范源码：competition/graph/dsu.hpp；完整独立代码如下；测试：tests/round02/test_graph_personal.cpp

```
#include <bits/stdc++.h>
using namespace std;
using ll = long long;

// PG02. 直接保留 DSU.cpp / cf/xian/B.cpp 的 fa/rk 写法。
// 1<=x<=n<N；rk 是秩，不是集合大小。多测重新 init(n)。
const ll N = 200010;
ll fa[N], rk[N];

void init(ll n) {
    for (ll i = 1; i <= n; i++) {
        fa[i] = i;
        rk[i] = 1;
    }
}

int find(ll x) {
    return fa[x] == x ? x : fa[x] = find(fa[x]);
}

void unite(ll x, ll y) {
    ll fx = find(x), fy = find(y);
    if (fx == fy) return;
    if (rk[fx] < rk[fy]) fa[fx] = fy;
    else {
        fa[fy] = fx;
        if (rk[fx] == rk[fy]) rk[fx]++;
    }
}

bool same(ll x, ll y) {
    return find(x) == find(y);
}
```

【PG03】原队列 BFS 与最短路径恢复

识别：每走一步代价相同，求最少步数或一条最短路径。

思路：保留 bfs.cpp 的 queue、入队即 visited；首次入队时写 dist 和 fa。

证明：FIFO 按层扩展；首次发现 v 时所有更短层都已扫描，因此 dist[u]+1 就是最短距离。

约定：1<=start<=n<200010；点 1..n，有向/无向均可；init 只清图，bfs 重置访问/距离/父亲。不可达 dist=-1；path 返回点序列。

调用：init(4);
g[1]={2};
g[2]={3};
bfs(1);
auto p=path(3);

例子：1->2->3、孤立 4：从 1 出发 dist[3]=2，path(3)=[1,2,3]，dist[4]=-1。

复杂度：O(n+m)时间、O(n+m)空间。

易错：出队才标记会重复入队。；带 0 权或不同边权不能继续用普通 BFS。；有向边不要自动反向 push。

来源：template/bfs.cpp；template/graph/bfs_path.hpp

规范源码：competition/graph/bfs.hpp；完整独立代码如下；测试：tests/round02/test_graph_personal.cpp

```
#include <bits/stdc++.h>
using namespace std;

// PG03. bfs.cpp 的队列/入队即标记；加 dist/fa 供最短路回溯。
const int N = 200010;
int n, dist[N], fa[N];
vector<int> g[N];
bool visited[N];

void init(int count) {
    n = count;
    for (int i = 1; i <= n; ++i) g[i].clear();
}

void bfs(int start) {
    fill(visited + 1, visited + n + 1, false);
    fill(dist + 1, dist + n + 1, -1);
    fill(fa + 1, fa + n + 1, 0);
    queue<int> q;
    visited[start] = true;
    dist[start] = 0;
    q.push(start);
    while (!q.empty()) {
        int u = q.front(); q.pop();
        for (int v : g[u]) {
            if (!visited[v]) {
                visited[v] = true;
                dist[v] = dist[u] + 1;
                fa[v] = u;
                q.push(v);
            }
        }
    }
}

vector<int> path(int t) {
    if (t < 1 || t > n || dist[t] == -1) return {};
    vector<int> ans;
    for (; t; t = fa[t]) ans.push_back(t);
    reverse(ans.begin(), ans.end());
    return ans;
}
```

【PG04】显式栈 DFS 与连通块

识别：无向图分组件、记录 DFS 父亲和遍历序；n 可达 20 万时避免深递归。

思路：dfs.cpp 每次沿下一条邻边递归，改为存(u,下一条邻边下标)的显式栈帧。

证明：每个栈帧与递归调用一一对应；visited 保证点只首次进入一次，扫完整个组件后再选新根。

约定：0<=n<200010，点 1..n；dfs_forest 重置状态并遍历所有组件。comp 从 1 开始，根 fa=0。有向输入只表示遍历森林，不能当强连通分量。

调用：init(4);
g[1]={2};
g[2]={1};
g[3]={4};
g[4]={3};
dfs_forest();

例子：边 1-2、3-4：dfs_forest 后 cnt=2，comp[1]=comp[2]，comp[1]!=comp[3]。

复杂度：O(n+m)时间和空间。

易错：只 dfs(1)会漏断图。；仅跳过父亲只适用于树，普通图要 visited。；树深度 20 万不依赖系统递归栈。

来源：template/dfs.cpp；template/graph/dfs_forest.hpp

规范源码：competition/graph/dfs.hpp；完整独立代码如下；测试：tests/round02/test_graph_personal.cpp

```
#include <bits/stdc++.h>
using namespace std;

// PG04. dfs.cpp 的标记后沿邻边访问；栈帧避免长链递归爆栈。
const int N = 200010;
int n, fa[N], comp[N], cnt;
vector<int> g[N], order;
bool visited[N];

void init(int count) {
    n = count;
    for (int i = 1; i <= n; ++i) g[i].clear();
}

void dfs(int start) {
    vector<pair<int, int>> st{{start, 0}};
    visited[start] = true; comp[start] = cnt;
    order.push_back(start);
    while (!st.empty()) {
        int u = st.back().first;
        int& i = st.back().second;
        if (i == (int)g[u].size()) { st.pop_back(); continue; }
        int v = g[u][i++];
        if (visited[v]) continue;
        visited[v] = true; comp[v] = cnt; fa[v] = u;
        order.push_back(v);
        st.push_back({v, 0});
    }
}

void dfs_forest() {
    fill(visited + 1, visited + n + 1, false);
    fill(fa + 1, fa + n + 1, 0);
    fill(comp + 1, comp + n + 1, 0);
    cnt = 0; order.clear();
    for (int s = 1; s <= n; ++s) if (!visited[s]) {
        ++cnt;
        dfs(s);
    }
}
```

【PG05】全局数组 Dijkstra

识别：稀疏图非负权单源/多源最短路，需要 dist/fa 与路径。

思路：ROUND01 堆松弛不变，把 Result 成员改为熟悉的 g/dist/fa 全局数组。

证明：最小距离候选出堆时，非负边不能通过尚未出堆点改出更短路径；过期距离快照跳过。

约定：0<=n<200010，点 1..n；0<=w<INF=LLONG_MAX/4；距离>=INF 视为不可达。dijkstra 返回 false 表示非法端点/源点/边权，false 后结果作废；合法空源集合全 INF。

调用：init(3);
g[1]={{2,3},{3,1}};
g[3]={{2,1}};
bool ok=dijkstra(1);
auto p=path(2);

例子：1->2 权 3，1->3 权 1，3->2 权 1：dist[2]=2，路径 1,3,2。

复杂度：堆优化 O((n+m)log(n+m))时间，O(n+m)空间。

易错：负边即使不在源点可达区也拒绝。；INF 不是任意大的真实距离；总路径长度须小于 INF。；等长不改父亲，避免零环导致回溯成环。

来源：template/graph/dijkstra.hpp

规范源码：competition/graph/dijkstra.hpp；完整独立代码如下；测试：tests/round02/test_graph_personal.cpp

```
#include <bits/stdc++.h>
using namespace std;
using ll = long long;

// PG05. graph/dijkstra.hpp 的同一松弛式，个人全局 g/dist/fa 版。
const int N = 200010;
const ll INF = LLONG_MAX / 4;
int n, fa[N];
ll dist[N];
vector<pair<int, ll>> g[N];

void init(int count) {
    n = count;
    for (int i = 1; i <= n; ++i) g[i].clear();
}

// 点 1..n；0<=w<INF；距离>=INF 视为不可达。
bool dijkstra(const vector<int>& sources) {
    fill(dist + 1, dist + n + 1, INF);
    fill(fa + 1, fa + n + 1, 0);
    for (int s : sources) if (s < 1 || s > n) return false;
    for (int u = 1; u <= n; ++u)
        for (auto [v, w] : g[u])
            if (v < 1 || v > n || w < 0 || w >= INF) return false;
    using State = pair<ll, int>;
    priority_queue<State, vector<State>, greater<State>> q;
    for (int s : sources) if (dist[s] != 0) {
        dist[s] = 0; q.push({0, s});
    }
    while (!q.empty()) {
        auto [du, u] = q.top(); q.pop();
        if (du != dist[u]) continue;
        for (auto [v, w] : g[u]) {
            if (w > INF - du) continue;
            if (du + w < dist[v]) {
                dist[v] = du + w; fa[v] = u;
                q.push({dist[v], v});
            }
        }
    }
    return true;
}
```

【PG05】全局数组 Dijkstra 续

```
bool dijkstra(int start) { return dijkstra(vector<int>{start}); }

vector<int> path(int t) {
    if (t < 1 || t > n || dist[t] == INF) return {};
    vector<int> ans;
    for (; t; t = fa[t]) ans.push_back(t);
    reverse(ans.begin(), ans.end());
    return ans;
}
```

【PG06】个人树上倍增 LCA

识别：同一棵树反复查询最近公共祖先、向上 k 步、两点边数。

思路：复用原 g/fa/dep/order，up[u][k]=2^k 级祖先。

证明：up 转移将两段 2^(k-1)跳跃拼接；先等深后从高位同时上跳，最后两点的父亲就是第一个共同祖先。

约定：依赖 PG01；1<=n<200010，树双向存储。build_lca(root)必须在建图或换根后调用；非树 false。jump 越根/非法点返回 0，distance 非法返回-1。LOG=20，扩大 N 时同步调整。

调用：init(4);
g[1]={2};
g[2]={1,3,4};
g[3]={2};
g[4]={2};
bool ok=build_lca(1);
int a=lca(3,4);

例子：1-2、2-3、2-4，根 1：lca(3,4)=2，jump(3,2)=1，distance(3,4)=2。

复杂度：预处理 O(n log n)，查询 O(log n)，空间 O(n log n)。

易错：根祖先为 0，不是根自身。；不能拿任意图的 DFS 树距离当原图最短路。；换根或重新 init 后必须重新 build_lca。

来源：template/graph/lca.hpp；template/tree_storage.cpp

规范源码：competition/graph/lca.hpp；完整独立代码如下；测试：tests/round02/test_graph_personal.cpp

```
#include <bits/stdc++.h>
using namespace std;
using ll = long long;

// PG01. 原 tree_storage.cpp 的双向邻接表；点 1..n，1<=n<N。
const int N = 200010;
int n;
vector<ll> g[N];
ll fa[N], dep[N], sz[N];
vector<ll> order; // 父亲先于儿子；逆序可做子树 DP。

void init(int count) {
    n = count;
    for (int i = 1; i <= n; ++i) {
        g[i].clear();
        fa[i] = sz[i] = 0;
        dep[i] = -1;
    }
    order.clear();
}

void read_tree(istream& in = cin) {
    int count;
    in >> count;
    init(count);
    for (int i = 1; i <= n - 1; ++i) {
        ll u, v;
        in >> u >> v;
        g[u].push_back(v);
        g[v].push_back(u);
    }
}

// 双向对称的邻接表由调用方保证。非树返回 false。
bool root_tree(int root = 1) {
    order.clear();
    if (root < 1 || root > n) return false;
    ll edges = 0;
    for (int i = 1; i <= n; ++i) {
        fa[i] = 0; dep[i] = -1; sz[i] = 1;
        edges += (ll)g[i].size();
```

【PG06】个人树上倍增 LCA 续

```
    }
    if (edges != 2LL * (n - 1)) return false;
    vector<ll> st{root};
    dep[root] = 0;
    while (!st.empty()) {
        ll u = st.back(); st.pop_back();
        order.push_back(u);
        for (ll v : g[u]) if (dep[v] == -1) {
            fa[v] = u; dep[v] = dep[u] + 1;
            st.push_back(v);
        }
    }
    if ((int)order.size() != n) return false;
    for (int i = n - 1; i > 0; --i) {
        ll u = order[i];
        sz[fa[u]] += sz[u];
    }
    return true;
}

// PG06. 复用 PG01 的 g/fa/dep/order，不重新存一份树。
const int LOG = 20; // N=200010 时足够；扩大 N 时同步调整。
int up[N][LOG];
bool lca_ready = false;

bool build_lca(int root = 1) {
    lca_ready = root_tree(root);
    if (!lca_ready) return false;
    fill(up[0], up[0] + LOG, 0);
    for (ll u : order) {
        up[u][0] = (int)fa[u];
        for (int k = 1; k < LOG; ++k)
            up[u][k] = up[up[u][k - 1]][k - 1];
    }
    return true;
}

int jump(int u, ll k) {
    if (!lca_ready || u < 1 || u > n || k < 0 || k > dep[u]) return 0;
    for (int i = 0; i < LOG; ++i) if (k >> i & 1) u = up[u][i];
    return u;
}

int lca(int u, int v) {
    if (!lca_ready || u < 1 || u > n || v < 1 || v > n) return 0;
    if (dep[u] < dep[v]) swap(u, v);
    u = jump(u, dep[u] - dep[v]);
    if (u == v) return u;
    for (int k = LOG - 1; k >= 0; --k) if (up[u][k] != up[v][k]) {
        u = up[u][k]; v = up[v][k];
    }
    return up[u][0];
}

ll distance(int u, int v) {
    int a = lca(u, v);
    return a ? dep[u] + dep[v] - 2 * dep[a] : -1;
}
```

【PG07】四邻接网格 BFS

识别：迷宫、棋盘上每步一格且代价都是 1。

思路：网格坐标就是点，合法四邻格就是隐式边，不必显式存邻接表。

证明：四邻扩展与单位边权图 BFS 等价；先边界检查再读取 g 避免越界。

约定：1<=n,m<1010；g 行、列均 1-indexed，每行前补空格；#障碍，其余可走。源点出界或障碍时全-1。

调用：n=2;
m=3;
g[1]=" ...";
g[2]=" .#.";
bfs(1,1);

例子：2x3 网格“...” “.#.”，从(1,1)到(2,3)最短 3 步；障碍格距离-1。

复杂度：O(nm)时间/空间。

易错：string 默认 0 下标，忘补空格会整体错位。；先判边界再访问 g[nx][ny]。；斜走、传送门、钥匙状态改变建边或状态，不能原样套。

来源：template/bfs.cpp

规范源码：competition/graph/grid_bfs.hpp；完整独立代码如下；测试：tests/round02/test_graph_personal.cpp

```
#include <bits/stdc++.h>
using namespace std;

// PG07. 矩形网格，g[1..n][1..m]，# 为障碍；四邻接单位代价。
const int N = 1010;
int n, m, dist[N][N];
string g[N]; // 每行读入后前面补一个空格。
const int dx[4] = {1, -1, 0, 0};
const int dy[4] = {0, 0, 1, -1};

void bfs(int sx, int sy) {
    for (int i = 1; i <= n; ++i)
        fill(dist[i] + 1, dist[i] + m + 1, -1);
    if (sx < 1 || sx > n || sy < 1 || sy > m || g[sx][sy] == '#')
        return;
    queue<pair<int, int>> q;
    dist[sx][sy] = 0; q.push({sx, sy});
    while (!q.empty()) {
        auto [x, y] = q.front(); q.pop();
        for (int k = 0; k < 4; ++k) {
            int nx = x + dx[k], ny = y + dy[k];
            if (nx < 1 || nx > n || ny < 1 || ny > m) continue;
            if (g[nx][ny] == '#' || dist[nx][ny] != -1) continue;
            dist[nx][ny] = dist[x][y] + 1;
            q.push({nx, ny});
        }
    }
}
```

【PG08】多源 BFS

识别：每个点距最近医院/出口/起火点的最少步数。

思路：把所有源同时入队，等价于一个虚拟起点零代价连到所有源。

证明：第 0 层就是全部源；分层不变式给出 min_s dist(s,v)，不需要每个源单独 BFS。

约定：依赖 PG03；sources 每个点在 1..n，可重复或为空；所有源距离 0，共用 dist/fa/path。

调用：init(5);
for(int i=1;i<5;++i){g[i].push_back(i+1);g[i+1].push_back(i);}
bfs(vector<int>{1,5,1});

例子：链 1-2-3-4-5，sources={1,5,1}：dist=[0,1,2,1,0]。

复杂度：O(n+m+源数)时间，O(n+m)空间。

易错：每个源依次调用单源 bfs 会覆盖之前结果。；源点时间不同不能都设 0，需要最短路或分层事件。；空源集合应全部不可达。

来源：template/bfs.cpp

规范源码：competition/graph/multisource_bfs.hpp；完整独立代码如下；测试：tests/round02/test_graph_personal.cpp

```
#include <bits/stdc++.h>
using namespace std;

// PG03. bfs.cpp 的队列/入队即标记；加 dist/fa 供最短路回溯。
const int N = 200010;
int n, dist[N], fa[N];
vector<int> g[N];
bool visited[N];

void init(int count) {
    n = count;
    for (int i = 1; i <= n; ++i) g[i].clear();
}

void bfs(int start) {
    fill(visited + 1, visited + n + 1, false);
    fill(dist + 1, dist + n + 1, -1);
    fill(fa + 1, fa + n + 1, 0);
    queue<int> q;
    visited[start] = true;
    dist[start] = 0;
    q.push(start);
    while (!q.empty()) {
        int u = q.front(); q.pop();
        for (int v : g[u]) {
            if (!visited[v]) {
                visited[v] = true;
                dist[v] = dist[u] + 1;
                fa[v] = u;
                q.push(v);
            }
        }
    }
}

vector<int> path(int t) {
    if (t < 1 || t > n || dist[t] == -1) return {};
    vector<int> ans;
    for (; t; t = fa[t]) ans.push_back(t);
    reverse(ans.begin(), ans.end());
    return ans;
}

// PG08. 复用 PG03 的数组；所有源点同时从距离 0 开始。
void bfs(const vector<int>& sources) {
    fill(visited + 1, visited + n + 1, false);
```

【PG08】多源 BFS 续

```
    fill(dist + 1, dist + n + 1, -1);
    fill(fa + 1, fa + n + 1, 0);
    queue<int> q;
    for (int s : sources) if (!visited[s]) {
        visited[s] = true; dist[s] = 0; q.push(s);
    }
    while (!q.empty()) {
        int u = q.front(); q.pop();
        for (int v : g[u]) if (!visited[v]) {
            visited[v] = true; dist[v] = dist[u] + 1;
            fa[v] = u; q.push(v);
        }
    }
}
```

【PG09】0-1 BFS

识别：边权仅 0/1，如免费移动与一次付费翻转。

思路：0 权松弛入队头，1 权松弛入队尾；用距离改善而非 visited 一次性锁死。

证明：deque 保留当前最小层及下一层；有效弹出按距离非降序，等价于只含 0/1 边的 Dijkstra。

约定：1<=start<=n<200010；每条边 w 必须是 0 或 1，函数不校验。不可达 dist=1e9。

调用：init(3);
g[1]={{2,1},{3,0}};
g[3]={{2,0}};
bfs01(1);

例子：上述三边中 dist[2]=0；普通 BFS 按边数会给 1。

复杂度：O(n+m)时间和空间。

易错：1->2 权 1、1->3 权 0、3->2 权 0 时，2 初次发现不是最短。；有 2 权时不能继续保证队列序。；必须跳过过期距离快照。

来源：template/graph/dijkstra.hpp

规范源码：competition/graph/zero_one_bfs.hpp；完整独立代码如下；测试：tests/round02/test_graph_personal.cpp

```
#include <bits/stdc++.h>
using namespace std;

// PG09. 点 1..n；权只能为 0 或 1。有向/无向都可。
const int N = 200010, INF = 1000000000;
int n, dist[N], fa[N];
vector<pair<int, int>> g[N];

void init(int count) {
    n = count;
    for (int i = 1; i <= n; ++i) g[i].clear();
}

void bfs01(int start) {
    fill(dist + 1, dist + n + 1, INF);
    fill(fa + 1, fa + n + 1, 0);
    deque<pair<int, int>> q; // (距离快照,点)，弹出时丢掉过期状态。
    dist[start] = 0; q.push_front({0, start});
    while (!q.empty()) {
        auto [du, u] = q.front(); q.pop_front();
        if (du != dist[u]) continue;
        for (auto [v, w] : g[u]) if (du + w < dist[v]) {
            dist[v] = du + w; fa[v] = u;
            if (w == 0) q.push_front({dist[v], v});
            else q.push_back({dist[v], v});
        }
    }
}
```

【PG10】Floyd 全源最短路

识别：点少、多次两点最短路查询；可有负边但无负环。

思路：逐步放开可经过的中间点 k，dist[i][j]取不经过 k 或经 k 两部分之和。

证明：最短路按是否经过 k 分成两类；无负环可选简单最短路。k 必须是最外层才能维护中间点集合不变式。

约定：0<=n<510，|边权|<=1e12；先 init，再 add_edge 取重边最小值，无向边加两次。floyd 有任何负环返回 false，此时全部结果不使用；不可达 INF。

调用：init(3);
add_edge(1,2,4);
add_edge(2,3,-2);
add_edge(1,3,9);
bool ok=floyd();

例子：1->2 权 4，2->3 权-2，1->3 权 9：dist[1][3]=2；另加 3->2 权 1 形成负环，返回 false。

复杂度：O(n^3)时间、O(n^2)空间。

易错：不可达 INF 不参与加法。；负环可能使递推数值迅速变负；用 i128 并截到-INF 后返回 false。；不能把同一 k 放最内层当标准 Floyd。

来源：template/graph/dijkstra.hpp

规范源码：competition/graph/floyd.hpp；完整独立代码如下；测试：tests/round02/test_graph_personal.cpp

```
#include <bits/stdc++.h>
using namespace std;
using ll = long long;

// PG10. n<N, |边权|<=1e12；负环返回 false，此时不使用距离结果。
const int N = 510;
const ll INF = LLONG_MAX / 4;
int n;
ll dist[N][N];

void init(int count) {
    n = count;
    for (int i = 1; i <= n; ++i)
        for (int j = 1; j <= n; ++j) dist[i][j] = (i == j ? 0 : INF);
}

void add_edge(int u, int v, ll w) { dist[u][v] = min(dist[u][v], w); }

bool floyd() {
    for (int k = 1; k <= n; ++k)
        for (int i = 1; i <= n; ++i) if (dist[i][k] != INF)
            for (int j = 1; j <= n; ++j) if (dist[k][j] != INF) {
                __int128 sum = (__int128)dist[i][k] + dist[k][j];
                dist[i][j] = min(dist[i][j], (ll)max(sum, -(__int128)INF));
            }
    for (int i = 1; i <= n; ++i) if (dist[i][i] < 0) return false;
    return true;
}
```

【PG11】二分图染色与冲突检测

识别：每条约束要求两端不同组；判断能否只用两种颜色。

思路：沿用 col[v]=3-col[u]，遍历每个组件；给已着色邻点补相异检查。

证明：边要求两端相异，沿路径颜色由长度奇偶唯一决定；出现同色边恰好构成奇环矛盾。

约定：0<=n<200010，无向边双向添加；col=1/2。成功时所有点着色；失败时剩余 col 可能为 0。支持断图、重边；自环立即失败。

调用：init(3);
g[1]={2};
g[2]={1,3};
g[3]={2};
bool ok=bipartite();

例子：链 1-2-3 可染 1,2,1；再加 3-1 失败；两个平行 1-2 边仍可行。

复杂度：O(n+m)时间/空间。

易错：只给未着色点染色但不检查已着色边，会把三角形误判可二分。；原 6.16 题可能自带可行结构，其专用代码不能直接充当通用判定。；断图必须枚举所有起点。

来源：template/graph_paint.cpp；cf/date/6/6.16/2_bi_paint_.cpp

规范源码：competition/graph/bipartite.hpp；完整独立代码如下；测试：tests/round02/test_graph_personal.cpp

```
#include <bits/stdc++.h>
using namespace std;

// PG11. graph_paint.cpp / 6.16 的 1/2 染色，补冲突检测。
const int N = 200010;
int n, col[N];
vector<int> g[N];

void init(int count) {
    n = count;
    for (int i = 1; i <= n; ++i) g[i].clear();
}

bool bipartite() {
    fill(col + 1, col + n + 1, 0);
    for (int s = 1; s <= n; ++s) if (!col[s]) {
        queue<int> q;
        col[s] = 1; q.push(s);
        while (!q.empty()) {
            int u = q.front(); q.pop();
            for (int v : g[u]) {
                if (!col[v]) {
                    col[v] = 3 - col[u]; q.push(v);
                } else if (col[v] == col[u]) return false;
            }
        }
    }
    return true;
}
```

【PG12】有向/无向环恢复

识别：不只判断有环，还要输出一个闭合环；无向图可能有重边或自环。

思路：显式 DFS 用 0/1/2 表示未进入/栈内/结束；碰到栈内祖先后沿 fa 恢复。

证明：回边连接当前点与栈内祖先，树父链加回边形成环；无向只跳过父边 id，第二条平行边仍能闭合长度 2 的环。

约定：0<=n<200010，输入 m 条边的唯一 id 为 0..m-1；无向双向记录共用同一个 id。find_cycle(directed)返回是否找到；cycle 首尾相同，cycle_edges 对应每一步，边号不重复。

调用：init(2);
g[1]={{2,0},{2,1}};
g[2]={{1,0},{1,1}};
bool ok=find_cycle(false);

例子：两点两条平行边 id0/id1 可恢复[1,2,1]；自环恢复[u,u]；有向 1->2,1->3,2->3 无环。

复杂度：O(n+m)时间/空间。

易错：无向图跳过所有 v==fa[u]会漏两条平行边构成的环。；有向图不能把指向已结束点的边判成环。；简单基环图专用剥叶算法仍见 G05，不必强行替换。

来源：template/graph/dfs_forest.hpp；template/advanced/bridges.hpp

规范源码：competition/graph/cycle.hpp；完整独立代码如下；测试：tests/round02/test_graph_personal.cpp

```
#include <bits/stdc++.h>
using namespace std;

// PG12. 输入边号为 0..m-1；无向的两个方向共用同一个 id。
const int N = 200010;
int n, color[N], fa[N], pe[N];
vector<pair<int, int>> g[N]; // (终点,边号)
vector<int> cycle, cycle_edges; // 首尾同点，边号与相邻点逐一对应。

void init(int count) {
    n = count;
    for (int i = 1; i <= n; ++i) g[i].clear();
}

bool find_cycle(bool directed) {
    fill(color + 1, color + n + 1, 0);
    fill(fa + 1, fa + n + 1, 0);
    fill(pe + 1, pe + n + 1, -1);
    cycle.clear(); cycle_edges.clear();
    for (int s = 1; s <= n; ++s) if (!color[s]) {
        vector<pair<int, int>> st{{s, 0}};
        color[s] = 1;
        while (!st.empty()) {
            int u = st.back().first;
            int& i = st.back().second;
            if (i == (int)g[u].size()) {
                color[u] = 2; st.pop_back(); continue;
            }
            auto [v, id] = g[u][i++];
            if (!directed && id == pe[u]) continue;
            if (!color[v]) {
                fa[v] = u; pe[v] = id; color[v] = 1;
                st.push_back({v, 0});
            } else if (color[v] == 1) {
                for (int x = u; x != v; x = fa[x]) {
                    cycle.push_back(x); cycle_edges.push_back(pe[x]);
                }
                cycle.push_back(v);
                reverse(cycle.begin(), cycle.end());
                reverse(cycle_edges.begin(), cycle_edges.end());
                cycle.push_back(v); cycle_edges.push_back(id);
                return true;
            }
        }
    }
    return false;
}
```

【PG13】无权树直径与路径

识别：求树上最长简单路径的边数及端点。

思路：从任一点找最远端 a，再从 a 找最远端 b；第二次父亲回溯得到直径路径。

证明：以初始点 s 为根，最远 a 的深度不小于任意直径端点 x/y。取 t=LCA(x,y)：若 a 在 t 子树内，选与 a 不同分支的端点；否则 a 到端点的分叉不深于 t。所得 a 到该端点距离至少为原直径，故 a 是某条直径端点；第二遍最大距离即直径。

约定：依赖 PG01。只接受无权无向树；tree_diameter 非树返回-1。单点长度 0、路径[1]；成功后 diameter_a/b 和 diameter_path 有效。只读 g/n，不修改 PG01 的父亲/深度。

调用：init(4);
g[1]={2,3,4};
g[2]={1};
g[3]={1};
g[4]={1};
int len=tree_diameter();

例子：星形 1-2、1-3、1-4，直径长度 2，可返回任意两片叶子的路径；20 万链长度 199999。

复杂度：O(n)时间和空间。

易错：本模块是无权版，不把 BFS 步数当带权距离。；非负权树可把遍历距离改为权和后用两遍法，但需另核验溢出；负边树不能直接套。；多个最远点可能并列，返回任意一条直径，不保证字典序。

来源：template/tree_storage.cpp；template/bfs.cpp

规范源码：competition/graph/tree_diameter.hpp；完整独立代码如下；测试：tests/round02/test_graph_personal.cpp

```
#include <bits/stdc++.h>
using namespace std;
using ll = long long;

// PG01. 原 tree_storage.cpp 的双向邻接表；点 1..n，1<=n<N。
const int N = 200010;
int n;
vector<ll> g[N];
ll fa[N], dep[N], sz[N];
vector<ll> order; // 父亲先于儿子；逆序可做子树 DP。

void init(int count) {
    n = count;
    for (int i = 1; i <= n; ++i) {
        g[i].clear();
        fa[i] = sz[i] = 0;
        dep[i] = -1;
    }
    order.clear();
}

void read_tree(istream& in = cin) {
    int count;
    in >> count;
    init(count);
    for (int i = 1; i <= n - 1; ++i) {
        ll u, v;
        in >> u >> v;
        g[u].push_back(v);
        g[v].push_back(u);
    }
}

// 双向对称的邻接表由调用方保证。非树返回 false。
bool root_tree(int root = 1) {
    order.clear();
    if (root < 1 || root > n) return false;
    ll edges = 0;
    for (int i = 1; i <= n; ++i) {
```

【PG13】无权树直径与路径 续

```
        fa[i] = 0; dep[i] = -1; sz[i] = 1;
        edges += (ll)g[i].size();
    }
    if (edges != 2LL * (n - 1)) return false;
    vector<ll> st{root};
    dep[root] = 0;
    while (!st.empty()) {
        ll u = st.back(); st.pop_back();
        order.push_back(u);
        for (ll v : g[u]) if (dep[v] == -1) {
            fa[v] = u; dep[v] = dep[u] + 1;
            st.push_back(v);
        }
    }
    if ((int)order.size() != n) return false;
    for (int i = n - 1; i > 0; --i) {
        ll u = order[i];
        sz[fa[u]] += sz[u];
    }
    return true;
}

// PG13. 无权树直径：长度是边数。非树返回 -1。
int diameter_a, diameter_b;
vector<ll> diameter_path;

int farthest(int s, vector<int>& d, vector<int>& parent) {
    d.assign(n + 1, -1); parent.assign(n + 1, 0);
    queue<int> q; q.push(s); d[s] = 0;
    int best = s;
    while (!q.empty()) {
        int u = q.front(); q.pop();
        if (d[u] > d[best]) best = u;
        for (ll v : g[u]) if (d[v] == -1) {
            d[v] = d[u] + 1; parent[v] = u; q.push((int)v);
        }
    }
    return best;
}

int tree_diameter() {
    diameter_path.clear(); diameter_a = diameter_b = 0;
    if (n < 1) return -1;
    ll edges = 0;
    for (int u = 1; u <= n; ++u) edges += (ll)g[u].size();
    if (edges != 2LL * (n - 1)) return -1;
    vector<int> d, parent;
    diameter_a = farthest(1, d, parent);
    for (int u = 1; u <= n; ++u) if (d[u] == -1) {
        diameter_a = 0;
        return -1;
    }
    diameter_b = farthest(diameter_a, d, parent);
    for (int u = diameter_b; u; u = parent[u]) diameter_path.push_back(u);
    reverse(diameter_path.begin(), diameter_path.end());
    return d[diameter_b];
}
```

【PG15】DAG 路径计数与最长路

识别：转移关系有向无环；求从固定起点的路径数和最长边数。

思路：Kahn 处理完全部前驱后，把 ways 加向后继、把 dist 最大化。

证明：每条到 v 的非空路径有唯一最后一条边 u->v；拓扑序保证所有前驱已完成，路径计数可加且最长路可取 max。

约定：1<=s<=n<200010，1<=mod<=LLONG_MAX；不可达 dist=-1/ways=0，起点含空路径。重边按不同边选择计数。有环返回 false，结果作废。

调用：init(4);
g[1]={2,3};
g[2]={4};
g[3]={4};
bool ok=dag_dp(1,1000000007);

例子：菱形 1->2/3->4：dist[4]=2，ways[4]=2；连续 63 层每层两条重边有 2^63 条路径。

复杂度：O(n+m)时间/空间。

易错：有向有环时最长简单路不能照搬此 DP。；mod=1 时 ways[s]=0 但 s 仍可达，不能用 ways==0 判断可达。；两项模数接近 LLONG_MAX 时加法先转 i128。

来源：cf/1704E.cpp；template/graph/topological_sort.hpp

规范源码：competition/graph/dag_dp.hpp；完整独立代码如下；测试：tests/round02/test_graph_personal.cpp

```
#include <bits/stdc++.h>
using namespace std;
using ll = long long;

// PG15. DAG 上从 s 出发的最长边数、路径数；重边视为不同路径选择。
const int N = 200010;
int n, indeg[N], dist[N];
ll ways[N];
vector<int> g[N];

void init(int count) {
    n = count;
    for (int i = 1; i <= n; ++i) g[i].clear();
}

// 1<=s<=n<N, 1<=mod<=LLONG_MAX。有环返回 false，结果作废。
bool dag_dp(int s, ll mod) {
    fill(indeg + 1, indeg + n + 1, 0);
    fill(dist + 1, dist + n + 1, -1);
    fill(ways + 1, ways + n + 1, 0);
    for (int u = 1; u <= n; ++u) for (int v : g[u]) ++indeg[v];
    queue<int> q;
    for (int u = 1; u <= n; ++u) if (!indeg[u]) q.push(u);
    dist[s] = 0; ways[s] = 1 % mod;
    int cnt = 0;
    while (!q.empty()) {
        int u = q.front(); q.pop(); ++cnt;
        for (int v : g[u]) {
            if (dist[u] != -1) {
                dist[v] = max(dist[v], dist[u] + 1);
                ways[v] = (ll)(((__int128)ways[v] + ways[u]) % mod);
            }
            if (--indeg[v] == 0) q.push(v);
        }
    }
    return cnt == n;
}
```

【PM01】扩展欧几里得与一般模逆元

识别：求 ax+by=gcd(a,b)，或模数不一定质数的逆元。

思路：从 b*x1+(a%b)*y1=g 倒推 x=y1,y=x1-(a/b)*y1。

证明：把 a%b=a-(a/b)b 代回即可得到 Bezout 等式；ax≡1(mod m)有解当且仅当 gcd(a,m)=1。

约定：|a|,|b|<=1e18；exgcd 返回非负 gcd 和 ll 系数，(0,0)取 x=1,y=0。inverse_mod 要求 2<=mod<=1e18，逆元归一到[0,mod)，不存在返回-1。

调用：ll x,y;
ll d=exgcd(30,18,x,y);
ll inv=inverse_mod(3,8);

例子：30*(-1)+18*2=6；3 在模 8 下逆元为 3，2 在模 8 下无逆元。

复杂度：O(log max(|a|,|b|))时间、递归栈同阶。

易错：费马 a^(p-2)必须模数为质数且 a 非零；复合模数用本函数。；不支持 LLONG_MIN 输入，契约上限 1e18。；整数解存在不代表存在满足非负/区间约束的解，还需解参数范围。

来源：template/gcd.cpp；template/ksm.cpp

规范源码：competition/math/exgcd.hpp；完整独立代码如下；测试：tests/round02/test_math_geometry.cpp

```
#include <bits/stdc++.h>
using namespace std;
using ll = long long;

// PM01. |a|,|b|<=1e18；返回非负 gcd，且 a*x+b*y=gcd。
ll exgcd(ll a, ll b, ll& x, ll& y) {
    if (b == 0) {
        x = (a < 0 ? -1 : 1); y = 0;
        return a < 0 ? -a : a;
    }
    ll x1, y1;
    ll d = exgcd(b, a % b, x1, y1);
    x = y1;
    y = (ll)((__int128)x1 - (__int128)(a / b) * y1);
    return d;
}

// 2<=mod<=1e18，|a|<=1e18；不存在逆元返回 -1。
ll inverse_mod(ll a, ll mod) {
    a %= mod;
    if (a < 0) a += mod;
    ll x, y;
    if (exgcd(a, mod, x, y) != 1) return -1;
    x %= mod;
    return x < 0 ? x + mod : x;
}
```

【PM02】线性筛、判素、分解与约数

识别：批量小范围判素/最小质因数，或少量<=1e12 整数试除分解、列约数。

思路：线性筛每个合数由其最小质因数唯一生成；试除持续除尽某个质因数，剩余>1 即为质数。

证明：筛循环遇整除就 break，避免重复最小质因数分解；试除结束时若剩数合成则必有不超过其平方根的因子，与循环停止矛盾。

约定：sieve:1<=n<1000010，primes[1..cnt]和 lp[1..n]；lp[1]=0。is_prime 负数/0/1 为 false。factorize/divisors 要求 1<=n<=1e12；筛与试除函数互不依赖。

调用：sieve(1000);
bool p=is_prime(97);
auto f=factorize(60);
auto d=divisors(60);

例子：60=2^2*3*5，共(2+1)(1+1)(1+1)=12 个约数；sieve(5)质数为 2,3,5。

复杂度：筛 O(n)时空；试除 O(sqrt n)时间；约数生成 O(tau(n)log tau(n))额外排序。

易错：1 不是质数；factorize(1)为空，divisors(1)=[1]。；试除单个 1e12 约 1e6 步，大量查询应改筛预处理或更强算法。；多测 sieve 必须重置 cnt 与当前 1..n 的 lp。

来源：template/Mobius_inversion.cpp

规范源码：competition/math/primes.hpp；完整独立代码如下；测试：tests/round02/test_math_geometry.cpp

```
#include <bits/stdc++.h>
using namespace std;
using ll = long long;

// PM02. 沿用 Mobius_inversion.cpp 的线性筛循环。
const int N = 1000010;
int primes[N], lp[N], cnt;

void sieve(int n) { // 1<=n<N; lp[x] 为最小质因数，lp[1]=0。
    fill(lp, lp + n + 1, 0); cnt = 0;
    for (int i = 2; i <= n; ++i) {
        if (!lp[i]) lp[i] = i, primes[++cnt] = i;
        for (int j = 1; j <= cnt && primes[j] <= n / i; ++j) {
            lp[i * primes[j]] = primes[j];
            if (i % primes[j] == 0) break;
        }
    }
}

// 单个数试除；n<=1e12 时最多约 1e6 次，不能批量处理大量大数。
bool is_prime(ll n) {
    if (n < 2) return false;
    for (ll p = 2; p <= n / p; ++p) if (n % p == 0) return false;
    return true;
}

vector<pair<ll, int>> factorize(ll n) { // 1<=n<=1e12。
    vector<pair<ll, int>> ans;
    for (ll p = 2; p <= n / p; ++p) if (n % p == 0) {
        int c = 0;
        do { n /= p; ++c; } while (n % p == 0);
        ans.push_back({p, c});
    }
    if (n > 1) ans.push_back({n, 1});
    return ans;
}

vector<ll> divisors(ll n) { // 包括 1 和 n，升序。
    vector<ll> ans{1};
    for (auto [p, c] : factorize(n)) {
        int old = (int)ans.size();
        ll power = 1;
        for (int k = 1; k <= c; ++k) {
```

【PM02】线性筛、判素、分解与约数 续

```
            power *= p;
            for (int i = 0; i < old; ++i) ans.push_back(ans[i] * power);
        }
    }
    sort(ans.begin(), ans.end());
    return ans;
}
```

【PM03】负数向下/向上整除

识别：不等式取整、负数坐标分桶、二分边界与容量除法。

思路：C++向零截断；有余数且商应为负则 floor 减一，商应为正则 ceil 加一。

证明：a=bq+r，|r|<|b|；r/b 的符号确定真实商在 q 哪一侧，最多需要改 1。

约定：a,b 为 ll，b!=0；排除 a=LLONG_MIN,b=-1（数学商 2^63 超出 ll）。floor_div/ceil_div 返回数学下取整/上取整。

调用：ll l=floor_div(-7,3);
ll r=ceil_div(-7,3);

例子：floor_div(-7,3)=-3，ceil_div(-7,3)=-2；floor_div(7,-3)=-3。

复杂度：O(1)时间/空间。

易错：(a+b-1)/b 只在特定非负范围成立且 a+b-1 可能溢出。；负数 a%b 可能为负。；不能用浮点 ceil 处理完整 ll 精确边界。

来源：cf/ICPC/JAKARTA_2018/H.cpp

规范源码：competition/math/integer_division.hpp；完整独立代码如下；测试：tests/round02/test_math_geometry.cpp

```
#include <bits/stdc++.h>
using namespace std;
using ll = long long;

// PM03. b!=0，排除 a=LLONG_MIN,b=-1（结果超出 ll）。
// C++ / 向零截断；根据非零余数的符号补一次修正。
ll floor_div(ll a, ll b) {
    ll q = a / b, r = a % b;
    if (r != 0 && ((r > 0) != (b > 0))) --q;
    return q;
}

ll ceil_div(ll a, ll b) {
    ll q = a / b, r = a % b;
    if (r != 0 && ((r > 0) == (b > 0))) ++q;
    return q;
}
```

【PM04】基础求和与成对差贡献

识别：连续整数/平方和/等差数列，或所有无序对绝对差的和。

思路：闭式计算总和；排序后 a[i]在前 i 项差中为正、在后 n-1-i 项差中为负，系数 2i-n+1。

证明：每对 i<j 贡献 a[j]-a[i]，按元素收集可得线性系数；平方和可用(n+1)^3-n^3 差分求和验证。

约定：全部返回__int128。sum1:0<=n<=1e18；sum2:0<=n<=1e12；arithmetic_sum 的 first/step 为 ll、0<=count<=1e9；pair_abs_sum 长度<=2e5、|值|<=1e18。

调用：auto s=sum1(100);
auto q=sum2(100);
auto a=arithmetic_sum(3,2,5);
auto d=pair_abs_sum({1,3,6});

例子：a={1,3,6}：差 2+5+3=10；系数-2,0,2，得到-2+12=10。

复杂度：前三项 O(1)；成对差排序 O(n log n)、空间 O(n)拷贝。

易错：必须先转 i128 再乘；最后才转换不能修复已溢出中间值。；pair_abs_sum 排序了拷贝；公式使用 0 下标 i。；不同函数的 n 上界不同，不把 sum1 的上界套给 sum2。

来源：cf/date/7/7.28/1.cpp

规范源码：competition/math/sums.hpp；完整独立代码如下；测试：tests/round02/test_math_geometry.cpp

```
#include <bits/stdc++.h>
using namespace std;
using ll = long long;
using i128 = __int128;

// PM04. 先提升类型再乘。返回 i128，转 ll 前检查结果范围。
i128 sum1(ll n) { // 0<=n<=1e18。
    return (i128)n * (n + 1) / 2;
}

i128 sum2(ll n) { // 0<=n<=1e12。
    return (i128)n * (n + 1) * (2 * n + 1) / 6;
}

i128 arithmetic_sum(ll first, ll step, ll count) { // 0<=count<=1e9。
    return (i128)count * first + (i128)step * count * (count - 1) / 2;
}

// 源于 cf/date/7/7.28/1.cpp; n<=2e5, |a[i]|<=1e18。
i128 pair_abs_sum(vector<ll> a) {
    sort(a.begin(), a.end());
    ll n = (ll)a.size();
    i128 ans = 0;
    for (ll i = 0; i < n; ++i) ans += (i128)a[i] * (2 * i - n + 1);
    return ans;
}
```

【PM05】奇偶、模不变量与环上可达

识别：操作使某个量只增加固定步长；判断同余类是否保持、模环上能否到目标。

思路：模 mod 增加 step 能生成的剩余类由 gcd(step,mod)刻画；奇偶就是 mod=2。

证明：若 t 可达则差为 k*step+q*mod，必被 gcd 整除；反过来 Bezout 给出整数 k，把 k 加上模环周期可变成非负次数，因此充分。

约定：norm_mod/congruent 要求 1<=mod<=LLONG_MAX；a/b 可完整 ll。reachable_mod 要求|step|<=1e18、1<=mod<=1e18，start/target 可完整 ll；允许模环反复绕回，次数为非负整数。

调用：ll r=norm_mod(-3,5);
bool ok=reachable_mod(1,7,4,10);
bool same=congruent(-3,2,5);

例子：模 10 每次+4，从 1 可达{1,5,9,3,7}，可到 7 但不可到 2；负数-3 归一模 5 为 2。

复杂度：归一化 O(1)，环可达 O(log max(|step|,mod))。

易错：不变量一般只给必要条件，不能自动证明可达。；有限区间内只许+x/+y 时 gcd 整除未必充分；例如步长 4/6 从 0 不能到 2。；模环中 step=0 只可达原剩余类。

来源：cf/2/873/B.cpp；cf/2/750/A.cpp；cf/3/1109/3.cpp

规范源码：competition/math/invariants.hpp；完整独立代码如下；测试：tests/round02/test_math_geometry.cpp

```
#include <bits/stdc++.h>
using namespace std;
using ll = long long;

// PM05. mod>0；负数也统一为 [0,mod) 的同余类。
ll norm_mod(ll a, ll mod) {
    ll r = a % mod;
    return r < 0 ? r + mod : r;
}

bool congruent(ll a, ll b, ll mod) {
    return norm_mod(a, mod) == norm_mod(b, mod);
}

// 模 mod 的环上反复 +step（次数>=0）；|step|,mod<=1e18,mod>=1。
// 这里允许在环上绕回；有限区间内只能向前走不是这个模型。
bool reachable_mod(ll start, ll target, ll step, ll mod) {
    ll d = gcd(step, mod);
    return ((__int128)target - start) % d == 0;
}
```

【PM06】整数点积、叉积与线段相交

识别：整数坐标判转向、共线、点在线段上、闭线段相交；需要精确判定。

思路：叉积符号判两侧；一般相交要求双方严格跨立，零叉积转为包围盒内的在线段判定。

证明：两段各自跨过另一条所在直线时交点落在双方区间；共线时退化为一维区间重叠。差分<=2e18，两个乘积绝对值和<=8e36，小于有符号 128 位上界。

约定：|每个坐标|<=1e18；Point 存 ll，先提升到__int128 再减。dot/cross 以 a 为共同起点；orientation 返回-1/0/1。segments_intersect 把端点接触、共线重叠与退化点段都算相交。

调用：Point a{0,0},b{2,2},c{0,2},d{2,0};
bool ok=segments_intersect(a,b,c,d);
int turn=orientation(a,b,c);

例子：(0,0)-(2,2)与(0,2)-(2,0)相交；(0,0)-(1,0)与(2,0)-(3,0)不相交；点段相交按同样规则。

复杂度：全部 O(1)时间/空间。

易错：任意完整 ll 坐标不在契约内；其差的叉积可能超过 128 位。；只比较 orientation 乘积<=0 会把共线但分离的段误判相交。；整型坐标用精确符号；浮点三角与 epsilon 规则另见对应浮点模块。

来源：template/math/integer_bits.hpp

规范源码：competition/math/integer_geometry.hpp；完整独立代码如下；测试：tests/round02/test_math_geometry.cpp

```
#include <bits/stdc++.h>
using namespace std;
using ll = long long;
using i128 = __int128;

// PM06. |坐标|<=1e18。差值<=2e18；叉积/点积绝对值<=8e36。
// 必须先转 i128 再减；本模块不接受任意完整 ll 坐标范围。
struct Point { ll x, y; };

i128 dot(Point a, Point b, Point c) { // (b-a) dot (c-a)。
    return ((i128)b.x - a.x) * ((i128)c.x - a.x)
         + ((i128)b.y - a.y) * ((i128)c.y - a.y);
}

i128 cross(Point a, Point b, Point c) { // (b-a) cross (c-a)。
    return ((i128)b.x - a.x) * ((i128)c.y - a.y)
         - ((i128)b.y - a.y) * ((i128)c.x - a.x);
}

int sign(i128 x) { return (x > 0) - (x < 0); }
int orientation(Point a, Point b, Point c) { return sign(cross(a, b, c)); }

bool on_segment(Point a, Point b, Point p) {
    return cross(a, b, p) == 0
        && min(a.x, b.x) <= p.x && p.x <= max(a.x, b.x)
        && min(a.y, b.y) <= p.y && p.y <= max(a.y, b.y);
}

bool segments_intersect(Point a, Point b, Point c, Point d) {
    int c1 = orientation(a, b, c), c2 = orientation(a, b, d);
    int c3 = orientation(c, d, a), c4 = orientation(c, d, b);
    if (c1 == 0 && on_segment(a, b, c)) return true;
    if (c2 == 0 && on_segment(a, b, d)) return true;
    if (c3 == 0 && on_segment(c, d, a)) return true;
    if (c4 == 0 && on_segment(c, d, b)) return true;
    return c1 * c2 < 0 && c3 * c4 < 0;
}
```

【PSD01】Fenwick：个人 add/query/rangeSum

识别：单点加、前缀和、可相减的区间和。

思路：按 lowbit 把前缀划成互不重叠块，修改只访问包含该点的块。

证明：query 访问的 lowbit 块恰好划分[1,x]，两个前缀相减得到闭区间和。

约定：0<=n<2^30 且内存足够；1 下标，add 只能 1..n；query(0)=0；空 rangeSum 返回 0；所有中间量在 ll 内。

调用：Fenwick fen(n); fen.add(pos,delta); fen.query(r); fen.rangeSum(l,r); 完整 main 见 competition/data_structure/examples/fenwick.cpp

例子：完整可运行例程：competition/data_structure/examples/fenwick.cpp；样例见 segment_sources.json。

复杂度：更新/查询 O(log n)，逐点初始化 O(n log n)，空间 O(n)。

易错：add(0,x)非法；add 是增量，赋新值需 new-old。；query(x)是前缀，不是原来的 query(l,r)；区间用 rangeSum。；不可用两个前缀 min 相减得到区间 min。

来源：template/Fenwick_tree.cpp

规范源码：competition/data_structure/fenwick.hpp；完整独立代码如下；测试：tests/round02/test_segment.cpp

```
#include <bits/stdc++.h>
using namespace std;
using ll = long long;

// 保留旧 Fenwick 的 n、tr、add 和 lowbit 循环。
// 1 下标；add 是增量，query(x) 是前缀，rangeSum(l,r) 是闭区间。
// 0<=n<2^30 且内存足够；所有节点、前缀及中间值须在 ll 范围。
struct Fenwick {
    int n;
    vector<ll> tr;
    explicit Fenwick(int size) : n(size), tr(size + 1) {
        assert(size >= 0);
    }
    void add(int x, ll val) {
        assert(1 <= x && x <= n);
        for (; x <= n; x += x & -x) tr[x] += val;
    }
    ll query(int x) const {
        assert(0 <= x && x <= n);
        ll res = 0;
        for (; x; x -= x & -x) res += tr[x];
        return res;
    }
    ll rangeSum(int l, int r) const {
        if (l > r) return 0;
        assert(1 <= l && r <= n);
        return query(r) - query(l - 1);
    }
};
```

接在上述代码后面的完整 main（已独立运行样例）：

样例输入：3 4 / 3 -2 4 / 2 1 3 / 1 2 5 / 3 2 / 2 2 3 /
样例输出：5 / 6 / 7 /

```
int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, q;
    cin >> n >> q;
    Fenwick fen(n);
    for (int i = 1; i <= n; ++i) { ll x; cin >> x; fen.add(i, x); }
    while (q--) {
        int op, l, r; ll val;
        cin >> op;
        if (op == 1) { cin >> l >> val; fen.add(l, val); }
        else if (op == 2) { cin >> l >> r; cout << fen.rangeSum(l, r) << '\n'; }
        else { cin >> r; cout << fen.query(r) << '\n'; }
    }
}
```

【PSD02】单点树公共骨架与区间和

识别：单点赋值，任意区间求和；后续需要换 Node 信息。

思路：父区间等于左右连续区间拼接，保存可合并的区间和。

证明：每次修改重算该叶到根路径；查询按从左到右的互不相交段合并。

约定：1<=n<N；1 下标闭区间；modify 是赋值；a 仅是建树输入；所有中间和须在 ll 内。

调用：读取 a[1..n]后 build(1,1,n)；具体可编译调用见 competition/data_structure/examples/point_sum.cpp。

例子：完整可运行例程：competition/data_structure/examples/point_sum.cpp；样例见 segment_sources.json。

复杂度：建树 O(n)，修改/查询 O(log n)，空间 O(n)。

易错：modify 为赋值；加 delta 需用当前值+delta。；n=0 不能 build；查询不接受 L>R。；a 仅供建树，修改后请读 tree 或另行同步 a。

来源：template/segmen tree1.cpp；cf/segment_tree_cf_edu/1.cpp

规范源码：competition/data_structure/segment_point_sum.hpp；完整独立代码如下；测试：tests/round02/test_segment.cpp

```
#include <bits/stdc++.h>
using namespace std;
using ll = long long;
struct Node { ll sum; };
Node make_node(ll val) { return {val}; }
Node merge(const Node& x, const Node& y) { return {x.sum + y.sum}; }
// 公共单点树骨架：先放一个 Node / make_node / merge，再接本文件。
// 每个 segment_point_*.hpp 均已准确包含所需节点与本骨架，可直接 include。
// 保留旧代码 a/tree/node/l/r/mid 和递归结构；update 改名 modify。
// 非空树，1<=n<N；1<=pos<=n；查询必须 1<=L<=R<=n。
// a 仅为建树输入缓冲；modify 后当前值以 tree 为准。
const int N = 200000 + 5;
ll a[N];
Node tree[N * 4];

void pushup(int node) {
    tree[node] = merge(tree[node * 2], tree[node * 2 + 1]);
}
void build(int node, int l, int r) {
    if (l == r) {
        tree[node] = make_node(a[l]);
        return;
    }
    int mid = (l + r) >> 1;
    build(node * 2, l, mid);
    build(node * 2 + 1, mid + 1, r);
    pushup(node);
}
void apply(int node, ll val) {
    tree[node] = make_node(val);
}
void modify(int node, int l, int r, int pos, ll val) {
    if (l == r) {
        apply(node, val);
        return;
    }
    int mid = (l + r) >> 1;
    if (pos <= mid) modify(node * 2, l, mid, pos, val);
    else modify(node * 2 + 1, mid + 1, r, pos, val);
    pushup(node);
}
Node query(int node, int l, int r, int L, int R) {
    if (L <= l && r <= R) return tree[node];
    int mid = (l + r) >> 1;
    if (R <= mid) return query(node * 2, l, mid, L, R);
    if (L > mid) return query(node * 2 + 1, mid + 1, r, L, R);
    return merge(query(node * 2, l, mid, L, R),
                 query(node * 2 + 1, mid + 1, r, L, R));
}
```

【PSD02】单点树公共骨架与区间和 续

接在上述代码后面的完整 main（已独立运行样例）：

样例输入：3 3 / 3 -2 4 / 2 1 3 / 1 2 5 / 2 1 3 /
样例输出：5 / 12 /

```
int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, q;
    cin >> n >> q;
    for (int i = 1; i <= n; ++i) cin >> a[i];
    build(1, 1, n);
    while (q--) {
        int op;
        cin >> op;
        if (op == 1) {
            int pos; ll val;
            cin >> pos >> val;
            modify(1, 1, n, pos, val);
        } else {
            int l, r;
            cin >> l >> r;
            auto ans = query(1, 1, n, l, r);
            cout << ans.sum << '\n';
        }
    }
}
```

【PSD03】单点树：区间最小值

识别：点值会变化，在线查询任意区间的最低值。

思路：Node 只存 mn，父节点取左右最小值。

证明：每个元素恰属一个查询子段，min 满足结合律。

约定：沿用 PSD02 边界；允许负数及 LLONG_MIN；无空区间查询。

调用：读取 a[1..n]后 build(1,1,n)；具体可编译调用见 competition/data_structure/examples/point_min.cpp。

例子：完整可运行例程：competition/data_structure/examples/point_min.cpp；样例见 segment_sources.json。

复杂度：建树 O(n)，修改/查询 O(log n)，空间 O(n)。

易错：最小值不能把不相交区间返回 0。；旧 4e18 有限哨兵不足以覆盖所有 ll；本版用分支查询避免哨兵。

来源：template/segmen tree1.cpp；cf/segment_tree_cf_edu/1.cpp；cf/segment_tree_cf_edu/2.cpp

规范源码：competition/data_structure/segment_point_min.hpp；完整独立代码如下；测试：tests/round02/test_segment.cpp

```
#include <bits/stdc++.h>
using namespace std;
using ll = long long;
struct Node { ll mn; };
Node make_node(ll val) { return {val}; }
Node merge(const Node& x, const Node& y) { return {min(x.mn, y.mn)}; }
// 公共单点树骨架：先放一个 Node / make_node / merge，再接本文件。
// 每个 segment_point_*.hpp 均已准确包含所需节点与本骨架，可直接 include。
// 保留旧代码 a/tree/node/l/r/mid 和递归结构；update 改名 modify。
// 非空树，1<=n<N；1<=pos<=n；查询必须 1<=L<=R<=n。
// a 仅为建树输入缓冲；modify 后当前值以 tree 为准。
const int N = 200000 + 5;
ll a[N];
Node tree[N * 4];

void pushup(int node) {
    tree[node] = merge(tree[node * 2], tree[node * 2 + 1]);
}
void build(int node, int l, int r) {
    if (l == r) {
        tree[node] = make_node(a[l]);
        return;
    }
    int mid = (l + r) >> 1;
    build(node * 2, l, mid);
    build(node * 2 + 1, mid + 1, r);
    pushup(node);
}
void apply(int node, ll val) {
    tree[node] = make_node(val);
}
void modify(int node, int l, int r, int pos, ll val) {
    if (l == r) {
        apply(node, val);
        return;
    }
    int mid = (l + r) >> 1;
    if (pos <= mid) modify(node * 2, l, mid, pos, val);
    else modify(node * 2 + 1, mid + 1, r, pos, val);
    pushup(node);
}
Node query(int node, int l, int r, int L, int R) {
    if (L <= l && r <= R) return tree[node];
    int mid = (l + r) >> 1;
    if (R <= mid) return query(node * 2, l, mid, L, R);
    if (L > mid) return query(node * 2 + 1, mid + 1, r, L, R);
    return merge(query(node * 2, l, mid, L, R),
                 query(node * 2 + 1, mid + 1, r, L, R));
}
```

【PSD03】单点树：区间最小值 续

接在上述代码后面的完整 main（已独立运行样例）：

样例输入：3 3 / 5 2 7 / 2 1 3 / 1 2 8 / 2 1 3 /
样例输出：2 / 5 /

```
int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, q;
    cin >> n >> q;
    for (int i = 1; i <= n; ++i) cin >> a[i];
    build(1, 1, n);
    while (q--) {
        int op;
        cin >> op;
        if (op == 1) {
            int pos; ll val;
            cin >> pos >> val;
            modify(1, 1, n, pos, val);
        } else {
            int l, r;
            cin >> l >> r;
            auto ans = query(1, 1, n, l, r);
            cout << ans.mn << '\n';
        }
    }
}
```

【PSD04】单点树：区间最大值与阈值搜索

识别：点改区间最大值，或找[L,R]里第一个至少为 x 的位置。

思路：mx<x 的整段绝不含答案；其余段先查左儿子。

证明：剪去不相交段与不达标段，按下标递增搜索，第一个叶就是最左合法位置。

约定：沿用 PSD02 边界；first_ge 允许 L>R 并返回-1；值可为任意 ll。

调用：读取 a[1..n]后 build(1,1,n)；具体可编译调用见 competition/data_structure/examples/first_ge.cpp。

例子：完整可运行例程：competition/data_structure/examples/first_ge.cpp；样例见 segment_sources.json。

复杂度：建树 O(n)，修改/区间查询/first_ge 均 O(log n)，空间 O(n)。

易错：max 不能用 0 代表空段，全负区间会错误。；first_ge 只查给定[L,R]；必须先左后右才是第一个。

来源：template/segmen tree1.cpp；cf/segment_tree_cf_edu/1.cpp

规范源码：competition/data_structure/segment_point_max.hpp；完整独立代码如下；测试：tests/round02/test_segment.cpp

```
#include <bits/stdc++.h>
using namespace std;
using ll = long long;
struct Node { ll mx; };
Node make_node(ll val) { return {val}; }
Node merge(const Node& x, const Node& y) { return {max(x.mx, y.mx)}; }
// 公共单点树骨架：先放一个 Node / make_node / merge，再接本文件。
// 每个 segment_point_*.hpp 均已准确包含所需节点与本骨架，可直接 include。
// 保留旧代码 a/tree/node/l/r/mid 和递归结构；update 改名 modify。
// 非空树，1<=n<N；1<=pos<=n；查询必须 1<=L<=R<=n。
// a 仅为建树输入缓冲；modify 后当前值以 tree 为准。
const int N = 200000 + 5;
ll a[N];
Node tree[N * 4];

void pushup(int node) {
    tree[node] = merge(tree[node * 2], tree[node * 2 + 1]);
}
void build(int node, int l, int r) {
    if (l == r) {
        tree[node] = make_node(a[l]);
        return;
    }
    int mid = (l + r) >> 1;
    build(node * 2, l, mid);
    build(node * 2 + 1, mid + 1, r);
    pushup(node);
}
void apply(int node, ll val) {
    tree[node] = make_node(val);
}
void modify(int node, int l, int r, int pos, ll val) {
    if (l == r) {
        apply(node, val);
        return;
    }
    int mid = (l + r) >> 1;
    if (pos <= mid) modify(node * 2, l, mid, pos, val);
    else modify(node * 2 + 1, mid + 1, r, pos, val);
    pushup(node);
}
Node query(int node, int l, int r, int L, int R) {
    if (L <= l && r <= R) return tree[node];
    int mid = (l + r) >> 1;
    if (R <= mid) return query(node * 2, l, mid, L, R);
    if (L > mid) return query(node * 2 + 1, mid + 1, r, L, R);
    return merge(query(node * 2, l, mid, L, R),
                 query(node * 2 + 1, mid + 1, r, L, R));
}
```

【PSD04】单点树：区间最大值与阈值搜索 续

```
// 在闭区间 [L,R] 中找第一个 a[pos]>=val；不存在或 L>R 返回 -1。
int first_ge(int node, int l, int r, int L, int R, ll val) {
    if (L > R || R < l || r < L || tree[node].mx < val) return -1;
    if (l == r) return l;
    int mid = (l + r) >> 1;
    int pos = first_ge(node * 2, l, mid, L, R, val);
    if (pos != -1) return pos;
    return first_ge(node * 2 + 1, mid + 1, r, L, R, val);
}
```

接在上述代码后面的完整 main（已独立运行样例）：

样例输入：4 4 / 8 1 8 2 / 2 2 4 8 / 2 2 2 8 / 1 2 9 / 2 2 4 8 /
样例输出：3 / -1 / 2 /

```
int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, q;
    cin >> n >> q;
    for (int i = 1; i <= n; ++i) cin >> a[i];
    build(1, 1, n);
    while (q--) {
        int op; cin >> op;
        if (op == 1) {
            int pos; ll val;
            cin >> pos >> val;
            modify(1, 1, n, pos, val);
        } else {
            int l, r; ll val;
            cin >> l >> r >> val;
            cout << first_ge(1, 1, n, l, r, val) << '\n';
        }
    }
}
```

【PSD05】单点树：区间 gcd

识别：修改一个数，询问一段数共同的最大公约数。

思路：叶子统一非负，父节点 gcd(left.g,right.g)。

证明：gcd 结合律使分段方式无关；取绝对值不改变共同正因数。

约定：沿用 PSD02 边界；负数先取绝对值，gcd(0,0)=0；LLONG_MIN 不在契约内。

调用：读取 a[1..n]后 build(1,1,n)；具体可编译调用见 competition/data_structure/examples/point_gcd.cpp。

例子：完整可运行例程：competition/data_structure/examples/point_gcd.cpp；样例见 segment_sources.json。

复杂度：建树 O(n log V)，修改/查询 O(log n log V)，空间 O(n)。

易错：std::gcd 返回非负；LLONG_MIN 的绝对值不能用 ll 表示，不支持。；区间加无法只用旧 gcd 推出新 gcd；[2,4]与[2,6]原 gcd 同为 2，加 2 后分别为 2 和 4。

来源：template/segmen tree1.cpp；cf/segment_tree_cf_edu/1.cpp

规范源码：competition/data_structure/segment_point_gcd.hpp；完整独立代码如下；测试：tests/round02/test_segment.cpp

```
#include <bits/stdc++.h>
using namespace std;
using ll = long long;
// gcd 统一非负；禁止 LLONG_MIN，因为其绝对值不能用 ll 表示。
struct Node { ll g; };
Node make_node(ll val) {
    assert(val != LLONG_MIN);
    return {val < 0 ? -val : val};
}
Node merge(const Node& x, const Node& y) { return {gcd(x.g, y.g)}; }
// 公共单点树骨架：先放一个 Node / make_node / merge，再接本文件。
// 每个 segment_point_*.hpp 均已准确包含所需节点与本骨架，可直接 include。
// 保留旧代码 a/tree/node/l/r/mid 和递归结构；update 改名 modify。
// 非空树，1<=n<N；1<=pos<=n；查询必须 1<=L<=R<=n。
// a 仅为建树输入缓冲；modify 后当前值以 tree 为准。
const int N = 200000 + 5;
ll a[N];
Node tree[N * 4];

void pushup(int node) {
    tree[node] = merge(tree[node * 2], tree[node * 2 + 1]);
}
void build(int node, int l, int r) {
    if (l == r) {
        tree[node] = make_node(a[l]);
        return;
    }
    int mid = (l + r) >> 1;
    build(node * 2, l, mid);
    build(node * 2 + 1, mid + 1, r);
    pushup(node);
}
void apply(int node, ll val) {
    tree[node] = make_node(val);
}
void modify(int node, int l, int r, int pos, ll val) {
    if (l == r) {
        apply(node, val);
        return;
    }
    int mid = (l + r) >> 1;
    if (pos <= mid) modify(node * 2, l, mid, pos, val);
    else modify(node * 2 + 1, mid + 1, r, pos, val);
    pushup(node);
}
Node query(int node, int l, int r, int L, int R) {
    if (L <= l && r <= R) return tree[node];
    int mid = (l + r) >> 1;
    if (R <= mid) return query(node * 2, l, mid, L, R);
    if (L > mid) return query(node * 2 + 1, mid + 1, r, L, R);
    return merge(query(node * 2, l, mid, L, R),
                 query(node * 2 + 1, mid + 1, r, L, R));
}
```

【PSD05】单点树：区间 gcd 续

接在上述代码后面的完整 main（已独立运行样例）：

样例输入：3 3 / -6 10 0 / 2 1 3 / 1 2 -9 / 2 1 2 /
样例输出：2 / 3 /

```
int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, q;
    cin >> n >> q;
    for (int i = 1; i <= n; ++i) cin >> a[i];
    build(1, 1, n);
    while (q--) {
        int op;
        cin >> op;
        if (op == 1) {
            int pos; ll val;
            cin >> pos >> val;
            modify(1, 1, n, pos, val);
        } else {
            int l, r;
            cin >> l >> r;
            auto ans = query(1, 1, n, l, r);
            cout << ans.g << '\n';
        }
    }
}
```

【PSD06】单点树：最大值及出现次数

识别：同时询问区间最优值及达到最优值的元素数。

思路：左右最大值不等时只保留更大一侧，相等时次数相加。

证明：所有最大值只可能来自最大值等于父最大值的子段。

约定：沿用 PSD02 边界；单个叶 cnt=1；cnt 不超过 n；值可为任意 ll。

调用：读取 a[1..n]后 build(1,1,n)；具体可编译调用见 competition/data_structure/examples/max_count.cpp。

例子：完整可运行例程：competition/data_structure/examples/max_count.cpp；样例见 segment_sources.json。

复杂度：建树 O(n)，修改/查询 O(log n)，空间 O(n)。

易错：子段最大值不等时不能把两边次数相加。；点改叶子必须把 cnt 重置为 1。

来源：template/segmen tree1.cpp；cf/segment_tree_cf_edu/1.cpp

规范源码：competition/data_structure/segment_max_count.hpp；完整独立代码如下；测试：tests/round02/test_segment.cpp

```
#include <bits/stdc++.h>
using namespace std;
using ll = long long;
struct Node { ll mx; int cnt; };
Node make_node(ll val) { return {val, 1}; }
Node merge(const Node& x, const Node& y) {
    if (x.mx > y.mx) return x;
    if (x.mx < y.mx) return y;
    return {x.mx, x.cnt + y.cnt};
}
// 公共单点树骨架：先放一个 Node / make_node / merge，再接本文件。
// 每个 segment_point_*.hpp 均已准确包含所需节点与本骨架，可直接 include。
// 保留旧代码 a/tree/node/l/r/mid 和递归结构；update 改名 modify。
// 非空树，1<=n<N；1<=pos<=n；查询必须 1<=L<=R<=n。
// a 仅为建树输入缓冲；modify 后当前值以 tree 为准。
const int N = 200000 + 5;
ll a[N];
Node tree[N * 4];

void pushup(int node) {
    tree[node] = merge(tree[node * 2], tree[node * 2 + 1]);
}
void build(int node, int l, int r) {
    if (l == r) {
        tree[node] = make_node(a[l]);
        return;
    }
    int mid = (l + r) >> 1;
    build(node * 2, l, mid);
    build(node * 2 + 1, mid + 1, r);
    pushup(node);
}
void apply(int node, ll val) {
    tree[node] = make_node(val);
}
void modify(int node, int l, int r, int pos, ll val) {
    if (l == r) {
        apply(node, val);
        return;
    }
    int mid = (l + r) >> 1;
    if (pos <= mid) modify(node * 2, l, mid, pos, val);
    else modify(node * 2 + 1, mid + 1, r, pos, val);
    pushup(node);
}
Node query(int node, int l, int r, int L, int R) {
    if (L <= l && r <= R) return tree[node];
    int mid = (l + r) >> 1;
    if (R <= mid) return query(node * 2, l, mid, L, R);
    if (L > mid) return query(node * 2 + 1, mid + 1, r, L, R);
    return merge(query(node * 2, l, mid, L, R),
                 query(node * 2 + 1, mid + 1, r, L, R));
}
```

【PSD06】单点树：最大值及出现次数 续

接在上述代码后面的完整 main（已独立运行样例）：

样例输入：4 3 / 5 5 1 5 / 2 1 4 / 1 2 0 / 2 1 4 /
样例输出：5 3 / 5 2 /

```
int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, q;
    cin >> n >> q;
    for (int i = 1; i <= n; ++i) cin >> a[i];
    build(1, 1, n);
    while (q--) {
        int op;
        cin >> op;
        if (op == 1) {
            int pos; ll val;
            cin >> pos >> val;
            modify(1, 1, n, pos, val);
        } else {
            int l, r;
            cin >> l >> r;
            auto ans = query(1, 1, n, l, r);
            cout << ans.mx << ' ' << ans.cnt << '\n';
        }
    }
}
```

【PSD07】单点树：最大非空连续子段和

识别：点改后询问一段内部的最大连续子段和。

思路：保存 sum、pref、suff、best，跨中点候选是左 suff+右 pref。

证明：任一最优子段恰好属于全在左、全在右、跨中点三类之一。

约定：非空子段；全负返回最大负数。sum/pref/suff/best 及相加中间量均在 ll 内；无任意区间加。

调用：读取 a[1..n]后 build(1,1,n)；具体可编译调用见 competition/data_structure/examples/max_subarray.cpp。

例子：完整可运行例程：competition/data_structure/examples/max_subarray.cpp；样例见 segment_sources.json。

复杂度：建树 O(n)，修改/查询 O(log n)，空间 O(n)。

易错：只存 sum 与 best 无法计算跨中点最优值。；全负结果是最大负数；允许空段时应在输出取 max(0LL,best)。；任意区间加不能只修改这四个字段。

来源：template/segmen tree1.cpp；cf/segment_tree_cf_edu/1.cpp

规范源码：competition/data_structure/segment_max_subarray.hpp；完整独立代码如下；测试：tests/round02/test_segment.cpp

```
#include <bits/stdc++.h>
using namespace std;
using ll = long long;
// 非空连续子段：全负时返回最大的那个负数。
struct Node { ll sum, pref, suff, best; };
Node make_node(ll val) { return {val, val, val, val}; }
Node merge(const Node& x, const Node& y) {
    return {x.sum + y.sum,
            max(x.pref, x.sum + y.pref),
            max(y.suff, y.sum + x.suff),
            max({x.best, y.best, x.suff + y.pref})};
}
// 公共单点树骨架：先放一个 Node / make_node / merge，再接本文件。
// 每个 segment_point_*.hpp 均已准确包含所需节点与本骨架，可直接 include。
// 保留旧代码 a/tree/node/l/r/mid 和递归结构；update 改名 modify。
// 非空树，1<=n<N；1<=pos<=n；查询必须 1<=L<=R<=n。
// a 仅为建树输入缓冲；modify 后当前值以 tree 为准。
const int N = 200000 + 5;
ll a[N];
Node tree[N * 4];

void pushup(int node) {
    tree[node] = merge(tree[node * 2], tree[node * 2 + 1]);
}
void build(int node, int l, int r) {
    if (l == r) {
        tree[node] = make_node(a[l]);
        return;
    }
    int mid = (l + r) >> 1;
    build(node * 2, l, mid);
    build(node * 2 + 1, mid + 1, r);
    pushup(node);
}
void apply(int node, ll val) {
    tree[node] = make_node(val);
}
void modify(int node, int l, int r, int pos, ll val) {
    if (l == r) {
        apply(node, val);
        return;
    }
    int mid = (l + r) >> 1;
    if (pos <= mid) modify(node * 2, l, mid, pos, val);
    else modify(node * 2 + 1, mid + 1, r, pos, val);
    pushup(node);
}
```

【PSD07】单点树：最大非空连续子段和 续

```
Node query(int node, int l, int r, int L, int R) {
    if (L <= l && r <= R) return tree[node];
    int mid = (l + r) >> 1;
    if (R <= mid) return query(node * 2, l, mid, L, R);
    if (L > mid) return query(node * 2 + 1, mid + 1, r, L, R);
    return merge(query(node * 2, l, mid, L, R),
                 query(node * 2 + 1, mid + 1, r, L, R));
}
```

接在上述代码后面的完整 main（已独立运行样例）：

样例输入：4 3 / -5 -2 -7 -1 / 2 1 4 / 1 2 6 / 2 1 4 /

样例输出：-1 / 6 /

```
int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, q;
    cin >> n >> q;
    for (int i = 1; i <= n; ++i) cin >> a[i];
    build(1, 1, n);
    while (q--) {
        int op;
        cin >> op;
        if (op == 1) {
            int pos; ll val;
            cin >> pos >> val;
            modify(1, 1, n, pos, val);
        } else {
            int l, r;
            cin >> l >> r;
            auto ans = query(1, 1, n, l, r);
            cout << ans.best << '\n';
        }
    }
}
```

【PSD08】频率线段树：查第 k 个

识别：动态维护每个值/位置的出现次数，问第 k 个元素落在哪。

思路：左子树总频率若不足 k，跳过整段并把 k 减去左总量。

证明：非负性保证前缀和单调，左段包含的正是最前面的 left.sum 个元素。

约定：所有频率非负，总频率在 ll 内；k 从 1 开始；k<=0 或 k>总量返回-1；modify 为频率赋值。

调用：读取 a[1..n]后 build(1,1,n)；具体可编译调用见 competition/data_structure/examples/frequency.cpp。

例子：完整可运行例程：competition/data_structure/examples/frequency.cpp；样例见 segment_sources.json。

复杂度：建树 O(n)，修改/query/kth 均 O(log n)，空间 O(n)。

易错：负频率破坏前缀单调；如[2,-1,2]不能据左段总和可靠排位。；这是第 k 个元素所在下标；若下标来自离散化，还需反查原值。

来源：template/segmen tree1.cpp；cf/segment_tree_cf_edu/1.cpp

规范源码：competition/data_structure/segment_frequency.hpp；完整独立代码如下；测试：tests/round02/test_segment.cpp

```
#include <bits/stdc++.h>
using namespace std;
using ll = long long;
// 每个位置存非负频率；修改是把该位置频率改成 val。
struct Node { ll sum; };
Node make_node(ll val) { assert(val >= 0); return {val}; }
Node merge(const Node& x, const Node& y) { return {x.sum + y.sum}; }
// 公共单点树骨架：先放一个 Node / make_node / merge，再接本文件。
// 每个 segment_point_*.hpp 均已准确包含所需节点与本骨架，可直接 include。
// 保留旧代码 a/tree/node/l/r/mid 和递归结构；update 改名 modify。
// 非空树，1<=n<N；1<=pos<=n；查询必须 1<=L<=R<=n。
// a 仅为建树输入缓冲；modify 后当前值以 tree 为准。
const int N = 200000 + 5;
ll a[N];
Node tree[N * 4];

void pushup(int node) {
    tree[node] = merge(tree[node * 2], tree[node * 2 + 1]);
}
void build(int node, int l, int r) {
    if (l == r) {
        tree[node] = make_node(a[l]);
        return;
    }
    int mid = (l + r) >> 1;
    build(node * 2, l, mid);
    build(node * 2 + 1, mid + 1, r);
    pushup(node);
}
void apply(int node, ll val) {
    tree[node] = make_node(val);
}
void modify(int node, int l, int r, int pos, ll val) {
    if (l == r) {
        apply(node, val);
        return;
    }
    int mid = (l + r) >> 1;
    if (pos <= mid) modify(node * 2, l, mid, pos, val);
    else modify(node * 2 + 1, mid + 1, r, pos, val);
    pushup(node);
}
Node query(int node, int l, int r, int L, int R) {
    if (L <= l && r <= R) return tree[node];
    int mid = (l + r) >> 1;
    if (R <= mid) return query(node * 2, l, mid, L, R);
    if (L > mid) return query(node * 2 + 1, mid + 1, r, L, R);
    return merge(query(node * 2, l, mid, L, R),
```

【PSD08】频率线段树：查第 k 个 续

```
                query(node * 2 + 1, mid + 1, r, L, R));
}

// k 从 1 开始；返回第 k 个元素所在下标，非法 k 返回 -1。
// 必须保证所有频率非负，且总频率不超过 LLONG_MAX。
int kth(int node, int l, int r, ll k) {
    if (k <= 0 || k > tree[node].sum) return -1;
    if (l == r) return l;
    int mid = (l + r) >> 1;
    if (k <= tree[node * 2].sum) return kth(node * 2, l, mid, k);
    return kth(node * 2 + 1, mid + 1, r, k - tree[node * 2].sum);
}
```

接在上述代码后面的完整 main（已独立运行样例）：

样例输入：4 4 / 0 2 0 3 / 2 1 / 2 5 / 1 2 0 / 2 1 /

样例输出：2 / 4 / 4 /

```
int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, q;
    cin >> n >> q;
    for (int i = 1; i <= n; ++i) cin >> a[i];
    build(1, 1, n);
    while (q--) {
        int op; cin >> op;
        if (op == 1) {
            int pos; ll val;
            cin >> pos >> val;
            modify(1, 1, n, pos, val);
        } else {
            ll k; cin >> k;
            cout << kth(1, 1, n, k) << '\n';
        }
    }
}
```

【PSD09】单点树：多个可合并统计量

识别：一段同时问和、最小值、最大值、gcd。

思路：各字段各自形成可合并统计量，Node 存它们的乘积状态。

证明：每个字段独立正确且查询划分相同，整组答案同时正确。

约定：沿用 PSD02 边界；受 gcd 约束禁止 LLONG_MIN；所有和和中间量在 ll 内；不支持区间加。

调用：读取 a[1..n]后 build(1,1,n)；具体可编译调用见 competition/data_structure/examples/point_stats.cpp。

例子：完整可运行例程：competition/data_structure/examples/point_stats.cpp；样例见 segment_sources.json。

复杂度：建树 O(n log V)，修改/查询 O(log n log V)，空间 O(n)。

易错：可合并不等于可做 lazy；gcd 不支持从 gcd 本身处理区间加。；矩阵等非交换 merge 必须保留左右顺序。

来源：template/segmen tree1.cpp；cf/segment_tree_cf_edu/1.cpp

规范源码：competition/data_structure/segment_point_stats.hpp；完整独立代码如下；测试：tests/round02/test_segment.cpp

```
#include <bits/stdc++.h>
using namespace std;
using ll = long long;
// 一起维护的字段须都能从左右子段独立合并；禁止 LLONG_MIN。
struct Node { ll sum, mn, mx, g; };
Node make_node(ll val) {
    assert(val != LLONG_MIN);
    return {val, val, val, val < 0 ? -val : val};
}
Node merge(const Node& x, const Node& y) {
    return {x.sum + y.sum, min(x.mn, y.mn), max(x.mx, y.mx), gcd(x.g, y.g)};
}
// 公共单点树骨架：先放一个 Node / make_node / merge，再接本文件。
// 每个 segment_point_*.hpp 均已准确包含所需节点与本骨架，可直接 include。
// 保留旧代码 a/tree/node/l/r/mid 和递归结构；update 改名 modify。
// 非空树，1<=n<N；1<=pos<=n；查询必须 1<=L<=R<=n。
// a 仅为建树输入缓冲；modify 后当前值以 tree 为准。
const int N = 200000 + 5;
ll a[N];
Node tree[N * 4];

void pushup(int node) {
    tree[node] = merge(tree[node * 2], tree[node * 2 + 1]);
}
void build(int node, int l, int r) {
    if (l == r) {
        tree[node] = make_node(a[l]);
        return;
    }
    int mid = (l + r) >> 1;
    build(node * 2, l, mid);
    build(node * 2 + 1, mid + 1, r);
    pushup(node);
}
void apply(int node, ll val) {
    tree[node] = make_node(val);
}
void modify(int node, int l, int r, int pos, ll val) {
    if (l == r) {
        apply(node, val);
        return;
    }
    int mid = (l + r) >> 1;
    if (pos <= mid) modify(node * 2, l, mid, pos, val);
    else modify(node * 2 + 1, mid + 1, r, pos, val);
    pushup(node);
}
Node query(int node, int l, int r, int L, int R) {
    if (L <= l && r <= R) return tree[node];
```

【PSD09】单点树：多个可合并统计量 续

```
    int mid = (l + r) >> 1;
    if (R <= mid) return query(node * 2, l, mid, L, R);
    if (L > mid) return query(node * 2 + 1, mid + 1, r, L, R);
    return merge(query(node * 2, l, mid, L, R),
                 query(node * 2 + 1, mid + 1, r, L, R));
}
```

接在上述代码后面的完整 main（已独立运行样例）：

样例输入：4 3 / 6 -12 9 3 / 2 1 4 / 1 2 15 / 2 1 3 /

样例输出：6 -12 9 3 / 30 6 15 3 /

```
int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, q;
    cin >> n >> q;
    for (int i = 1; i <= n; ++i) cin >> a[i];
    build(1, 1, n);
    while (q--) {
        int op;
        cin >> op;
        if (op == 1) {
            int pos; ll val;
            cin >> pos >> val;
            modify(1, 1, n, pos, val);
        } else {
            int l, r;
            cin >> l >> r;
            auto ans = query(1, 1, n, l, r);
            cout << ans.sum << ' ' << ans.mn << ' ' << ans.mx
                 << ' ' << ans.g << '\n';
        }
    }
}
```

【PSD10】区间加：和、最小值、最大值

识别：在线对一段整体加 x，再询问该段或任意子段的和/最小/最大。

思路：整段加 x 使 sum 增加 len*x，mn/mx 各增加 x；lazy 记录已计入本节点尚未传子的增量。

证明：整段平移保持大小顺序，标记相加表示时间上的连续平移；下传后子树与父树状态一致。

约定：1<=n<N；修改和查询是合法非空闭区间；a 为初始输入；和、标记、长度*x 及中间量在 ll 内。

调用：读取 a[1..n]后 build(1,1,n)；具体可编译调用见 competition/data_structure/examples/range_add.cpp。

例子：完整可运行例程：competition/data_structure/examples/range_add.cpp；样例见 segment_sources.json。

复杂度：建树 O(n)，区间修改/查询 O(log n)，空间 O(n)。

易错：查询部分覆盖也必须 pushdown。；sum 加 len*x，mn/mx 只加 x。；build 清 lazy 后才可多测复用；不要直接调用旧 update 跳过标记。

来源：template/Segment tree2.cpp；cf/date/6/6.9/1_special_segment_tree.cpp

规范源码：competition/data_structure/segment_range_add.hpp；完整独立代码如下；测试：tests/round02/test_segment.cpp

```
#include <bits/stdc++.h>
using namespace std;
using ll = long long;

// 继承旧 Segment tree2 / 6.9 的全局树与递归写法。
// 非空树，1<=n<N；修改/查询须为 [1,n] 内非空闭区间。
// 所有和、标记、长度*增量及中间值须在 ll 范围。
const int N = 200000 + 5;
struct Node { ll sum, mn, mx; };
ll a[N], lazy[N * 4];
Node tree[N * 4];
Node merge(const Node& x, const Node& y) {
    return {x.sum + y.sum, min(x.mn, y.mn), max(x.mx, y.mx)};
}
void pushup(int node) {
    tree[node] = merge(tree[node * 2], tree[node * 2 + 1]);
}
void apply(int node, int l, int r, ll val) {
    tree[node].sum += (r - l + 1LL) * val;
    tree[node].mn += val;
    tree[node].mx += val;
    lazy[node] += val;
}
void pushdown(int node, int l, int r) {
    if (lazy[node] == 0 || l == r) return;
    int mid = (l + r) >> 1;
    apply(node * 2, l, mid, lazy[node]);
    apply(node * 2 + 1, mid + 1, r, lazy[node]);
    lazy[node] = 0;
}
void build(int node, int l, int r) {
    lazy[node] = 0;
    if (l == r) {
        tree[node] = {a[l], a[l], a[l]};
        return;
    }
    int mid = (l + r) >> 1;
    build(node * 2, l, mid);
    build(node * 2 + 1, mid + 1, r);
    pushup(node);
}
void modify(int node, int l, int r, int L, int R, ll val) {
    if (L <= l && r <= R) {
        apply(node, l, r, val);
        return;
    }
    pushdown(node, l, r);
    int mid = (l + r) >> 1;
```

【PSD10】区间加：和、最小值、最大值 续

```
    if (L <= mid) modify(node * 2, l, mid, L, R, val);
    if (R > mid) modify(node * 2 + 1, mid + 1, r, L, R, val);
    pushup(node);
}
Node query(int node, int l, int r, int L, int R) {
    if (L <= l && r <= R) return tree[node];
    pushdown(node, l, r);
    int mid = (l + r) >> 1;
    if (R <= mid) return query(node * 2, l, mid, L, R);
    if (L > mid) return query(node * 2 + 1, mid + 1, r, L, R);
    return merge(query(node * 2, l, mid, L, R),
                 query(node * 2 + 1, mid + 1, r, L, R));
}
```

接在上述代码后面的完整 main（已独立运行样例）：

样例输入：3 4 / 3 -2 4 / 2 1 3 / 1 1 2 5 / 2 1 3 / 2 2 2 /

样例输出：5 -2 4 / 15 3 8 / 3 3 3 /

```
int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, q;
    cin >> n >> q;
    for (int i = 1; i <= n; ++i) cin >> a[i];
    build(1, 1, n);
    while (q--) {
        int op, l, r;
        cin >> op >> l >> r;
        if (op == 1) {
            ll val; cin >> val;
            modify(1, 1, n, l, r, val);
        } else {
            auto ans = query(1, 1, n, l, r);
            cout << ans.sum << ' ' << ans.mn << ' ' << ans.mx << '\n';
        }
    }
}
```

【PSD11】区间赋值与区间加：和

识别：对区间覆盖成 x，也可能随后整体加 x；问任意区间和。

思路：has_set 独立记录赋值是否存在；标签表示先可选 set，再 add。

证明：新 set 覆盖全部旧操作，新 add 接在旧操作后；下传先 set 后 add，等价于原时间顺序。

约定：1<=n<N；合法非空闭区间；true 表示赋值、false 表示加；所有中间量在 ll 内；允许赋值 0。

调用：读取 a[1..n]后 build(1,1,n)；具体可编译调用见 competition/data_structure/examples/set_add.cpp。

例子：完整可运行例程：competition/data_structure/examples/set_add.cpp；样例见 segment_sources.json。

复杂度：建树 O(n)，修改/查询 O(log n)，空间 O(n)。

易错：不能用 set!=0 判断有无赋值，set=0 也是有效操作。；set(v)后 add(d)得到 v+d；add(d)后 set(v)只剩 v。；下传先 set 再 add，新的 set 必须清掉旧 add。

来源：template/Segment tree2.cpp；cf/date/6/6.9/1_special_segment_tree.cpp

规范源码：competition/data_structure/segment_set_add.hpp；完整独立代码如下；测试：tests/round02/test_segment.cpp

```
#include <bits/stdc++.h>
using namespace std;
using ll = long long;

// 在旧区间加树上扩展赋值；true=赋值，false=加。
// 非空树，1<=n<N；区间须合法；所有中间值须在 ll 范围。
const int N = 200000 + 5;
struct Tag { bool has_set; ll set, add; };
ll a[N], tree[N * 4];
Tag lazy[N * 4];
void pushup(int node) {
    tree[node] = tree[node * 2] + tree[node * 2 + 1];
}
void apply(int node, int l, int r, ll val, bool is_set) {
    if (is_set) {
        tree[node] = (r - l + 1LL) * val;
        lazy[node] = {true, val, 0}; // 新赋值覆盖之前的赋值和增量。
    } else {
        tree[node] += (r - l + 1LL) * val;
        lazy[node].add += val;
    }
}
void pushdown(int node, int l, int r) {
    if (l == r) return;
    int mid = (l + r) >> 1;
    if (lazy[node].has_set) { // 下传必须先 set，再 add。
        apply(node * 2, l, mid, lazy[node].set, true);
        apply(node * 2 + 1, mid + 1, r, lazy[node].set, true);
    }
    if (lazy[node].add != 0) {
        apply(node * 2, l, mid, lazy[node].add, false);
        apply(node * 2 + 1, mid + 1, r, lazy[node].add, false);
    }
    lazy[node] = {};
}
void build(int node, int l, int r) {
    lazy[node] = {};
    if (l == r) { tree[node] = a[l]; return; }
    int mid = (l + r) >> 1;
    build(node * 2, l, mid);
    build(node * 2 + 1, mid + 1, r);
    pushup(node);
}
void modify(int node, int l, int r, int L, int R, ll val, bool is_set) {
    if (L <= l && r <= R) {
        apply(node, l, r, val, is_set);
        return;
    }
```

【PSD11】区间赋值与区间加：和 续

```
    pushdown(node, l, r);
    int mid = (l + r) >> 1;
    if (L <= mid) modify(node * 2, l, mid, L, R, val, is_set);
    if (R > mid) modify(node * 2 + 1, mid + 1, r, L, R, val, is_set);
    pushup(node);
}
ll query(int node, int l, int r, int L, int R) {
    if (L <= l && r <= R) return tree[node];
    pushdown(node, l, r);
    int mid = (l + r) >> 1;
    ll ans = 0;
    if (L <= mid) ans += query(node * 2, l, mid, L, R);
    if (R > mid) ans += query(node * 2 + 1, mid + 1, r, L, R);
    return ans;
}
```

接在上述代码后面的完整 main（已独立运行样例）：

样例输入：4 6 / 1 2 3 4 / 1 1 4 0 / 2 2 3 5 / 3 1 4 / 2 1 4 2 / 1 2 3 0 / 3 1 4 /

样例输出：10 / 4 /

```
int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, q;
    cin >> n >> q;
    for (int i = 1; i <= n; ++i) cin >> a[i];
    build(1, 1, n);
    while (q--) {
        int op, l, r;
        cin >> op >> l >> r;
        if (op <= 2) {
            ll val; cin >> val;
            modify(1, 1, n, l, r, val, op == 1);
        } else cout << query(1, 1, n, l, r) << '\n';
    }
}
```

【STL01】可执行 STL 容器与算法短例

识别：选择容器、排序/二分/去重/计数、读写字符串，现场查小段调用。

思路：先按需要的操作选容器，再让每个 API 落在一个可运行的具体任务中。

证明：比较器保持严格弱序才能排序；排序后相同元素相邻，unique+erase 才能得到去重序列；lower/upper 界定整个等值段。

约定：GNU++20；每个花括号是独立短例，断言给出预期。算法区间半开；查询前满足排序/非空/下标等条件。示例不是统一封装类。

调用：stl_container_recipes(); stl_algorithm_recipes(); stl_string_recipes(); // 学习/验证；比赛抄所需局部花括号。

例子：全部短例由 test_stl.cpp 调用：容器、升降序重复段、稳定排序、第 k 小、去重、删除奇数、循环移位、aab 的 3 种排列、64 位前后缀、递归 lambda、字符串 npos。

复杂度：array/vector 访问 O(1)；有序集合 O(log n)；哈希平均 O(1)最坏 O(n)；sort O(n log n)；算法扫描 O(n)，nth_element 平均 O(n)。

易错：reserve 不改变 vector.size，扩容使旧指针/引用/迭代器失效。；cmp 不能写<=；降序排序后查找也必须 greater。；multiset.erase(value)删除全部等值项；只删一次要 find+erase(iterator)。；accumulate 用 0LL；partial_sum 输入元素类型也要足够宽。；unordered 容器没有有序前驱，reserve 也不能保证防止恶意碰撞。

来源：template/data_sturcture.cpp

规范源码：competition/stl/recipes.hpp；完整独立代码如下；测试：tests/round02/test_stl.cpp

```
#include <bits/stdc++.h>
using namespace std;
using ll = long long;

// 可执行短例：每个花括号可单独抄用；assert 给出具体结果。
inline void stl_container_recipes() {
    { // array / vector：定长与动态序列。
        array<int, 4> a{3, 1, 2, 0};
        sort(a.begin(), a.end());
        assert(a.front() == 0 && a.back() == 3);
        a.fill(7); assert(a[2] == 7);
        vector<int> b; b.reserve(8); // reserve 不改变 size。
        b.push_back(3); b.emplace_back(4); b.resize(4, 9);
        b.pop_back(); b.erase(b.begin());
        assert((b == vector<int>{4, 9}));
        b.clear(); assert(b.empty()); // 多测清逻辑长度，容量可复用。
    }
    { // deque：两端增删，仍可 O(1)按下标访问。
        deque<int> q;
        q.push_back(2); q.push_front(1); q.push_back(3);
        q.pop_front(); q.pop_back();
        assert(q.size() == 1 && q[0] == 2);
    }
    { // pair / tuple：字典序排序和结构化绑定。
        vector<pair<int, int>> tasks{{2, 1}, {1, 7}, {1, 3}};
        sort(tasks.begin(), tasks.end());
        auto [time, id] = tasks.front();
        auto state = make_tuple(time, id, string("ready"));
        auto [t, i, label] = state;
        assert(t == 1 && i == 3 && label == "ready");
        assert(get<1>(state) == 3);
    }
    { // stack：左括号等待最近的右括号。
        string s = "(()())";
        stack<int> left;
        bool ok = true;
        for (int i = 0; i < int(s.size()); ++i) {
            if (s[i] == '(') left.push(i);
            else if (left.empty()) { ok = false; break; }
            else left.pop();
        }
        assert(ok && left.empty());
```

【STL01】可执行 STL 容器与算法短例 续

```
    }
    { // queue：先进先出，BFS 已发现结点的处理顺序。
        queue<int> q;
        q.push(1); q.push(2);
        vector<int> order;
        while (!q.empty()) { order.push_back(q.front()); q.pop(); }
        assert((order == vector<int>{1, 2}));
    }
    { // 三种常用堆，以及结构体自定义优先级。
        priority_queue<ll> big;
        priority_queue<ll, vector<ll>, greater<ll>> small;
        priority_queue<pair<ll, int>, vector<pair<ll, int>>,
                       greater<pair<ll, int>>> pairs;
        for (ll x : {2LL, 8000000000LL, 3LL}) { big.push(x); small.push(x); }
        assert(big.top() == 8000000000LL && small.top() == 2);
        pairs.push({4000000000LL, 9}); pairs.push({4000000000LL, 3});
        assert(pairs.top() == make_pair(4000000000LL, 3));
        struct Job { int deadline, id; };
        auto cmp = [](const Job& a, const Job& b) {
            return tie(a.deadline, a.id) > tie(b.deadline, b.id);
        };
        priority_queue<Job, vector<Job>, decltype(cmp)> pq(cmp);
        pq.push({4, 1}); pq.push({2, 9}); pq.push({2, 3});
        assert(pq.top().deadline == 2 && pq.top().id == 3);
    }
    { // set 去重；multiset 保留重复；删除一个要用迭代器。
        set<int> s{3, 1, 3}; assert(s.size() == 2);
        assert(s.contains(1) && !s.contains(2)); // C++20 等值存在性。
        assert(s.find(1) != s.end() && s.find(9) == s.end());
        s.insert(2); assert(s.count(2) == 1);
        assert(s.erase(2) == 1); // set 无重复；按值删除返回删除数。
        auto it = s.lower_bound(2); assert(it != s.end() && *it == 3);
        multiset<int> ms{2, 2, 5};
        auto pos = ms.find(2);
        if (pos != ms.end()) ms.erase(pos);
        assert(ms.count(2) == 1); // ms.erase(2)会删所有 2。
        ms.insert(2);
        assert(ms.erase(2) == 2 && !ms.contains(2)); // 按值删全部副本。
    }
    { // map 计频；multimap 存同一时间的多项事件。
        map<string, int> cnt;
        for (string x : {"b", "a", "b"}) ++cnt[x];
        assert(cnt.begin()->first == "a" && cnt.at("b") == 2);
        assert(cnt.find("c") == cnt.end()); // 查询不想插入时不用[]。
        multimap<int, string> events{{2, "B"}, {1, "A"}, {2, "C"}};
        auto [l, r] = events.equal_range(2);
        int total = 0;
        for (auto it = l; it != r; ++it) ++total;
        assert(total == 2);
    }
    { // 哈希去重/计频；只要等值查询，不要求前驱或排序。
        unordered_set<int> seen;
        unordered_map<int, int> cnt;
        seen.max_load_factor(0.7f); seen.reserve(100);
        cnt.max_load_factor(0.7f); cnt.reserve(100);
        for (int x : {4, 4, -1}) { seen.insert(x); ++cnt[x]; }
        assert(seen.size() == 2 && cnt.at(4) == 2);
    }
}

inline void stl_algorithm_recipes() {
    { // 自定义结构体排序；同分按 id 升序，必须严格小于。
        struct Node { int score, id; };
        vector<Node> a{{9, 2}, {7, 1}, {9, 1}};
        sort(a.begin(), a.end(), [](const Node& x, const Node& y) {
            if (x.score != y.score) return x.score > y.score;
            return x.id < y.id;
```

【STL01】可执行 STL 容器与算法短例 续

```
    });
    assert(a[0].id == 1 && a[1].id == 2);
    stable_sort(a.begin(), a.end(), [](const Node& x, const Node& y) {
        return x.score < y.score;
    });
    assert(a[0].id == 1 && a[1].id == 1 && a[2].id == 2);
}
{ // 升序找等值段；降序必须传同一比较器。
    vector<int> a{1, 2, 2, 5};
    auto l = lower_bound(a.begin(), a.end(), 2);
    auto r = upper_bound(a.begin(), a.end(), 2);
    assert(r - l == 2 && binary_search(a.begin(), a.end(), 5));
    sort(a.begin(), a.end(), greater<int>());
    auto dl = lower_bound(a.begin(), a.end(), 2, greater<int>());
    auto dr = upper_bound(a.begin(), a.end(), 2, greater<int>());
    assert(dl - a.begin() == 1 && dr - a.begin() == 3);
    assert(binary_search(a.begin(), a.end(), 5, greater<int>()));
}
{ // 第 k 小只需分隔，不要求其余位置有序；k 为 0-based。
    vector<int> a{7, 1, 4, 4, 9};
    int k = 2;
    nth_element(a.begin(), a.begin() + k, a.end());
    assert(a[k] == 4);
    for (int i = 0; i < k; ++i) assert(a[i] <= a[k]);
    for (int i = k + 1; i < int(a.size()); ++i) assert(a[i] >= a[k]);
}
{ // unique 只压相邻重复；erase 才实际缩短容器。
    vector<int> a{3, 1, 3, 2, 2};
    sort(a.begin(), a.end());
    a.erase(unique(a.begin(), a.end()), a.end());
    assert((a == vector<int>{1, 2, 3}));
    a.erase(remove_if(a.begin(), a.end(),
        [](int x) { return x % 2; }), a.end());
    assert((a == vector<int>{2}));
}
{ // 反转/循环左移；排列枚举必须包含初始排列。
    vector<int> a{1, 2, 3, 4};
    reverse(a.begin(), a.end());
    rotate(a.begin(), a.begin() + 1, a.end());
    assert((a == vector<int>{3, 2, 1, 4}));
    string p = "aab";
    int ways = 0;
    do { ++ways; } while (next_permutation(p.begin(), p.end()));
    assert(ways == 3); // 初始有序时，重复元素不会重复枚举。
    assert(prev_permutation(p.begin(), p.end()) == false);
}
{ // 64 位前缀/后缀；partial_sum 的内部类型取决于输入类型。
    vector<long long> a{2000000000LL, 2000000000LL, -1};
    auto sum = accumulate(a.begin(), a.end(), 0LL);
    vector<long long> pre(a.size()), suf(a.size());
    partial_sum(a.begin(), a.end(), pre.begin());
    partial_sum(a.rbegin(), a.rend(), suf.rbegin());
    assert(sum == 3999999999LL && pre[1] == 4000000000LL);
    assert(suf[1] == 1999999999LL);
    assert(*min_element(a.begin(), a.end()) == -1);
    assert(*max_element(a.begin(), a.end()) == 2000000000LL);
}
{ // 递归 lambda：参数 self 传自身，枚举每项选/不选。
    vector<int> a{1, 3}, sums;
    auto dfs = [&](auto&& self, int i, int sum) -> void {
        if (i == int(a.size())) { sums.push_back(sum); return; }
        self(self, i + 1, sum);
        self(self, i + 1, sum + a[i]);
    };
    dfs(dfs, 0, 0); sort(sums.begin(), sums.end());
    assert((sums == vector<int>{0, 1, 3, 4}));
}
```

【STL01】可执行 STL 容器与算法短例 续

```
}

inline void stl_string_recipes() {
    string s = "abC abC";
    size_t pos = s.find("abC", 1);
    assert(pos != string::npos && pos == 4);
    string part = s.substr(pos, 3); // 第二参数是长度，不是右端点。
    reverse(part.begin(), part.end());
    assert(part == "Cba");
    for (char& c : part) c = char(tolower((unsigned char)c));
    assert(part == "cba");
    for (char& c : part) c = char(toupper((unsigned char)c));
    assert(part == "CBA"); // 按字节/当前 locale；不处理 UTF-8 汉字。
    assert(s.find("xyz") == string::npos);
}
```

【STL02】堆与有序集合的典型题型

识别：反复取最优、流式 TopK、在线/滑窗中位数、取消任务、查前后继或分配最小够用资源。

思路：堆只维护下一次最优；有序集合多维护全局顺序换来前后继/删除；两半结构把中位数变成边界元素。

证明：TopK 淘汰候选中最小者不会丢失前 k 大；双堆/双集合保持小半<=大半且大小差<=1，边界就是下中位数。

约定：Huffman 权重非负且总成本 ll 不溢出；TopK 的 k>=0；中位数查询非空。延迟堆 id 为 1..n，每次 init 清空。窗口 k=0 或 k>n 返回空，极差不溢出。

调用：ll cost=huffman_cost(a); auto big=top_k_largest(a,k); auto small=top_k_smallest(a,k); median_init(); for(ll x:a) median_add(x); ll med=median_lower(); auto windows=window_lower_medians(a,k);

例子：Huffman{1,2,3,4}成本 19；{3,1,9,9,-2}取前 3 大得到 9,9,3，取前 3 小得到-2,1,3；{1,4,7}中 4 的严格邻居为 1/7；资源{2,2,5}依次要 2/3 后剩 2。

复杂度：Huffman O(n log n)；TopK O(n log(k+1))、空间 O(k)；双堆插入 O(log n)；延迟堆 Q 次总 O(Q log Q)、空间 O(Q)。前后继/资源 O(log n)，窗口 O(n log k)。

易错：堆只能看顶，不能用它直接查任意前驱或删除中间元素。；中位数定义为第(n+1)/2 小，即偶数长度取下中位数；平均两个中值需防溢出。；过期项可能大量堆积，延迟删除的空间按操作数而非当前活跃数估计。；只需窗口最值时单调队列可 O(n)，需要中位数才要更丰富的维护。

来源：template/data_sturcture.cpp

规范源码：competition/stl/heaps_sets.hpp；完整独立代码如下；测试：tests/round02/test_stl.cpp

```
#include <bits/stdc++.h>
using namespace std;
using ll = long long;

// 每次合并两份的代价是其和，所有权重非负；总和/答案不溢出 ll。
inline ll huffman_cost(const vector<ll>& a) {
    priority_queue<ll, vector<ll>, greater<ll>> q(a.begin(), a.end());
    ll ans = 0;
    while (q.size() > 1) {
        ll x = q.top(); q.pop();
        ll y = q.top(); q.pop();
        ans += x + y; q.push(x + y);
    }
    return ans;
}
// 流式保留最大的 k 个，重复值保留；输出降序，k 可超过元素数。
inline vector<ll> top_k_largest(const vector<ll>& a, int k) {
    assert(k >= 0);
    priority_queue<ll, vector<ll>, greater<ll>> q;
    for (ll x : a) {
        q.push(x);
        if (int(q.size()) > k) q.pop();
    }
    vector<ll> ans;
    while (!q.empty()) { ans.push_back(q.top()); q.pop(); }
    reverse(ans.begin(), ans.end());
    return ans;
}

// 流式保留最小的 k 个，用大根堆淘汰当前最大者；输出升序。
inline vector<ll> top_k_smallest(const vector<ll>& a, int k) {
    assert(k >= 0);
    priority_queue<ll> q;
    for (ll x : a) {
        q.push(x);
        if (int(q.size()) > k) q.pop();
    }
    vector<ll> ans;
    while (!q.empty()) { ans.push_back(q.top()); q.pop(); }
    reverse(ans.begin(), ans.end());
    return ans;
}
```

【STL02】堆与有序集合的典型题型 续

```
// 动态只插入的下中位数：low 有 ceil(n/2)个小值，high 存其余。
inline priority_queue<ll> low;
inline priority_queue<ll, vector<ll>, greater<ll>> high;
inline void median_init() { low = {}; high = {}; }
inline void median_add(ll x) {
    if (low.empty() || x <= low.top()) low.push(x);
    else high.push(x);
    if (low.size() < high.size()) {
        low.push(high.top()); high.pop();
    }
    if (low.size() > high.size() + 1) {
        high.push(low.top()); low.pop();
    }
}
inline ll median_lower() { assert(!low.empty()); return low.top(); }

// 可改优先级/取消任务：旧条目留在堆里，到堆顶时按版本丢弃。
// id 为 1..n；相同值时较大 id 优先；版本次数必须能用 ll 表示。
inline priority_queue<tuple<ll, int, ll>> task_heap;
inline vector<ll> version;
inline vector<bool> active;
inline void lazy_init(int n) {
    assert(n >= 0);
    task_heap = {}; version.assign(n + 1, 0); active.assign(n + 1, false);
}
inline void lazy_set(int id, ll value) {
    assert(1 <= id && id < int(version.size()));
    active[id] = true;
    task_heap.emplace(value, id, ++version[id]);
}
inline void lazy_erase(int id) {
    assert(1 <= id && id < int(version.size()));
    active[id] = false;
}
inline optional<pair<ll, int>> lazy_top() {
    while (!task_heap.empty()) {
        auto [value, id, stamp] = task_heap.top();
        if (!active[id] || stamp != version[id]) task_heap.pop();
        else return pair<ll, int>{value, id};
    }
    return nullopt;
}

// 严格前驱< x 与严格后继> x；不存在时返回 nullopt。
inline pair<optional<ll>, optional<ll>>
strict_neighbors(const set<ll>& s, ll x) {
    optional<ll> before, after;
    auto it = s.lower_bound(x);
    if (it != s.begin()) before = *prev(it);
    it = s.upper_bound(x);
    if (it != s.end()) after = *it;
    return {before, after};
}
// 每份资源只用一次；选容量>=need 的最小容量，保留更大资源。
inline optional<ll> assign_at_least(multiset<ll>& slots, ll need) {
    auto it = slots.lower_bound(need);
    if (it == slots.end()) return nullopt;
    ll value = *it; slots.erase(it); return value;
}
// 定长滑窗极差。k=0/空数组返回空；非空窗口 max-min 不溢出 ll。
inline vector<ll> window_spread(const vector<ll>& a, int k) {
    assert(k >= 0);
    vector<ll> ans;
    if (!k || k > int(a.size())) return ans;
    multiset<ll> window;
    for (int i = 0; i < int(a.size()); ++i) {
        window.insert(a[i]);
```

【STL02】堆与有序集合的典型题型 续

```
        if (i >= k) window.erase(window.find(a[i - k]));
        if (i + 1 >= k) ans.push_back(*window.rbegin() - *window.begin());
    }
    return ans;
}
// 双 multiset 滑窗下中位数，支持删除；left 比 right 多 0 或 1 个。
inline vector<ll> window_lower_medians(const vector<ll>& a, int k) {
    assert(k >= 0);
    vector<ll> ans;
    if (!k || k > int(a.size())) return ans;
    multiset<ll> left, right;
    auto balance = [&]() {
        while (left.size() > right.size() + 1) {
            auto it = prev(left.end()); right.insert(*it); left.erase(it);
        }
        while (left.size() < right.size()) {
            auto it = right.begin(); left.insert(*it); right.erase(it);
        }
    };
    for (int i = 0; i < int(a.size()); ++i) {
        if (left.empty() || a[i] <= *left.rbegin()) left.insert(a[i]);
        else right.insert(a[i]);
        balance();
        if (i >= k) {
            auto it = left.find(a[i - k]);
            if (it != left.end()) left.erase(it);
            else right.erase(right.find(a[i - k]));
            balance();
        }
        if (i + 1 >= k) ans.push_back(*left.rbegin());
    }
    return ans;
}
```

【TRS01】实数二分与三分

识别：实数答案可行性单调，或目标函数在区间内单谷。

思路：二分保留可行/不可行分界；三分通过两点函数值删去不含最小值的一侧。

证明：若 m1<m2 且 f(m1)<=f(m2)，单谷且无非最优平台保证右侧 m2 以外不会更优，因此可令 r=m2。

约定：二分 check 仅 false->true 且右端可行；三分函数连续，最小区间外严格下降/上升，可全程单调，不允许非最优平台。边界和 r-l 有限；check/f 数值可靠。

调用：auto x=real_first_true(0,2,[](long double t){return t*t>=2;}); auto p=ternary_min(-100,100,[](long double t){return (t-3)*(t-3)+7;});

例子：check(x):x*x>=2，在[0,2]返回 sqrt(2)近似；f(x)=(x-3)^2+7 在[-100,100]最小点 3。

复杂度：固定 200 轮，O(200*一次 check/f)，空间 O(1)。

易错：二分需要谓词单调，三分需要目标单峰，两者不互相替代。；双谷函数或非最优平台会误删最优点；迭代更多不会修复模型。；200 轮后仍受浮点精度与函数计算误差限制；不能把相等输出当成精确根。

来源：template/Ternary_Search.cpp

规范源码：competition/runtime/real_search.hpp；完整独立代码如下；测试：tests/round02/test_runtime.cpp

```
#include <bits/stdc++.h>
using namespace std;

// check 在 [l,r] 上只能 false -> true，check(r) 为 true。
// 返回转折点近似值；边界、r-l、check 运算须有限。
template<class Check>
long double real_first_true(long double l, long double r, Check check) {
    assert(l <= r && check(r));
    for (int it = 0; it < 200; ++it) {
        long double mid = l + (r - l) / 2;
        if (check(mid)) r = mid;
        else l = mid;
    }
    return r;
}
// f 连续且先严格下降后严格上升，可有连续平底最小区间。
// 也允许全程单调；不得在非最优值处存在平坦区间。
// 数值噪声/多峰函数无保证；200 次不能修复错误模型。
template<class Function>
long double ternary_min(long double l, long double r, Function f) {
    assert(l <= r);
    for (int it = 0; it < 200; ++it) {
        long double m1 = l + (r - l) / 3;
        long double m2 = r - (r - l) / 3;
        if (f(m1) <= f(m2)) r = m2;
        else l = m1;
    }
    return l + (r - l) / 2;
}
```

【TRS02】个人数组字符串 Trie：前缀频次与整词频次

识别：大量单词共享前缀，查询以 p 开头有多少串或完整字符串出现次数。

思路：一个结点对应从根到它的前缀；沿用户原 tr/cnt/dep/tot 插入路径累加前缀频次，终点额外记整词。

证明：每个插入串恰好经过它的每个前缀结点一次，因此 cnt 是前缀拥有数；只在终点加 word_end 可区分前缀和整词。

约定：小写 a..z，根 0；tr[u][c]=0 表示边不存在。TRIE_N=500005，非根节点<=500004；每次清空后总插入长度<TRIE_N 是充分容量条件。重复串计重，空串记根；计数不超过 int。

调用：trie_init(); trie_insert("ab"); trie_insert("abc"); int p=trie_prefix("a"); int exact=trie_count("ab");

例子：插 ab、abc、ab；prefix(a)=3、count(ab)=2、count(a)=0；init 后查询 a 为 0。

复杂度：插入/查询 O(串长)，清空 O(26*已用节点)，固定空间约 55.4 MiB。

易错：cnt[u]是前缀出现数，word_end[u]才是完整词数。；只设 tot=0 而不清 tr 会沿旧边走，污染下一组。；总字符数决定容量，不是单词个数；根也占一格。；字符 Trie 不是处理整数按位异或的二进制 Trie。

来源：cf/ICPC/2026ICPCWeb1/L.cpp

规范源码：competition/string/trie.hpp；完整独立代码如下；测试：tests/round02/test_game_string.cpp

```
#include <bits/stdc++.h>
using namespace std;

// 源自 cf/ICPC/2026ICPCWeb1/L.cpp 的 tr/cnt/dep/tot 写法。
// 根为 0，0 号儿子表示不存在；最多 TRIE_N-1 个非根结点。
// 每次 trie_init 后，所有新插入串总长<TRIE_N 是充分容量条件。
const int TRIE_N = 500005;
inline int tr[TRIE_N][26], cnt[TRIE_N], dep[TRIE_N];
inline int word_end[TRIE_N], tot;

inline void trie_init() {
    for (int u = 0; u <= tot; ++u) {
        fill(tr[u], tr[u] + 26, 0);
        cnt[u] = dep[u] = word_end[u] = 0;
    }
    tot = 0;
}
// 只接收'a'..'z'；重复串重复计数；空串在根处结束。
inline void trie_insert(const string& s) {
    int u = 0;
    ++cnt[0];
    for (char ch : s) {
        int c = ch - 'a';
        assert(0 <= c && c < 26);
        if (!tr[u][c]) {
            assert(tot + 1 < TRIE_N);
            tr[u][c] = ++tot;
            dep[tot] = dep[u] + 1;
        }
        u = tr[u][c];
        ++cnt[u];
    }
    ++word_end[u];
}
inline int trie_node(const string& s) {
    int u = 0;
    for (char ch : s) {
        int c = ch - 'a';
        assert(0 <= c && c < 26);
        if (!tr[u][c]) return -1;
        u = tr[u][c];
    }
    return u;
}
inline int trie_prefix(const string& s) {
    int u = trie_node(s);
    return u == -1 ? 0 : cnt[u];
}
inline int trie_count(const string& s) {
    int u = trie_node(s);
    return u == -1 ? 0 : word_end[u];
}
```

【X01】前缀 XOR、子数组计数与离线区间异或

识别：连续区间 XOR 查询、统计 XOR 为 k 的子数组，或全部更新后只输出最终数组。

思路：两次出现的前缀抵消；统计时查 p^k；区间更新只在两端翻转标记。

证明：p[r]^p[l-1]中 1..l-1 出现两次。p[r]^p[l-1]=k 等价于 p[l-1]=p[r]^k，只统计过去前缀。差分标记在 l 开始、r+1 取消。

约定：a 至少为{0}，采用 a[1..n]且 n<=1e9；值为 unsigned long long 完整 64 位。range_xor 仅接收合法非空区间；离线操作{l,r,x}满足 1<=l<=r<=n。{}不是空数组表示。

调用：vector<ull> a={0,3,1,2}; auto p=xor_prefix(a); ull x=range_xor(p,1,3); ll cnt=count_subarray_xor(a,0); a=xor_offline_update(a,{{1,2,7}});

例子：cf/1/1124/C.cpp 先写出了 p[r]^p[l-1]；真实 2268C 还需要 AND 区间最大值，完整解见 X11。

复杂度：前缀 O(n)/查询 O(1)；哈希计数期望 O(n)、最坏 O(n²)；离线更新 O(n+q)。空间 O(n)。

易错：cnt[0]=1；先查询后插入，否则 k=0 计入空区间。；不适合混合在线区间修改/区间查询；哈希有最坏退化，可用 map 得 O(n log n)。；计数用 ll；值用 ull，避免符号位比较混入。

来源：cf/1/1124/C.cpp

规范源码：competition/xor/prefix_xor.hpp；完整独立代码如下；测试：tests/round02/test_xor.cpp

```
#include <bits/stdc++.h>
using namespace std;
using ll = long long;
using ull = unsigned long long;

// a[1..n]; a[0] 不使用。来源：cf/1/1124/C.cpp 的前缀抵消观察。
vector<ull> xor_prefix(const vector<ull>& a) {
    vector<ull> p(a.size());
    for(size_t i = 1; i < a.size(); i++) p[i] = p[i - 1] ^ a[i];
    return p;
}
ull range_xor(const vector<ull>& p, int l, int r) {
    return p[r] ^ p[l - 1]; // 1 <= l <= r <= n
}
ll count_subarray_xor(const vector<ull>& a, ull k) {
    unordered_map<ull, ll> cnt;
    cnt[0] = 1;
    ull p = 0;
    ll ans = 0;
    for(size_t i = 1; i < a.size(); i++) {
        p ^= a[i];
        auto it = cnt.find(p ^ k);
        if(it != cnt.end()) ans += it->second;
        cnt[p]++; // 先查询，避免 k=0 时计入空区间
    }
    return ans;
}
// 操作 {l,r,x}：将 a[l..r] 都 ^= x。仅需最终数组时使用。
vector<ull> xor_offline_update(vector<ull> a,
                               const vector<array<ull, 3>>& q) {
    vector<ull> d(a.size() + 1);
    for(auto [l, r, x] : q) {
        d[l] ^= x;
        d[r + 1] ^= x;
    }
    ull tag = 0;
    for(size_t i = 1; i < a.size(); i++) {
        tag ^= d[i];
        a[i] ^= tag;
    }
    return a;
}
```

【X02】按位贡献：数对与所有子数组

识别：暴力中有所有数对，或求所有子数组 XOR 的和/再次 XOR。

思路：每位 one/zero 决定多少数对使该位为 1；子数组变成 n+1 个前缀的数对。

证明：XOR 贡献 one*zero；AND 贡献 C(one,2)；OR 贡献 one*zero+C(one,2)。每对前缀唯一对应一个非空子数组。元素 a[i]参与 i(n-i+1)个子数组，再次 XOR 时仅奇数次保留。

约定：a 至少{0}、1-indexed、n<=1e9、元素 uint64。返回 GNU __uint128_t；pair_bit_sums 依次为 i<j 的 XOR/OR/AND 普通和。subarray_xor_sum 为普通加法和，all_subarray_xor 才是再次异或。

调用：vector<ull> a={0,1,2,3}; auto ans=pair_bit_sums(a); cout<<to_string_u128(ans[0]); auto sum=subarray_xor_sum(a); ull x=all_subarray_xor(a);

例子：cf/4/1090/E_xor_find_pattern.cpp 的双循环提供数对识别信号；原题求 max 而非 sum，本模块是明确分开的扩展练习。

复杂度：数对/子数组普通和 O(64n)；再次 XOR O(n)；额外空间 O(64)。

易错：先提升到 u128 再乘。n<=1e9 保证总和小于 2^128，不能无限泛化。；i<j 与有序对不同；OR/AND 不得照搬 one*zero。；把所有子数组 XOR 值相加，与把它们再次 XOR 是不同目标。

来源：cf/4/1090/E_xor_find_pattern.cpp

规范源码：competition/xor/bit_contribution.hpp；完整独立代码如下；测试：tests/round02/test_xor.cpp

```
#include <bits/stdc++.h>
using namespace std;
using ull = unsigned long long;
using u128 = __uint128_t;

// a[1..n]，返回所有 i<j 的 XOR 和、OR 和、AND 和。
// n <= 1e9，元素为 uint64；答案使用 GNU unsigned __int128。
array<u128, 3> pair_bit_sums(const vector<ull>& a) {
    u128 n = a.size() - 1;
    array<u128, 3> ans{};
    for(int k = 0; k < 64; k++) {
        u128 one = 0;
        for(size_t i = 1; i < a.size(); i++) one += (a[i] >> k) & 1;
        u128 zero = n - one, bit = u128(1) << k;
        u128 both = 0;
        if(one >= 2) both = one * (one - 1) / 2;
        ans[0] += one * zero * bit;
        ans[1] += (one * zero + both) * bit;
        ans[2] += both * bit;
    }
    return ans;
}
// 所有子数组 XOR 值的普通加法和；统计 n+1 个前缀中不同位数对。
u128 subarray_xor_sum(const vector<ull>& a) {
    ull cnt[64][2]{};
    for(int k = 0; k < 64; k++) cnt[k][0] = 1;
    ull p = 0;
    for(size_t i = 1; i < a.size(); i++) {
        p ^= a[i];
        for(int k = 0; k < 64; k++) cnt[k][(p >> k) & 1]++;
    }
    u128 ans = 0;
    for(int k = 0; k < 64; k++)
        ans += u128(cnt[k][0]) * cnt[k][1] * (u128(1) << k);
    return ans;
}
// 将所有子数组 XOR 值再 XOR；第 i 个元素出现 i*(n-i+1) 次。
ull all_subarray_xor(const vector<ull>& a) {
    ull ans = 0;
    size_t n = a.size() - 1;
    for(size_t i = 1; i <= n; i++)
        if((i & 1) && ((n - i + 1) & 1)) ans ^= a[i];
    return ans;
}
```

【X02】按位贡献：数对与所有子数组 续

```
string to_string_u128(u128 x) {
    string s;
    do { s += char('0' + x % 10); x /= 10; } while(x);
    reverse(s.begin(), s.end());
    return s;
}
```

【X03】XOR 构造、未知元素与最高不同位

识别：给总 XOR 补最后一项；互异非负数构造；排列整体 XOR；连续值域的 OR。

思路：末项等于目标异或已知总量；互异构造给最后三项留两个自由高位；区间 OR 保留公共高位、填满最高不同位及以下。

证明：前三部分低于 2^60，最后三数的高两位分别 01、10、11，因此互异且不与先前数冲突；其总 XOR 为缺口。区间跨越最高不同位的边界，必能覆盖其下每个 1 位。

约定：known 至少{0}、1-indexed。xor_distinct 只承诺 1<=n<=1e6、target<2^60、互异非负、每项<2^62；失败返回 false 且 a 清空。interval_or 要求 0<=l<=r<=ULLONG_MAX。

调用：vector<ull> a; bool ok=xor_distinct(5,7,a); ull missing=xor_missing({0,3,5},6); ull x=xor_1_to_n(8); ull y=interval_or(4,6);

例子：cf/2/1129/B.cpp 真实 2261B：相邻较小值递增，答案为[min,max]整数区间 OR；原掩码循环保留。

复杂度：未知元素/构造 O(n)；xor_1_to_n O(1)；interval_or O(64)。

易错：互异的两数 XOR=0 不可能；允许重复时两数相等即可。；上界小、正数、排列等限制不能直接套自由高位。；xor_1_to_n(ULLONG_MAX)走 n%4=3 分支，不发生 n+1 溢出。；mx|低位 mask 最大化的是特定 OR 目标，不是任意最大 XOR。

来源：cf/2/1129/B.cpp

规范源码：competition/xor/xor_construct.hpp；完整独立代码如下；测试：tests/round02/test_xor.cpp

```
#include <bits/stdc++.h>
using namespace std;
using ull = unsigned long long;

// known[1..n] 是已知元素，总 XOR 为 target，求唯一未知元素。
ull xor_missing(const vector<ull>& known, ull target) {
    for(size_t i = 1; i < known.size(); i++) target ^= known[i];
    return target;
}
ull xor_1_to_n(ull n) {
    if(n % 4 == 0) return n;
    if(n % 4 == 1) return 1;
    if(n % 4 == 2) return n + 1;
    return 0;
}
// 1<=n<=1e6，target<2^60；构造互异非负数，全部小于 2^62。
// 不承诺正数、较小上界或排列；这些限制必须另证。
bool xor_distinct(int n, ull target, vector<ull>& a) {
    a.clear();
    if(n < 1 || n > 1000000 || target >= (1ULL << 60)) return false;
    if(n == 2 && target == 0) return false;
    a.assign(n + 1, 0);
    if(n <= 2) { a[n] = target; return true; }
    ull last = target;
    for(int i = 1; i <= n - 3; i++) a[i] = ull(i), last ^= a[i];
    a[n - 2] = 1ULL << 60;
    a[n - 1] = 1ULL << 61;
    a[n] = last ^ a[n - 2] ^ a[n - 1];
    return true;
}
// 继承 cf/2/1129/B.cpp 的 mx^mn + 低位全 1 掩码；0<=l<=r。
ull interval_or(ull l, ull r) {
    ull d = l ^ r, mask = 0;
    while(d) {
        mask = (mask << 1) | 1;
        d >>= 1;
    }
    return r | mask;
}
```

【X04】数组节点二进制 Trie：插入、删除、最大/最小 XOR

识别：从集合里选一个数与 x 异或最优；维护重复数；最大 XOR 数对/子数组。

思路：从最高位到最低位决定答案；最大优先异位，最小优先同位，仅走 cnt>0 的路径。

证明：高位 1 的权重大于所有低位之和；若有活跃候选能获得当前更优位，丢弃另一分支不会损失最优解。查询前缀 XOR 则将最大子数组变为历史前缀中最大数对。

约定：先 trie_init(width,max_nodes)，width=1..64、max_nodes 包含根且<=INT_MAX；值非负并装入 width 位；同时存储数<=LLONG_MAX。init 失败不改旧树；insert 容量失败不改树。删除一个副本，不回收节点。便捷 max 函数输入 a 至少{0}、n<=1e6，并重置全局 Trie。

调用：trie_init(64,10000); trie_insert(7); trie_insert(7); ull ans; bool ok=trie_query(3,ans); trie_query(3,ans,false); trie_erase(7); ull pair=max_xor_pair({0,1,2,3}); ull sub=max_xor_subarray({0,1,2,3});

例子：cf/4/1090/E_xor_find_pattern.cpp 的 O(n²)在 2218E 约束足够；此模块服务更大 n，不替换正确原解。

复杂度：每次操作 O(width)；容量按累计新路径计，最多 1+width*插入次数。tree 的 2 个 int+cnt 的 ll 约 16 字节/节点，vector 扩容还有余量。最大数对/子数组 O(64n)。

易错：查询返回 XOR 值，不是被选的数。；空树查询/删除返回 false；删除到 0 的旧节点不可再视为候选。；删除不回收节点；长期滑窗按历史不同路径估容量或分批 init。；用 1ULL<<63；不得右移/左移 64 位；如题域<2^30 可用 30 位显著节省内存。；普通 Trie 不能直接求(x^y)&动态 mask 最优；mask 为 0 的位不能随意只选一支，2268C 见 X11。

来源：cf/4/1090/E_xor_find_pattern.cpp

规范源码：competition/xor/binary_trie.hpp；完整独立代码如下；测试：tests/round02/test_xor.cpp

```
#include <bits/stdc++.h>
using namespace std;
using ll = long long;
using ull = unsigned long long;

// 数组节点：0 为空指针，1 为根。容量包含根，不含 0 号哨兵。
vector<array<int, 2>> tree;
vector<ll> cnt;
int bits = 64;
size_t capacity = 1;
bool trie_init(int width, size_t max_nodes) {
    if(width < 1 || width > 64 || max_nodes < 1 ||
       max_nodes > size_t(INT_MAX)) return false;
    bits = width; capacity = max_nodes;
    tree.assign(2, {0, 0}); cnt.assign(2, 0);
    return true;
}
bool trie_fits(ull x) { return bits == 64 || (x >> bits) == 0; }
bool trie_insert(ull x) {
    if(!trie_fits(x) || tree.empty()) return false;
    int p = 1;
    // 先检查新增节点数，容量不足时不留下半次插入。
    for(int k = bits - 1; k >= 0; k--) {
        int c = int((x >> k) & 1);
        if(!tree[p][c]) {
            if(size_t(k + 1) > capacity - (tree.size() - 1)) return false;
            break;
        }
        p = tree[p][c];
    }
    p = 1; cnt[p]++;
    for(int k = bits - 1; k >= 0; k--) {
        int c = int((x >> k) & 1);
        if(!tree[p][c]) {
            tree[p][c] = int(tree.size());
            tree.push_back({0, 0}); cnt.push_back(0);
        }
        p = tree[p][c]; cnt[p]++;
    }
    return true;
}
bool trie_erase(ull x) {
```

【X04】数组节点二进制 Trie：插入、删除、最大/最小 XOR 续

```
    if(!trie_fits(x) || tree.empty() || !cnt[1]) return false;
    int p = 1, path[65], len = 0;
    path[len++] = p;
    for(int k = bits - 1; k >= 0; k--) {
        p = tree[p][(x >> k) & 1];
        if(!p || !cnt[p]) return false;
        path[len++] = p;
    }
    for(int i = 0; i < len; i++) cnt[path[i]]--;
    return true; // 不回收节点；相同路径可复用，换批次 init()
}
// 返回 XOR 值而非被选元素。空树/超位宽返回 false，不修改 ans。
bool trie_query(ull x, ull& ans, bool maximum = true) {
    if(!trie_fits(x) || tree.empty() || !cnt[1]) return false;
    int p = 1;
    ans = 0;
    for(int k = bits - 1; k >= 0; k--) {
        int c = int((x >> k) & 1), want = c ^ int(maximum);
        int q = tree[p][want];
        if(!q || !cnt[q]) want ^= 1;
        if(c != want) ans |= 1ULL << k;
        p = tree[p][want];
    }
    return true;
}
// a[1..n]，n<=1e6；无数对/空数组按 0 返回；会重置全局 Trie。
ull max_xor_pair(const vector<ull>& a) {
    trie_init(64, 1 + 64 * (a.size() - 1));
    ull ans = 0, value = 0;
    for(size_t i = 1; i < a.size(); i++) {
        if(trie_query(a[i], value)) ans = max(ans, value);
        trie_insert(a[i]); // 先查后插，保证不同下标
    }
    return ans;
}
ull max_xor_subarray(const vector<ull>& a) {
    trie_init(64, 1 + 64 * a.size());
    trie_insert(0); // 空前缀，允许子数组从 1 开始
    ull p = 0, ans = 0, value = 0;
    for(size_t i = 1; i < a.size(); i++) {
        p ^= a[i];
        trie_query(p, value); ans = max(ans, value);
        trie_insert(p);
    }
    return ans;
}
```

【X05】图上的 XOR 约束

识别：关系 value[u]^value[v]=w；需要找一组值或判断矛盾。

思路：每个分量选根为 0，沿边推出邻居值，回边核验。

证明：任意边给出 value[v]=value[u]^w。树路径唯一确定相对值；所有已赋值边一致当且仅当各环边权 XOR 为 0。每个连通分量整体 XOR 任意 t 仍满足全部边。

约定：1-indexed 图，先 init(n)，边端点合法；w 为 uint64。允许重边、自环。xor_assign 返回 true 才可使用完整 value/comp/components；false 时只是部分状态。

调用：xor_graph_init(3); xor_add_edge(1,2,5); xor_add_edge(2,3,7); bool ok=xor_assign();

例子：由 XOR 自反和路径抵消推导的新模型；不是六份旧训练代码中的既有实现。

复杂度：O(n+m)时间、O(n+m)空间；BFS 不依赖递归栈。

易错：不同连通分量的根可以分别选择；并非全图只剩一个自由变量。；若另有限制 0<=value<=K，K 非 2^B-1 时，自由平移可能越界，不能直接宣称 2^(Bc)种解。；在线加约束可考虑带 XOR 势能的 DSU；静态只需 BFS。

来源：cf/1/1124/B.cpp

规范源码：competition/xor/graph_xor.hpp；完整独立代码如下；测试：tests/round02/test_xor.cpp

```
#include <bits/stdc++.h>
using namespace std;
using ull = unsigned long long;

vector<vector<pair<int, ull>>> g;
vector<ull> value;
vector<int> comp;
int components = 0;
void xor_graph_init(int n) {
    g.assign(n + 1, {}); value.assign(n + 1, 0);
    comp.assign(n + 1, 0); components = 0;
}
void xor_add_edge(int u, int v, ull w) {
    g[u].push_back({v, w}); g[v].push_back({u, w});
}
bool xor_assign() {
    fill(comp.begin(), comp.end(), 0);
    components = 0;
    for(int s = 1; s < int(g.size()); s++) {
        if(comp[s]) continue;
        queue<int> q;
        comp[s] = ++components; value[s] = 0; q.push(s);
        while(!q.empty()) {
            int u = q.front(); q.pop();
            for(auto [v, w] : g[u]) {
                if(!comp[v]) {
                    comp[v] = components; value[v] = value[u] ^ w;
                    q.push(v);
                } else if((value[u] ^ value[v]) != w) return false;
            }
        }
    }
    return true; // 仅 true 时 value 为完整解；每分量可整体再异或任意 t
}
```

【X06】XOR 线性基：插入与最大子集 XOR

识别：从任意若干个数选子集，求可达 XOR/最大子集 XOR；长期进阶。

思路：把数当 GF(2)上的向量，以最高位为主元消元。

证明：每次异或已有向量不改变可表示空间；消成 0 说明相关，否则新增未有的最高主元。查询从高到低选择能增大结果的主元，低主元无法影响已确定高位。

约定：先 basis_init；uint64 全 64 位；空子集允许，故空基最大值 0。basis_insert 返回是否增加秩；rk 为独立向量数。basis_max(seed)求 seed XOR 任意子集的最大值。

调用：basis_init(); basis_insert(1); basis_insert(2); basis_insert(4); ull ans=basis_max();

例子：例{1,2,4}：最大数对 6，最大子集 7；不能用线性基替换 2218E 数对目标。

复杂度：每次插入/查询 O(64)，空间 O(64)。

易错：Trie 选一个已存元素，线性基选任意子集，答案不同。；普通基不直接支持删除、k 小、区间查询；这些需要额外结构。；P2，不挤占赛前 P0 复习时间；FWT 仍 OPTIONAL。

来源：cf/4/1090/E_xor_find_pattern.cpp

规范源码：competition/xor/linear_basis.hpp；完整独立代码如下；测试：tests/round02/test_xor.cpp

```
#include <bits/stdc++.h>
using namespace std;
using ull = unsigned long long;

// b[k] 的最高位为 k；P2，维护任意子集 XOR，不是从集合选一个数。
ull b[64];
int rk = 0;
void basis_init() { fill(b, b + 64, 0); rk = 0; }
bool basis_insert(ull x) {
    for(int k = 63; k >= 0; k--) {
        if(((x >> k) & 1) == 0) continue;
        if(b[k]) x ^= b[k];
        else { b[k] = x; rk++; return true; }
    }
    return false; // 消成 0：原数已在已有数的 XOR 张成空间内
}
ull basis_max(ull x = 0) {
    for(int k = 63; k >= 0; k--)
        if((x ^ b[k]) > x) x ^= b[k];
    return x;
}
```

【X07】我的偶数 popcount 计数模型（2268B）

识别：相邻两个数同时 XOR 3k(k=1..5)，值在 0..15；求能变为 3 的倍数的最多位置，单点修改。

思路：每个位置 popcount 奇偶不变；偶数类可以全部变到 3 的倍数。

证明：操作掩码张成 8 个偶 popcount 数；相邻路径能给任意两位置加同一掩码。若有奇类留它吸收总 XOR；若全偶类，保留两个位置，任意偶类总 XOR 都能写成两个 3 倍数之 XOR。

约定：a[1..n]、n>=2、0<=a[i],x<16；原 good(int)仅在此域复用。修改 pos 合法；ans 必须是当前 all_meat_count。

调用：vector<ll> a={0,10,3}; ll ans=all_meat_count(a); all_meat_update(a,ans,1,0);

例子：cf/1/1124/B.cpp 已正确实现；补证明而不重写算法。例[5,0]可到[6,3]，两位置都成功。

复杂度：初始 O(n)，每次更新 O(1)，空间 O(n)。

易错：popcount 奇偶不是整数奇偶：3 是奇数但 popcount 偶。；只证不变量是必要条件，还需可达性；只把前 n-1 位置清 0 不足以保证最后值是 3 倍数，例如总 XOR=5。；不应为原 good 的 int 接口报错：原题值<16，完全足够。

来源：cf/1/1124/B.cpp

规范源码：competition/xor/popcount_inventory.hpp；完整独立代码如下；测试：tests/round02/test_xor.cpp

```
#include <bits/stdc++.h>
using namespace std;
using ll = long long;
using ull = unsigned long long;

// cf/1/1124/B.cpp：原判定原样。2268B 限定 0<=x<16、n>=2。
bool good(int x) {
    return __builtin_popcount(x) % 2 == 0;
}
ll all_meat_count(const vector<ll>& a) {
    ll ans = 0;
    for(size_t i = 1; i < a.size(); i++) if(good(int(a[i]))) ans++;
    return ans;
}
void all_meat_update(vector<ll>& a, ll& ans, int p, ll x) {
    if(good(int(a[p]))) ans--;
    a[p] = x;
    if(good(int(a[p]))) ans++;
}
```

【X08】我的最大数对双循环（2218E）

识别：选当前元素 x、所有元素^=x、删除 x，做至剩一项；原题 sum n<=3105。

思路：公共 XOR 标签在下一次操作时被抵消，最后只与最后删除者和幸存者两个原值有关。

证明：当前值为原值^t，选原值 a[i]后其他值变成 a[j]^a[i]；最后为两个不同原下标的 XOR。任何一对都可保留至最后，故枚举数对既是上界也能达到。

约定：a 至少{0}、1-indexed、非负值；原 2218E 为 2<=n<=3105、a[i]<=1e9。模板不足两项返回 0 约定。

调用：ull ans=max_xor_pair_quadratic({0,1,2,3});

例子：cf/4/1090/E_xor_find_pattern.cpp，真实题 The 67th XOR Problem，CF2218E。

复杂度：O(n²)时间，O(1)额外空间；原题最多约 482 万对。

易错：二重循环不能凭文件名判 TLE；必须核对真实 3105 规模。；max 元素和 min 元素不一定给最大 XOR：{1,2,3}极值异或 2，最优 1^2=3。；不把本地验证说成已核验在线 AC。

来源：cf/4/1090/E_xor_find_pattern.cpp

规范源码：competition/xor/max_pair_quadratic.hpp；完整独立代码如下；测试：tests/round02/test_xor.cpp

```
#include <bits/stdc++.h>
using namespace std;
using ll = long long;
using ull = unsigned long long;

// cf/4/1090/E_xor_find_pattern.cpp：2218E 原 O(n²)足够，保留双循环。
ull max_xor_pair_quadratic(const vector<ull>& a) {
    ull ans = 0;
    for(size_t i = 1; i < a.size(); i++)
        for(size_t j = i + 1; j < a.size(); j++)
            ans = max(ans, a[i] ^ a[j]);
    return ans;
}
```

【X09】我的 XOR-gun：最高位分类与前缀枚举

识别：非降序正整数数组，相邻两项可合并为 XOR；最少几次使序列出现下降。

思路：最终相邻两项对应原来的相邻两段；段内合并成本等于长度减 1。30 种最高位，超过 60 项必有连续三项最高位相同。

证明：三项最高位相同时，右两项 XOR 消去该位，必小于左项，所以 1 次足够；初始非降序排除 0 次。小规模枚举所有相邻两段，成本 r-l-1 覆盖全部可能首次逆序。

约定：a[1..n]，2<=n<=1e5，1<=a[i]<=1e9 且非降序；无解返回-1。

调用：int ans=xor_gun({0,1,2,4,6,20});

例子：cf/date/5.15/3.cpp = CF1457D；[1,2,4,6,20]答案 2，[1,2,3]无解。

复杂度：读入 O(n)；n>60 直接 1，否则 $O(n^3)$枚举、O(n)空间。

易错：正数、非降序、上界 1e9 都是 60 阈值的前提；61 个 0 不适用。；原 ca 逐段扫描实际为 $O(n^4)$，但 60 封顶仍是可用旧解；没有伪造 TLE。；只比较两个连续区间，不可随意跳选下标。

来源：cf/date/5.15/3.cpp

规范源码：competition/xor/xor_gun.hpp；完整独立代码如下；测试：tests/round02/test_xor.cpp

```
#include <bits/stdc++.h>
using namespace std;
using ll = long long;
using ull = unsigned long long;

// cf/date/5.15/3.cpp：1457D，正数、非降序、a[i]<=1e9。
// 原 ca 每次扫区间；仅替成前缀查询，O(n^4)降到 O(n^3)。
int xor_gun(const vector<ll>& a) {
    int n = int(a.size()) - 1;
    if(n > 60) return 1;
    vector<ll> p(n + 1);
    for(int i = 1; i <= n; i++) p[i] = p[i - 1] ^ a[i];
    int ans = n;
    for(int l = 1; l <= n; l++)
        for(int r = l + 1; r <= n; r++)
            for(int m = l + 1; m <= r; m++)
                if((p[m - 1] ^ p[l - 1]) > (p[r] ^ p[m - 1]))
                    ans = min(ans, r - l - 1);
    return ans == n ? -1 : ans;
}
```

【X10】我的 AND 排列：补码块与同最高位环

识别：构造 1..n 排列，要求 p[i]!=i，分别要求 i&p[i]=0 或!=0。

思路：零 AND：从右端找全 1 掩码，块内互补配对。非零 AND：保留原 6/7 元素基础环，高位区间内循环移位。

证明：零 AND 时奇数下标必须映射偶数，n 奇数不可能；n 偶数每个补码块边界为奇/偶，递归剩余仍为偶数。非零 AND 时 2 的幂只能映射自身，不允许；n=3,5 有候选冲突；其余原构造的每条环边都有共同 1 位。

约定：1<=n<=1e5；返回 false 表示对应构造不存在且输出向量清空；true 给 1-indexed 排列。

调用：vector<ll> p,q; bool ok0=and_zero_perm(6,p); bool ok1=and_nonzero_perm(6,q);

例子：cf/date/7/7.9/2_and.cpp = CF909F AND-permutations；n=6 两种都可构造，n=8 非零 AND 排列不存在。

复杂度：O(n)时间/空间。

易错：不能把 i&p[i]!=0 误解为相邻排列元素 AND!=0。；p[i]!=i 必须独立检验；最后高位块只有一个元素时恰为 2 的幂障碍。；真实 909F 为进阶构造，原代码可以直接收录，无需换成陌生算法。

来源：cf/date/7/7.9/2_and.cpp

规范源码：competition/xor/and_permutations.hpp；完整独立代码如下；测试：tests/round02/test_xor.cpp

```
#include <bits/stdc++.h>
using namespace std;
using ll = long long;
using ull = unsigned long long;

// cf/date/7/7.9/2_and.cpp：909F 的两段原构造，仅抽出输入输出。
bool and_zero_perm(int n, vector<ll>& p) {
    p.clear();
    if(n % 2 == 1) return false;
    p.assign(n + 1, 0);
    ll r = n;
    while(r > 0) {
        ll mask = 1;
        while(mask < r) mask = mask * 2 + 1;
        ll l = mask - r;
        for(ll i = l; i <= r; i++) p[i] = mask ^ i;
        r = l - 1;
    }
    return true;
}
bool and_nonzero_perm(int n, vector<ll>& q) {
    q.clear();
    if(n < 6 || (n & (n - 1)) == 0) return false;
    q.assign(n + 1, 0);
    vector<ll> a = {1, 3, 2, 6, 4, 5};
    if(n != 6) a.push_back(7);
    for(ll i = 0; i < ll(a.size()); i++) q[a[i]] = a[(i + 1) % a.size()];
    for(ll l = 8; l <= n; l <<= 1) {
        ll r = min(ll(n), 2 * l - 1);
        for(ll i = l; i < r; i++) q[i] = i + 1;
        q[r] = l;
    }
    return true;
}
```

【X11】我的 Energy Intervals 补全（2268C / 2269E）

识别：最大化长度至少 2 区间的 (区间 XOR & 区间最大值)，n 大且值<2^18。

思路：沿原前缀异或观察，先把 AND 提出；按位贪心答案，将问题变成受区间最大值限制的互补前缀存在性。

证明：最大笛卡尔树结点 v 统一负责跨 v 且以 a[v]为最大值的区间。候选 mask 须被 a[v]包含，且两前缀在 mask 投影后的 XOR 等于 mask。小并大枚举较小侧，完整保留另一侧频次，明确扣除(v-1,v)单元素对。每个下标进入较小侧时所在规模至少倍增。

约定：a[1..n]、2<=n<=2e5、0<=a[i]<2^width、1<=width<=18；energy_max 重建全局状态；energy_check 仅供内部调用。n<2 按 0 约定。

调用：int ans=energy_max({0,1,7,3,7,2});

例子：cf/1/1124/C.cpp 真实题意已经核验，原 solve 为空。官方样例[1,7,3,7,2]答案 6；此完整解为 P2，不当成普通最大子数组 XOR。

复杂度：O(width*n*log n + width*2^width)时间，O(n+2^width)空间；显式栈。

易错：普通最大 XOR Trie 忽略动态 AND 掩码会错：对 0 查询{4,3}最大 XOR 是 4，mask=3 后为 0，另一个候选可得 3。；(v-1,v)永远会通过自身 a[v]包含 mask 的检查，但它是禁止的单元素区间。；重复前缀必须维护频次，不能简单删除一个值的全部存在性。；单调数组的笛卡尔树深度 n；使用显式栈防递归爆栈。

来源：cf/1/1124/C.cpp

规范源码：competition/xor/energy_intervals.hpp；完整独立代码如下；测试：tests/round02/test_xor.cpp

```
#include <bits/stdc++.h>
using namespace std;

// 2268C / 2269E。延续原前缀 p 思路，补全原本为空的 solve。
// a[1..n], n>=2, 0<=a[i]<2^width, 1<=width<=18。
vector<int> a, p, s, L, R, cnt;
vector<array<int, 2>> ch;
int root = 0;
bool energy_check(int mask) {
    int n = int(a.size()) - 1;
    for(int i = 0; i <= n; i++) s[i] = p[i] & mask;
    fill(cnt.begin(), cnt.end(), 0);
    // {结点,阶段,是否保留频次}；显式栈避免单调数组递归 n 层。
    vector<array<int, 3>> st{{root, 0, 1}};
    while(!st.empty()) {
        auto [v, stage, keep] = st.back(); st.pop_back();
        int left = ch[v][0], right = ch[v][1];
        bool heavy_right = R[v] - v > v - L[v];
        int heavy = heavy_right ? right : left;
        int light = heavy_right ? left : right;
        if(stage == 0) {
            st.push_back({v, 1, keep});
            if(heavy) st.push_back({heavy, 0, 1});
            if(light) st.push_back({light, 0, 0});
            continue;
        }
        bool allowed = (a[v] & mask) == mask;
        if(heavy_right) {
            cnt[s[v]]++; // 对侧端点为[v..R]
            if(allowed) for(int x = L[v] - 1; x < v; x++) {
                int need = s[x] ^ mask;
                int forbidden = (x == v - 1 && need == s[v]);
                if(cnt[need] > forbidden) return true;
            }
            for(int x = L[v]; x < v; x++) cnt[s[x]]++;
        } else {
            cnt[s[L[v] - 1]]++; // 对侧端点为[L-1..v-1]
            if(allowed) for(int y = v; y <= R[v]; y++) {
                int need = s[y] ^ mask;
                int forbidden = (y == v && need == s[v - 1]);
                if(cnt[need] > forbidden) return true;
            }
            cnt[s[L[v] - 1]]--;
```

【X11】我的 Energy Intervals 补全（2268C / 2269E） 续

```
            for(int y = v; y <= R[v]; y++) cnt[s[y]]++;
        }
        // 此时恰好保存前缀下标[L[v]..R[v]]。
        if(!keep) for(int i = L[v]; i <= R[v]; i++) cnt[s[i]]--;
    }
    return false;
}
int energy_max(const vector<int>& input, int width = 18) {
    a = input;
    int n = int(a.size()) - 1;
    if(n < 2) return 0;
    p.assign(n + 1, 0); s.resize(n + 1);
    L.resize(n + 1); R.resize(n + 1); ch.assign(n + 1, {0, 0});
    cnt.assign(1 << width, 0);
    vector<int> st;
    for(int i = 1; i <= n; i++) {
        p[i] = p[i - 1] ^ a[i];
        int last = 0;
        while(!st.empty() && a[st.back()] < a[i]) {
            last = st.back(); st.pop_back();
        }
        if(!st.empty()) ch[st.back()][1] = i;
        ch[i][0] = last; st.push_back(i);
    }
    root = st.front();
    vector<int> order{root};
    for(size_t i = 0; i < order.size(); i++)
        for(int v : ch[order[i]]) if(v) order.push_back(v);
    reverse(order.begin(), order.end());
    for(int v : order) {
        L[v] = ch[v][0] ? L[ch[v][0]] : v;
        R[v] = ch[v][1] ? R[ch[v][1]] : v;
    }
    int ans = 0;
    for(int k = width - 1; k >= 0; k--)
        if(energy_check(ans | (1 << k))) ans |= 1 << k;
    return ans;
}
```

基础 STL 逐项速查：场景、调用、复杂度与坑

以下每项都有受测调用；完整代码按模块编号查前面的代码索引。几个字符就能使用的 API 在此集中比较，避免为每个 API 另建算法文件。

【TIP_C_ARRAY】array

用途：固定长度的连续存储、统一填值与排序。

实际调用：array<int,4>；sort(begin,end)；front/back；fill；operator[]

复杂度：访问 O(1)，填充 O(n)，排序 O(n log n)。

易错：长度属于类型；默认未初始化的局部数值 array 元素不自动清零。

已检查例子：array_vector 短例排序 0/1/2/3 并 fill(7)，断言 front/back 与元素值。

代码与来源：STL01；competition/stl/recipes.hpp

【TIP_C_VECTOR】vector

用途：按下标访问，末尾动态增长，批量清理多测数据。

实际调用：reserve/push_back/emplace_back/resize/pop_back/erase/clear/empty

复杂度：按下标 O(1)，尾插摊还 O(1)，中间增删 O(n)。

易错：reserve 不改变 size；扩容使旧引用/指针/迭代器失效；erase 使删除点之后失效。

已检查例子：实际执行 reserve(8)、resize(4,9)、删除与 clear，结果{4,9}后为空。

代码与来源：STL01；competition/stl/recipes.hpp

【TIP_C_DEQUE】deque

用途：需要两端插删且保留随机访问。

实际调用：push_front/push_back/pop_front/pop_back/operator[]

复杂度：两端操作与随机访问 O(1)，中间操作 O(n)。

易错：不要长期保存跨修改的迭代器；front/back/pop 前保证非空。

已检查例子：双端加入 1/2/3，再各删一端，断言仅剩 2。

代码与来源：STL01；competition/stl/recipes.hpp

【TIP_C_STRING】string

用途：查找模式、截取、反转及按字节改变大小写。

实际调用：find(pattern,pos)/substr(pos,len)/reverse/tolower/toupper；getline

复杂度：截取/反转 O(处理长度)，字符串查找一般不能当 O(1)。

易错：find 失败为 npos；substr 第二参数是长度；ctype 先转 unsigned char；UTF-8 汉字不是单字节字符。

已检查例子：string 短例检查 pos=4、Cba/cba/CBA 与 npos；IO 测试保留整行前导空格。

代码与来源：STL01、S03；competition/stl/recipes.hpp、string/basic_string.hpp

【TIP_C_PAIR】pair

用途：把排序关键字和编号绑定，或堆中存距离与点号。

实际调用：vector<pair<int,int>>排序；auto [time,id]；pair<ll,int>小根堆

复杂度：固定字段比较 O(1)，容器算法复杂度另计。

易错：默认字典序先 first 后 second；交换字段会改变优先级。

已检查例子：pair_tuple 短例首项(1,3)；Dijkstra 对 Floyd 及 ROUND01 独立对照。

代码与来源：STL01、PG05；competition/stl/recipes.hpp、competition/graph/dijkstra.hpp

【TIP_C_TUPLE】tuple

用途：状态有三个及以上字段，并需要字典序或结构化绑定。

实际调用：make_tuple/get<1>/auto [t,i,label]；tuple<priority,id,version>

复杂度：固定字段数比较 O(1)。

易错：auto 复制，auto&才能改原成员；多个字段的排序含义必须明确。

已检查例子：tuple 解包与 get 断言；版本堆 20000 次修改/取消对 map 全扫描。

代码与来源：STL01、STL02；competition/stl/recipes.hpp、competition/stl/heaps_sets.hpp

【TIP_C_STACK】stack

用途：最近进入且尚未处理的对象先处理，如左括号或显式 DFS。

实际调用：push/top/pop/empty；括号泛用函数见 BM07

复杂度：基础栈操作 O(1)，每元素一次进出合计 O(n)。

易错：pop 不返回元素；空栈不能 top/pop；不能只用固定合法例声称覆盖所有括号输入。

已检查例子：STL 固定合法例实际运行；BM07 补通用括号模型与测试，PG04 显式 DFS 对照连通块。

代码与来源：STL01、PG04、BM07；competition/stl/recipes.hpp、competition/graph/dfs.hpp、competition/models/greedy_construct.hpp

【TIP_C_QUEUE】queue

用途：按被发现顺序处理状态，适合无权 BFS。

实际调用：push/front/pop/empty；bfs(start)与路径恢复

复杂度：基础操作 O(1)，BFS O(n+m)。

易错：首次入队即标记/定距；pop 不会返回队首；加权图不能无条件普通 BFS。

已检查例子：queue 短例顺序 1/2；PG03 实际 BFS 与独立全源距离对照。

代码与来源：STL01、PG03；competition/stl/recipes.hpp、competition/graph/bfs.hpp

【TIP_C_HEAP_MAX】priority_queue<ll>大根堆

用途：反复取当前最大值，或最小 k 项中淘汰最大者。

实际调用：priority_queue<ll>；push/top/pop；top_k_smallest(a,k)

复杂度：top O(1)，push/pop O(log n)。

易错：默认 top 是最大值；top/pop 前非空；数值可能超 int 时用 ll。

已检查例子：显式 8e9 最大值断言；最小 k 项 30000 次前缀与升序排序截断比较。

代码与来源：STL01、STL02；competition/stl/recipes.hpp、competition/stl/heaps_sets.hpp

【TIP_C_HEAP_MIN】priority_queue<ll,...,greater<ll>>小根堆

用途：反复取最小值，或最大 k 项中淘汰最小者。

实际调用：priority_queue<ll,vector<ll>,greater<ll>>；top_k_largest

复杂度：top O(1)，push/pop O(log n)。

易错：greater 令最小值在堆顶，不能用大根堆淘汰 TopK 最大项。

已检查例子：显式小根堆 top=2；最大 k 项 30000 次前缀对降序排序截断。

代码与来源：STL01、STL02；competition/stl/recipes.hpp、competition/stl/heaps_sets.hpp

【TIP_C_HEAP_PAIR】pair 小根堆与自定义 cmp

用途：距离相同按编号，或多个字段构成任务优先级。

实际调用：priority_queue<pair<ll,int>,vector<pair<ll,int>>,greater<pair<ll,int>>>；Job lambda cmp

复杂度：固定字段比较 O(1)，堆操作 O(log n)。

易错：cmp(a,b)为 true 表示 a 优先级低于 b；不能用<=，须严格弱序。

已检查例子：两项 4e9 同值时编号 3 先于 9；Job 按 deadline/id 排列；Dijkstra 真实调用 pair 堆。

代码与来源：STL01、PG05；competition/stl/recipes.hpp、competition/graph/dijkstra.hpp

【TIP_C_SET】set 全基础 API

用途：动态维护唯一键且需要顺序、阈值或邻居。

实际调用：insert/erase/find/count/contains(C++20)/lower_bound/upper_bound/begin/end/empty/clear

复杂度：查找/插入/按键删除 O(log n)，端点访问 O(1)。

易错：成员 lower_bound 才是对数搜索；find 可能为 end；只查询避免非法解引用；重复 insert 不增加元素。

已检查例子：recipes 显式 contains、find 命中/未命中、insert、count、erase 返回数；前后继边界与 MEX 多测 clear 实测。

代码与来源：STL01、STL02、MEX02；competition/stl/recipes.hpp、competition/stl/heaps_sets.hpp、competition/game/mex.hpp

【TIP_C_MULTISET】multiset 重复值与单删/全删

用途：重复资源、滑窗或按重数计算的动态序列。

实际调用：find 后检查 end 再 erase(it)；erase(value)全删并返回数；count/contains

复杂度：查找 O(log n)；按值删 k 个 O(log n+k)，按迭代器摊还 O(1)。

易错：erase(value)删所有相等副本；不能对 end 做 erase；滑窗只删旧位置的一份。

已检查例子：先删一份 2 断言仍有一份，再插回并 erase(2)断言恰删 2 份；窗口 2000 组含重复随机对照。

代码与来源：STL01、STL02；competition/stl/recipes.hpp、competition/stl/heaps_sets.hpp

【TIP_C_MAP】map 与 operator[]插入

用途：有序键映射到频次/状态，输出需要按键排序。

实际调用：++cnt[x]；at(已存在键)；find(缺失键)==end；begin

复杂度：查找/插入/删除 O(log n)。

易错：[]会为不存在键插入默认值；只查询用 find/contains；at 缺失会抛异常。

已检查例子：a/b 计数中 b=2，begin 为 a，find(c)为 end；缺失查询没有新增键。

代码与来源：STL01；competition/stl/recipes.hpp

【TIP_C_MULTIMAP】multimap 重复键

用途：同一时间多个事件或一个键对应多个独立值。

实际调用：multimap<int,string>；equal_range(key)遍历[l,r)

复杂度：等值范围定位 O(log n)，遍历 k 项 O(k)。

易错：不能当 map 用[]；重复键的所有值由 equal_range 取得，end 不可解引用。

已检查例子：时间 2 对应 B/C，实际遍历并断言 2 项。

代码与来源：STL01；competition/stl/recipes.hpp

【TIP_C_UNORDERED_SET】unordered_set

用途：只需要等值存在性与去重，不需要顺序。

实际调用：max_load_factor/reserve/insert/size

复杂度：平均查询/插删 O(1)，最坏 O(n)，空间 O(n)。

易错：无前驱/排序语义；rehash 使迭代器失效；reserve 不保证防恶意碰撞。

已检查例子：4/4/-1 插入后 size=2，已编译执行；容量预留实际调用。

代码与来源：STL01；competition/stl/recipes.hpp

【TIP_C_UNORDERED_MAP】unordered_map

用途：按任意值计频，仅关心等值键。

实际调用：max_load_factor/reserve/++cnt[x]/at

复杂度：平均 O(1)，最坏 O(n)。

易错：[]同样可能插入；不依赖遍历顺序；复杂度不能写无条件 O(1)。

已检查例子：4/4/-1 的频次断言 cnt.at(4)=2。

代码与来源：STL01；competition/stl/recipes.hpp

【TIP_A_SORT】sort / 自定义 cmp / lambda

用途：多关键字排序，为贪心、二分和去重建立顺序。

实际调用：sort(score 降序,id 升序)

复杂度：O(n log n)。

易错：不能<=，不能把 EPS 近似相等塞进排序；同分是否需要 id 必须明确。

已检查例子：自定义 Node 排序检查 9 分的 id1 先于 id2。

代码与来源：STL01；competition/stl/recipes.hpp

【TIP_A_STABLE】stable_sort

用途：比较等价的元素要保留进入算法时的相对顺序。

实际调用：stable_sort(score 升序)

复杂度：可用额外内存时 O(n log n)，不足时比较上界 O(n log² n)。

易错：稳定相对的是比较器等价关系，不一定是 operator==。

已检查例子：同为 9 分的 id1/id2 在 stable_sort 后仍按原相对顺序。

代码与来源：STL01；competition/stl/recipes.hpp

【TIP_A_LOWER_UPPER】lower_bound / upper_bound

用途：查等值段或第一个>=/>阈值的位置。

实际调用：lower_bound/upper_bound；r-l

复杂度：随机访问已排序区间 O(log n)比较。

易错：返回 end 合法，解引用前判断；set 使用成员版本。

已检查例子：{1,2,2,5}查 2 的左右界差为 2。

代码与来源：STL01；competition/stl/recipes.hpp

【TIP_A_BINARY】binary_search

用途：只判断已排序区间中是否存在指定值。

实际调用：binary_search(a.begin(),a.end(),5)

复杂度：随机访问已排序区间 O(log n)。

易错：未排序或比较器不一致时不保证结果。

已检查例子：升序和降序都实际搜索到 5。

代码与来源：STL01；competition/stl/recipes.hpp

【TIP_A_DESCENDING】降序下的二分比较器

用途：数据按大到小排列，仍想定位等值范围。

实际调用：sort(greater<int>())；bounds/search 传 greater

复杂度：O(n log n)排序，O(log n)查询。

易错：降序 lower 为第一个<=x，upper 为第一个<x；遗漏 greater 会错。

已检查例子：降序{5,2,2,1}中 2 的左右界索引 1/3。

代码与来源：STL01；competition/stl/recipes.hpp

【TIP_A_NTH】nth_element

用途：只需第 k 小或按 k 分区，不需完整有序。

实际调用：nth_element(begin,begin+k,end)

复杂度：平均 O(n)。

易错：k 为 0-based 且必须落在[0,n)；其余元素不会自动排序。

已检查例子：固定数组第 2 号元素为 4，左右分区不等式实际断言。

代码与来源：STL01；competition/stl/recipes.hpp

【TIP_A_UNIQUE】sort + unique + erase

用途：值域离散化或构造去重集合。

实际调用：sort；erase(unique(...),end)

复杂度：O(n log n)。

易错：unique 只压相邻重复且不缩容器长度，必须最后 erase。

已检查例子：{3,1,3,2,2}得到{1,2,3}。

代码与来源：STL01；competition/stl/recipes.hpp

【TIP_A_ERASE_REMOVE】erase-remove / remove_if

用途：批量删除符合谓词的元素。

实际调用：erase(remove_if(...),end)

复杂度：O(n)。

易错：remove 只是移动保留项并返回新逻辑尾；没有 erase 仍保留原 size。

已检查例子：删除奇数后只剩{2}。

代码与来源：STL01；competition/stl/recipes.hpp

【TIP_A_REVERSE】reverse

用途：镜像、双向处理或构造翻转。

实际调用：reverse(begin,end)

复杂度：O(n)。

易错：区间半开；子串与子序列不同，反转不会自动解决所有对称约束。

已检查例子：{1,2,3,4}反转再 rotate 得到{3,2,1,4}，回文模块也实测。

代码与来源：STL01；competition/stl/recipes.hpp

【TIP_A_ROTATE】rotate

用途：将一个前缀循环挪到末尾。

实际调用：rotate(begin,begin+k,end)

复杂度：O(n)。

易错：k 需位于合法分割位置；不是普通交换两个元素。

已检查例子：反转数组向左转 1 得到{3,2,1,4}。

代码与来源：STL01；competition/stl/recipes.hpp

【TIP_A_PERMUTE】next_permutation / prev_permutation

用途：小规模全排列枚举与构造。

实际调用：排序初态+do/while next_permutation

复杂度：每次 O(n)，总最多 O(n*n!)。

易错：先处理初始排列；重复元素从排序态出发可避免重复；n 大不能枚举。

已检查例子：aab 实际得到 3 种；next 到头复位后 prev 返回 false。

代码与来源：STL01；competition/stl/recipes.hpp

【TIP_A_ACCUMULATE】accumulate

用途：一趟求和/聚合，防止 int 中间值溢出。

实际调用：accumulate(...,0LL)

复杂度：O(n)。

易错：初值 0 会选择 int 累加类型；事后赋给 ll 不能补救。

已检查例子：2e9+2e9-1 实际输出 3999999999。

代码与来源：STL01；competition/stl/recipes.hpp

【TIP_A_PARTIAL_SUM】partial_sum

用途：一次构造全部前缀/后缀累计。

实际调用：partial_sum 普通迭代器与反向迭代器

复杂度：O(n)。

易错：中间类型取决于输入元素，不只输出类型；输入用 ll。

已检查例子：前缀第二项 4e9，后缀第二项 1999999999。

代码与来源：STL01；competition/stl/recipes.hpp

【TIP_A_MIN_MAX】min_element / max_element

用途：从非空区间找最值或其位置。

实际调用：*min_element；*max_element

复杂度：O(n)。

易错：空区间返回 end，不可解引用；比较次数不是 O(log n)。

已检查例子：固定 ll 数组最小-1、最大 2e9。

代码与来源：STL01；competition/stl/recipes.hpp

【TIP_A_RECURSIVE_LAMBDA】递归 lambda

用途：局部回溯需捕获数组与答案容器。

实际调用：dfs(dfs,0,0)；self(self,...)

复杂度：例子 O(2^n)，栈深 O(n)。

易错：给递归 lambda 明确返回类型；深递归仍受栈限制。

已检查例子：{1,3}子集和得到 0/1/3/4。

代码与来源：STL01；competition/stl/recipes.hpp

【TIP_A_PREFIX_SUFFIX】前缀/后缀信息

用途：区间和，或枚举中点把左右信息合并。

实际调用：PrefixSum(a).sum(l,r)；partial_sum 正/反向

复杂度：预处理 O(n)，区间和 O(1)。

易错：1 下标前缀用 pre[r]-pre[l-1]；STL 示例为 0 下标，不能混用。

已检查例子：ROUND01 前缀差分随机 oracle 与 STL 大数前后缀样例。

代码与来源：CO02、STL01；core/basics.hpp、competition/stl/recipes.hpp

【TIP_A_BITS】lowbit / popcount / ctz / clz

用途：最低位、1 的数量、最低/最高有效位的位置。

实际调用：lowbit/bit_count/lowest_bit_index；countr_zero/countl_zero；非零时__builtin_ctzll/clzll

复杂度：固定 64 位 O(1)。

易错：GNU ctz/clz(0)未定义；C++20 对应 count 函数对 0 返回 64；1ULL<<k 要求 k<64。

已检查例子：test_integer_reference_recipes 对 mask0..255 核验 count 与非零 builtin；显式 0 和 12 边界。

代码与来源：M01、STL01；math/integer_bits.hpp、competition/stl/recipes.hpp

【TIP_A_MASKS】mask、子集与限定全集补集

用途：枚举某集合所有子集，或求只在 B 位内的补集。

实际调用：sub=(sub-1)&mask；sub==0 后 break；full^mask

复杂度：子集 O(2^popcount(mask))，全部 mask O(2^B)。

易错：不在 0 处退出会循环回原 mask；~mask 会把高位也翻转，有限全集用 full^mask。

已检查例子：所有 8 位 mask 枚举子集数=2^popcount，补集与原集交 0 且并 full。

代码与来源：M01、STL01；math/integer_bits.hpp、competition/stl/recipes.hpp

【TIP_A_GCD_LCM】gcd / lcm

用途：公因子、公倍数与周期合并。

实际调用：gcd_abs(-6,4)；lcm_checked(6,4)

复杂度：O(log max(|a|,|b|))。

易错：ll 最小值不能直接 abs；lcm 先除后乘仍可能溢出，checked 返回 optional。

已检查例子：原 M01 边界+随机测试，STL 引用例显式结果 2 和 12。

代码与来源：M01；math/integer_bits.hpp

【TIP_A_DIV_MOD】ceil_div / floor_div / 负数除模

用途：有负数的上/下取整，以及规范非负余数。

实际调用：floor_div/ceil_div；norm_mod；-7/3 与-7%3

复杂度：单次 O(1)。

易错：C++除法向 0 截断；b 不能 0；LLONG_MIN/-1 的数学商超 ll；模数须符合接口契约。

已检查例子：PM03/PM05 独立数学随机 oracle；STL 显式-7/3=-2、余数-1、归一化 2。

代码与来源：PM03、PM05、M01；competition/math/integer_division.hpp、competition/math/invariants.hpp、math/integer_bits.hpp

【TIP_A_I128】__int128 防溢出与输出

用途：乘积/累计可能超过 ll，但在 128 位范围内。

实际调用：(__int128)x*n；to_string_i128；特殊输入先 cin 读十进制文本

复杂度：固定宽度运算 O(1)，十进制输出 O(位数)。

易错：转换必须发生在乘法之前；不为原 read 扩展接口；本轮未宣称提供任意长度/i128 解析器。

已检查例子：runtime 实际输出-63000000000、0 与-2^127；输入示例使用 ll 和 cin 字符串。

代码与来源：CO02、IO02；core/basics.hpp、core/main.cpp

【TIP_A_INVALIDATION】迭代器失效、多测清空

用途：容器在遍历中变化或跨多个测试复用。

实际调用：erase 返回值/clear；median_init/lazy_init/trie_init/mex_init

复杂度：清空按已用大小计；Trie O(26*已用结点)。

易错：vector 重分配/erase 导致失效；unordered rehash 使迭代器失效；仅重置 tot 不清 Trie 边会污染下一组。

已检查例子：容器 clear、多个堆 init、MEX300 轮 init、Trie 满容量后 init 再查询全空。

代码与来源：STL01、STL02、TRS02、MEX02；competition/stl/recipes.hpp、competition/stl/heaps_sets.hpp、competition/string/trie.hpp、competition/game/mex.hpp

【TIP_A_COST】时间/空间复杂度估计与容器选择

用途：约束决定 n^2、n log n 或线性方案，防止只记单个 API。

实际调用：同样任务比较 sort/heap/set/计数数组；可用容量显式估计

复杂度：n=2e5 时 n^2约 4e10，n log n 为数百万量级；时限需结合常数。

易错：哈希不保证最坏 O(1)；map/set 每节点内存较大；1e8 级操作不是跨机器统一阈值。

已检查例子：STL 模型分别与扫描/排序 oracle 比较；各模块元数据有实际复杂度与测试规模。

代码与来源：STL01、STL02、PSD01、DS05；competition/stl/recipes.hpp、competition/stl/heaps_sets.hpp、competition/data_structure/fenwick.hpp、data_structure/sparse_table.hpp